Singl.eView™

Operational Support Guide V2.0

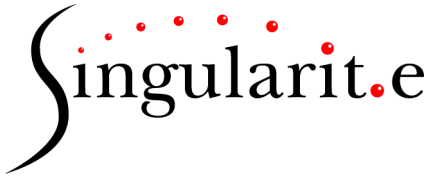
for

Singl.eView Convergent Billing ICP

and

Singl.eView Convergent Billing IP/Data





Printing History
First Printing, August, 2000

Document Number
IP373-55-017, V2.0

Disclaimer

Although every effort has been made to ensure the accuracy of the information contained in this document, ADC Software Systems Ireland Limited makes no warranty, expressed or implied, with respect to the quality, correctness, reliability, currency, accuracy, or freedom from error of this document or the products it describes. ADC Software Systems Ireland Limited may make improvements and/or changes to the products and services described in this document at any time. ADC Software Systems Ireland Limited disclaims all liability for any direct, indirect, incidental, consequential, special, or exemplary damages resulting from the use of the information in this document or from the use of any products described in this document. Mention of any other product not owned by ADC does not constitute an endorsement of that product by ADC Software Systems Ireland Limited. Data used in examples and sample data files are intended to be fictional and any resemblance to real persons or companies is entirely coincidental.

Persons reading this document should rely on their own enquiries in making any decisions touching on their own interests.

The following is a trademark of ADC Software Systems Ireland Limited:

Singl.eView™

Copyright Information

©2000 by ADC Software Systems Ireland Limited. All rights reserved. Apart from any use permitted under the Copyright Act, no part may be reproduced by any process without the written permission of ADC Software Systems Ireland Limited, IDA Business Park, Dangan, Galway, Ireland.

Preface ..... **xi**
Welcome to Singl.eView Convergent Billing xi
Who Should Read this Guide xi
Before You Begin xii
Other Singl.eView Documentation xii
Conventions Used in this Guide xiii

Part 1 – Topics for Singl.eView ICP and Singl.eView IP/Data Functionality

Chapter – 1 Startup and Shutdown Procedures **1-1**
Starting and Stopping ORACLE 1-1
Starting ORACLE 1-1
Stopping ORACLE 1-2
Starting and Stopping TUXEDO 1-3
Starting TUXEDO 1-3
Stopping TUXEDO 1-3
Starting and Stopping Singl.eView Convergent Billing 1-4
Starting Singl.eView Convergent Billing and TUXEDO 1-4
Stopping Singl.eView Convergent Billing and TUXEDO 1-4
Starting Singl.eView Convergent Billing processes 1-4
Stopping all Singl.eView Convergent Billing processes 1-4
Starting and stopping individual Singl.eView Convergent Billing processes 1-5

Chapter 2 – Scheduling Tasks **2-1**
Key Terms 2-2
Configuration 2-3
Scheduled Task Despatcher (STD) process 2-3
Task Launch and Monitor (TLM) process 2-5
Creating a Schedule 2-7
Fields 2-7
Saving 2-13
Using Schedules 2-15
Monitoring a schedule 2-15
Scheduling a future one-off task 2-16
Scheduling an immediate one-off task 2-16
Troubleshooting 2-17
No task was run 2-17
Too many tasks are running 2-18
Error when saving a schedule 2-18
Tips and Recommendations 2-19

Table of Contents

Chapter 3 – Rating 3-1

Configuration 3-1
ENM process for switch files 3-1
FTP process 3-3
Batch validation 3-3
ENM process for rentals 3-5
Other rating processes 3-5
Sizing 3-7
Singl.eView ICP Event Fix-Up Script 3-7
Troubleshooting 3-9
Tips and Recommendations 3-11

Chapter 4 – Client Requirements 4-1

Key Terms 4-1
Billing Schedule Parameters 4-3
How Many Invoice Cycles? 4-5
How Many Billing Schedules? 4-6
Maximising billing performance 4-6
Maximum number of schedules possible 4-6
Customers per schedule 4-7
Billing duration estimates 4-7
Schedule Naming Conventions 4-8
Schedule Planning 4-10
Days/Month 4-10
Hours/Day 4-10
Determining the window proportions 4-10
Allowance for reprocessing 4-10
Allowance for other server processes 4-11
Assigning Customers To Billing Schedules 4-12
Selecting the right schedule 4-12
Invoice Messages 4-13
Message locations 4-13
Invoice message naming convention 4-14
Default message 4-14
Pre-Billing Checks 4-15
Check for DVP errors 4-15
All events rated and errors corrected 4-15
Invoice messages set up 4-15
Environment checks 4-15
Managing Billing Schedules 4-16
Schedule progress chart 4-16
Execution of Billing Schedules 4-18
Billing outputs 4-18
Task status 4-18
Task output text 4-18
Number of invoices generated 4-19
log.out file 4-19
Managing Billing Outputs 4-19

RGP event files 4-19
Debug details 4-19
Post Billing Checks 4-20
DVP 4-20
Spot check invoice details 4-20
Spot check invoice images 4-21
Reports 4-22
Apply Invoices 4-22
Regular schedule 4-22
Schedule naming convention 4-23
Immediate task 4-24
Printing and Other Outputs 4-24
Troubleshooting 4-25
Performance 4-26
Number of CPUs 4-26
Number of customers per schedule 4-26

Chapter 5 – Printing and Other Outputs 5-1
Key Terms 5-2
Overview 5-3
When to modify a GOP schedule configured for printed output 5-3
Main GOP components 5-3
Singl.eView ICP and IP/Data Outputs 5-4
Output image type 5-4
Output Device Configuration 5-5
Printing invoices or treatment letters 5-5
Emailing invoices 5-5
Printing reports 5-6
Re-printing invoices 5-7
Output Method Types 5-8
Output method types specific to Singl.eView ICP and Singl.eView IP/Data 5-8
Standard output method types 5-8
Output Methods 5-9
Example 5-9
GOP Schedule Configuration for Printed Invoices 5-10
Naming convention 5-10
Email Invoices Schedule Type 5-11
Scheduling 5-11
Actions triggered by running the Email Invoices schedule 5-12
Output results 5-13
Troubleshooting 5-13
Re-generating Email Invoices 5-14
Printing and Reprinting 5-15
Printing 5-15
Reprinting 5-15
Troubleshooting for Printed Invoices 5-16
Failed print task 5-16
Other failed output task 5-16

Table of Contents

Chapter 6 – Singl.eView ICP Downstream Interfaces 6-1
- Overview 6-2
  - CNAM 6-2
  - E911 6-3
  - LIDB 6-4
- Configuration 6-6
  - Maintaining the Update Capture Options function 6-6
  - Configuring the Update Capture schedule 6-10
  - Maintaining the interface derived attribute tables 6-12
- Outputs 6-13
  - Sending files by FTP 6-13
- When to Run CLEC Update Capture Extraction 6-14
- Recommendations 6-14

Chapter 7 – Treatment 7-1
- Treatment Queries 7-2
  - Promise to pay 7-3
- Using the Treatment Engine 7-5
  - Schedule types 7-5
- Tips and Recommendations 7-6
- Treatment Report 7-6

Chapter 8 – PC Operations 8-1
- Clearing Out Temporary Invoice Files 8-1
- Caching 8-2
  - Caching static data 8-2
  - Caching commonly used queries 8-2
  - Turning on caching 8-3
  - Saving your query cache 8-3
  - When to clear the query cache 8-4
  - How to clear the query cache 8-4
  - Troubleshooting 8-5
  - Recommendations 8-5

Chapter 9 – Database Auditing 9-1
- Why Audit? 9-1
  - Security 9-1
  - Problem solving 9-1
- How to Audit a Table 9-2
  - Creating the audit configuration item 9-2
  - Full versus partial auditing 9-4
  - Running the audit schedule 9-4
  - Changing audit configuration 9-5
  - Stopping auditing on a table 9-5
- Accessing Audit Data 9-6
  - Audit search form 9-6
  - Audit report 9-8

Interpreting Audit Data 9-11
Date 9-11
Operator 9-11
Operation 9-11
Record details 9-11
Maintaining Audit Tables 9-12
Archiving audit data 9-12
Archive schedules 9-12
Recommendations 9-13

Chapter 10 – Database Archiving 10-1
Table Naming Conventions 10-1
The Archiver 10-3
Exporting 10-3
Importing 10-4
Entity replacement strategies 10-5
Examples 10-9
Exporting 10-9
Importing 10-10

Chapter 11 – Database Validation 11-1
Overview 11-1
Performance 11-1
Referential integrity 11-1
Pro-active management 11-2
DVP Checks for Singl.eView IP/Data 11-2
Customised DVP 11-2
Configuration 11-3
DVP schedule configuration 11-3
Local DVP schedule configuration 11-4
Schedule type to run all DVPs 11-5
Execution 11-5
Full DVP execution 11-5
Limited DVP execution 11-5
Multiple DVP execution 11-6
Sample limited DVP usage 11-7
Output Results 11-8
Sample output 11-9
Identifying errors 11-10
Correcting errors 11-10
Correcting Errors for the Singl.eView IP/Data DVP 11-11
When to Run the DVP 11-11
When not to run the DVP 11-11
Recommendations 11-12

Table of Contents

Chapter 12 – Table Partitioning 12-1
- Overview 12-1
- Creating Partitions 12-2
  - Determining the number of partitions 12-2
  - Creating partitions 12-2
- Rotating Partitions 12-3
- Partition Maintenance 12-5
  - Create and drop partition 12-5
  - Sort partition 12-5
  - Archiving partition 12-5

Chapter 13 – General Housekeeping 13-1
- Overview 13-1
  - Switching files 13-1
  - Purging server files 13-2
  - Purging client files 13-2

Chapter 14 – Backup and Recovery 14-1
- Overview 14-1
- Backup Strategy 14-2
  - Points to consider 14-2
  - Possible disaster scenarios 14-2
- Singl.eView ICP Specific Backup Issues 14-3
- Types of Backup 14-4
  - Medium to low availability 14-4
  - High availability 14-5

Chapter 15 – Proactive Monitoring 15-1
- Machine Monitoring 15-1
  - CPU activity 15-1
  - Disk space 15-1
- Server Process Monitoring 15-2
  - Daemon processes 15-2
  - Task output 15-2
  - log.out 15-3
  - ULOG 15-3
  - Monitoring of the Singl.eView RADIUS Accounting Real Time Rating process 15-4
  - Database validation 15-5
- Client Monitoring 15-5
- TUXEDO Monitoring 15-7
  - Printclient 15-7
  - Printserver 15-8
- Database Monitoring 15-9

Chapter 16 – Schedule Planner 16-1
Example Scenario 16-1
Rating considerations 16-2
Billing considerations 16-2
Schedules 16-5
Schedule Plan 16-8
Monthly schedule plan 16-8
Weekly schedule plan 16-16
Daily schedule plan 16-17
Daily time windows 16-19

Chapter 17 – Payment Interfaces and Schedule Types 17-1
Overview of Outbound Payments and Confirmation Letter Schedules 17-2
PAP Auto Payment Generation Schedule Type 17-3
Scheduling 17-3
Actions triggered by running the PAP Auto Payment Generation schedule 17-5
Output results 17-7
PAP Payment File Generation Schedule Type – Pre-authorised Payments 17-8
Scheduling 17-8
Actions triggered by running the PAP Payment File Generation schedule for pre-authorised payments 17-8
Output results 17-9
PAP Revoke Process Schedule Type 17-10
Scheduling 17-10
Actions triggered by running the PAP Revoke Process schedule 17-11
Output results 17-11
PAP Confirm Letter Generation Schedule Type 17-12
Scheduling 17-12
Actions triggered by running the PAP Confirm Letter Generation schedule 17-13
Output results 17-14
PAP Confirm Letter Revoke Schedule Type 17-14
Scheduling 17-14
Actions triggered by running the PAP Confirm Letter Revoke schedule 17-14
Output results 17-15
Process Lockbox Files Schedule Type – Inbound Payments 17-15
Scheduling 17-15
Actions triggered by running the Process Lockbox Files schedule 17-16
Output results 17-17
FTP Exchange Schedule Type 17-18
Scheduling 17-18
Actions triggered by running the FTP Exchange schedule 17-19
Output results 17-20

Part 2 – Specific Topics for Singl.eView IP/Data Functionality

Chapter 18 – RADIUS Accounting Real Time Rating Process 18-1
Introduction to the RADIUS Server and Real Time Rating 18-2
The RADIUS server and real time rating 18-3
RADIUS protocol 18-3
The RADIUS Accounting Real Time Rating Process and Singl.eView Convergent Billing 18-4
RADIUS Accounting start and stop records 18-5
Internal and External Components of the RADIUS Accounting Real Time Rating Process 18-8
RADIUS Accounting Real Time Rating process components overview 18-8
External components 18-9
Internal components 18-10
Sockets and the RADIUS Accounting Real Time Rating Process 18-12
Socket Components 18-13
File Components of the RADIUS Accounting Real Time Rating Process 18-14
Configuration File 18-14
Secrets File 18-14
Data Dictionary 18-14
Restart Data 18-14
Log File 18-15
Batch File 18-15
Error Batch File 18-15
Understanding and Running the RADIUS Accounting Real Time Rating Process 18-15
Starting the RADIUS Accounting Real Time Rating process 18-17
Controlling the RADIUS Accounting Real Time Rating process after start up 18-18
RADIUS Accounting Real Time Rating process Input Thread 18-21
RADIUS Accounting Real Time Rating process Output Thread 18-22
RADIUS Accounting Real Time Rating process Controller Thread 18-23
RADIUS Accounting Real Time Rating Socket Manager 18-23
Restart Data File 18-25
Batched output and error streams 18-26
Log File 18-26
Archiving RADIUS Accounting start and stop records on the RADIUS server 18-26

Chapter 19 – Singl.eView IP/Data Updates for Vertex 19-1
Singl.eView IP/Data Category, Service, and Tax Codes 19-2
Category and service codes 19-2
Tax codes 19-2
CTX150 Program 19-3
Transaction File Generator spreadsheet 19-3
Using the Transaction File Generator Spreadsheet and Running CTX150 19-4
Generating the transaction file 19-5
Transferring the transaction file and running CTX150 19-6
Troubleshooting 19-7

Configuration Checklist for Updates to Vertex 19-9
✓ CLEC_NE TAXING_ CATEGORY reference type 19-9
✓ CLEC_NE_ TAXED_ SERVICE_IP reference type 19-10
✓ dCLEC_NE_ TaxFields& derived attribute 19-10
✓fCMN_TAX_ GatherSvcChg function 19-11
✓dCMN_SVC_ TAX_Group$ 19-13

Chapter 20 – Automatic Provisioning: Troubleshooting Guide 20-1
Troubleshooting Aids for Automatic Provisioning 20-2
How Automatic Provisioning is Triggered in Singl.eView Convergent Billing 20-2
Troubleshooting Aids: Pop-up Error Messages 20-7
Troubleshooting Aids: UNIX Log Directory 20-8
Troubleshooting Aids: Customer Hierarchy Form 20-9
Troubleshooting Aids: Service Detail Form 20-10
Troubleshooting Aids: Provisioning Error Queries 20-11
Error Messages Generated by EPM provisioning functions 20-12

Chapter 21 – IP/Data Event Feeds and Normalised Event Layouts 21-1
Non-Native Event Feeds 21-2
RADIUS Accounting Server Log Data Feed 21-2
XACCT RADIUS Accounting CSV Data Feed 21-3
DILs for IP/Data Event Feeds 21-4
Normalised Event Fields – All Usage Events 21-4
Normalised Event Fields – Dialup Usage 21-9
Related reports 21-11
Normalised Event Fields – DSL Internet Usage 21-11
Normalised Event Fields – Data Services Usage 21-13
Related reports 21-14
Normalised Event Fields – GVAS 21-15
Normalised Event Fields – Web Hosting 21-16
Normalised Event Fields – Payments Gateway 21-18

Chapter 22 – Speed Value Management 22-1
Master Reference Types for Speed Values 22-3
Reference codes 22-5
Abbreviation 22-6
Code Label for single-speed values 22-6
Code Label for dual-speed values 22-7
Enabled? 22-7
Description 22-7
IP_SPEED_INSTANCES and IP_DUALSPEED_INSTANCES Reference Types 22-8
Speed Reference Type Instances 22-10
Adding New Speed Value Reference Type Instances 22-11
Adding and Changing Speed Values 22-14
Update Speed Reference Types Schedule Types 22-15
Scheduling 22-15
Actions triggered by running the Update Speed Reference Types schedule types 22-16
Output results 22-17

Part 3 – Security for Web Enablement

Chapter 23 – Security for Web Enablement 23-1
Security and Web Enablement 23-2
Authentication 23-2
Passwords 23-2
Sessions 23-2
Authorisation 23-3
Setting up Web Users 23-3
Permissions 23-5
Servlets 23-6
Entity Classes 23-7
Entity Subclasses 23-7
Entities 23-7
Roles 23-9
Derived attributes for role creation 23-9
Encryption 23-12
Auditing 23-13

Appendix A – Configuration File Reference Information and Example

Appendix B – Outbound File Format for Pre-authorised and Credit Card Payments

Appendix C – Inbound File Format for Lockbox

Appendix D – Configuration for Process Lockbox Files Schedule Type

Appendix E – DILs and Configuration Entities for IP/Data Event Feeds

Preface

Welcome to Singl.eView Convergent Billing

Singl.eView Convergent Billing has been designed specifically for new generation customer care and billing environments to increase your competitive advantage by making it easier to design, deliver, and bill the products and services your customers want. It provides modules for billing mediation, rating, discounting, and bill production.

It is based on industry standard architecture, relational database technology and interfaces, to ensure portability across operating platforms.

Who Should Read this Guide

This document details how to operate and maintain Singl.eView Convergent Billing ICP (hereafter referred to as Singl.eView ICP) and Singl.eView Convergent Billing IP/Data (hereafter referred to as Singl.eView IP/Data). It covers areas such as starting and stopping the system, configuring and performing rating and billing, maintaining the database, and performing system backups. For detail on reports, refer to the *Reports Guide V2.0 for Singl.eView Convergent Billing ICP and Singl.eView Convergent Billing IP/Data*.

The audience for this document is anyone involved (from both the deployment and client teams) with the operation and maintenance of Singl.eView ICP and Singl.eView IP/Data.

Before You Begin

This guide assumes that you have some experience in using:

- Windows 95 or Windows 98
- Singl.eView Convergent Billing
- UNIX.

It is also assumed that you have a thorough knowledge of Singl.eView ICP concepts.

Other Singl.eView Documentation

This guide is part of the Singl.eView ICP and Singl.eView IP/Data deployment documentation suite. It provides Singl.eView ICP and Singl.eView IP/Data specific information on the operation and maintenance of Singl.eView Convergent Billing. It is supplemented by Singl.eView Convergent Billing core product documentation. In particular, the *System Administration Guide*, *System Operations Guide* and *Reports Guide for Singl.eView Convergent Billing* contain related information.

If information contained in this manual conflicts in any way with information in a core product document, this manual should take precedence.

Conventions Used in this Guide

The following conventions are used in this guide:

File ⇨ Exit	Indicates that you should open the **File** menu and select the **Exit** option.
`UNIX output`	In interactive and code examples, this typeface is used to indicate system output.
`UNIX input`	In interactive examples, this bold typeface is used to show user input.
Ctrl+F4	In procedures, a key sequence indicates that you must hold down the **Ctrl** key while you press another key. In this example, you must hold down the **Ctrl** key and press function key **F4**.

Use of a mouse in Singl.eView Convergent Billing is identical to that in other Windows 95 or Windows 98 applications. The following terminology is used in this guide to describe the mouse functions.

Select	Highlight one or more characters to perform an operation on those characters.
Click	Point to an object, then press and release the left mouse button.
Double-Click	Point to an object, then press and release the left mouse button twice in quick succession.
Drag	Point to an object, hold down the left mouse button, reposition the mouse, and release the mouse button.

Preface

Part 1
Topics for Singl.eView ICP and Singl.eView IP/Data Functionality

This part of the *Operational Support Guide* covers all topics that apply to both Singl.eView ICP and Singl.eView IP/Data. It consists of the following seventeen chapters:

- *Chapter 1, Startup and Shutdown Procedures*
- *Chapter 2, Scheduling Tasks*
- *Chapter 3, Rating*
- *Chapter 4, Billing*
- *Chapter 5, Printing and Other Outputs*
- *Chapter 6, Singl.eView ICP Downstream Interfaces*
- *Chapter 7, Treatment*
- *Chapter 8, PC Operations*
- *Chapter 9, Database Auditing*
- *Chapter 10, Database Archiving*
- *Chapter 11, Database Validation*
- *Chapter 12, Table Partitioning*
- *Chapter 13, General Housekeeping*
- *Chapter 14, Backup and Recovery*
- *Chapter 15, Proactive Monitoring*
- *Chapter 16, Schedule Planner*
- *Chapter 17, Payment Interfaces and Schedule Types*

Chapter 1
Startup and Shutdown Procedures

This section describes the procedures for starting and stopping Singl.eView Convergent Billing server processes. The following information is included:

- Starting and stopping ORACLE
- Starting and stopping TUXEDO
- Starting and stopping Singl.eView Convergent Billing
- Starting and stopping individual Singl.eView Convergent Billing processes.

For more information on managing processes, refer to *Chapter 6, Managing Server Process* in the *System Administration Guide for Singl.eView Convergent Billing*.

Starting and Stopping ORACLE

This section provides information on starting and stopping the ORACLE database. It is necessary to be logged on as database owner.

Starting ORACLE

To start the ORACLE database:

1. Log in as root.
2. Log in to the database home directory with the command

   `su – <database owner>`
3. Identify the ORACLE_SID of the running database using **`ps –ef`**, or by referring to /etc/oratab.
4. Issue the following command to set the environment variables

   `. oraenv`

5. Identify the database to shut down by entering the ORACLE_SID from step 3 at the prompt.
6. Connect to the server manager with the command

   svrmgrl
7. Connect to the database with the command

   connect internal
8. Issue the following command to start up the database

   startup
9. Quit the server manager with the command

   Exit

Stopping ORACLE

To shut down the ORACLE database:

1. Log in as root.
2. Use standard procedures to warn any logged on users.
3. Shut down the associated Singl.eView Convergent Billing instance before proceeding to shut down the database.
4. Log into the database home directory with the command

   su - <database owner>
5. Identify the ORACLE_SID of the running database using **ps –ef**, or by referring to /etc/oratab.
6. Issue the following command to set the environment variables

   . oraenv
7. Enter the ORACLE_SID from step 5 at the prompt to identify the database to be shut down.
8. Connect to the server manager with the command

   svrmgrl
9. Connect to the database with the command

   connect internal
10. Issue the following command to shutdown the database

    shutdown

11. This command will time out if users are still logged on. In this case, issue the following command

```
shutdown immediate
```

12. Quit the server manager with the command

```
exit
```

Starting and Stopping TUXEDO

This section provides information on starting and stopping the TUXEDO server. It is necessary to be logged in as the Singl.eView Convergent Billing owner.

Starting TUXEDO

To start the TUXEDO server, enter the following command:

```
tmboot –y
```

Note: Make sure the Singl.eView Convergent Billing instance has been shutdown before trying to shutdown TUXEDO.

Stopping TUXEDO

To shutdown the TUXEDO server, enter the following command:

```
tmshutdown –y
```

Note: These commands normally do not need to be run, since they are automatically run when starting and stopping the Singl.eView Convergent Billing server.

Starting and Stopping Singl.eView Convergent Billing

This section provides information on starting and shutting down processes. It is necessary to be logged in as the Singl.eView Convergent Billing owner.

Starting Singl.eView Convergent Billing and TUXEDO

To start Singl.eView Convergent Billing, including all the necessary TUXEDO processes, enter the following command:

```
startup_express
```

This command first executes a **`tmboot -y`** command to start the TUXEDO processes, then the command **`ecp "biBmpStart&()"`** to start the process.

Stopping Singl.eView Convergent Billing and TUXEDO

To stop Singl.eView Convergent Billing, including all the TUXEDO processes, enter the following command:

```
shutdown_express
```

This command first executes the command **`ecp "biBmpStop&()"`** to stop the processes, then the command **`tmshutdown -y`** to stop TUXEDO.

Starting Singl.eView Convergent Billing processes

To start the Backend Monitor Process (BMP) which starts all processes:

1. Stop all processes with the command

   ```
   ps –ef | grep <SavilleExpress owner>
   ```

2. Verify that the TUXEDO processes are running.

3. Start the processes with the command

   ```
   ecp "biBmpStart&()"
   ```

Stopping all Singl.eView Convergent Billing processes

To stop all processes, enter the command

```
ecp "biBmpStop&()"
```

Starting and stopping individual Singl.eView Convergent Billing processes

Individual processes are started and stopped using the following ECP commands.

To start a process, use the command

```
ecp "<startfunction>"
```

where `<startfunction>` is taken from the table below, and the parameter `ProcessNum&` is the number of the process to stop. For example, to start ENM process number 3, enter

```
ecp "biEnmStart&(3)"
```

ECP Functions for Starting Individual Processes

Process	Function	Description
Event Rating Broker (ERB) process and any children (ENM, ERT and ERO) with the ENABLED attribute set to TRUE	**`biBkrStart&(ProcessNum&)`**	Starts the specified ERB process.
Event Normalis-ation (ENM) process	**`biEnmStart&(ProcessNum&)`**	Converts raw data in a variety of input formats.
Event Rating Output (ERO) process	**`biEroStart&(ProcessNum&)`**	Takes the normalised events from the ENM process, and charges from the ERT process and sends them to a remote host, to the database, or to SQL*Loader files.

Process	Function	Description
Event Rating (ERT) process	**biErtStart&(ProcessNum&)**	Applies tariffs to the normalised events to generate charges.
Event Socket Interface (ESI) process	**biEsiStart&(ProcessNum&)**	Provides a single bi-directional TCP/IP socket interface between a remote host and the rating engine.
Scheduled Task Dispatcher (STD) process	**biStdStart&(ProcessNum&)**	Generates and queues tasks in accordance with their schedule.
Task Launch and Monitor (TLM) process	**biTlmStart&(ProcessNum&)**	Launches tasks when instructed by the STD process.

To stop a process use the command

```
ecp "<stopfunction>"
```

where `<stopfunction>` is taken from the table below, and the parameter `ProcessNum&` is the number of the process to stop. For example, to stop the ERB and all its running children, enter:

```
ecp "biBkrStop&(1)"
```

ECP Functions for Stopping Individual Processes

Process	Function
ERB and any children (ENM, ERT and ERO) with the ENABLED attribute set to TRUE	**biBkrStop&(ProcessNum&)**
ENM	**biEnmStop&(ProcessNum&)**
ERO	**biEroStop&(ProcessNum&)**
ERT	**biErtStop&(ProcessNum&)**
ESI	**biEsiStop&(ProcessNum&)**
STD	**biStdStop&(ProcessNum&)**
TLM	**biTlmStop&(ProcessNum&)**

Note: Do not stop the ERB process while rating is in progress, because the ERT process will not shut down correctly.

Chapter 2
Scheduling Tasks

Most server tasks in Singl.eView Convergent Billing are run automatically, under the control of the Scheduled Task Dispatcher (STD) process and the Task Launch and Monitor (TLM) process. The client workstation's Scheduler controls these processes.

The Scheduler can be used to set up tasks that run regularly at predefined intervals, or tasks that run only once, either immediately or at a future date and time. Once set up, these tasks run automatically at the chosen time without further intervention, and a message notifies the person responsible on completion.

This chapter describes how to create and modify schedules, and to monitor the tasks that the schedules create.

Key Terms

The table below lists the key terms used in the area of scheduling tasks.

Term	Description
Schedule	A schedule is a Singl.eView Convergent Billing object that defines all the information necessary to perform a one-off action or to repeatedly perform a task at regular intervals. A schedule specifies: • When a task is to be run for the first time. • How often the task should be repeated, and at what intervals. • What program the task runs to perform its function, and what parameters are passed to it.
Task	A task is a Singl.eView Convergent Billing object that is created by a schedule to perform an action. A schedule may create many tasks, so that an action is repeated at well defined intervals.

Configuration

The Singl.eView Convergent Billing client can create schedules, but it is server processes that create the tasks for a schedule and run the tasks when they are due to be run. These are the STD and TLM processes. One instance of each of these processes needs to be configured and run within the Singl.eView ICP and Singl.eView IP/Data environment.

The STD and TLM processes are configured by Singl.eView Convergent Billing configuration items. The configuration items are preconfigured with the Singl.eView Convergent Billing product and the processes will be automatically started and stopped when the Singl.eView Convergent Billing server is started and stopped (for more information, refer to *Chapter 1, Startup and Shutdown Procedures*). Some of the parameters associated with the STD and TLM processes configuration items may need modification. The parameters required for these two processes are described in the tables below.

Scheduled Task Despatcher (STD) process

Parameters Associated with the STD Process

Parameter	Description
`TLM_SERVICE_NAME`	TCP/IP service name used by the TLM process for inter-process communication. This service name must be defined in the systems */etc/services* file. Example: `tlmATP_DEV`
`MAIL_COMMAND`	System mail command used to send email notifications when tasks have completed. Example: **`/usr/sbin/Mail -s "Server Response"`** **Note:** The above command is only available on Silicon Graphics servers. On HP servers, the nearest available command is: **`/usr/bin/mailx`**
`HOST_NAME`	Name of the host machine on which the STD process will run.
`TLM_HOST_NAME`	Name of the host machine on which the TLM process will run.

Parameter	Description
TRACE_FILE	Full path name of the file in which trace information is written. Example: $ATA_DATA_SERVER_LOG/std1.trc
SERVICE_NAME	TCP/IP service name used by the STD process for inter-process communication. This service name must be defined in the system's */etc/services* file. Example: stdATP_DEV
WORKING_DIRECTORY	Full path to the working directory to be used by the STD process. This directory must exist on the server. Example: $ATA_DATA_SERVER/build
DEBUG_LEVEL	Specifies the level of tracing to be produced. The following two parameters are normally left blank, but can be used to turn on tracing information, which is written to the file specified by the TRACE_FILE parameter.
DEBUG_LEVEL_FLAG	Command line flag passed to the STD process to turn on tracing. This must be set to -d if tracing is to be turned on.
ENABLED	Should always be set to TRUE. By setting it to TRUE, the STD process will be automatically started/stopped when the server processes are started/stopped. If set to FALSE, the STD process will have to be manually started.
PROCESS_NAME	Name of the STD process. This must be set to std. After the server processes are started, a list of the running system processes will show that a process called std is running.

Task Launch and Monitor (TLM) process

Parameters Associated with the TLM Process

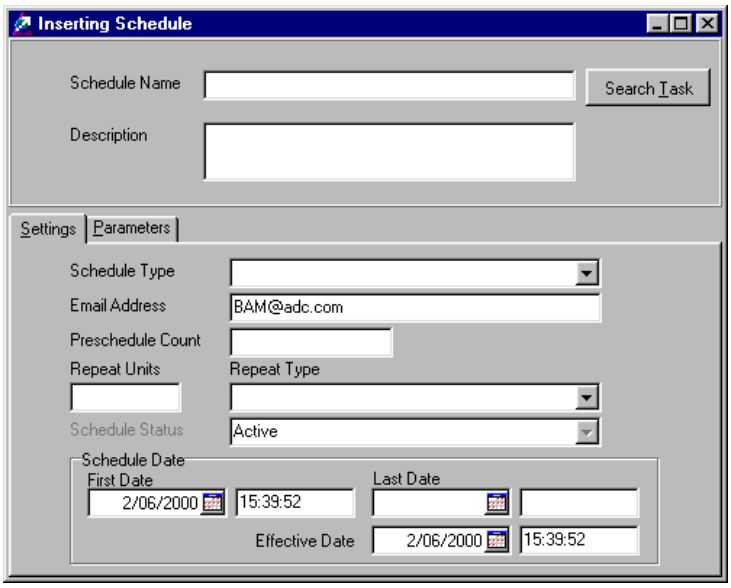
Parameter	Description
WORKING_DIRECTORY	Full path to the working directory used by the TLM process. This directory must exist on the server. Example: $ATA_DATA_SERVER_INPUT
SERVICE_NAME	TCP/IP service name used by the TLM process for inter-process communication. This service name must be defined in the system's */etc/services* file. Example: tlmATP_DEV
TRACE_FILE	Full path name of the file in which trace information is written. Example: $ATA_DATA_SERVER_LOG/tlm1.trc
ENABLED	Should always be set to TRUE so the TLM process will be automatically started/stopped when the server processes are started/stopped. If set to FALSE, the TLM process will have to be manually started.
PROCESS_NAME	Name of the TLM process. This must be set to tlm After starting up the server processes, a list of the running system processes will show that a process called tlm is running.
SEQUENCE_NR_FLAG	Should be left blank.
DEBUG_LEVEL	Specifies the level of tracing to be produced. The following two parameters are normally left blank, but can be used to turn on tracing information, which is written to the file specified by the TRACE_FILE parameter.

Parameter	Description
DEBUG_LEVEL_FLAG	Command line flag passed to the TLM process to turn on tracing. This must be set to -d if tracing is to be turned on.
WAIT_PERIOD	Time (in seconds) that the TLM allows for the tasks that it launches to start up in. Example: 10

Creating a Schedule

To create a schedule:

1. Select the **Server Tasks ⇨ Schedules** option. The *Schedule Search* form is displayed.
2. Click the **New…** button. The *Schedule Definition* form is displayed in Insert mode.



Fields

Field Name	Description
Schedule Name	Name used to uniquely identify this schedule.
Description	Description of the schedule's function. It is important that this be populated with descriptive, meaningful information, to inform users of the purpose of the schedule.

Settings tab

This tab defines all of the parameters that the schedule needs to know to be able to create and run tasks. The key to working with schedules is the ability to set the fields on this tab correctly so that the desired result is achieved. Setting the fields takes practice and the best way to understand how the fields work is to create some schedules and run some tasks and observe what happens.

Field Name	Description
Schedule Type	Tells the schedule the name of the process that will be run when tasks are created for this schedule, and the parameters that the process will take. There are many schedule types pre-defined with the Singl.eView ICP product. For example, the schedule type used to rate event files is called 'Rate Files', while the schedule type used to generate a customer's invoice is called 'Invoice Generation'. New schedule types can also be created. The selection of this field determines the fields that are displayed on the *Parameters* tab.
Email Address	Email address (or list of email addresses) to which an email message will be sent when a task created by this schedule completes. The body of the email message contains the standout output of the task. **Note:** If it is necessary to change email addresses to which a number of schedules are frequently sent, it is recommended that an email alias be set up on the server and the alias entered in the *Email Address* field. For example, there may be a number of schedules to be always sent to, say, the Billing Operations Manager. Instead of entering the address of this manager in each schedule, an email alias (eg. bill_ops) could be created so that only this needs to be changed to point to a new manager's email address. For information on how to set up a mail alias, consult documentation provided for the server system.

Field Name	Description
Preschedule Count	When a new schedule is created or an existing schedule modified, the tasks for the schedule can be regenerated. This parameter specifies how many tasks will be pre-generated for the schedule. For example, if a schedule is running a task once a month and *Preschedule Count* is set to 3, then when the schedule is saved and tasks regenerated, 3 tasks will be created. The first task will be scheduled to run on the date specified by the *First Date* parameter (which will be immediately, if the *First Date* parameter is in the past), the second task one month from the *First Date*, and the third task two months from the *First Date*. Once a task completes, the STD process will create a new task for the schedule (as long as it is not past the *Last Date* parameter). In short, this parameter specifies the number of pending tasks to create. The only reason why more than one task would be prescheduled is that some of the characteristics of the tasks generated by a schedule may need to be changed (and therefore the tasks would need to be generated in advance). If no characteristics need changing, the *Preschedule Count* is set to 1 and the STD process will create the next task whenever the current task completes. An example of prescheduling multiple tasks so they can be updated in advance is described below. A Billing Operations Manager knows that he will be taking long service leave for three months later in the year. He usually receives email notifications for an 'Invoice Generation' schedule, but would like to send them to a co-worker while he is away. To do this, he sets the *Preschedule Count* to 12, so that the schedule generates 12 monthly tasks for the next year in advance. He then updates the 3 tasks for the months he will be away and changes the *Email Address* to the relevant co-worker.

Field Name	Description
Repeat Units	Multiplier to use in conjunction with the *Repeat Type*. For example, a *Repeat Type* of *Month Day* and a *Repeat Unit* of *2* means that tasks will be scheduled every two months.
Repeat Type	Repeat interval for creating future tasks, such as day, week, or month. This is used in conjunction with the defined *Repeat Unit* (*n* in the following examples). The repeat types are: *Second* Every n seconds. *Minute* Every n minutes. *Hour* Every n hours. *Day* Every nth day at the start time. *Week* Every nth week. *Month Day* Every nth month, on the same day of the month as the *First Date*. *Month End Day* Every nth month, using the same number of days from the end of the month as the *First Date*. *Month Start* Every nth month, on the same day of the same week as the *First Date*. *Month End* Every nth month, on the same day of the same week as the *First Date*, calculated back from the end of the month.

Field Name	Description
Schedule Status	A schedule is either 'Active' or 'Complete'. An active schedule creates tasks as required; a complete schedule is effectively turned off, and does not create tasks. When a *Last Date* for a schedule is set, the STD process will not create any tasks to run on any dates after the *Last Date*. Once the last task (which will run prior to the *Last Date*) for this schedule completes, the *Schedule Status* is automatically set to 'Complete' by the STD process. If a *Last Date* for a schedule is not set, it will keep generating tasks forever; however, it can be manually set to 'Complete' to stop it from generating new tasks.
First Date	Start date and time for the first task generated by this schedule. If this is earlier than the date and time at which the schedule is submitted (saved), the first task runs immediately (assuming that tasks are regenerated when the schedule is saved). The *Repeat Units* and *Repeat Type* fields are used in conjunction with the *First Date* used, to derive the value of the run date for all subsequent tasks (after the first task) relating to the schedule.
Last Date	The STD process will not create any tasks for the schedule to be run after this date. Once the last task completes, the schedule's status changes to 'Complete'. If the *Last Date* is not defined, the schedule continues indefinitely.

Field Name	Description
Effective Date	Date and time used by some tasks to retrieve information from the database. For example, if a billing task is to be run in early September, and only data for August used, the *Effective Date* would be set to 31st August. The *Effective Date* for the schedule is used as a basis for computing the effective date of all tasks created for the schedule. The *Repeat Units* and *Repeat Type* fields are used to derive the value of the *Effective Date* for all tasks relating to the schedule. **Note:** The *Effective Date* is not used in any way to determine when tasks will run; this is determined entirely by the *First Date*, *Last Date*, *Repeat Units* and *Repeat Type*. An *Effective Date* is passed as a parameter to all tasks run by the Scheduler and it is up to the task to determine how to use the date. Different processes (such as different schedule types) use the parameter in different ways, and some processes may not use it at all.

Parameters tab

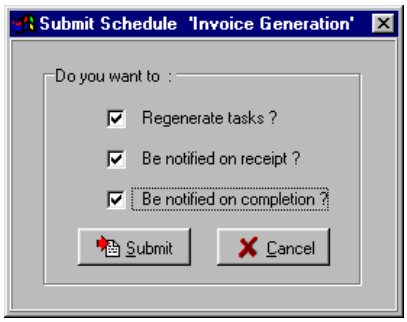
This tab provides fields used to specify values for the parameters that will be passed to tasks created for this schedule.

Note: An *Effective Date* is another (implicit) parameter that is always passed to every task created for a schedule.

The fields on the *Parameters* tab are dependent on the schedule type that has been chosen for the schedule. For example, the 'Rate Files' schedule type has different parameters from the 'Invoice Generation' schedule type.

Saving

Clicking the **Save** button on the toolbar saves the form after all the fields have been specified for a new schedule, and the following window is displayed:



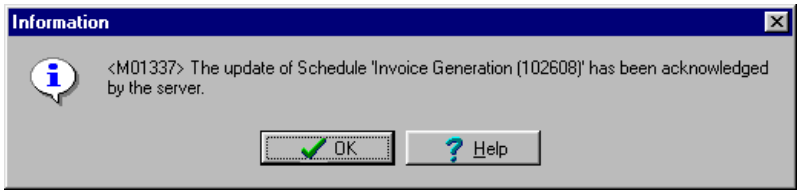
Notes on the checkboxes

Regenerates tasks?
This tells the STD process to discard (delete) any pending tasks that it has created, but which have not yet run, and regenerate new tasks based on the fields defined in the schedule (ie. *Preschedule Count*, *First Date*, *Last Date*, *Repeat Units*, *Repeat Type*).

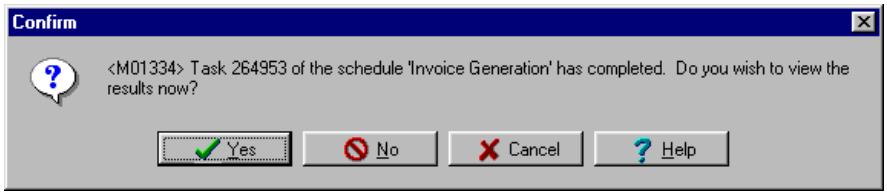
If the schedule is being saved for the first time, there will not be any existing tasks to discard and the STD process will generate new tasks. This box should be selected when a new schedule is created. If it is not selected, the STD process will not generate any tasks, and no tasks will ever run for this schedule.

If an existing schedule is being updated, this box is usually selected; any pending tasks are deleted, and new tasks are generated using data from the updated schedule.

Be notified on receipt?
A pop-up window is displayed to acknowledge that the update of the schedule has been acknowledged by the STD process.



Be notified on completion?
By selecting this box and remaining logged in until the first task for the schedule completes, a pop-up window is displayed.



If the **Yes** button is clicked, the *Task Details* form is displayed, which shows the output generated by the task.

Alternatively, the *Task Search* form can be used to determine if a task for a particular schedule is pending, running or complete.

Using Schedules

Monitoring a schedule

To monitor a schedule, use the following methods:

- Choose *Be notified on completion?* check box in the *Submit Schedule* form which opens when a new or modified schedule is saved.
- Use the *Task Search* form which is displayed by selecting the menu item **Server Tasks ⇨ Tasks.**
- Click the **Search Task** button on the *Schedule Definition* form in View Only mode. This method is useful for looking at tasks for a schedule already displayed.
- Click the **Search Task** button on the *Schedule Definition* form in View Only or Insert mode to search for tasks that meet certain criteria (for example, if all of the tasks belong to a specific schedule). The tasks are displayed in a list. Double-click a task to open the *Task Details* form, which displays all the details for that particular task, including the output that the task generated.

The *Task Status* field indicates whether the task has yet to be run, whether it is currently running, or whether it has completed. The status values for tasks are:

- Pending

  The task has been created, but has yet to be run (the *Start Date* of the task is when the task will be run).

- Running

  The task is currently running.

- Success

  The task has completed and was successful.

- Failure

  The task has completed, but has failed.

If a task fails, it can be resubmitted by selecting the **Re-submit** button on the *Task Details* form. This runs the task again using the same data values. Some of the task fields can be updated before resubmitting a schedule. For example, the *Email Address* can be updated so that the email notification is sent to a different person when the task completes. The *First Date* and the *Effective Date* can also be updated. If the *First Date* is changed to a date/time in the future and the task resubmitted, the task status will revert to 'Pending' and the task will be run at that date/time in the future.

Scheduling a future one-off task

To schedule a future one-off task:

1. Set the *First Date* to the date and time to run the task.
2. Set the *Last Date* to be shortly after the *First Date*; for example, 5 minutes.
3. Set the *Preschedule Count* and the *Repeat Units* to 1, and the *Repeat Type* to a long interval; for example, *Month Day*.
4. Select the *Regenerate task?* check box when saving the schedule.

Scheduling an immediate one-off task

Scheduling an immediate one-off task is similar to scheduling a future one-off task, except that the *First Date* is set to the current date/time instead of in the future.

However, because there is a frequent need to run a single task immediately, Singl.eView Convergent Billing provides a simpler way to do it.

To create an immediate task, select the menu item **Server Tasks ⇨ Immediate Task**. The *Schedule Immediate Task* form is displayed where the following are defined:

- *Schedule Type*

  The type of task to be run (eg. 'Rate Files').

- *Email Address*

  The email address for notification email.

- *Effective Date*

  The effective date to be passed to the task.

- *Parameters*

These are specific to the schedule type selected (and will only appear after selecting the schedule type and moving to another field).

When this form is saved, a single task is run immediately.

Note: When an immediate task is created using the Client, a schedule is actually created that will generate a single task to run immediately. The *Last Date* of the schedule is set to a date/time one second greater than the *First Date* of the schedule. This causes the schedule to generate one task and then become complete. The name of a schedule created to run an immediate task is derived as:

<Schedule ID>-<Operator Name>-<Schedule Type Name>

For example, 6085-PJV-aapjvls

Troubleshooting

The following section describes some common mistakes the user can make with the Scheduler (for example, a task has not run at a particular time), and their prevention.

No task was run

If a task does not run, it is usually a result of not regenerating tasks when saving the schedule. If a schedule is being created or an existing schedule updated, and tasks need to be generated using the new schedule details, these tasks need to be regenerated.

If tasks have been regenerated but will still not run, check that the:

- *First Date/Time* are correct. For example, is 1/4/99 specified instead of 1/3/99, or 10:00:00 pm instead of 10:00:00 am?
- *Repeat Units* and *Repeat Type* are set correctly. For example, a task may have been specified to run every day, whereas it should be run every hour.
- *Schedule Status* is 'Active'. For example, a schedule has completed as the *Last Date* has passed. More tasks might need to be run for this schedule, so the schedule is updated and the *First Date* changed to now and the *Last Date* to some time in the future. If the schedule is saved and tasks regenerated, but the *Schedule Status* not reset from 'Complete' back to 'Active', no tasks will be generated for the schedule.

A less obvious reason for a task not running occurs in the following situation.

A schedule is created to run a task once a month and the first task is to run immediately, so the *First Date* is set to 10 minutes ago. If the current time is 10:15, the *First Date* is set for 10:05. If the *First Date* is any time in the past, the first task will run immediately, with a *Start Date* of the current time. The *Start Date* of the task is **not** the *First Date* of the schedule, if the *First Date* is already in the past.

The schedule is saved and the first task runs immediately. The task fails, and it is found that one of the parameters on the *Parameters* tab of the schedule was specified incorrectly. The parameter in the schedule must be fixed and tasks regenerated so that the first task will run immediately.

After the parameter is fixed and the *First Date* moved to 10:10, the schedule can be saved and tasks regenerated. Despite these measures, the task does not run immediately. The reason is that the STD process has already run a task for this schedule on this day with a *Start Date* greater than the *First Date* of the schedule (ie. the task that failed), so it does not generate another one. It will generate a pending task to be run at 10:10 in one month's time (assuming the *Preschedule Count* is set to 1).

Even though the *First Date* of the Schedule was increased, the task that was run when the *First Date* of the schedule was originally set to 10:05 actually ran with a *Start Date* of 10:15.

To run a task immediately after updating the schedule, the *First Date* of the schedule must be set to a time greater than 10:15.

Too many tasks are running

The *Repeat Type* of the schedule needs to be checked to see if tasks have been accidentally specified to run every second instead of every month.

Too many tasks could also be running if a *First Date* that is a long time in the past has been specified. When the *First Date* of a schedule is in the past, the first task for the schedule will run immediately. If the calculated date for the second task (*First Date + Repeat Units * Repeat Type)* is also in the past, it will run immediately after the first task has completed. Similarly, other tasks for the schedule will also run immediately if they were to be run in the past.

For example, if a schedule is to be run once a day starting from today, the *First Date* is set to the current date. Perhaps the year has been mistyped (eg. 1998 instead of 1999) and so the *First Date* for the schedule is set to one year ago. This will cause the STD process to run the first task immediately, then the second one, then the third, until 366 tasks (ie. one for every day in the last year, plus one for today) have been run, and it has finally caught up to the current date. A task will also be generated for tomorrow, but it will still be pending and will only run tomorrow at the specified time.

Error when saving a schedule

If an error message window appears when a schedule is saved, the most likely cause is that the STD and/or the TLM process/es is/are not running. As these tasks should be automatically started when the server is started, this should never occur. Check to see if these tasks are running using the UNIX ps command

```
ps -ef | grep std
```

Tips and Recommendations

- Every schedule must be assigned a schedule type. A schedule type determines the action performed (process run) when tasks are run for the schedule.
- Remember to regenerate tasks when creating and updating schedules.
- Create an immediate task to perform a one-off task immediately.
- The *Effective Date* in schedules does not control when tasks are run; it is a parameter that is passed to a task. The task is free to use it in whatever way it wants (including ignoring it altogether).
- Tasks can be resubmitted if they fail.
- The *Task Search* form can be used to monitor the status of tasks for a schedule.
- The *Preschedule Count* should be set to one (1), unless multiple tasks need to be pre-generated for the purpose of editing them in advance.
- Make sure the *Last Date* of a schedule is set if no more tasks are to be run after a certain date.
- A schedule can be manually stopped from generating or running any more tasks by changing its status from 'Active' to 'Complete'. Similarly, a schedule can be re-activated (so that it starts running tasks again) by changing its status from 'Complete' to 'Active'.

Chapter 3
Rating

This chapter describes the operational aspects of rating that are specific to the Singl.eView ICP product.

For details on operational aspects that relate to rating, refer to the *System Operations Guide for Singl.eView Convergent Billing*.

For details on how to configure Singl.eView Convergent Billing rating processes, refer to the *System Administration Guide for Singl.eView Convergent Billing*.

Configuration

At least two Event Normalisation (ENM) processes must be configured to run Singl.eView ICP:

1. ENM process for switch files

   This ENM processes the event files generated by the Singl.eView ICP network switches. Multiple switch file ENMs may need to be configured, depending on the number and type of network switches.

2. ENM process for rentals

   This ENM processes rental events generated by the Rental Generation Process (RGP) and the Rental Adjustment Process (RAP).

ENM process for switch files

Every Singl.eView ICP network will have at least one switch that generates event data records for customer calls. However, larger networks may have multiple network switches and not all switches may come from the same switch vendor. This means that the switches may produce records in different formats.

A separate process is required for each different record format that needs to be processed. Therefore, if a network has two switches from different vendors, two ENM processes will need to be configured to process the event files generated by the two switches.

Even if there is just one switch, multiple ENM processes could be configured so that the processing of event files is divided between these ENM processes to increase performance on machines that have multiple processors (CPUs).

Each ENM process requires three directories:

- Input Directory

  Event files generated by the switch that are ready for processing by Singl.eView ICP are placed into this directory. The environment variable `$ATA_DATA_SERVER_INPUT` points to the default system input directory. To better manage event files from different switches, a subdirectory should be created within this directory for each ENM process that will process event files (eg. `$ATA_DATA_SERVER_INPUT/dms100`).

- Error Directory

  Corrupted events detected by the ENM process are written to files in this directory. The environment variable `$ATA_DATA_SERVER_ERROR` points to the default system error directory. To better manage event files from different switches, a subdirectory should be created within this directory for each ENM process that will process event files (eg. `$ATA_DATA_SERVER_ERROR/dms100`).

- Archive Directory

  An event file is moved from the Input directory to this directory after it has been processed. The environment variable `$ATA_DATA_SERVER_ARCHIVE` points to the default system archive directory. To better manage event files from different switches, a subdirectory should be created within this directory for each ENM process that will process event files (eg. `$ATA_DATA_SERVER_ARCHIVE/dms100`).

The 'Input', 'Error', and 'Archive' directories used by an ENM process are defined in the configuration item for that process. There are a number of other parameters that must be assigned specific values for an ENM process configuration item that will be used to process events from a network switch. These parameters are shown below (with examples assuming that the switch is a Nortel DMS 100 called dms100).

Parameter	Example Value
INPUT_METHOD	file
HOME_DIR	$ATA_DATA_SERVER_INPUT/dms100
ERROR_DIR	$ATA_DATA_SERVER_ERROR/dms100
ARCHIVE_DIR	$ATA_DATA_SERVER_ARCHIVE/dms100
FILE_TYPE	CLEC AMADMS 100
EVENT_TYPE	CLEC Usage

Note: An ENM process configuration item includes other parameters that must also be configured.

FTP process

Singl.eView ICP provides a file transfer process that uses the Internet FTP protocol to automatically transfer files from a remote machine to the Singl.eView Convergent Billing server. This process can be used to automatically transfer event files from the switch or mediation system to the appropriate ENM process input directory.

Batch validation

Many network switches produce event datafeeds that are made up of batches of events. A batch of events is defined by:

- A header record
- Zero, one or more event records
- A trailer record.

The header record typically identifies the date and time that the batch was created. Some datafeeds also contain header records that contain a sequence number indicating the number of the batch (unique for the switch that produced the batch).

The trailer record usually contains a field that provides a count of the number of event records that should have been in the batch. This value can be used to check that the correct number of records have been processed and therefore whether the batch is valid.

Singl.eView Convergent Billing keeps track of the batches that it processes from the various switch datafeeds. The following uniquely identifies each batch:

- Batch source

  This is typically a physical switch. Each source that generates event batches must be identified by a different name.

- Date and time that the batch was created by the source

  This is typically provided as a field in the header record of the batch.

When files are being processed, batch validation is performed to ensure that:

- The same batch (as identified by the criteria described above) is not processed twice by the system.
- The number of event records that are processed matches the record count in the trailer record. If the record count does not match, the batch will be marked as invalid. A separate schedule can be run to automatically revoke the processing of all files that contained invalid batches. These files must then be fixed and processed again.

Schedule types

The following are Singl.eView ICP specific schedule types for batch validation:

- Revoke Event File & Batch

  Use this schedule type to revoke an event file for a datafeed that performs batch validation. This schedule type replaces the use of the Singl.eView Convergent Billing schedule type 'Revoke Event File'.

- Revoke File/Batch & move back

  Use this schedule type to revoke an event file for a datafeed that performs batch validation. The event file is also moved from the archive directory back to the input directory. This schedule replaces the use of the Singl.eView Convergent Billing schedule type 'Revoke File and move back'.

- Revoke Files with Invalid Batches

  Use this schedule to revoke the processing of all files that contained invalid batches. A schedule of this type should be run once a day after all rating has completed for the day. Running this schedule will revoke any files containing invalid batches that have been processed in the last day. These files will then need to be manually checked and fixed by billing operations staff.

ENM process for rentals

The Singl.eView Convergent Billing billing engine uses the RGP and RAP processes to charge subscribers for rental (recurring) charges, and for activation and cancellation charges. These processes generate event files that must also be processed by an ENM process. The event files generated by these processes is in a format known as Native Format.

An ENM process that will process the event files generated by the RGP and RAP processes must be configured.

The ENM process configured to process rental events also requires 'Input', 'Error', and 'Archive' directories (for more information, refer to *ENM process for switch files* earlier in this chapter). However, for this ENM process, the sub-directories would be given a different name, such as 'Rentals'.

There are a number of parameters that must be assigned specific values for an ENM process configuration item, which is an ENM process that processes events generated by the RGP and RAP processes. These parameters are shown below (with examples assuming that the subdirectory is called 'Rentals').

Parameter	Example Value
INPUT_METHOD	file
HOME_DIR	$ATA_DATA_SERVER_INPUT/rentals
ERROR_DIR	$ATA_DATA_SERVER_ERROR/rentals
ARCHIVE_DIR	$ATA_DATA_SERVER_ARCHIVE/rentals
FILE_TYPE	Singl.eView Convergent Billing Native
EVENT_TYPE	CLEC MRC+NRC

Note: An ENM process configuration item includes other parameters that must also be configured.

Other rating processes

Event Rating Broker (ERB) process

The Event Rating Broker (ERB) process controls the rating engine processes (ENMs, ERTs, EROs). A single instance of this process needs to be configured.

A sample configuration item that comes with Singl.eView Convergent Billing can be used. The name (and type) of the configuration item is the ERB process.

The ERB process configuration requires a TCP/IP service name to be configured. This service name must also be defined in the system's `/etc/services` file.

Event Rating (ERT) process

The Event Rating (ERT) process takes normalised events generated by an ENM process and uses tariffs configured in the system to generate one or more charges for the event.

The number of ERT processes configured for the system depends on a number of factors (for more information, refer to *Sizing* later in this chapter).

However, a working Singl.eView ICP system only needs to have a single ERT process configured (even though it has at least two ENM processes configured). The two different types of ENM process can both use the same instance of the ERT process to charge the normalised events that they generate. However, for performance reasons, the system might be configured to use more than one ERT process.

Event Rating Output (ERO) process

The Event Rating Output (ERO) process is responsible for outputing the normalised events and charges generated by the ENM and ERT processes. An ERO process can output events and charges directly into the database to a file that can be uploaded into the database using the SQL Loader program, or output directly to a TCP/IP socket.

A working Singl.eView ICP system only needs to have a single ERO process configured with all events and charges sent directly to the database. This means the ERO process should have its `OUTPUT_METHOD` parameter set to direct. Also, the `LOAD_PARAMETERS`, `COMPLETED_DIR`, `OUTPUT_HOST`, and `OUTPUT_SERVICE` parameters can be left blank because they are not relevant when the `OUTPUT_METHOD` is set to direct. For performance reasons, the system could be configured to use more than one ERO process.

Sizing

The number of rating processes configured for a system to achieve maximum performance depends on many factors. This section provides guidelines for configuring the rating processes so that the best performance is achieved.

To increase performance when rating switch files (to reduce the time it takes to rate all the switch files generated), a machine with multiple processors must be used. There is no point in configuring multiple ENM processes to run simultaneously on a machine with one CPU because they will all be sharing it. In this case, one ENM process would be more effective.

Note: It is unlikely that a Singl.eView ICP machine will have only one CPU. For performances reasons, machines would have at least 2 CPUs.

However, on a machine with multiple CPUs, multiple ENM processes can be configured to run simultaneously because they will be run on separate CPUs, with each ENM process utilising the full capacity of the CPU.

Note: The operating system allocates processes to different CPUs. This does not need to be configured in Singl.eView ICP.

Singl.eView ICP Event Fix-Up Script

Singl.eView Convergent Billing date ranged entities are date ranged down to a granularity of a day. This leads to an issue when customers change their telephone number.

For example, a customer might enter a shopfront at 1:30pm and ask to have his telephone number changed. In Singl.eView Convergent Billing, this change can only be marked to occur at 12:00 midnight. One way to handle this would be to make the change effective at 12 midnight that night, 10.5 hours later. This might be acceptable for something like a telephone number, but the customer might want to use the new number immediately instead of waiting until midnight.

This issue is handled in Singl.eView ICP by having the change made effective at 12:00 midnight from the previous night, 13.5 hours previously. However, the customer may have made calls on his old number during the morning, before changing over to the new number. Any such calls will fail to be rated by the rating engine (as the service lookup by the ERT process will fail).

The CLEC Event Fix-Up script is used to fix and re-rate all such events (ie. those events that fail to rate as they were made prior to, but on the same day as, a telephone number changeover).

The script must be run after all the rating tasks have completed. This would be once a day after all of the rating tasks have finished for that day. It will adjust and re-rate the relevant error events generated by these rating tasks.

The script is run by creating a schedule or an immediate task of type 'CLEC Fix Changeover Errors'. The parameters for this schedule type are described in the table below.

Parameter	Description
ENM Process	Used to re-rate the events that have been adjusted. It must be configured to process events in Native Format. For example, the ENM process used to process rental events could be used.
Retention Period (days)	Whenever a telephone number changeover occurs, the Singl.eView ICP system records the exact date and time of the changeover. This enables the script to fix any events that were generated for that service on the same day, but prior to the changover. However, there may be a time delay between when the event is generated by the switch and when the event is actually processed by Singl.eView ICP. For example, the event may only be processed by Singl.eView Convergent Billing 3 days after the event is generated, because it takes that amount of time for the tape to be transported from the switch to the data centre. This parameter indicates how long Singl.eView ICP keeps a record of the telephone number changeovers. It should reflect the maximum delay expected between when the switch generates the event and when the event is processed by Singl.eView Convergent Billing.

Troubleshooting

Details on how to fix and re-process corrupted or error events generated by the rating engine are contained in the *System Operations Guide for Singl.eView Convergent Billing*.

Some sample problems and resolutions follow.

Problem

An event file has been put in the input directory for an ENM process and a rating task has been run, but it has not processed the file.

Resolution

Check that the file has been placed in the correct directory, and that this is the directory specified as the `HOME_DIR` parameter in the ENM process configuration item.

Another reason that this problem might occur involves the effective date of the rating task. A rating task uses the effective date parameter to restrict the files that it processes from the input directory. It will only process a file from the input directory if its last modification time is less than the effective date. This problem does not usually occur when periodic rating tasks are created by a schedule. However, it could occur if the file is copied to the input directory and then an immediate task to rate the file is run.

Problem

A file has been placed in the input directory and has been processed, but the entire file appears to be corrupt. (No records are successfully processed and the entire file ends up in the error directory.)

Resolution

Check that the file has been copied to the correct input directory and has been processed by the correct ENM process. This problem requires that the entire file be revoked, copied to the correct input directory, and processed again using the correct ENM process.

If the event file has been processed by the correct ENM process, other reasons for the entire file appearing corrupt could be that:

- The switch has been reconfigured and it is generating events in a slightly different format.
- The switch has been upgraded with a new version of software that generates events in a different format.

Problem

Files were not rated and the `log.out` file indicates that one of the rating processes (ENM, ERT, ERO) required to rate the file is not running.

Resolution

If the rating process is not for temporary or testing purposes, ensure that the `Enabled` parameter in the configuration item is set to TRUE. This ensures that the process will be automatically started when the Singl.eView Convergent Billing server is started.

If the rating process has shutdown, check the `log.out` file for reasons why this has occurred. For example, the configuration item for an ENM process may be configured incorrectly. The rating process can then be individually started from the UNIX command line by using Singl.eView Convergent Billing TRE functions and the **`ecp`** command. For example, to start the ENM process with a sequence number of 4, issue the following command (and check `log.out` to ensure the ENM process starts successfully):

```
ecp "biEnmStart&(4)"
```

Other TRE functions for starting and stopping rating processes are:

- **`biEnmStop&`**

  Stops an individual ENM process.

- **`biErtStart&`**

  Starts an individual ERT process.

- **`biErtStop&`**

  Stops an individual ERT process.

- **`biEroStart&`**

  Starts an individual ERO process.

- **`biEroStop&`**

  Stops an individual ERO process.

- **`biBkrStart&`**

  Starts the ERB process and all the ENM, ERT and ERO processes which it controls.

- **`biBkrStop&`**

  Stops the ERB process and all the ENM, ERT, and ERO processes which it controls.

Tips and Recommendations

- In ENM process configuration items, the `Internal ERO` parameter is not used and should always be left blank.
- Set the `Enabled` parameter of any ENM, ERO, and ERT processes that are configured to TRUE. This ensures that they will be automatically started when the Singl.eView Convergent Billing server is started.
- If required, Singl.eView Convergent Billing TRE functions can be called from the UNIX command line using the **`ecp`** program to manually start and stop rating processes; for example:

  ```
  ecp "biEnmStart&(4)"
  ```

- The CLEC Event Fix-Up script should be run daily after all rating processes have completed for that day. The script must be run (though it could be more or less than once a day), otherwise customers may not be billed for calls that they make on a day in which they change their telephone number (only for calls made on that day, before the changeover occurred).
- A daily check should be made on error events. Billing operations staff can fix and re-rate error events using the Singl.eView Convergent Billing client, either individually or in bulk.
- The Singl.eView Convergent Billing `log.out` file (in the `$ATA_DATA_SERVER_LOG` directory) contains a record of the event files that have been processed by an ENM process and any errors that occurred.

Rating

Chapter 4
Client Requirements

This chapter describes the operational aspects of billing that are specific to the Singl.eView ICP product. It also describes any relevant sizing and performance recommendations.

For details on the core operational aspects that relate to billing, refer to the *Systems Operations Guide for Singl.eView Convergent Billing.*

For details on how to configure the Singl.eView Convergent Billing processes, refer to the *System Administration Guide for Singl.eView Convergent Billing*.

Key Terms

Term	Description
Invoice Cycle	Billing period a customer is to be billed for each cycle. The period is defined by specifying the day of the month the customer is to be billed from each time. For example, an invoice cycle may be the 5th of every month.
Billing Schedule	Configured schedule with required parameters to generate invoices each month for customers for a specific invoice cycle date. There can be many billing schedules all configured with the same invoice cycle date. This allows you to spread your customer base across many schedules all with the same invoice cycle date.

Term	Description
Billing Task/Run	Instance of a billing schedule that has run or is pending to be run. For example, a billing schedule may be configured with an invoice cycle of the 5th of every month. An instance of that schedule running would be 05/05/1999.
Invoice Generation	Name of the schedule type used by billing schedules.
RGP	Rental Generation Process. Generates the recurring/rental charges for each service and product.
RAP	Rental Adjustment Process. Generates pro-rated charges for partial rental charges where applicable.
BGP	Bill Generation Process. Totals call, rental and one-off charges, payments, and account adjustments for a customer. Also applies discounts/taxes to charges and totals, and applies rules from product promotions to a customer's product charges. The process determines a customer's new account balance to be invoiced.
IGP	Invoice Generation Process. Substitutes billing details into invoice templates to create the final invoice image for each customer. Invoices can be of a summarised format or a full itemised format.
GOP	General Output Process. Sends invoices either to a local printer or to a tape or file so that they can be printed externally.
Apply Invoices	Posts billing details generated by a bill schedule to a customer's account and associates un-allocated funds to outstanding invoices. Unallocated funds include any payment or adjustment amounts not yet associated with an invoice. Funds are associated with outstanding invoices based on the priority of the outstanding receivable amounts remaining on each invoice. Invoices with higher priority receivable amounts outstanding will be allocated funds first in preference to other outstanding invoices.

Billing Schedule Parameters

The table below describes each parameter in the 'Invoice Generation' schedule and the recommendation for Singl.eView ICP billing schedules.

Parameter	Required Value
BGP command line options	Nothing required to be entered, but debug options can be specified as required.
IGP command line options	**`-p $ATA_DATA_SERVER/invoices`**
Generate Rental Events?	Yes. Although this is optional within the billing schedule, it **must** be run for Singl.eView ICP billing schedules because they are billing customers with products that contain rental tariffs.
RGP Process Number	1
RGP Command Line Options	Nothing required to be entered, but debug options can be specified as required.
Perform Rental Adjustments?	Yes. Although this is optional within the billing schedule, it must be run for Singl.eView ICP billing schedules because they are billing customers with Singl.eView ICP products that contain rental tariffs with pro-rating options.
RAP Process Number	1
RAP Command Line Options	Nothing required to be entered, but debug options can be specified as required.

Parameter	Required Value
Output Invoices?	Recommend that this be set to 'No' initially. When confident that the billing processes are running smoothly, and there is no need to check each stage before running the next, set this value to 'Yes'. Another reason to set this to 'No' is that the billing schedule will complete in a shorter period of time.
Output Method	Only requires a value if *Output Invoices* above is set to 'Yes'. If 'Yes', select an *Output Method* suitable for printing invoices.
Output Device	Only required if *Output Invoices* is set to 'Yes'. If 'Yes', select the appropriate output device.
Directory for temporary data	Only required if Output Invoices is set to 'Yes'. Specify a directory to which Singl.eView Convergent Billing can write.
Apply Invoices?	It is recommend that this be set to 'No' and a separate schedule be run to apply invoices once the billing run has been checked, and everything is correct. If the billing processes are running smoothly, and there is no need to check before applying invoices, set this value to 'Yes'. **Note:** Once the 'Apply Invoices' schedule has run, it can be revoked only if no payments or adjustments have been processed against the customer's accounts since the 'Apply Invoices' task has run.

How Many Invoice Cycles?

The invoice cycle is the period between individual bill runs. Options would be monthly, bi-monthly, or quarterly. The invoice cycle is generally a monthly cycle that starts on the same day of every month. There can be as many cycles as there are days in the month, but it is not recommend that invoice cycles start on 29th of the month or later.

The first day of the cycle is generally what the cycle is known as or named after (eg. 5th of the month cycle).

The period of an invoice cycle and its start date will depend on individual requirements. Some factors to help determine the invoice cycles include:

- Distribution of work load on billing inquiries centre

  If all customers were billed with the same invoice cycle date, all of the invoices would be issued on the same date. A side effect of issuing all of the invoices on the same day would be the amount of billing inquiry calls made by customers when they receive their invoices, because the calls would all be made around the same time. By having different invoice cycles throughout the month (and therefore different issue dates for invoices), the load of calls to the billing inquiries centre are spread across the whole month. One way to achieve this is to split customers across invoice cycles alphabetically. For instance, all customers whose names start with a letter between A and L are assigned to a different invoice cycle than customers with names between the letters M and Z.

- Different cycles for geographic regions

  It may be necessary to have different invoice cycles for each geographic region through which the customer base extends.

  Benefits of this would be the savings in postal deliveries to a region if all invoices for a region are sent together. Another benefit is that CSRs can allocate new customers to their geographic regions.

  Disadvantages of this would be the impact on the billing inquiries centre if the region was a large region and all customers received their invoices at the same time.

- Different cycles for each customer type

  All residential customers may be assigned to a different invoice cycle from corporate customers.

  Benefits of doing this are that CSRs will know which invoice cycle to apply to a new customer.

How Many Billing Schedules?

The number of schedules required will depend on many factors. These include:

- How many customers are currently in the database?
- What is the projected size of the customer base (how big will the anticipated database be)?
- What invoice cycles are required, and how will customers be allocated to them (alphabetically, geographically, customer type, etc.)?
- What are the operational windows available daily/monthly for running server processes (number of nights available per month for running billing schedules and the number of hours available each night)?
- How many CPUs are available on the production hardware?
- Should schedules be configured to meet current database and hardware configuration and more hardware and schedules added as the database grows?
- Alternatively, should schedules and hardware be set up now to meet the projected growth sizes? This would mean that there would be many schedules running, processing a small number of customers each. As the database grows, these schedules will start to run at their maximum capacity.
- What is the average event throughput per day or month for each customer?

Maximising billing performance

In Singl.eView Convergent Billing, maximum billing throughput is achieved by running billing schedules in parallel. The number of parallel billing schedules should equal the number of CPUs on the production machine.

Maximum number of schedules possible

To determine the maximum number of billing schedules possible per month for a given installation, multiply the number of nights available for processing each month with the number of available CPUs.

Nights available for processing	x	No. of CPUs	=	Maximum number of schedules

The table below shows the number of billing schedules able to be run per month based on different numbers of CPUs and available processing nights.

Example: Maximum schedules per month

Number nights/month	3 CPUs	4 CPUs	5 CPUs	6 CPUs
20	60	80	100	120
15	45	60	75	90
10	30	40	50	60

Note: A disadvantage of defining the maximum number of billing schedules is that it becomes harder to maintain them. For optimum performance, it is important that the amount of processing on each billing schedule is roughly the same (ie. roughly the same number of customers and events in each billing schedule). As the number of billing schedules increases, it becomes harder to balance the load across all of them.

Customers per schedule

The number of customers assigned to an invoice cycle must first be determined. This number is then divided by the number of schedules that can be run each night (this will be dependent on the number of available CPUs and the average duration of each schedule).

Number of customers per invoice cycle	/	Maximum number of schedules that can be run each night	=	Average number of customers per schedule

If a large percentage of the customers are assigned to the same invoice cycle, schedules may need to be established to run on a number of nights for that one invoice cycle date.

For example, if all of the customers in the database have an invoice cycle of the 5th of every month, it may not be feasible to bill them all in one night. Schedules may have to be configured to run on the 5th, 6th and 7th of every month, all processing customers with an invoice cycle date of the 5th.

Billing duration estimates

The duration of each billing schedule will be dependent on the number of customers to be processed in that schedule, the number of events per customer to be processed, and the number of CPUs available.

Schedule Naming Conventions

Since there can be multiple schedules configured for each invoice cycle, a schedule naming convention is important so that operators can tell at a glance the invoice cycle a schedule relates to. A naming convention will assist CSRs to select the correct schedule when assigning a new customer, especially if there are multiple schedules per invoice cycle; for example, there are 3 invoice cycles occurring on the 5th, 15th and 20th of every month. Two methods of dealing with multiple schedules and invoice cycles follow.

Example 1

The 5th of the month cycle is for residential customers, 15th of the month cycle is for business customers, and the 20th of the month is for all other customers. For each invoice cycle, there may be 3 billing schedules.

To ensure an even distribution of customers across each schedule for a given cycle, the customers could be split alphabetically.

Suggested schedule names could be:

Schedule 1 'Residential (5th) - A-I'

Schedule 2 'Residential (5th)- J-Q'

Schedule 3 'Residential (5th)- R-Z'

Schedule 4 'Business (15th)- A-I'

Schedule 5 'Business (15th)- J-Q'

Schedule 6 'Business (15th)- R-Z'

Schedule 7 'Other (20th) - A-I'

Schedule 8 'Other (20th)- J-Q'

Schedule 9 'Other (20th)- R-Z'

Example 2

An invoice cycle can be set up for each different region of the country. For example, the 5th of the month for Northern region customers, 15th of the month for Central region customers, and 20th of the month for Southern region customers. Each cycle has 3 schedules with the customers split alphabetically between them.

Schedule 1	'Northern (5th) - A-K'
Schedule 2	'Northern (5th)- L-Z'
Schedule 3	'Central (15th)- A-K'
Schedule 4	'Central (15th)- L-Z'
Schedule 5	'Southern (20th)- A-K'
Schedule 6	'Southern (20th)- L-Z'



A naming convention is also recommended for the corresponding 'Apply Invoices' and 'General Bulk Output Process' schedules that need to be established to clarify which billing schedule they relate to.

Schedule Planning

No more than twenty (20) days per month should be used for running billing schedules, and no more than twelve (12) hours per day used for running server processes. Billing schedules need to be established to run within the following operational windows.

Days/Month

If the maximum (20) number of days per month for server processing is being used, there are only ten (10) days free for any revoking and reprocessing of server tasks, if required. This model will only work if the time it takes to either revoke a billing task, or to reprocess it, is less than the available processing hours per day.

If it takes more than half the available processing hours to either revoke or re-run a billing task, the activity will need to be spread over two (2) days. For example, one night would be needed to revoke a billing task and another night to re-run it. Therefore, no more than a third of days per month should be utilised for scheduled processing, to allow for contingencies for the revoking and re-running of every task each month. This would be a worst case scenario.

Hours/Day

Although it is recommended that no more than twelve (12) hours per day be used for server processing, a safer model is to plan for a maximum of ten (10) hours per day. This allows for a two (2) hour buffer for any schedules that may run over time.

Determining the window proportions

The fewer billing schedules that are configured to handle the entire customer base, the longer each one will take to process. The more billing schedules that are configured, the less time each schedule will take. Determining the right balance will depend on the hardware requirements, the database size, and the number of invoice cycles required.

Allowance for reprocessing

For contingency reasons, a number of days per month or hours per night should be set aside for reprocessing billing schedules.

If the maximum number of days per month for processing (20, for instance) have been used, then there are approximately ten (10) days left in the month for reprocessing billing schedules, if required. It is recommended these process free days be spread evenly amongst the scheduled billing tasks, so that if a scheduled billing task fails or needs to be reprocessed, it can be scheduled into the next free day.

For example, if there were twenty (20) nights per month allocated to processing billing schedules, every 3rd night would be free (ie. two nights of processing with one night free).

Alternatively, if the duration of each billing task is less than a third of the number of processing hours available each night, a billing task that has run can be revoked and re-run all in one night.

Allowance for other server processes

When planning how many free nights per month or hours per day to use for processing billing tasks, all the other required daily processes that need to be scheduled need to be considered as well.

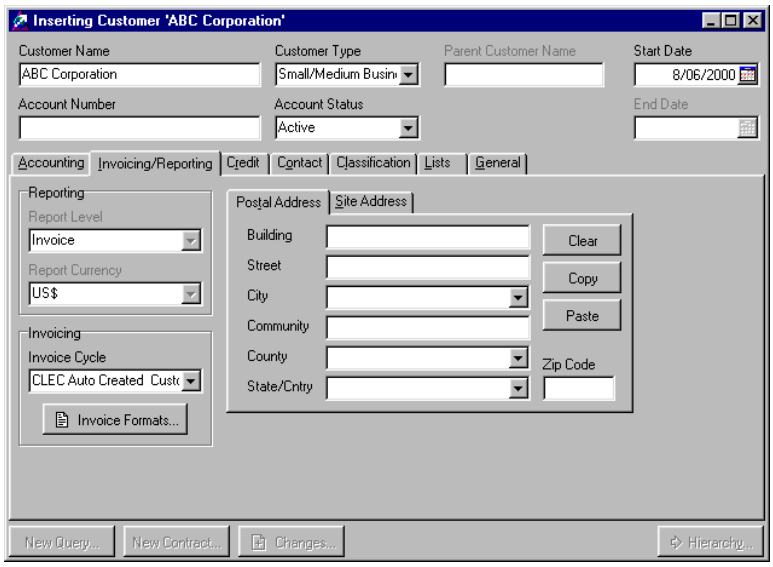
The table below shows a sample day usage.

Operation	Start Time	End Time
CSR activity	8.00am	6.00pm
Rating	6.00pm	8.00pm
Daily Housekeeping (back-ups, DVP, archiving, etc.)	8.00pm	10.00pm
Other Server Processes (Billing, reports, upload to GL, dunning)	10.00pm	5.00am
Buffer Zone	5.00am	8.00am

Assigning Customers To Billing Schedules

Multiple schedules are established for each invoice cycle. Once they have been established, customers can be assigned to them.

The *Invoice Cycle* field is a mandatory field on the *Customer Detail* form's *Invoice/Reporting* tab. The *Invoice Cycle* field actually refers to the billing schedule in which a customer will be processed.



When Customer Service Representatives (CSRs) create a new customer, they must select an appropriate billing schedule (for more information, refer to *Schedule Naming Conventions* earlier in this chapter).

Selecting the right schedule

Since there can be multiple schedules configured for each invoice cycle, the CSR may not know which one to assign the new customer to unless a schedule naming convention is obvious (for more information, refer to *Schedule Naming Conventions* earlier in this chapter).

If multiple schedules are established for an invoice cycle with customers split alphabetically across them, additional schedules will have to be configured for any new customers when the schedule sizes reach their maximum limits. In this situation, the System Administrator needs to provide CSRs with a new strategy for assigning customers to billing schedules.

Invoice Messages

Generally, the only form of communication between a service provider and its customers is the invoice that is sent monthly. Consequently, the invoice also acts as an advertising medium for new products, services or current deals that the service provider has to offer. A naming scheme for invoice messages allows the placement and formatting of messages on the Singl.eView Convergent Billing invoice.

Message locations

The following table shows the location of different message types on the invoice.

Invoice Location	Message Type	Remarks
First Page	First Page	These are the most visible messages on the Invoice. Examples of First Page messages are a Final Bill message, a Past Due message or a New Customer message. Depending on the number of subtotals printed in the Customer Summary of Charges, up to 15 lines of messages may be printed on the first page. For example, if there are 10 subtotals, 5 lines are left for first page messages. If there is enough space, the first page message will be continued in the Account Statement section, printed before any third page messages.
Second Page	PUC Message	Public Utilities Commission (PUC) messages provide information about where users can file complaints about bill payments and late payment fees. By law, these messages must appear on the second page. The information in the PUC message varies depending on the customer's market and state.

Invoice Location	Message Type	Remarks
Account Statement	Third Page	Third Page messages are printed first in the Account Statement section before any Payments and Adjustments. Third Page messages are customer specific.
Node Details	Marketing Message	Marketing Messages may be printed at the top of each node's details before the Customer Node Summary of Charges section if the Invoice has more than one customer node. Marketing Messages can be made node or location specific. If there is only one customer node on the invoice, the marketing messages are printed after the customer's third page messages in the Account Statement section.

Invoice message naming convention

For invoice messages, the following naming convention must be followed:

- Messages in the different sections appear in the sort order of the names of the messages.
- If the invoice message name contains 'bold', the message will be printed in bold print.
- In all of the rules that state 'if the name contains', case is insensitive. For example, CLEC_PUC_CAPLA_bold and clec_PUC_PA_Bold are both PUC messages and will both print in bold print.
- If the name contains clec_p1, the message type is first page.
- If the name contains clec_puc, the message type is PUC.
- If the name contains clec_p3, the message type is third page.
- If the name does not contain clec_p1, clec_puc or clec_p3, the message type is marketing.

Default message

Singl.eView ICP comes with the default invoice message 'InvMessageOne'. This must be replaced or renamed (if it is to be used) before going into production.

Pre-Billing Checks

Although there are no mandatory pre-requisites required prior to running the billing schedule, the following is a recommended list of checks to perform:

Check for DVP errors

- Run the DVP process to check for any database integrity violations that may have an impact on the billing process.

All events rated and errors corrected

- Check that all required call data files to be processed for this billing period have been rated successfully.
- Check that all events that rated in error have been corrected.

Note: It is not mandatory that all data files are rated and events in error corrected prior to billing. If they are not rated or corrected for the current billing period, they will be included in the next billing process as soon as they are corrected. This is a recommended practice.

Invoice messages set up

- Invoice messages to appear on the current month's invoices have been set up.

Environment checks

- Check that all required server processes are up and running. This includes, as a minimum, the BMP, TLM, STD, ERB, ERO, ERT, and ENM 1 processes.

Managing Billing Schedules

It is important to know what stage each invoice schedule is at in the complete end to end dataflow. If the system is configured for maximum throughput (billing schedules are configured for each CPU for every night of the processing window), there could be anywhere from 30 to 120 billing schedules running each month.



To ensure that each stage of every billing schedule has been completed successfully, it is recommend that a manual checklist be completed after each stage of each schedule.

Schedule progress chart

A sample billing schedule progress chart may look like the example provided below.

Note: This is a sample billing schedule process chart only. A checklist should be tailored to incorporate all of the checks and tasks unique to the installation.

Billing Schedules for the month of _____MAY_____ year ___1999___

Billing Schedule	Pre Billing Checks	BGP			IGP			Post Billing Checks	GOP			Apply Invoices	Billing Reports Run
		Success	Fail		Success	Fail			Success	Fail			
			Revoke	Re-Run		Revoke	Re-Run			Revoke	Re-Run		
5th of month - schedule 1	5/5/99	5/5/99	-	-	6/5/99	5/5/99	6/5/99	6/5/99	6/5/99			6/5/99	6/5/99
5th of month - schedule 2													
15th of month - schedule 1													
15th of month - schedule 2													
15th of month - schedule 3													
20th of month - schedule 1													
25th of month - schedule 1													

Execution of Billing Schedules

Billing schedules will be set up to run automatically at the specified times. Upon completion of each billing task, a message is emailed to the address specified in the schedule. This message contains details of the outcome of the task.

During execution, different processes running on the server can be seen. When listing all processes running (**`'ps -ef'`**), listings for the RGP, BGP and IGP processes can seen as they are being run. Once they have completed, the process no longer appears in the listing.

The stage a process is at can be monitored by regularly checking the tail of the `log.out` file, which resides in the `$ATA_DATA_SERVER_LOG` directory.

Billing outputs

The following should be checked at the completion of each task:

- Task status
- Task output text
- Number of invoices generated
- `log.out` file.

Task status

The task status will either be 'Success' or 'Failure'. If the status is 'Failure', the task output text should be checked for details to determine at which point the task failed. If the status is 'Success', all other billing outputs should be checked to confirm the success. The 'Success' status means that each stage of the billing process could be run to completion. There still may be problems and warnings with a particular stage.

Task output text

The output text will contain details of each step of the billing task. If the task failed, then the output text will indicate at which step it failed.

For example, the BGP may have completed successfully, but the IGP may have failed. Knowing what point a billing task failed will help determine what corrective action is required. In this example, the IGP is re-run, but not the entire billing process.

Number of invoices generated

The total number of invoices generated by the schedule is listed in the output text. This number includes any statements that were generated. It is important to note this number. When applying invoices or revoking invoices, the number of invoices recorded in those processes should match the number in the original billing schedule.

log.out file

The `log.out` file is the master log file for all server processes. It contains details of all steps in the billing process. Warning and error messages are fully detailed in the log. Most importantly, the sequence of events is visible to determine which errors occurred first. Any other system wide error messages that may have had an impact on the billing process are also visible.

Every error message that can occur in the `log.out` file can be searched online through the client. Select **Maintenance ⇨ Data Definition ⇨ Error Messages**. Each error message has three pieces of information:

- Actual error message
- Cause of the error
- Action to follow to rectify it.

Managing Billing Outputs

RGP event files

Each billing task produces a number of RGP event files. These files are rated during the billing process to produce the rental charges. Once the RGP files have been processed, they are moved into an archive directory.

Note: RGP files should be deleted regularly from this directory. There is no need to keep these files.

Debug details

Within the billing schedule parameters, a debug level for the BGP, IGP, RGP, and RAP processes can be specified.

Some debug levels will generate more debug information than others will. All debug information is included in the output text of the billing task. All task output is stored in the `TASK_QUEUE_RESULT` table. This table should be monitored closely and old records from old billing tasks should be archived, as the table becomes full.

Debug levels in a billing schedule should only be set when developing or testing new configuration (for example, products and tariffs). If some events are not being billed correctly, debug information in the billing schedule processes will help identify problems in the configuration. Including debug details in the billing schedule will have two adverse impacts on performance:

- The task queue result table will fill up very quickly.
- Additional logging will require more system resources and will increase the duration of the billing schedule.

For more information, refer to *Chapter 6, Managing Server Processes* in *System Administration Guide for Singl.eView Convergent Billing.*

Post Billing Checks

The checks that should be performed after each billing task has completed will vary for each installation. It is important to confirm that a billing task was successful, as many other processes such as treatment processes and reports, depend on this information. Suggested checks include:

- Running the DVP
- Spot checking invoices
- Running post billing reports.

DVP

Run the DVP process to identify any integrity constraint violations that may have been introduced by the billing process.

Spot check invoice details

To spot check individual invoice details online to ensure that the monthly rental charges generated are of the correct amount for that period:

1. Select the **Accounts ⇨ Invoices** option. The *Invoice Search* form is displayed.
2. Select 'Issue Date' in the *Search Field*.
3. Enter the value of the billing period in the *Search Criteria* field.
4. Click the **Search** button. A list of invoices that match the specified criteria is displayed.
5. Select an invoice from the list and click the **Detail…** button. The *Invoice Detail* form is displayed in View Only mode.

6. Click the **Service Details…** button to view details of the services included on this invoice. The *Service Details for Invoice* form is displayed in View Only mode with the appropriate services displayed.

7. Click the **Call Details…** button to list details of the charges included in this service. The *Call Details* form is displayed with the appropriate usage details.

To check the monthly recurring charges for a service:

1. Open the *Call Details* form by following the previous procedure.

2. Using the *Invoice Text* column, identify the *Duration* and *Charge* details for the service rental charges.

3. Confirm that the rental charge is correct and that the correct rate for the period shown has been used by checking the appropriate Service Monthly Recurring Charge table.

The following conditions apply to monthly recurring charges:

- For a full monthly charge, the *Duration* field displays the value 1.
- For a partial monthly change, the *Duration* field displays the value as a fraction of 1.
- The *Call Start Date and Time* column defines the range of the value entered in the *Duration* field.
- The *Charge* field is determined by the rate of a monthly period multiplied by the value in the *Duration* field.

Spot check invoice images

To spot check selected invoice images to ensure the layout is correct and that the required invoice messages are displayed:

- Select the **Accounts ⇨ Invoices** option. The *Invoice Search* form is displayed.
- Use a *Search Field* of 'Issue Date' and enter the value of the relevant billing period.
- Click on any invoice in the list to view the details.
- Click the *Format* tab.
- Click the **Invoice Image…** button. This will submit a task to load the image into Acrobat Reader.
- Scroll though the pages to check that the layout and invoices messages are intact.

Reports

Run the Billing Reconciliation Report (RBRC_CLEC). This has a breakdown of call, rental, and one-off charges. The report total is the total of all charges generated for all customers within the bill run. The report total will be the amount of money the service provider can expect to collect from customers in that billing schedule for that billing period. Check that this amount is within the expected range for that period.

Apply Invoices

The 'Apply Invoices' schedule takes the billing details generated by the BGP and posts them to the customer's account. Until this is done, the effects of a billing run will not be visible on a customer's account history.

Once all post billing checks have been completed for a billing task and it has been confirmed a success, the invoices can be applied to the customers' accounts.



'Apply Invoices' must be run **after** each billing task and **before** the execution of the next task for that billing schedule.

Note: Once the 'Apply Invoices' schedule has run, it cannot be easily re-run. It can only be revoked if no payments or adjustments have been processed against the customer's accounts since it was run.

'Apply Invoices' can be run as a regular schedule or as an immediate task. For every billing schedule that is run, there **must** be a corresponding 'Apply Invoices' schedule configured (or an immediate task run for that schedule).

Regular schedule

If 'Apply Invoices' is run as a regular schedule, it is recommended that it be scheduled to run at least a couple of days after the scheduled execution of the corresponding billing schedule. This will allow a couple of days for any reprocessing that may be required.

If the billing schedule cannot be re-run before the scheduled running of the 'Apply Invoices' task, the 'Apply Invoices' schedule will have to be intercepted and start time of the next 'Apply Invoices' task changed so that it runs at a time after the expected completion of the re-running of the billing task.

To change the start time of an 'Apply Invoices' task:

1. Select the **Server Tasks ⇨ Schedules** option. The *Schedule Search* form is displayed.
2. Click the **New…** button. The *Schedule Definition* form is displayed in Insert mode.
3. Select 'Apply Invoices' schedule in *Schedule Type* field.
4. Click the **Search Task** button. The *Task Search* form is displayed.
5. Click the **Search…** button to display all tasks.
6. Select the next pending task.
7. Click the **Detail…** button. The *Task Details* form is displayed.
8. Click the **Update** button on the toolbar. The Task Details form is displayed in Update mode.
9. Change the *Start Date* forward to the required value.
10. Click the **Save** button on the toolbar.

Note: Do not click the **Re-submit** button. This button is only for re-submitting tasks that have already run. It is not for submitting pending tasks.

Schedule naming convention

If 'Apply Invoices' is run as a normal schedule, it is important to name the schedule appropriately so that it is clear to which billing schedule it relates.

If there are three billing schedules for one invoice cycle date with the schedules split up alphabetically, the suggested 'Apply Invoice' schedule names might be as in the following table.

Billing Schedule	Corresponding Apply Invoices Schedule
5th of month cycle - A-I	Apply Invoices 5th/month A-I
5th of month cycle - J-Q	Apply Invoices 5th/month J-Q
5th of month cycle - R-Z	Apply Invoices 5th/month A-Z

Immediate task

To avoid having to change the start time of scheduled tasks, it may be preferable to run 'Apply Invoices' as an immediate task as required. It will therefore not run automatically.

If this method is chosen, it is important that manual procedures are in place to ensure that this step is not omitted. It is recommended that a billing manager spreadsheet be used (for more information, refer to sample spreadsheet in *Schedule progress chart* earlier in this chapter).

Printing and Other Outputs

This topic is covered in detail in *Chapter 5, Printing and Other Outputs*. The following points should be noted in relation to this chapter:

- For each billing schedule, there must be a corresponding GOP schedule configured.
- The GOP schedule must be run after the successful completion of a billing task, and before the running of the next billing task of a billing schedule.
- A naming convention should be used to identify the billing schedule to which a GOP schedule corresponds.

Troubleshooting

The following table lists each billing process that can be run and the corresponding process required to revoke it if it needs to be undone.

Billing Process/Stage to be Revoked	Schedule Type to Use to Revoke	Schedule Type to Re-run
Invoice Generation (RGP, BGP, IGP)	**Revoke Invoices** Revokes all billing details that were generated including the invoice images.	**Invoice Generation** Re-run the original invoice generation schedule for the required billing period.
IGP If just the IGP part of the Invoice Generation schedule failed.	**Revoke Invoice Images** Revokes the images, but not the billing details.	**Generate Invoice Images** Regenerates the invoice images by substituting the billing details into the invoice templates.
GOP If outputting to a printer, file or tape.	**Revoke Invoice Print Settings** Sets the printed flag back to null so that the GOP will pick all invoices up the next time to output them to the required device.	**GOP** Re-run the GOP again, as before.
Apply Invoices	**Unapply Invoices** **Note:** Although this process exists, it is not always feasible to use. 'Unapply Invoices' can only be run if no payments or adjustments have been processed against customer's accounts since the 'Apply Invoices' script has run.	**Apply Invoices**

Performance

This section is a summary of all the factors that impact the performance of billing schedules.

Number of CPUs

The more CPUs available, the more billing schedules that can be run in parallel. This allows the customer base to be split across multiple schedules to run in parallel, instead of running single schedules in succession. The net effect is that more customers are processed in the same amount of time as the running of one schedule.

Number of customers per schedule

The number of customers per schedule will be restricted by the average duration of each schedule, the number of events per customer per month to be processed, and the hours available each night in the processing window.

If it is required for the total duration of each schedule to be less than a third of the nightly processing window, the number of customers per schedule will have to be limited. This will allow room for revoking and reprocessing a schedule all within one night, if required. Alternatively, it may be desirable to process as many customers as time permits in the one nightly processing window, leaving spare days between that schedule and the next one, to allow room for any reprocessing required.

Chapter 5
Printing and Other Outputs

This chapter describes the operational aspects of printing and other output methods that are specific to Singl.eView ICP and Singl.eView IP/Data. It describes sizing and performance recommendations. Also included is information on emailing of invoices.

For details on the operational aspects that relate to general output processes, refer to *System Operations Guide for Singl.eView Convergent Billing*. For details on how to configure the Singl.eView Convergent Billing general output processes, refer to the *System Administration Guide for Singl.eView Convergent Billing.*

Key Terms

The table below lists the key terms that are used in Singl.eView Convergent Billing in the area of printing and other outputs.

Term	Description
GOP	General Output Process.
Output Device	Defines the UNIX command used to send the output to the required device.
Output Method Type	Specifies the database views used to retrieve the data for output, and defines any changes the output process is to make to the database (eg. to set a Printed indicator on the invoice table).
Output Method	Specifies the output method type and the output devices to use during the output process. It allows selection of the order in which the output is printed or written to a file (eg. invoices can be printed by state and postal code, for ease of posting). Different output devices for different sets of invoices can also be selected (eg. a different printer could be used for each state or region).

Overview

The final stage in the billing end to end dataflow is the physical output of the invoice images so they can be sent to customers; alternatively, an email is created with the invoice image attached, to be sent to a SMTP server so that the invoices can be emailed to the customer. In the case of printed output, the invoices can be printed locally or output to a file or tape to be sent elsewhere for printing and distribution. The server process responsible for this stage of billing is the General Output Process (GOP).

A GOP schedule must be established for each corresponding billing schedule (ie. if an installation has 20 billing schedules, there must be 20 GOP schedules established).

When to modify a GOP schedule configured for printed output

Once a GOP schedule is configured for printed output, it will only need to be modified if:

- An additional device for printing invoices is installed (eg. if two printers are being used to print a batch of invoices, instead of one).
- The printer device that the invoices are being sent to is changed.
- The sort order of the invoices is changed.
- The output options for a batch are changed (eg. switch from being output to a file to being printed locally, or vice versa).

Main GOP components

The three main components that need to be configured for the GOP include:

- Output Devices
- Output Method Types
- Output Methods.

Recommendations and requirements for each of these components are discussed later in this chapter.

Singl.eView ICP and IP/Data Outputs

The table below lists the Singl.eView ICP and Singl.eView IP/Data items that can be output to a printer, file, SMTP server, or other device, together with their output image type.

Singl.eView ICP Output Image	Output Image Type	Description
Invoices	`.tex.Z`	Compressed TeX image
Email Invoices	`.txt`	ASCII text file
Treatment letters	`.tex.Z`	Compressed TeX image
Reports	`.spf`	SQR Printer Format
Invoice reprints	`.tex.Z`	Compressed TeX image

Output image type

It is important to be aware of the output image type because it helps to define the printer run command to use when configuring Singl.eView Convergent Billing output devices to output the images.

Each image type requires a different run command to convert that image into either a postscript image for a printer to interpret, or a different format to be written to a file or other device. For example, if the output image type is compressed, the run command needs to uncompress it before sending it to the physical device. For emailing of invoices, the output image type must be an ASCII text file so that it can be read line by line and sent to the SMTP server.

For file output, the images may need to be concatenated together into the one physical file, or they may need to be output into separate files.

Output Device Configuration

For each type of Singl.eView ICP output, the required output device configuration is listed in the tables below.

Note: The output device names are suggestions only and `<printer>` is to be replaced with your local device name.

Printing invoices or treatment letters

The table below lists the output configuration for invoices and treatment letters.

Field Name	Value
Output Device	Singl.eView ICP Postscript Printer.
Output Image Type	`.tex.Z`
Description	Device to be used for printing invoices and treatment letters.
Run Command	**`zcat $file | print_inv - - | lp -d<printer>`**

Emailing invoices

The table below lists the output device configuration for emailing of invoices.

Field Name	Value
Output Device	CMN Email Invoice via SMTP
Output Image Type	`.txt`
Description	Creation of the Invoice to be sent via email.
Run Command	**`gop_smtp $ATA_DATA_SERVER/invoices/images/tmp ""`**

The syntax for the run command is

```
gop_smtp <temp_dir> <dir_path> | ""
```

where `gop_smtp` is the command for sending the output to the SMTP server, and `<temp_dir>` and `<dir_path>` (defined in the table below).

Parameter	Description	Example
`<temp_dir>`	Directory where temporary files are written prior to being sent to the SMTP server	`$ATA_DATA_SERVER/ invoices/images/tmp`
`<dir_path>`	Directory where the invoice files that are sent as attachments to the emails are to be read from. If `""` is specified, this directory defaults to the `CACHE_DIRECTORY` set in the `INVOICE_IMAGE` configuration item.	`"'`

Printing reports

The table below lists the output device configuration for reports.

Field Name	Value
Output Device	SQR Printer Output Device.
Output Image Type	`.spf`
Description	Device to be used for printing reports.
Run Command	**`convert_sqr PS $file | lp -onb -d<printer>`**

Re-printing invoices

The table below lists the output device configuration for re-printing invoices.

Field Name	Value
Output Device	Singl.eView ICP Reprint Printer.
Output Image Type	`.tex.Z`
Description	Singl.eView ICP Printer to use when reprinting an invoice. Places a 'COPY' watermark on the invoice.
Run Command	**`zcat $file | print_inv_watermark - - | lp -d<printer>`**

Output Method Types

When establishing output methods in Singl.eView ICP, the following output method types are supplied with each type of output. Some are Singl.eView ICP or Singl.eView IP/Data specific, and some are part of Singl.eView Convergent Billing.

Output method types specific to Singl.eView ICP and Singl.eView IP/Data

The table below contains output method types specific to Singl.eView ICP and Singl.eView IP/Data.

Output Method Type	Description
CLEC Print Treatment Letters	Print treatment letters for Singl.eView ICP customers.
Print Text Invoices	Prints invoices that contain text only (ie. non TeX images). This is to be used for the output of CLEC TAP outgoing files.
CMN Email Invoices	Sends invoices images to a SMTP email server to be emailed to customers.

Standard output method types

The table below includes Singl.eView Convergent Billing standard output method types.

Output Method Type	Description
Print Invoices	Print invoices from an invoice run.
Reprint Invoices	Print a set of invoices that have been registered for reprinting.

Output Methods

There are no specific Singl.eView ICP or Singl.eView IP/Data requirements for setting up output methods. The configuration of output methods will depend entirely on the unique requirements of each installation. Some questions to be considered to help determine how to set up output methods include:

- Are invoices printed internally or is the printing outsourced?
- If the printing is done internally, will the invoices be printed at one location or will each invoice batch be split up, so that invoices are printed on the printer within the branch office closest to their distribution point?
- If the printing is done internally, how many printers are required for the printing of all invoice schedules? If only one printer is available, print jobs will have to queue. If multiple printers are available, each schedule can print to a different printer in parallel.
- If invoices are sent to an external print vendor, in what medium and format are the invoice records transferred?
- How are the invoices sorted before they are printed or written to a file? For example, are they sorted alphabetically?

Example

A billing schedule generates invoices for residential customers with an invoice cycle on the 5th of every month. Within that batch of invoices there may be invoices for customers in the northern region of the country and invoices for customers in the southern region of the country. It may be preferable to configure an output method that sends all of the invoices for customers in the northern region to the local printer for local distribution, and all of the invoices for customers in the southern region to a file that can then be sent to your southern office for printing and local distribution.



For information on configuring and splitting invoice batches, refer to the *System Operations Guide for Singl.eView Convergent Billing.*

GOP Schedule Configuration for Printed Invoices

The GOP schedule takes three parameters:

- Schedule to output

  The corresponding billing schedule that has generated the invoices to be printed.

- Output method

  This defines which devices and output method types will be used to output the invoices. Select the appropriate method for the schedule being output.

- Directory for temporary data

  This is a working directly only. The contents of this directory are deleted at the end of the GOP task. Recommended directory is:

  `$ATA_DATA_SERVER_LOG` directory.

The GOP must be run for each billing task prior to running the next billing task for a billing schedule.

Naming convention

For every billing schedule that is configured, there must be a corresponding GOP schedule. It is important to use a naming convention for the GOP schedule so that each billing schedule can be linked to the appropriate GOP schedule.

Email Invoices Schedule Type

The Email Invoices schedule type sends emails with attached invoice images to a SMTP server. The SMTP server then sends the invoices to the customers. When this schedule is run, email invoices are generated for all customers that have email addresses recorded for the contact (reached through the **Details…** button on the *Contact* tab of *Customer Detail* form) and who have an *Invoice Format* of 'Email Invoices' on the *Invoice Format Definition* form.

For every Invoice Generation schedule that is configured, there must be a corresponding Email Invoices schedule. It is important to use a naming convention for the Email Invoices schedule, so that each Invoice Generation schedule can be linked to the appropriate Email Invoices schedule.



Before running the Email Invoices schedule type, the temp directory where temporary files are written prior to being sent to the SMTP server must be set up at `$ATA_DATA_SERVER/invoices/images/tmp`.

Scheduling

The Email Invoices schedule type should be scheduled to run after the IGP portion of the billing run is complete. From the *Results* tab of the Invoice Generation task that generated the invoice images, find out whether the invoice images have been generated successfully, and how many have been generated. This will help determine how and when to schedule the Email Invoices process.



If a large number of invoice images have been generated, processing them all at once can overburden the SMTP server.

To avoid overburdening the SMTP server when there are a large number of email invoices to generate:

- Schedule the Email Invoices schedule to run when the SMTP server is less busy

  or

- Use the *No. of Invoices to Process* field on the *Parameters* tab of the *Schedule Definition* form to limit the number of invoices processed by the Email Invoices schedule. More than one schedule would then have to be set up to process all the invoice images generated by the IGP.



Before running the Email Invoices schedule, ensure that the SMTP server is up and running.

Scheduling parameters

When the Email Invoices schedule type is configured, three mandatory parameters and one optional parameter must be specified on the *Parameters* tab of the *Schedule Definition* form:

1. The task ID for the Invoice Generation schedule that generated the invoice images should be entered in the *Schedule to output* field. The applicable task ID can be selected from the drop-down list.
2. The 'CMN Email Invoices Method' should be selected in the *Output Method* field.
3. The name of the `temp_directory` must be specified in the *Directory for temporary data* field. This is the directory where temporary files are written prior to being sent to the SMTP server. Enter **`$ATA_DATA_SERVER/invoices/images/tmp`.**
4. *No. of Invoices to Process* is an optional field that can be used to limit the number of invoices processed by the run and thereby avoid overburdening the SMTP server. For further information, refer to the suggestions earlier in this section.

Actions triggered by running the Email Invoices schedule

When the Email Invoices schedule is run, any invoice images that were generated by the Invoice Generation task that is specified in the *Schedule to output* field are converted to attachments to emails that are sent to the SMTP server. The SMTP server then emails the invoices to the customers. If a number has been entered in the *No. of Invoices to Process* field, either that number will be processed, or fewer will be processed if the total remaining is less than the number specified.



The Email Invoices process assumes that the SMTP server is up and running.

Output results

The success or failure of the run can be determined by checking the *Task Status* field and the *Results* tab of the *Task Details* form. The task ID is also displayed on the *Results* tab and should be noted in case the task needs to be viewed later. For more information on how to access this data, refer to *Using Schedules* in *Chapter 2, Scheduling Tasks*. Output results can also be sent by email (for more information, refer to *Fields* in *Chapter 2, Scheduling Tasks*).



A successful run does not indicate that the invoices have been successfully mailed to the customers, only that the emails have been successfully accepted by the SMTP server. The Email Invoices schedule assumes that the SMTP server is up and running.

The following is an example of the output text:

```
<S90006> gop_emails Started Thu Aug 24 16:57:32 2000.
<M06003> Connecting to database "SIP_DEV" as ""...
<M06313> 2 images output.
<S90007> gop_emails Completed Thu Aug 24 16:58:22 2000.
```

Troubleshooting

Check the number of images output. It should either be equal to the number of customers set up to receive email invoices for the bill cycle that was run, or if a *No. of Invoices to Process* was specified, it should be fewer than, or equal to, that number. It may be fewer than the number specified in the *No. of Invoices to Process* field if the total number of unprocessed invoices for that Invoice Generation task is less.

If the number of images output is not what was expected or if no images were output, the *Results* tab of the Invoice Generation task that generated the invoice images should be checked to ensure that the invoice images were successfully generated.

If the Invoice Generation task has failed to generate the images, the IGP command line options on the *Parameters* tab of the *Schedule Definition* form for the Invoice Generation task should be checked. In addition, ensure that the working directory specified in the IGP command line options exists and can be written to. Then run a task of type 'Regenerate Invoices'. For more information, refer to *Correcting Billing Errors* in *Chapter 3, Billing* of the *Systems Operations Guide for Singl.eView Convergent Billing*.

Re-Generating Email Invoices

If an email invoice has already been generated and there is a need to re-send the invoice to a customer, use the following procedure:

To re-generate email invoices:

1. Select the **Accounts** ⇨ **Invoices** option. The *Invoice Search* form is displayed.
2. Search for the invoice to be re-generated. For more information on how to search for an invoice, refer to the *Customer Service Representative User Guide for Singl.eView Convergent Billing*.
3. Open the *Invoice Detail* form for the selected invoice.
4. On the *Format* tab of the *Invoice Detail* form, select the CLEC Invoice or CMN Invoice format record.
5. Click the **Generate Image** button. A confirmation message will be displayed. Click the **OK** button.
6. A second confirmation message will be displayed. Click the **OK** button.
7. Click the **Invoice Image…** button to view the invoice.
8. On the *Format* tab of the *Invoice Detail* form, select the Email Invoice format record.
9. Repeat steps 5 through 7.

Printing and Reprinting

Printing

Printing a batch of invoices involves:

- Running the GOP to print invoices from the invoice tables for the most recent task of the specified billing schedule.
- Sending invoices to a printer or other output device.
- Setting an output flag within the invoice tables indicating that the invoices have been printed.

Reprinting

Reprinting invoices involves:

- A CSR flagging selected invoices where another copy of that invoice has been requested by a customer.
- Running the schedule 'Invoice Reprint Generation' to generate the invoice images for those selected invoices that have been flagged for reprinting.
- Running a GOP schedule using an 'Output Method Type' of **Print Invoices** and an 'Output Device' of **CLEC Reprint Printer** to print the invoices with a watermark indicating that they are a **copy** and not the original.

Troubleshooting for Printed Invoices

Failed print task

If a printing task fails for any reason (for example, if the toner was low in the printer or if the printer lost power during the print job), the entire batch needs to be reprinted. To reprint an entire batch:

1. Run an immediate task of 'Revoke Invoice Print Settings'. This will reset the printed flag of all invoices in that print batch so they are printed again.

   Note: If this flag is not reset when original GOP schedule is re-submitted, the system will not print any invoices because it will think that the invoices have already been printed.

2. Re-submit the original GOP schedule for that batch of invoices.

Other failed output task

If the file that a batch of invoices was output to is corrupted, or is accidentally deleted and needs to be recreated:

1. Run an immediate task to 'Revoke Invoice Print Settings'. This will reset the printed flag of all invoices in the invoice batch so that they can be output again.

   Note: If this flag is not reset, when the original GOP schedule is resubmitted it will not output any invoices because it will think that the invoices have already been output.

   Although the name of the flag infers that it is only set if invoices are printed, it actually refers to the output status of an invoice; as soon as an invoice has been output by the GOP, this flag is set.

2. Re-submit the original GOP schedule for that batch of invoices to recreate the file.

Chapter 6
Singl.eView ICP Downstream Interfaces

This section describes the operational aspects of downstream interfaces supported by Singl.eView ICP. The following information is included:

- Overview of CNAM, E911, and LIDB
- Configuring support in Singl.eView ICP
- Setting up schedules and configuration parameters
- Setting up and modifying function definitions
- Maintaining the interface derived attribute tables
- Monitoring schedules and frequency
- Delivering files via FTP.

Overview

Singl.eView ICP allows changes to service information to be detected and captured for subsequent extraction and output to downstream systems. The service details captured are stored in a Singl.eView Convergent Billing database table and then later extracted to a file.

The following service information records can be extracted:

- CNAM – Calling Line ID name presentation
- E911 – Emergency Services address information
- LIDB – Call Card PIN Number.

More details on each of these records are provided below. When certain changes occur to services, the changes can be detected and written to a database table. The data can then be extracted from the table, formatted, and output to a file and sent to the downstream system.

To support the generation of these files from an operational perspective, the following steps should be taken in Singl.eView Convergent Billing:

1. Set up configuration

   This involves modifying a set of options in a function definition and setting up schedules. For a detailed description of this process, refer to *Configuration* later in this chapter.

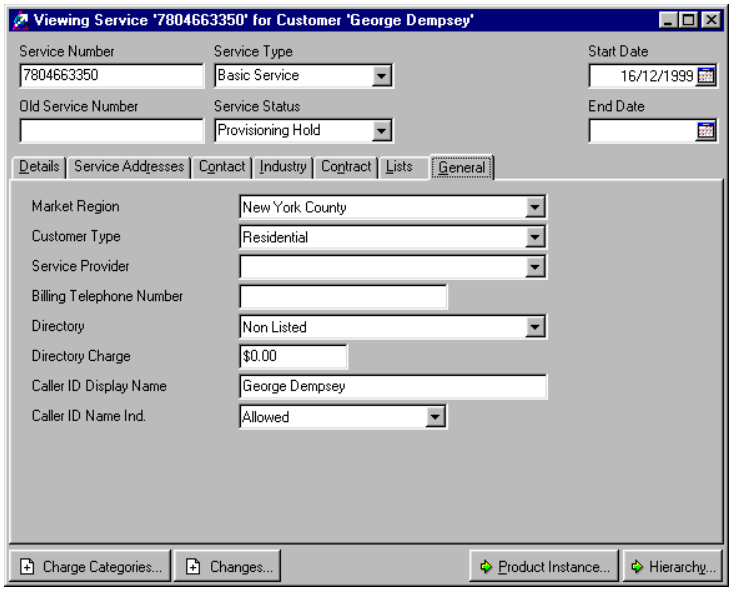
2. Create output directories

   The files generated after the schedule is run are placed in the `$ATA_DATA_SERVER/<capture type>` directory. For a detailed description of this process, refer to *Outputs* later in this chapter.

CNAM

The Calling Line Name record specifies the calling line name for a service. This is the name associated with a service that is displayed on a recipient's telephone, instead of the telephone number.

In Singl.eView ICP, the *Caller ID Display Name* is specified on the *General* tab of the *Service Detail* form.



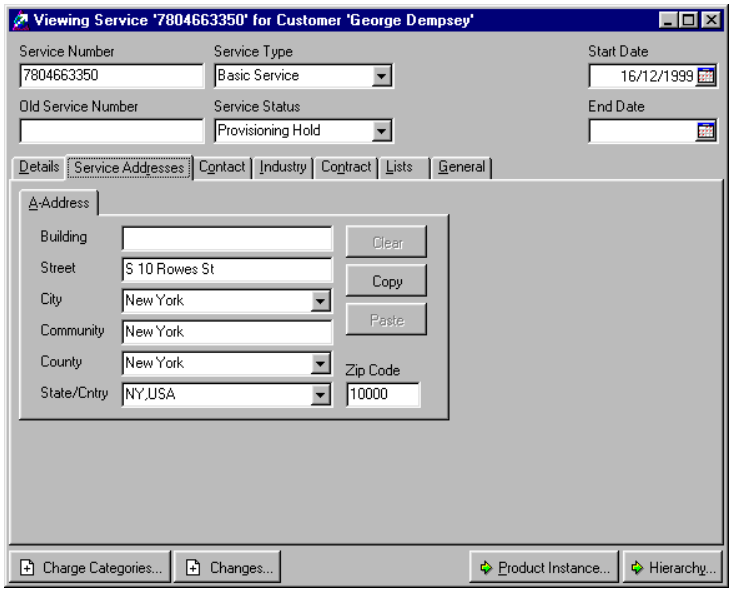
If the *Caller ID Display Name* is updated or revised, an update record is generated in the database table and will be extracted to the output file when the schedule is run. The following steps summarise the operations that occur when CNAM specific record changes are made to the service:

- Whenever a newly created service (for which CNAM capture is to be performed) leaves the 'Provision Hold' status (indicating that the service is to be provisioned or made active), an Insert CNAM record is generated.
- Any changes to the *Caller ID Display Name* or the *Caller ID Name Ind.* for a particular service (for which CNAM capture is to be performed) result in an Update CNAM record being generated.
- Whenever a service (for which CNAM capture is to be performed) is cancelled, a Delete CNAM record is generated.
- Whenever the telephone number (Service Number) of a service changes, a Delete CNAM record is generated with the old telephone number and an Insert CNAM record is generated with the new telephone number.

E911

The E911 record captures the address associated with a particular telephone number. The records are typically sent to service companies that provide a regulatory 911 Emergency service.

In Singl.eView ICP, the information for the E911 record is on the *Service Addresses* Tab of the *Service Detail* form.



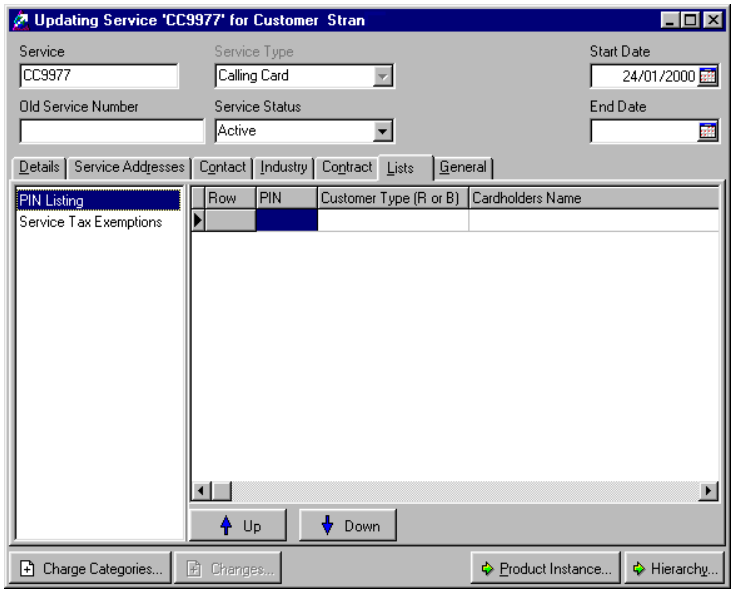
The following steps summarise the operations that occur when the E911 specific record changes are made to the service:

- Whenever a newly created service (for which E911 capture is to be performed) leaves the 'Provision Hold' status (indicating that the service is to be provisioned or made active), an Insert E911 record is generated.
- Any changes to the A-Address, the E911 Class Of Service (COS), or the customer name for a particular service (for which E911 capture is to be performed) result in an Update E911 record being generated.
- Whenever a service (for which E911 capture is to be performed) is cancelled, a Delete E911 record is generated.
- Whenever the telephone number (Service Number) of a service changes, a Delete E911 record is generated with the old telephone number and an Insert E911 record is generated with the new telephone number.

LIDB

The LIDB record captures the information for services that require PINs such as calling cards or travel cards.

In Singl.eView ICP, the information for the LIDB record is specified in the PIN Listing list on the *Lists* tab, and on the *Industry* tab of the *Service Detail* form for Calling Card and Travel Card services.



The following steps summarise the operations that occur when LIDB specific record changes are made to the service:

- Whenever a newly created service (for which LIDB capture is to be performed) leaves the 'Provision Hold' status (indicating that the service is to be provisioned or made active), an Insert LIDB record is generated for each PIN associated with the service.
- Any changes to the following information for a particular service (for which LIDB capture is to be performed) will result in an Update LIDB record being generated (for each PIN that is affected):
  - Certain details within the A-Address
  - Customer name
  - Blocking code
  - Card restrictions
  - Number of cards requested
  - Special indicator
  - Class code
  - PIN customer type (R or B)
  - PIN cardholder's name.
- Whenever a service (for which LIDB capture is to be performed) is cancelled, a Delete LIDB record is generated for each PIN associated with the service.

- Whenever the telephone number (service number) of a service changes, a Delete LIDB record is generated with the old telephone number and an Insert LIDB record is generated with the new telephone number for each PIN associated with the service.
- Whenever the PIN number associated with a service changes, a Delete LIDB record is generated with the old PIN number and an Insert LIDB record is generated with the new PIN number.

Configuration

The operational tasks required to configure Singl.eView Convergent Billing to manage the generation of CNAM, E911, and LIDB records include:

- Maintaining the client configurable parameters of the Update Capture Options function
- Configuring the update capture schedule
- Maintaining the derived attribute tables for each interface defining the type of service for which records are to be captured.

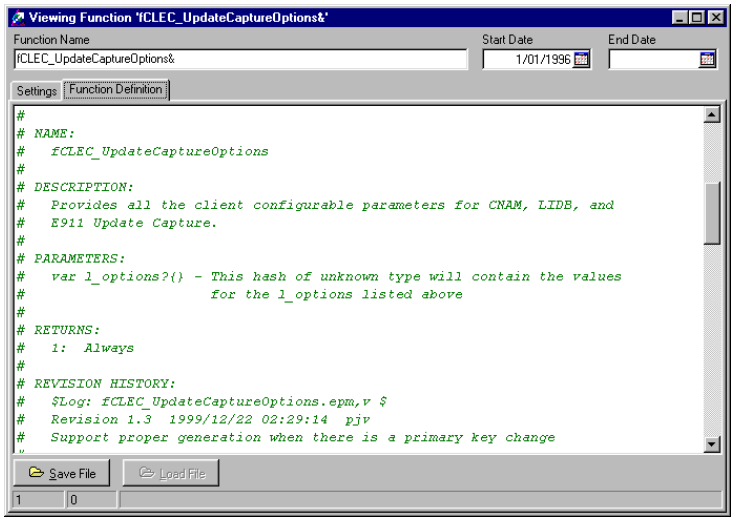
Maintaining the Update Capture Options function

This section describes how to maintain the Update Capture Options function. For a detailed description on how to create and maintain function definitions, refer to *Variables, Expressions, and Functions for Singl.eView Convergent Billing.*

The Update Capture Options function is maintained from the *Function Definition* form. To view the form, select the **Maintenance ⇨ Tariffs ⇨ Function** option, and search for the function

```
fCLEC_UpdateCaptureOptions&
```

The following form is displayed.



The `fCLEC_UpdateCaptureOptions&` function provides all the client configurable parameters for CNAM, E911, and LIDB update capture. The definition will need to be modified if CNAM, E911, or LIDB files are to be generated.

A sample of the CNAM parameters specified for this function definition is provided below.

```
### CNAM
#
l_options?{'CNAM_Capture'} := 'ON';
l_options?{'CNAM_Client_Number'} := '222222';
l_options?{'CNAM_Sequence_Nr'} := '00';
l_options?{'CNAM_Record_ID'} := '2';
```

The following table summarises the parameters and options that can be specified for the CNAM definition.

CNAM Parameter	Option	Description
`Capture`	ON/OFF	Enables/Disables the capture of CNAM records
`Client_Number`	6 digit number	Specifies the client number in the file
`Sequence_Nr`	2 digits	Specifies the sequence number of the CNAM record
`Record_ID`	1 digit	Specifies the ID of the record.

A sample of the LIDB parameters specified for this function definition is provided below.

```
### LIDB
#
l_options?{'LIDB_Capture'} := 'ON';
l_options?{'LIDB_Client_Number'} := '333333';
```

The following table summarises the parameters that can be specified.

LIDB Parameter	Option	Description
Capture	ON/OFF	Enables/Disables the capture of LIDB records
Client_Number	6 digit number	Specifies the client number in the file

A sample of the E911 parameters specified for this function definition is provided below.

```
### E911
#
l_options?{'E911_Capture'} := 'ON';
l_options?{'E911_Cell_ID'} := ' ';
l_options?{'E911_Sector_ID'} := ' ';
l_options?{'E911_TAR_Code'} := ' ';
l_options?{'E911_INP'} := ' ';
l_options?{'E911_Comments'} := ' ';
l_options?{'E911_County_ID'} := ' ';
l_options?{'E911_General_Use'} := ' ';
l_options?{'E911_Company_ID'} := ' ';
l_options?{'E911_Source_ID'} := ' ';
l_options?{'E911_Customer_Code'} := ' ';
l_options?{'E911_End_Of_Record'} := '*';
```

The following table summarises the parameters that can be specified.

E911 Parameter	Option	Description
Capture	ON/OFF	Enables/Disables the capture of E911 records
Cell_ID	6 digits	E911 cell Id
Sector_ID	1 digit	E911 sector ID
TAR_Code	6 digits	E911 tar code
INP	156 digits	E911 inp

E911 Parameter	Option	Description
Comments	30 digits	E911 comments
County_ID	4 digits	E911 county ID
General_Use	11 digits	E911 general use
Company_ID	5 digits	E911 company ID
Source_ID	1 digit	E911 source ID
Customer_Code	3 digit	E911 customer code
End_Of_Record	1 digit	E911 end of record

The E911 function also requires maintenance of the E911 Header and Trailer parameters. A sample of the Header/Trailer definition is provided below.

```
### E911 Header/Trailer
#
l_options?{'E911_HT_County_ID'} := 'AAA';
l_options?{'E911_HT_State'} := 'AAA';
l_options?{'E911_HT_General_Use'} := 'AAA';
l_options?{'E911_HT_Release_Number'} := 'AAA';
l_options?{'E911_HT_Format_Version'} := 'AAA';
l_options?{'E911_HT_End_Of_Record'} := '*';
l_options?{'E911_HT_Head_Ind'} := 'UHL';
l_options?{'E911_HT_Trail_Ind'} := 'UTL';
l_options?{'E911_HT_Company_Name'} := 'Test Company';
```

The following table summarises the header and trailer parameters that can be specified.

E911 Parameter	Option	Description
County_ID	4 digits	E911 county ID
State	2 digit	E911 state
General_Use	20 digits	E911general use
Release_Number	3 digit	E911 release number
Format_Version	1 digit	E911 format version
End_Of_Record	1 digit	E911 end of record
Head_Ind	5 digit	Header indicator
Trail_Ind	5 digit	Trailer indicator
Company_Name	50 digit	Company name

Configuring the Update Capture schedule

The 'CLEC Update Capture Extraction' schedule can be run either:

- From within the client via an immediate task
- As a regular schedule.

To configure the schedule:

1. Select the **Server Tasks ⇨ Schedules** option. The *Schedule Search* form is displayed.
2. Click the **New…** button. The *Schedule Definition* form is displayed.
3. Create a new schedule of type 'CLEC Update Capture Extraction'.
4. Set the *Repeat Units* to the required frequency for the process. For a daily process, the *Repeat Units* would be 1 and the *Repeat Type* would be *Day*.
5. Set the *First Date* and time for when the update capture is to run every time the task is repeated.
6. On the *Parameters* Tab, select the appropriate file type from the drop-down list in the *File Type* field.

Note: The configured schedule generates only one type of file depending on the selection in the *Parameters* tab. To generate CNAM, E911, and LIDB files, three separate schedules must be configured.

To view the 'CLEC Update Capture Extraction' schedule type:

1. Select the **Maintenance ⇨ Server ⇨ Schedule Types** option. The *Schedule Search* form is displayed.
2. Click the **Search** button.
3. Select 'CLEC Update Capture Extraction' schedule.
4. Click the **Detail…** button. The *Schedule Type Definition* form is displayed with the following details.

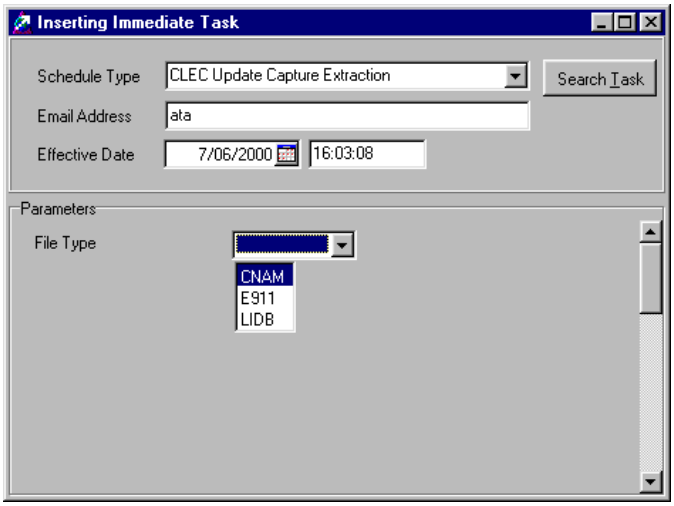
Field Name	Description
Schedule Type Name	CLEC Update Capture Extraction
Description	Extracts update capture information, formats it and sends it to a file or other output device
Task Type	General Server Task
Maximum Duration	3600 seconds
Maximum Tasks	1

Field Name	Description
Retain Tasks for	30
Task Complete Message	<M02004>
Task Validation	tqiCLEC Update Capture
Server Command	**uc_extract**

5. A new local schedule type with the features above can be created and renamed as 'CLEC Local Update Capture Extraction', if required.

To run 'CLEC Update Capture Extraction' as an immediate task:

1. Select the **Server Tasks ⇨ Immediate Task** option. The *Schedule Immediate Task* form is displayed.
2. Select 'CLEC Update Capture Extraction' from the drop-down list in the *Schedule Type* field.
3. On the *Parameters* tab, select the appropriate file type from the drop-down list in the *File Type* field.

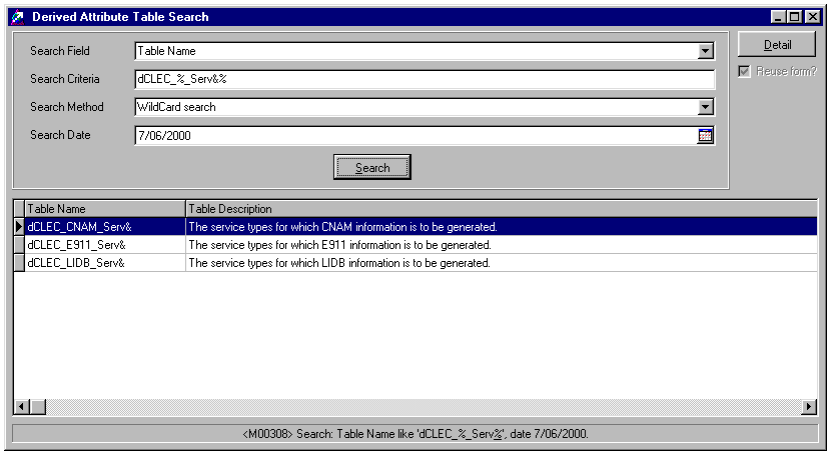


4. Click the **Save** button on the toolbar.

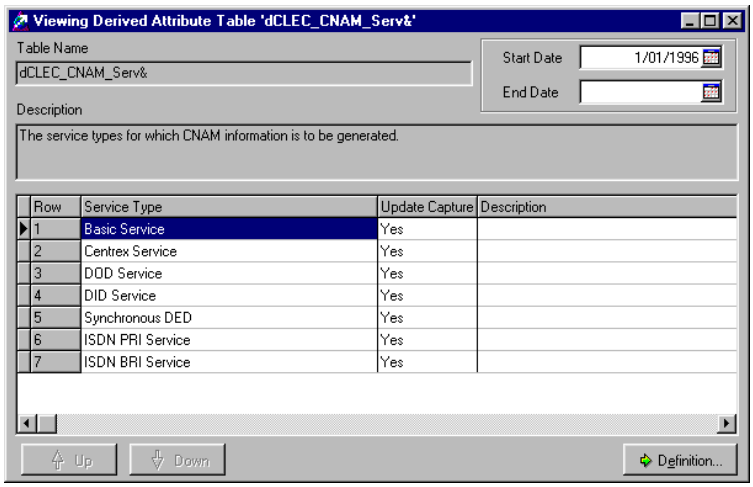
Maintaining the interface derived attribute tables

In Singl.eView ICP, three derived attribute tables have been created to define the service types for which records are to be captured. These tables may need to be modified if new service types are defined requiring CNAM, E911, or LIDB capture.

The three derived attribute tables are displayed below.



The `dCLEC_CNAM_Serv&` table defines the service types for which CNAM records are to be captured (if the `CNAM_Capture` flag is set to ON). The *Derived Attribute Definition* form is displayed below. Any new service types need to be added to the table.



The `dCLEC_E911_Serv&` table defines the service types for which E911 records are to be captured (if the `E911_Capture` flag is set to ON).

The dCLEC_LIDB_Serv& table defines the service types for which LIDB records are to be captured (if the LIDB_Capture flag is set to ON).

For more detail on working with and maintaining derived attributes, refer to the *Product Configuration Guide for Singl.eView Convergent Billing*.

Outputs

When the update capture schedule is completed, the files are output to a directory on the server associated with the file type.

If the directories are not already created on the server machine, they will need to be created. The following directories should be available to receive the output from the schedule:

- E911 records are output to the directory $ATA_DATA_SERVER/E911
- CNAM records are output to the directory $ATA_DATA_SERVER/CNAM
- LIDB records are output to the directory $ATA_DATA_SERVER/LIDB

The directory paths are specified by the environment variable $ATA_DATA_SERVER. To view the path specified for the directory, enter the UNIX command echo **$ATA_DATA_SERVER**. The directory paths specified are case sensitive.

The output files generated are date and time stamped. As a result, they are not overwritten when output to the specified directories.

Sending files by FTP

The FTP process transfers a set of files by FTP, and can be used for both inward and outward transfers. The following aspects of the process need to be setup in an operational support context:

- Inbound FTP sources/files/users/passwords etc.
- Outbound FTP destination/files/users/passwords etc.
- Transfer failure notification and retry options.

When to Run CLEC Update Capture Extraction

The 'CLEC Update Capture Extraction' schedule should be set up to run on a daily or weekly basis dependent on the requirements of your service provider.

Recommendations

It is recommended that generation of those records that are not required be turned off. For example, if only the generation of E911 records is to be done, the CNAM and LIDB record capture parameter should be turned off in the `fCLEC_UpdateCaptureOptions&` function definition.

Chapter 7
Treatment

Singl.eView ICP and Singl.eView IP/Data treatment is designed to manage customers who fail to pay their invoice by the specified due date. Each customer is assigned a customer type and credit class which will determine the criteria for that customer entering into, and progressing through, predefined treatment processes. For example, a residential, low credit risk customer may enter the treatment process five days after the due date on their invoice and a residential, high risk customer may enter the treatment process immediately once the due date is reached.

Once a customer has entered treatment, their case will be escalated through a series of stages called treatment steps. This escalation will continue until the customer reduces the amount owing to below the minimum collection amount (which is specified for each customer type and credit class), and makes a promise to pay the balance of the account. If a customer does not do this, the final stage of treatment is reached. Treatment tasks performed at each step, and the criteria a customer must meet to progress to each step, is defined by the individual organisation. The following list shows an example of some of the treatment steps that can be implemented (the order and number of steps can be modified for each customer type and credit class):

- Letter
- Phone call
- Suspend features
- Suspend service
- Apply security deposit
- Collection agency
- Disconnect service.

Treatment Queries

In Singl.eView ICP and Singl.eView IP/Data, a customer query of type 'Treatment' is used to store the current status of a customer in treatment. While in treatment, a customer will only have one customer query of type 'Treatment' open. This indicates the current treatment step, what actions have taken place or have yet to take place based on the present treatment step, and any other information that is configured. For example, it might be configured to show, based on the current balance, the dates on which the customer will continue to escalate through treatment. When a customer is not in treatment, they will have no open queries of type 'Treatment' .

A new 'Treatment' customer query is created when the 'Treatment Monitor' schedule is executed and determines that the customer's accounts are required for treatment. As a customer moves from one treatment step to the next, the 'Treatment Monitor' schedule closes the existing 'Treatment' customer query, and a new one is opened for the new step. When a customer moves out of treatment, the last remaining open 'Treatment' customer query is closed. A customer's full treatment history is available by examining the closed 'Treatment' queries for that customer.

Note: All treatment query creation, escalation, and closure will only occur during the execution of the 'Treatment Monitor' schedule or 'Promise To Pay Monitor' schedule.

The *General* tab on the *Customer Query* form contains two fields to store the current treatment step. Each query record uses another *General* field to store the next treatment step. If the Treatment Monitor process decides that it is time to escalate the customer to a new step, the *Next treatment level* field on the current treatment query is assigned the value for this new treatment level, and the query is closed. A new query is opened with the new treatment level.

If a CSR wishes to manually move the customer into a new treatment level, they can update the current treatment query, and set the *Next treatment level* field to the new treatment value. This will automatically close the edited query and create a new treatment query with the new treatment step details.

Note: The new treatment level is not applied until the 'Treatment Monitor' schedule is run. Even if the treatment schedule is run, the current level of treatment will be left to run its course before the new specified level is applied.

Promise to pay

A *Customer Query* form with a query type of 'Promise to Pay' has three date fields:

- *Opened*
- *Instalment End Date*
- *Final Date*.

Each date is stored within the database under an alias. The *Opened* field is the date and time that the 'Promise to Pay' query has been created. It has the same name within the database.

The *Instalment End Date* describes the date due of the 'Promise to Pay' queries. In the case of linked promises to pay, each promise will have a different *Instalment End Date* to reflect the time that each promise must be paid by, to keep the promise to pay as a whole. The *Instalment End Date* is known within the database as the 'Response Due Date'.

The *Final Date* is the date that the 'life span' of the promise to pay is completed. This date is known within the database as the 'Resolution Due Date'.

Note: When a promise to pay is not linked (ie. it is just a single promise to pay), then the *Instalment End Date* and *Final Date* will be the same.

Customers with open promises, in which the start and end dates of the promise span the date of the treatment schedule, are considered to be lifted out of treatment. If customers pay according to their promise, they are removed from treatment, but they may re-enter treatment at the starting level on a different date. If a customer breaks the promise, they are escalated to the next treatment step.

A promise can be linked to another promise by storing the previous promise ID on the *General* tab. If the promise is linked, the start date of the promise must be after the end date of the previous promise. A customer can only have one **active** promise at any specific date (there may be future or historical promises).

The *General* tab contains a percentage in the *Promise Threshold* field. This is set by a system default. When investigating a promise, the total amount of that promise (which includes the previous and next promises), is compared to the total amount of payments made over that period. If the total payment amount is *n%* or more of the total promise amount (where *n%* is the threshold percentage), the promise is kept. Payments accepted and rejected within the investigation period will be included in the total payment amount (since they will cancel each other out). Payments rejected within the investigation period but not received within the investigation period will not be considered for that total payment amount.

The *General* tab also contains the *Promise Outcome* field. If a promise has not yet been investigated, the field is blank. If a promise has been investigated, the field records whether the payment amount was enough to keep the promise or not. This field is required because linked promises remain open even after they have been investigated. The promise query is closed after the last promise in the series has been investigated.

When investigating a promise that is linked to another future promise, the customer's payment amount must be *n%* or more of the total promise amount, that *n%* representing a configurable amount. This percentage threshold defaults to a system setting, and appears on the *General* tab. The promise query cannot be progressed to any workforce.

A scheduled process will 'investigate' each promise on the end date of that promise. This schedule runs nightly over all promises to pay that end on or before that date where there is not yet an investigation outcome. If the total payment amount over that period is enough to meet the promise, the promise is marked accordingly.

Using the Treatment Engine

Schedule types

- Treatment Monitor

  This process examines customers and determines whether they are eligible to enter treatment, leave treatment, or progress to the next treatment level. This schedule will also control the processes that need to occur when a customer enters a particular treatment step (eg. sending treatment letters).

- Promise to Pay

  This process evaluates all open promises to determine whether they have been kept or not.

Operators entering a schedule can choose to filter that schedule by entering a set of criteria. The following chart shows the parameters available for filtering:

Parameter	Description
Customer Type	All customers, or customers of a specific type.
Optional WHERE Clause	Defines any customised filtering.
Credit Class	All customers, or customers of a specific credit class.
State	All customers, or customers with a site address in a specific state.

Additional parameters can be configured, and credit class and state can be removed.

Tips and Recommendations

- Run treatment tasks during the night when system activity is at a minimum.
- Ideally, treatment should be run on all customers once a day. If time/resource constraints prevent this, multiple treatment schedules can be created to perform treatment on certain subsets of customers.
- Treatment cannot easily be revoked. Ensure that all treatment parameters are specified correctly before performing a treatment task.
- For some businesses it may be standard policy not to send a customer any further letters once they have been suspended. For example, a customer should not be sent a warning letter for an overdue invoice if they are already suspended for another late invoice. By default, the treatment engine will still perform treatment actions on suspended customers (eg. sending a letter). To implement the functionality described by the above example, the WHERE clause parameter in the schedule needs to be set so that it prevents treatment from being performed on such customers.

Treatment Report

The Treatment Management Report (schedule type RTRMT_CMN) lists details of customers with overdue accounts.

The purpose of this report is to monitor overdue accounts, identify possible credit risks, and take action based on the treatment status.

For more information, refer to the Reports Guide for *Singl.eView Convergent Billing ICP and Singl.eView Convergent Billing IP/Data*.

Chapter 8
PC Operations

The role of System Administrator goes beyond the database and server. In the Singl.eView Convergent Billing three-tier architecture, client PCs require regular attention and configuration for optimum Singl.eView Convergent Billing performance.

Occasionally, the Singl.eView Convergent Billing System Administrator may need to instruct operators to clear out temporary files or refresh the PC cache to coincide with other global configuration activities.

The chapter discusses the importance of this role in the following areas:

- Clearing out temporary invoice files
- Caching.

Clearing Out Temporary Invoice Files

When invoice images are viewed within Singl.eView Convergent Billing, a `.spf` file that is used by Acrobat Reader is stored locally on the PC. These files are generally stored in the 'c:/windows/temp' directory. Without attention, this directory will grow and may take up valuable disk usage.



A System Administrator should send monthly reminders to operators to clear out their window's temporary directory.

Caching

Database queries are used to retrieve data such as screen details, an operator's security access levels, or database records. Much of this data is static and rarely, if ever, changes. However, operators perform the same queries daily, but the data being retrieved is constantly changing.

For efficiency and performance, both the regular queries used and the static data can be cached on the local PC to speed execution the next time that information is required.

Caching follows a simple 'first in, first out' rule. If the cache has reached its desired limit, the oldest queries are replaced with the latest queries.

The cache configuration details are saved automatically in the atlanta.ini file for that application.

Caching static data

The database queries and results used to retrieve static information from the database can be cached on the local PC. Next time that data is required, it can be retrieved from the local cache instead of retrieving it from the database.

Examples of static type data include:

- An operator's security access levels.
- Reference types such as customer, equipment, and service statuses.

Note: Examples of non-static data that is regularly queried but not cached includes regularly updateable data such as customer nodes, product instances and equipment.

Caching commonly used queries

Each time data needs to be retrieved from the database, a query is constructed first to retrieve the required data that the operator is allowed to see. The query is executed and the data is retrieved from the database.

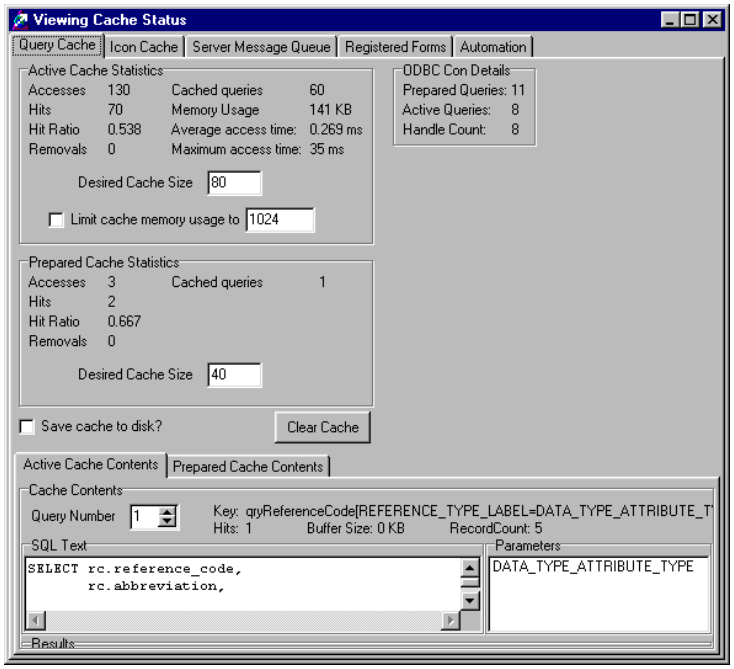
Operators can perform the same types of queries daily. The data they are retrieving is changing but the query used to retrieve that data is constant. For efficiency, the constructed query used to retrieve this data can be cached locally and used immediately the next time that query is required.

Examples of commonly used queries include:

- Residential customers
- Product types
- Quarantined equipment items.

Turning on caching

From the **File ⇨ Cache Status** menu item, set the *Desired Cache Size* to the required limit.



- The default cache size is 80.
- For no caching, set the Desired Cache Size to 0.
- The cache is cleared each time you log out of Singl.eView Convergent Billing.

Saving your query cache

Each time you log into Singl.eView Convergent Billing, your query cache is empty. The first time you perform an activity, the queries are re-constructed and executed to retrieve the required data. Hence, the first time an activity is performed, it will be slower than the subsequent executions of the same activity during that session.

Alternatively, you can chose to save your query cache to a file on your local PC. The next time you log into Singl.eView Convergent Billing, your query cache will be reloaded from the saved file automatically.

The saved cache file is specific to an operator. For example, if two different operators use the same PC and both have the option set to save their query cache to disk, two files will be present on the local PC each with the name of the operator in the file name.

The next time a specific operator logs in, Singl.eView Convergent Billing will check for the existence of a query cache file for that operator and use it if it is configured to do so.

To save the cache to disk, select the check box *Save cache to disk?* on the *Query Cache* form shown above.

The query cache file can be saved locally. The file will have a name made up of `<system name>_<operator name>.aqc`

For example, `atp_tst2_tah.aqc`

The query cache file location can be specified in the 'atlanta.ini' file. If it is not specified there, it will default to the 'c:\atlanta' directory. If this directory does not exist, the file will reside in the 'c:\' directory.

When to clear the query cache

By default, the query cache is cleared each time a user logs out of Singl.eView Convergent Billing, unless the option *Save cache to disk?* is turned on.

If a System Administrator changes an operator's security access levels while he or she is currently logged into Singl.eView Convergent Billing, the operator should clear his or her cache to ensure that all subsequent queries are re-cached to include their correct data access levels.

The query cache needs to be cleared if:

- Configuration has changed and an operator has saved his or her cache to disk.
- An operator is logged in while a configuration change is made and he or she needs to see or use the new configuration.

How to clear the query cache

To clear the query cache either:

- Click on the **Clear Cache** button on the *Cache Status* form.

  or

- Select **File ⇨ Refresh Security Settings** option. An hour glass icon will appear while the query cache is clearing and will disappear when it is complete.

  Refreshing the security settings clears the entire query cache and then reloads the queries used to retrieve an operator's security access levels. This option is preferable, as not all operators will have access to the *Cache Status* form.

Troubleshooting

If the response time for searches or other database activities is slow, check the local cache size. It may not be large enough to store all of the regularly used queries.

If the cache size is too small, it will appear as if no caching is occurring. The next time an activity, similar to a previous one, is performed, it is probable that those queries no longer exist in the cache and have to be reconstructed again. There is therefore no noticeable gain in performance.

Recommendations

A desired cache size of 100 is recommended for CSRs who perform regular customer hierarchy searches and product instance creations. Anything less than this may not achieve the desired performance gains.



Whether developing new Singl.eView Convergent Billing configuration components, or modifying existing ones, such as functions, reference types, or attributes, *Desired Cache Size* should be set to 0. This will prevent any of the queries used to retrieve the configuration data from being stored in the cache. It will not be necessary to continually refresh security settings after each change to see the results.

Note: Ensure that the cache limit is set back to the recommended size for general Singl.eView Convergent Billing operations.

Chapter 9
Database Auditing

Singl.eView Convergent Billing provides the ability to audit various tables in the database. The audit process records information about who made changes to a table, when the change was made, and what the record being changed looked like before the change.

This chapter explains how to turn auditing on or off. It explains how to use the *Audit Search* forms to retrieve information, and how to interpret the audit records. This chapter also discusses the housekeeping responsibilities required to keep the audit tables at manageable sizes.

At the end of this chapter there is a list of all tables that can be audited, and the recommended auditing requirements for each table in development and production environments.

Why Audit?

Audit information plays a very important role in:

- Monitoring database security
- Providing statistical analysis of tables
- Assisting in problem solving when errors occur due to changed data.

Security

Occasionally, the activities of an operator may need to be monitored to determine whether they are performing illegal operations or not. An organisation may need evidence of fraudulent behaviour before they can start legal action. The audit records will show who made particular changes to database records, and when those changes were made.

Problem solving

When errors occur in rating, billing, or CSR operations, the latest changes to data and who made them can be tracked to assist in rectifying the problem.

For example, one operator may have cancelled the wrong customer record in the system while another operator is trying to assign a product to that customer. To enable the second operator to determine how the customer record was cancelled, he or she can use the audit records to show who cancelled the customer record and when it was cancelled. The second operator can then contact the first operator to correct the situation.

How to Audit a Table

Auditing a table is a two-step process:

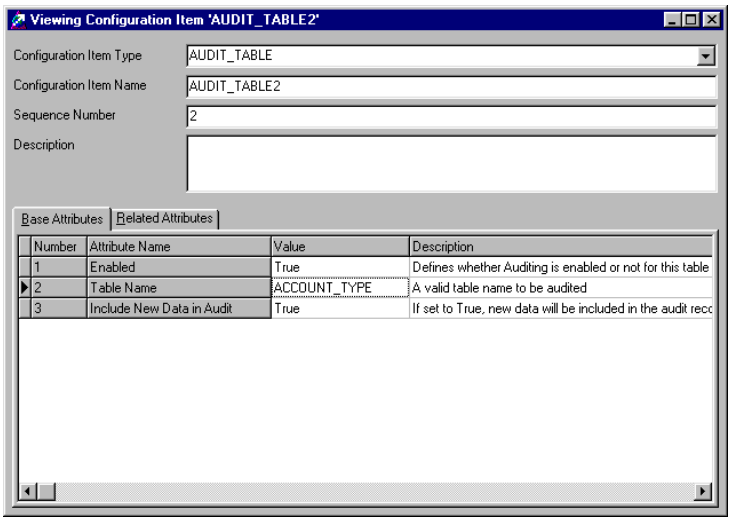
1. Creating a configuration item of type `AUDIT_TABLE` for the table to be audited.
2. Running the schedule 'Reconfigure Auditing' to establish the database triggers for auditing.

Creating the audit configuration item

To create the audit configuration item:

1. Select the **Maintenance ⇨ Server ⇨ Configuration Items** option. The *Configuration Item Search* form is displayed.
2. Click the **New…** button. The *Configuration Item Definition* form is displayed.
3. Select a *Configuration Item Type* of `AUDIT_TABLE` and tab to the next field. This action will automatically populate some of the other fields.
4. Replace the *Configuration Item Name* with the table name to be audited, and prefixed with the word `AUDIT`; for example, `AUDIT_CUSTOMER_NODE`.
5. Accept the default *Sequence Number*.

6. Complete the three audit attributes in the bottom grid.



7. Click the **Save** button on the toolbar.

8. Repeat this process for each table being audited.

The *Configuration Item Definition* form contains the fields described below.

Attribute	Description
`Enabled`	TRUE or FALSE Defines whether Auditing is enabled or not for this table. If set to TRUE, the schedule 'Reconfigure Auditing' will establish database triggers to audit activity on the table when the schedule is run. If set to FALSE, then the schedule 'Reconfigure Auditing' will disable the database triggers that audit activity on the table when the schedule is run.
`Table Name`	Name of table to be audited.
`Include New Data in Audit`	TRUE or FALSE If set to TRUE, the INSERTS into the table are audited as well. If set *to* FALSE, then only UPDATES/DELETES are audited.

Full versus partial auditing

The audit table configuration item asks whether new data should be included in the audit. If configured to TRUE, *insert activities*, *deletes*, and *updates* are captured. There are some tables in which it is not practical to audit all activity because the audit tables would fill up with data that is not useful.

For example, a full audit on the `NORMALISED_EVENT` table should not be performed, although capturing changes to event records might be desirable.

When deciding which tables to audit and the type of auditing required, the following points should be remembered:

- What is the daily growth of the table?
- Is size a concern? Is there enough table space to support all of the audit data?
- Is it a table of low importance where changes have little or no impact on key processes? If so, is there any benefit in auditing it?
- What is the driving force behind the auditing requirements? If corruption is an important issue, auditing everything fully might be preferable. Alternatively, only audit records for problem solving might be required because other records are kept in areas where limited auditing is sufficient.

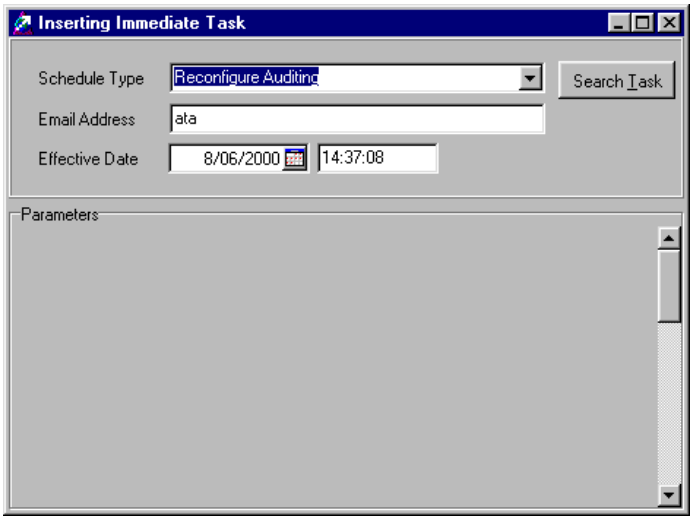


As a guide for which tables to audit and whether they should be fully or only partially audited, refer to *Recommendations* later in this chapter.

Running the audit schedule

To run an immediate task to establish the database triggers required for the auditing, follow these steps:

1. Select the **Server Tasks ⇨ Immediate Task** option. The *Schedule Immediate Task* form is displayed.
2. Select 'Reconfigure Auditing' from the drop-down list in the *Schedule Type* field.
3. Click the **Save** button on the toolbar.



This schedule drops all the existing audit triggers and creates them all again based on the parameters in the latest configuration items.

Changing audit configuration

To change the type of audit from full to partial or from partial to full:

1. Click the **Update** button on the toolbar to display the *Configuration Item Definition* form for the relevant table in Update mode.
2. Change the attribute `Include New Data in Audit` from TRUE to FALSE for partial auditing, or from FALSE to TRUE for full auditing.
3. Submit an immediate task of schedule type 'Reconfigure Auditing'.

Stopping auditing on a table

To stop auditing on a table:

4. Click the **Update** button on the toolbar to display the *Configuration Item Definition* form for the relevant table in Update mode.
1. Change the attribute `Enabled` from TRUE to FALSE to stop auditing.
2. Submit an immediate task of schedule type 'Reconfigure Auditing'.



Chose a convenient time to run the 'Reconfigure Auditing' schedule so that will not impact other database activities. If it is run during peak user times, there may be a chance that key audit information will not be captured while the database triggers are being dropped and re-created.

Accessing Audit Data

There are two different ways of accessing the audit data:

- By running an audit report.
- By using the *Audit Search* form.

Audit search form

To use the *Audit Search* form:

1. Select **Maintenance ⇨ Security ⇨ View Audit Records** option. The *Audit Search* form is displayed.

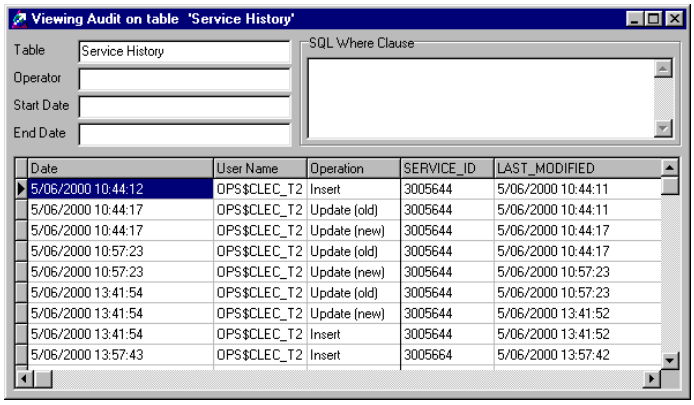


The *Audit Search* form fields are described in the following table.

Field Name	Description
Table	Shows the required table, selected from the drop down list. If this field is left blank, all tables will be searched.
Operator	To search for the activities of a specific operator, select the operator name from the drop down list. If this field is left blank, the search will cover all operators.
Start Date/ End Date	Specify the period of the search. Enter a date for the start of the period and a date for the end of the period. Only records within this period will be retrieved. If left blank, then all records will be returned.
SQL Where Clause	Use SQL Where Clause syntax to specify the exact condition to search on; for example, all adjustments made by a specific operator over 100 dollars. The Where clause would be… AMOUNT > 100
Reuse Last Audit Form	If this box is checked, the results screen will be refreshed for each search. If it is not checked, each search will generate a results window. This is useful for performing a number of different searches, and to compare the results from each search on the screen.

2. After completing the search parameters, click on the **Audit…** button in the bottom right hand corner of the form.

   The result form lists all records that meet the search criteria in chronological order.



3. Scroll across the window to see the full details of the audit records.

Audit report

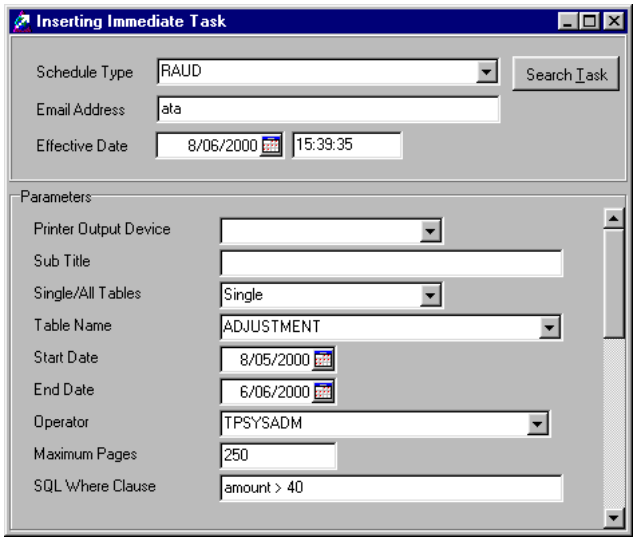
The audit report allows audit information on all tables, a specific table, a specific operator, a specific period of time, or any combination of all of these to be viewed.

It can be run by the scheduler on a regular basis or when required by submitting an immediate task.

To submit an immediate task and generate an audit report:

1. Select **Server Tasks ⇨ Immediate Task** option. The *Schedule Immediate Task* form is displayed.

2. Select 'RAUD' from the drop-down list in the *Schedule Type* field.



The parameters are described in the table below.

Parameter	Description	Reqd?
Printer Output Device	Select a device to print the report. If no value is entered, the report will not be printed. **Note:** Although the report is not printed, it will still be generated and can be viewed on line.	No
Sub Title	Enter a title relevant to the type of search being performed.	No
Single/All Tables	If *Single* is chosen, then a *Table Name* must be inserted in the next field. If *All Tables* is chosen, the report will include audit records that match all the other criteria for every single table that has auditing turned on.	Yes
Table Name	Mandatory for a Single Table report. Select the required table from the drop-down list.	Yes
Start Date End Date	The period the report covers must be specified. Enter a date for the start and end of the period. Only records within this period will be included in the report.	Yes
Operator	If activities of a specific operator are being reported, select the operator name from the drop-down list. The report will only include actions performed by this operator.	No
Maximum Pages	Some audit reports can be massive if the report parameters are not specific enough. To prevent large reports from being generated and from tying up the printer and other resources, the size of the report can be limited to a set number of pages. The report will only include data up to the page limit and then it will truncate. The default value is 250.	No

Parameter	Description	Reqd?
SQL Where Clause	Use SQL Where Clause syntax to specify the exact condition to search on; for example, all adjustments made by a specific operator over 100 dollars. The Where Clause would be… AMOUNT > 100.	No

3. Enter the required parameters.

4. Click the **Save** button on the toolbar to submit the report.

When the task is complete, the report can be viewed from within the task output by clicking on the **Report…** button in the top right hand corner of the *Task Details* form.

The report cam be printed with the report viewing tool.



To ensure that the report will include the anticipated data, query the relevant audit records online via the *Audit Search* form within. If audit records are queried online, the parameters can be experimented with until they are correct for the type of data required.

Interpreting Audit Data

Each audit record has four main sections:

- Date
- Operator
- Operation
- Record Details.

Date

The date/time the record was altered.

Operator

If the action is performed by an operator, the operator's name is recorded in this field.

If the action was performed by a server process (eg. rating or billing), then the table owner of the data is recorded as the operator. Table owners are prefixed with `OPS$`.

Operation

The type of action performed on the table. This is either 'Insert', 'Delete', 'Update(old)', or 'Update(new)'.

If a record is deleted from a table, then a copy of the record is written to the audit table with an operation of 'Delete'.

If a record is updated, two (2) records are written to the audit table. One is a snapshot of the record before the update was made. This record is labelled 'Update(old)', and a second record is written to the audit table after the update action with the new details. This record is labelled 'Update(new)'.

Record details

The full details of the record being inserted, deleted, or updated are listed in the audit record.



To see what was changed in a record update, scroll across the columns in the audit record on the *Audit Results* form and compare the values of each column between the 'Update(old)' entry and the 'Update(new)' entry.

Maintaining Audit Tables

Audit tables grow faster than normal tables because some audit actions result in two (2) records being created for each action. The table sizes need to be monitored and old data archived off at appropriate times.

Archiving audit data

There are four archive types within Singl.eView Convergent Billing for audit records. The audit tables are split into four groups alphabetically. Each group has an archive type to archive data from all tables within that group.

The four audit archive types are described in the following table.

Archive Type	Description
Audit1	Archive audit tables from `ACCOUNT_A` to `CUSTOMER_NODE_TYPE_CC_A`.
Audit2	Archive audit tables from `CUSTOMER_QUERY_A` to `NORMALISED_EVENT_FT_A`.
Audit3	Archive audit tables from `NORMALISED_EVENT_FT_HIST_A` to `SUBTOTAL_HISTORY_A`.
Audit4	Archive audit tables from `SUBTOTAL_TERM_A` to `WORK_FORCE_A`.

Archive schedules

Generally, schedules are created to run regularly to archive off old audit data. Alternatively, an immediate task can be run.

The effective date of the schedule is used to specify which audit records are archived. Records with an audit entry date prior to the schedule's effective date will be archived.

Reasons for keeping audit records and available table space will dictate the length of time audit records are kept in the database. It is recommended that audit data be kept for at least three months. Many customers archive data once or twice a year.



For more archiving information, refer to *Chapter 12, Table Partitioning*.

Recommendations

The table below lists all of the Singl.eView Convergent Billing tables that can be audited.

- A 'Yes' in the Audit column means that auditing that table is recommended.
- It is not recommended that some tables be audited. For example, auditing should not be turned on for the Normalized Event table because the table grows very large and the additional processing required for the auditing would have a negative impact on the performance of rating. Also, some tables have little or no operator control, and there would be no benefit gained from auditing them.
- A 'Yes' in the Include New Data column indicates that any rows added to the table along with any rows that have changed will be included in the audit. A 'No' in the Include New Data column indicates that only rows that have changed will be included in the audit, and not rows that have been added.

Table	Audit	Include New Data
ACCOUNT	Yes	Yes
ACCOUNT_HISTORY	Yes	Yes
ACCOUNT_TYPE	-	-
ADDRESS	-	-
ADDRESS_FORMAT	-	-
ADDRESS_HISTORY	Yes	No
ADJUSTMENT	Yes	Yes
ADJUSTMENT_BATCH	Yes	Yes
ADJUSTMENT_INVOICE	Yes	Yes
ADJUSTMENT_TYPE	-	-
ARCHIVE_RESULT	-	-
ARCHIVE_TYPE	-	-
ARCHIVE_TYPE_TABLE_LIST	-	-
ATAI_ERROR_NAME	-	-
ATLANTA_FORM	-	-

Table	Audit	Include New Data
ATLANTA_FORM_FIELD	-	-
ATLANTA_GROUP	Yes	No
ATLANTA_HELP_ENTRY	-	-
ATLANTA_OPERATOR	Yes	No
ATLANTA_PROFILE	Yes	No
ATLANTA_PROFILE_GROUP_MAP	Yes	No
ATLANTA_PROFILE_ROLE_MAP	Yes	No
ATLANTA_PROFILE_WF_MAP	Yes	No
ATLANTA_ROLE	Yes	No
ATLANTA_TABLE_PARTITION	-	-
ATTRIBUTE_TYPE	-	-
CHARGE	-	-
CHARGE_CATEGORY	-	-
CHARGE_OUTPUT_DEFN	-	-
CHARGE_OUTPUT_DEFN_HISTORY	-	-
CONFIGURATION_ATTRIBUTE	Yes	No
CONFIGURATION_ATTR_TYPE	-	-
CONFIGURATION_ITEM	-	-
CONFIGURATION_ITEM_TYPE	-	-
CONTRACT	-	-
CONTRACT_HISTORY	-	-
CONTRACT_TYPE	-	-
CONVERSION_RATE	Yes	Yes
CURRENCY	-	-
CURRENCY_HISTORY	Yes	No
CUSTOMER_CONTACT	Yes	No
CUSTOMER_NODE	-	-
CUSTOMER_NODE_CHARGE_CAT	-	-
CUSTOMER_NODE_CORRESPOND	Yes	Yes
CUSTOMER_NODE_HISTORY	Yes	Yes

Table	Audit	Include New Data
CUSTOMER_NODE_INV_FORMAT	Yes	No
CUSTOMER_NODE_LIST	-	-
CUSTOMER_NODE_LIST_ITEM	-	-
CUSTOMER_NODE_TYPE	Yes	No
CUSTOMER_NODE_TYPE_CC	-	-
CUSTOMER_QUERY	Yes	Yes
CUSTOMER_QUERY_PROGRESS	Yes	Yes
CUSTOMER_QUERY_TYPE	Yes	No
DELETE_CONSTRAINT	-	-
DERIVED_ATTRIBUTE	Yes	No
DERIVED_ATTRIBUTE_ARRAY	Yes	Yes
DERIVED_ATTRIBUTE_HISTORY	Yes	Yes
DERIVED_ATTRIBUTE_INDEX	Yes	Yes
DERIVED_ATTRIBUTE_RESULT	Yes	Yes
DERIVED_ATTRIBUTE_VALUE	Yes	Yes
DIL	Yes	No
DIL_HISTORY	Yes	Yes
DIL_REFERENCE	Yes	Yes
ENTITY_MAPPING	-	-
ENTITY_VALIDATION	Yes	No
ENTITY_VALIDATION_LIST	Yes	No
EQUIPMENT	Yes	Yes
EQUIPMENT_HISTORY	Yes	Yes
EQUIPMENT_TYPE	Yes	No
ERROR_GROUP	Yes	No
ERROR_MESSAGE	Yes	No
EVT_EQUIPMENT_CHANGEOVER	-	-
EXPRESSION_LIST	Yes	No
EXPRESSION_LIST_VALUE	Yes	Yes
FACILITY	Yes	No

Table	Audit	Include New Data
FACILITY_GROUP	Yes	No
FACILITY_GROUP_HISTORY	Yes	Yes
FAC_GROUP_COMPATIBILITY	Yes	Yes
FAC_GROUP_INSTANCE	Yes	No
FAC_GROUP_INSTANCE_HISTORY	Yes	No
FIN_CUST_REC_BUCKETS	-	-
FOREIGN_KEY	-	-
FUNCTION_DEFN	Yes	No
FUNCTION_DEFN_HISTORY	Yes	No
GL_CODE	Yes	No
GL_CODE_HISTORY	Yes	No
GL_UPLOAD		
ICON	Yes	No
INVOICE	-	-
INVOICE_CONTENTS	-	-
INVOICE_FORMAT	Yes	No
INVOICE_FORMAT_HISTORY	Yes	Yes
INVOICE_MESSAGE	Yes	Yes
INVOICE_MESSAGE_HISTORY	Yes	Yes
INVOICE_TYPE	Yes	No
INVOICE_TYPE_HISTORY	Yes	No
LIST_TYPE	Yes	No
LIST_TYPE_HISTORY	Yes	No
MENU_ROLE_MAP	Yes	Yes
NORMALISED_EVENT	-	-
NORMALISED_EVENT_ERROR	-	-
NORMALISED_EVENT_FILE	-	-
NORMALISED_EVENT_FT	Yes	Yes
NORMALISED_EVENT_FT_HIST	Yes	Yes
NORMALISED_EVENT_TYPE	Yes	Yes

Table	Audit	Include New Data
OUTPUT_DEVICE	Yes	Yes
OUTPUT_METHOD	Yes	Yes
OUTPUT_METHOD_TYPE	Yes	Yes
OUTPUT_SELECT	Yes	Yes
PAYMENT	Yes	Yes
PAYMENT_BATCH	Yes	Yes
PAYMENT_INVOICE	Yes	No
PAYMENT_TYPE	-	-
PERSON	Yes	No
PERSON_HISTORY	Yes	No
PERSON_TYPE	-	-
PRODUCT	Yes	No
PRODUCT_COMPATIBILITY	Yes	Yes
PRODUCT_FACILITY_GROUP	Yes	Yes
PRODUCT_GROUP	Yes	Yes
PRODUCT_GROUP_HISTORY	Yes	Yes
PRODUCT_HISTORY	Yes	Yes
PRODUCT_INSTANCE	Yes	Yes
PRODUCT_INSTANCE_EQUIP	Yes	No
PRODUCT_INSTANCE_HISTORY	Yes	No
PRODUCT_INSTANCE_SERVICE	Yes	No
PRODUCT_IN_PRODUCT_GROUP	-	-
PRODUCT_LIST	-	-
PRODUCT_NODE_TYPE	Yes	No
PRODUCT_SERVICE_TYPE	-	-
PRODUCT_SERVICE_TYPE_EQUIP	-	-
PRODUCT_SUBTOTAL	-	-
PRODUCT_TARIFF	-	-
REFERENCE_CODE	-	-
REFERENCE_TYPE	-	-

Table	Audit	Include New Data
RELEASE_INFO		
REPORT_REFERENCE_TYPE	-	-
REPORT_REFERENCE_TYPE_DEFS	-	-
REPROCESSED_FILE	Yes	No
RGP_FILE	-	-
RGP_NORMALISED_EVENT	-	-
SCHEDULE	-	-
SCHEDULE_TASK_TYPE	-	-
SERVICE	Yes	No
SERVICE_CHARGE_CATEGORY	-	-
SERVICE_HISTORY	Yes	No
SERVICE_LIST	-	-
SERVICE_LIST_ITEM	-	-
SERVICE_TYPE	-	-
SERVICE_TYPE_CC	-	-
SERVICE_TYPE_EQUIP_TYPE	-	-
SUBTOTAL	-	-
SUBTOTAL_HISTORY	-	-
SUBTOTAL_TERM	-	-
SUBTOTAL_VALUE	-	-
TABLE_IMPORT_ORDER	-	-
TARIFF	-	-
TARIFF_CHARGE	-	-
TARIFF_CHARGE_DEFN	-	-
TARIFF_CONDITION_DEFN	-	-
TARIFF_HISTORY	-	-
TARIFF_RECURRING	-	-
TASK_QUEUE	-	-
TASK_QUEUE_RESULT	-	-
TEMPLATE	Yes	No

Table	Audit	Include New Data
TEMPLATE_HISTORY	Yes	No
TEMPLATE_REFERENCE	-	-
VARIABLE_DEFN	-	-
VARIABLE_DEFN_DEPENDENCY	-	-
VARIABLE_DEFN_DIRECT	-	-
VARIABLE_DEFN_HISTORY	Yes	No
VERTEX_UPDATE	-	-
WORK_FORCE	-	-
XPSI_ADDR_VAL_COMMTAX21	-	-
XPSI_ADDR_VAL_MSAG	-	-
XPSI_ADDR_VAL_ZIP	-	-
XPSI_NE_BATCH	-	-
XPSI_TAX_ERROR	-	-
XPSI_UPDATE_CAPTURE	-	-

Chapter 10
Database Archiving

This chapter describes the operational aspects of database archiving specific to the Singl.eView ICP product. It also provides an overview of the database table naming conventions used by Singl.eView Convergent Billing.

For detailed information on the Singl.eView Convergent Billing archiver, refer to the *Systems Administration Guide for Singl.eView Convergent Billing.*

Database archiving involves the purging, saving and restoring of database entities. This is different from database backup and recovery, which involves the backup and recovery of the physical database tables. Archiving is concerned with the business objects (entities) provided by Singl.eView Convergent Billing and how they can be purged, saved and restored.

Table Naming Conventions

Singl.eView Convergent Billing entities represent business objects that are stored in a relational database. Each type of entity is typically implemented via multiple database tables. Having a basic understanding of the table model naming conventions used by Singl.eView Convergent Billing helps clarify the purpose of the table model and why a single type of entity needs to exist across many physical database tables.

A single entity normally contains the following tables (more complicated entities can be made up of more tables):

- `<Entity_Type>`, which contains a single row for each entity of this type. Only information about the entity that does not change with time is stored in this table.
- `<Entity_Type>_HISTORY`, which may contain multiple rows for each entity of this type. Each row captures the information relevant for the entity for a particular period of time.

  Note: Some entities are not date ranged; this means a history of the changes to the entity over time is not stored.

- <Entity_Type>_A, which is the audit table for the <Entity_Type> table. It contains information that can be used to audit the operations that have been performed on entities of this type.
- <Entity_Type>_HISTORY_A, which the audit table for the <Entity_Type>_HISTORY table. It contains information that can be used to audit the operations performed on entities of this type.

For each of the above tables, there is also an archive table that is used by the Singl.eView Convergent Billing archiver for importing and exporting entities. The name of the archive table is formed by appending _R to the above table names. For example, Singl.eView Convergent Billing function entities are implemented by the following tables:

- FUNCTION_DEFN
- FUNCTION_DEFN_HISTORY
- FUNCTION_DEFN_Af
- FUNCTION_DEFN_HISTORY_A
- FUNCTION_DEFN_R
- FUNCTION_DEFN_HISTORY_R
- FUNCTION_DEFN_A_R
- FUNCTION_DEFN_HISTORY_A_R

The Archiver

The Singl.eView Convergent Billing archiver can be used to export entities to a file and to import entities (saved to a file) into the database. The archiver performs both of these operations using the archive (_R) tables (for more information, refer to the *System Administration Guide for Singl.eView Convergent BIlling*.

When entities are exported to a file, they can be deleted from the database. This action is known as purging entities from the database. In fact, the archiver is typically used on a periodic basis to purge certain entities from the database when they are no longer needed. For example, the archiver can be used to purge customer entities that have been cancelled for at least ninety (90) days.

Exporting

Exporting entities occurs in two phases:

1. The entities specified by the archive command are copied from the relevant tables to the archive tables.
2. The archiver exports all of the data in the archive tables to a file and removes all of the data from the archive tables.

Command line options can be passed to the archiver so that it performs either one or both of these phases. For example, phase 1 could be executed a number of times to build up the archive tables with the data from multiple archive commands. A single archive command can then be issued to export all of the data in the archive tables to a single file.

Other commonly used command line options allow for the following when exporting entities:

- Purge (delete) the entities from the database. The default is to just copy the data; the entities still exist in the database after they are exported to file.

  Note: Care must be taken with the Purge option because if it is selected, the entities will be removed from the database.

- Export the data in an ASCII format. The default is to export the data in a binary format.
- Start with empty archive tables. Remove any existing data in the archive tables before performing phase 1.

- Report missing data. This checks that the data being exported does not rely on or reference other data in the database that is not being exported. For example, if an export is performed and it does not report any missing data, the exported file can be imported into another database and the import operation will not report any missing data. The export file is 'self-contained' and does not rely on any other data being present.
- Report on the data that currently exists in the archive tables.

Importing

The importing of entities occurs in two phases:

1. The archiver imports all the data from the file into the archive tables.
2. The archiver uses the information in the archive tables to populate the real database tables.

Command line options can be passed to the archiver so that it performs either one or both of these phases. Other commonly used command line options allow for the following when importing entities:

- Import the data in an ASCII format

  The default is to import the data in a binary format.
- Start with empty archive tables

  Remove any existing data in the archive tables.
- Report missing data

  This checks that the data being imported does not rely on or reference other data that is not currently in the database.
- Report on the data that currently exists in the archive tables.

An important aspect of importing is how the archiver handles populating the real database tables with the data from the archive tables. The archiver has to be able to answer the question 'If the entity already exists in the real table, how do I populate the real tables with the entity information from the archive tables?'

The archiver solves this problem by using a replacement strategy. A number of different replacement strategies exist and can be specified via a command line option.

What makes an entity unique?

To understand replacement strategies, it is important to understand how entities are uniquely identified. Every entity in Singl.eView Convergent Billing is uniquely identified by an internal identification number, or ID. Singl.eView Convergent Billing uses these IDs internally, but a Singl.eView Convergent Billing Client user does not need to know about them.

For example, a CSR may view a customer with the name of 'Harry Smith'. This customer might have an internal customer ID of 1245, but the CSR does not need to know this (and is never shown it). The name 'Harry Smith' does not uniquely identify the customer to the system; the customer ID of 1245 does. There may be another customer in the system with a name of 'Harry Smith,' but he would have a different customer ID. Also, a CSR could change the name from 'Harry Smith' to 'Harold Smith', but it still represents the same customer, identified by ID 1245.

The ID of an entity is its primary identifier, or its primary key. However, some entities also have other attributes, which must be unique. These additional attributes are known as unique keys. For example, for operator entities, the login name is unique because every operator must use a different name to login to the system. This is an example of a unique key. All operator entities also have an ID which is the primary key of the entity.

Entity replacement strategies

When entities are imported from a file, they are first imported into the archive tables. How does the system know whether an entity in the archive tables represents the same entity as one that is in the real database tables? The answer is that it uses the primary key (ID) of the entity. Unique keys (for those entities that have them) can also be used to detect whether two entities represent the same entity.

To illustrate how the different entity replacement strategies work, we will consider the example of an operator entity with the following details:

- Operator ID: 1245 (primary key)
- Operator Login Name: HAS (unique key)
- Operator Full Name: Harry Smith.

Note: The replacement strategies work in the same way for other entities. Operators are just used here as an example.

Replacing entities

Assume that the archive command was used to archive out the operator entity for Harry Smith to the file `hsmith.sql`. The Singl.eView Convergent Billing client was then used to update the operator full name from Harry Smith to Harold Andrew Smith, and the following archive command was run to import the operator entity from `hsmith.sql`.

```
archive -d -x -r 2 -f hsmith.sql
```

The replacement strategy of 2 indicates that if the archiver finds an entity from the file has the same primary key (ID) as an entity in the database, the database entity is said to already exist, and will be replaced by the version in the file.

In this example, the archiver detects that the operator entity already exists in the database, because they have the same primary key (an operator ID of 1245). After performing the above command, the operator entity for Harry Smith will be replaced with the version that was in the archive file `hsmith.sql`. As a result, the operator full name is changed from Harold Andrew Smith back to Harry Smith.

Remapping entities

Harry Smith exists as an operator in database A, but does not exist as an operator in database B. An operator exists in database B with the following details:

- Operator ID: 1245 (primary key)
- Operator Login Name: TPJ (unique key)
- Operator Full Name: Terry Jones.

How do we export the operator entity for Harry Smith out of database A and import it into database B, so that Harry Smith and Terry Jones are now operators in database B?

The problem with using a replacement strategy of 2 when importing Harry Smith into database B is that the system would detect Terry Jones and Harry Smith as the same operator entity because they have the same ID. A replacement strategy of 2 would cause the entity for Harry Smith to replace the Entity for Terry Jones, so Harry Smith is now an operator in database B, but Terry Jones is not.

A replacement strategy of 5 tells the archiver to remap the ID of any entity being imported when the primary key (ID) of the entity being imported already exists in the real entity tables. In this example, the following command is used:

```
archive -d -x -r 5 -f hsmith.sql
```

Instead of replacing the existing operator entity of Terry Jones, the archiver detects that the Harry Smith entity being imported has the same ID as the Terry Jones entity, and adds the Harry Smith entity to database B, but with a different ID (ie. the ID for the imported entity is remapped to a new value). For example, after the above command is run, the Harry Smith entity will exist in database B with the following details:

- Operator ID: 1387 (primary key)
- Operator Login Name: HAS (unique key)
- Operator Full Name: Harry Smith.

Merging entities

Harry Smith exists as an operator in database A, and also in database B, but with the following details:

- Operator ID: 1289 (primary key)
- Operator Login Name: HAS (unique key)
- Operator Full Name: Harold Andrew Smith.

This could happen if the Singl.eView Convergent Billing client was used to create the operator Harry Smith in both databases A and B. The person creating the operator has entered a different operator full name, and the system automatically allocates the operator ID based on a sequence number. The sequence number keeps growing larger as more operators are created.

Assume the `hsmith.sql` archive file has been exported from database A and is being imported into database B.

In this example, a replacement strategy of 2 will not work, as the primary keys (IDs) are different. In fact, the Harry Smith entity being imported (ID=1245) would be created as a different operator entity as the existing Harry Smith entity (ID=1289). However, a unique key violation error would occur, as two operator entities now exist with the same operator login name of HAS (which is disallowed because the operator login name is a unique key).

A replacement strategy of 6 tells the archiver to merge entities when an entity being imported has the same unique key as an entity that already exists in the real entity tables. For example, the following command is used:

```
archive -d -x -r 6 -f hsmith.sql
```

With this replacement strategy, the archiver recognises that the Harry Smith entities are the same because they have the same unique key (even though they have a different primary key). The archiver replaces the entity in the database with the entity data imported from the file, but keeps the original primary key (ID) used by the entity in the database.

After issuing the above command, the details for the Harry Smith operator entity would be:

- Operator ID: 1289 (original primary key of entity in database B; the imported entity had an ID of 1245, but this is discarded).
- Operator Login Name: HAS (unique key; same for both existing entity and the imported entity).
- Operator Full Name: Harry Smith (the data from the imported entity replaces the data in the existing database entity).

Other strategies

The replacement strategies described above are the ones most commonly used. The other replacement strategies available are shown below:

Strategy (-r flag)	Description
1	Replace duplicate rows only.
3	Truncate the real entity tables before the unarchive. **Note:** Be careful with this option because the data in the database table will be replaced by new data.
4	Truncate the real entity tables before the unarchive and do not re-enable constraints. **Note:** Be careful with this option because the data in the database table will be replaced by new data.
7	This strategy combines the remapping and merging replacement strategies as follows: • If the primary key of the imported entity is the same as an existing entity, it is remapped. • If the imported entity has a unique key that is the same as an entity already in the real entity tables, then it is merged with this entity. The data from the imported entity replaces the data in the existing entity.

Examples

Exporting

Example 1

The following command exports all of the operator entities to a binary file called `operator.sql`. The entities still exist in the database; they are not purged. Any data currently existing in the archive tables is deleted before copying the operator entities. The command is:

```
archive -x -t Operators -f operator.sql
```

To export the operator entities to an ASCII file, the command would be changed to:

```
archive -o -x -t Operators -f operator.sql
```

To purge the entities from the database, the command would be changed to:

```
archive -p -x -t Operators -f operator.sql
```

To restrict the operator entities archived to those whose login name begins with 'P', the command would be changed to:

```
archive -x -t Operators -f operator.sql -w
'operator_login_name like 'P%''
```

To copy the operator entities to the archive tables (phase 1) but not export the archive tables to file (phase 2), the following command would be used. Also, this command adds the operator entities to any existing data in the archive tables (the archive tables are not cleared). The command is:

```
archive -s -t Operators
```

Note: The `-s` option should always be used when archiving additional data to the archive tables. This would be an archive where the archive tables are not first cleared. The `-s` option tells the archiver not to copy the entity to the archive tables if it is already there. Without this option, an entity that is already in the archive tables will be copied again, resulting in two instances of the entity existing in the archive tables.

To export the existing data in the archive tables to file (phase 2 only), the following command would be used:

```
archive -f operator.sql
```

Example 2

To export and purge from the database all the product instances that have been cancelled for more than ninety (90) days, the following command would be used:

```
archive -x -p -t ProductInstancesFutureCan90 -f
cancelled.sql
```

Example 3

To export (in ASCII format) all entities that have security group ID of 1045, or are not associated with a security group, the following command is used. This command also checks if the resulting export contains any missing data:

```
archive -x -o -m -t GroupsAll -f gr1045.sql -g
'atlanta_group_id = 1045 or atlanta_group_id is
null'
```

Importing

Example 1

The following command imports all of the entities in the binary file `data.sql`. The archive tables are cleared before the data is imported from the file. Any entities that are in the file but already exist in the database will be replaced. The file version of the entity will replace the database version of the entity.

```
archive -d -x -r 2 -f data.sql
```

To import the entities from an ASCII file, the command would be changed to:

```
archive -o -d -x -r 2 -f data.sql
```

To import the entities into the archive tables (phase 1), but not update the real entities using the information in the archive tables (phase 2), the following command is used. This command also adds the entities to any existing data in the archive tables (ie. the archive tables are not cleared).

```
archive -d -e -f data.sql
```

To update the real entities using the information in the archive tables (phase 2 only), the following command is used. Existing database entities will be replaced. The command is:

```
archive -d -r 2
```

Example 2

To import entities from a file and remap them to new IDs when the ID already exists in the database, the following command is used:

```
archive -d -x -r 5 -f data.sql
```

Example 3

To import (in ASCII format) all entities in `data.sql` and check that the resulting database (after the import is performed) does not contain any missing data, the following command is used. Entities that already exist in the database are replaced by the entities imported from the file. The command is:

```
archive -o -m -d -x -r 2 -f data.sql
```

Chapter 11
Database Validation

This chapter explains the Singl.eView Convergent Billing database validation processes. It describes how to run these processes and how to identify and correct errors found by the processes. Also included are recommendations for how often and when, the validation processes should be run.

Overview

Singl.eView Convergent Billing comes with a database validation tool called the DVP (Data Validation Process). This tool identifies where the referential integrity of the database has been violated. Violations are reported as either warnings or errors that need to be corrected before they impact on other Singl.eView Convergent Billing processes.

Performance

Because of the size of some databases and the number of foreign key relationships that exist between each of the entities, it is not feasible to check all integrity rules at the point of data entry. These additional checks would add considerable time to the saving of each record to the database if they were made at commit time. For these performance reasons, the foreign key relationships are checked by the DVP at a time that is convenient for the installation.

Referential integrity

The DVP ensures data integrity across the foreign key relationships within the database. Many relationships exist between records and many operators and processes can be working with the records at different times. The DVP is run to ensure that all database activity has not affected the required relationships.

Pro-active management

The DVP plays an important part in pro-active error management. The earlier errors are detected, the less of an impact they will have on other Singl.eView Convergent Billing processes.

DVP Checks for Singl.eView IP/Data

Singl.eView IP/Data uses an additional DVP to check for data integrity in the speed value master reference types and speed value reference type instances. For more information on these reference types, refer to Chapter 22, Speed Value Management. When the DVP – IP Template schedule is run, the following checks are performed:

1. The master speed reference types are checked to ensure that the format of all code labels is valid.
2. A comparison is made of the master speed reference types to the reference type instances to ensure that no speed value records exist in the reference type instances that are not included in the applicable master reference type.

Customised DVP

Each installation of Singl.eView Convergent Billing may include an additional customised validation process that checks that the local configuration's data integrity rules have not been violated. These customised checks are run from the **`atai_dvp`** process.

Configuration

The DVP can be run:

- Directly from the command line on the server.
- From within the client via an immediate task.
- As a regular schedule.

If the DVP process is run from the command line of the server, no configuration is required.

If the DVP process is run regularly by the scheduler, the only configuration required is the setting up of the schedule. Additional schedules can be set up to run the Singl.eView IP/Data DVP or a customised DVP. If required, a schedule type can be configured to run the three DVPs together.

DVP schedule configuration

To configure DVP schedules:

1. Select the **Server Tasks ⇨ Schedules** option. The *Schedule Search* form is displayed.
2. Click on the **New…** button. The *Schedule Definition* form is displayed in Insert mode.
3. Create a new schedule of type 'Full Database Validation'.
4. Set the repeat units to the required frequency for this process. For a daily process, *Repeat Units* would be 1 and *Repeat Type* would be *Day*.
5. Set the *First Date* and time of the schedule for a time that the DVP is to be run every time the task is repeated.
6. Click the **Save** button on the toolbar.

There are no additional parameters to be supplied for the DVP.



Chose a run time that will not clash with other server processes or backup procedures. For more information, refer to *When to Run the DVP* later in this chapter.

Local DVP schedule configuration

If there is also a customised DVP process, a schedule type will have to be created for this process first. Once the schedule type is created, a schedule of that schedule type should be created to regularly run the customised DVP.

To configure a local DVP schedule:

1. Select the **Maintenance ⇨ Server ⇨ Schedule Types** option. The *Schedule Type Search* form is displayed.
2. Click the **New…** button. The *Schedule Type Definition* form is opened in Insert mode.
3. Create a new schedule type using the details in the following table.

Field Name	Details
Schedule Type Name	Local Database Validation
Description	Additional database validation checks for the customised implementation
Task Type	General Server Task
Maximum Duration	7200 seconds
Maximum Tasks	1
Retain Tasks for	30 days
Task Complete Message	<M02004>
Task Validation	tqFull Database Validation
Server Command	**`atai_dvp`**

4. Create a new schedule to run the customised DVP as per the standard DVP schedule created previously, but use a schedule type of 'Local Database Validation' and a compatible *First Date* and time.

Schedule type to run all DVPs

If one schedule that performs the full, Singl.eView IP/Data, and customised database validations is preferred, a schedule type with an appropriate server command must be created. To do this:

1. Either specify the following command string in the server command

   ```
   sh -c 'dvp; atai_dvp'; atai_dvp
   $ATAI_REL_SERVER_BIN/ip_dvp.txt
   ```

   or

   Create a script that runs all required DVP processes and call this script in the server command of the schedule type.

2. Create a schedule of this new schedule type to run when required.

Execution

The DVP can be run over the entire database (full DVP), or just selected tables (limited DVP).

Full DVP execution

- To do a full DVP, the schedule must be submitted.
- To run the DVP from the server command line, run the command

  `dvp`
- To run the DVP and redirect the output to a file, run

  `dvp > dvp.out`

  where `dvp.out` can be any file name.
- To run the customised DVP, run the command:

  `atai_dvp`.

Limited DVP execution

For large stable databases, a full DVP every time may not be feasible. For performance reasons, only key areas of the database need be checked, instead of every table, only errors and not warnings reported, and only the existence of a specific error condition noted.

To run a limited DVP, any combination of the following options can be passed to the DVP as parameters.

Option	Description
`-w`	Report on warnings only.
`-e`	Report on errors only.
`-s <error code>`	Suppress specified warning/error.
`-r <error code>`	Only include specified warning/error.
`-t <table name>`	Only report on warnings/errors for specified table.
`-d <table name>`	Do not report on warnings/errors for specified table.
`-f <table name>`	Start reporting from this table.
`-u <user/pass>`	Log on as specified user.
`-m <rows>`	Only display the first `<rows>` rows in error (default 50).
`-trace <debug level>`	Debug level. Currently only `<debug level = 1>` is supported which enables ORACLE tracing.
`-o<output file>`	Record the times in the specified output file.
`-v`	Verbose mode (usually for testing).

Note: Options `-t`, `-d`, `-s` and `-r` can be repeated as many times as required.

Multiple DVP execution

The DVP queries on the larger tables in the database can take a significant amount of time to run (tens of minutes). If Singl.eView Convergent Billing is running in a multi CPU environment, the DVP can be configured to spawn child processes, so that the queries can be run in parallel, thus reducing the total execution time of the DVP.

The parent DVP will attempt to run the largest queries first, thereby minimising the total execution time. Suppose the DVP consisted of four checks (in reality it is many more) and that three of those checks took 1 minute to execute, and the fourth took 3 minutes. If the environment consisted of two CPUs, the DVP could be configured to spawn two child processes. The first child process would start the largest query first, and complete it in 3 minutes. While that is running, the second child process would perform the three 1 minute checks, using the second CPU. Hence, the total execution time would be 3 minutes.

Without specifying any child processes, the queries would have been run sequentially and the total execution time would have been six (6) minutes.

Note: The number of child processes configured should never exceed the number of available CPUs. To do this would mean that two or more of the child processes would be competing against each other for CPU time, and the queries would not be running in parallel.

The following three command line options relate to multiple DVP execution:

Option	Description
`-wait<time>`	`Wait <time>` seconds before killing a child process and going on to process the next query. The default is to have no limit set, meaning that each query will run until it has finished. **Note:** This is only valid if the '`-c`' option has been used to spawn child processes.
`-sleep_max <time>`	The parent DVP will sleep a maximum of `<time>` seconds before attempting to process the next query. The default value is 5 seconds. **Note:** This is only valid if the '`-c`' option has been used to spawn child processes.
`-sleep_min <time>`	The parent DVP will sleep a minimum of `<time>` seconds before attempting to process the next query. The default value is 1 second. **Note:** This is only valid if the '`-c`' option has been used to spawn child processes.
`-c <max children>`	Allows the DVP to spawn up to `<max children>` child processes to perform the queries in parallel. **Note:** `<max children>`should not be greater than the number of available CPUs.

Sample limited DVP usage

Usage: DVP `[options] [file]`

Where `[options]` is any of the options in the table above, and `[file]` is the file with all the validation checks to be performed. The default file `dvp.sql` is always used. If only the existence of a specific customised error message needed to be checked, the `atai_dvp.sql` file would be used instead.

Example 1

If a large number of equipment items have been bulk loaded into the database, the equipment tables can be checked for validation errors. The following command is used:

```
dvp -t equipment -t equipment_history
```

Example 2

Old person records archived out some time ago may have been reloaded into the database. To check that this has not introduced any duplicate person details, use the following command:

```
dvp -t person_history -r E50807
```

Output Results

The output from each of the DVPs consists of a listing of each table that has been checked. Any validation errors found in a table are listed after the table name in the output.

If the DVP is run from the command line, by default the output is written to the screen. If the output is redirected to a file, only the validation errors are written to the file. The listing of each table being checked is still sent to the screen.

If the DVP is run from a schedule of an immediate task, the output is captured in the output text of the task.

The DVP output has three main parts:

1. The listing of each table being checked.
2. The listing of error and warning messages after each table.
3. A summary at the end of the validation queries that took the longest amount of time.

Sample output

An extract from the DVP output is shown below. It is a portion of the full output.

```
Checking table CURRENCY...
Checking table CURRENCY_HISTORY...
Checking table CUSTOMER_CONTACT...
Checking table CUSTOMER_NODE...
Checking table CUSTOMER_NODE_CHARGE_CAT...
Checking table CUSTOMER_NODE_HISTORY...
Checking table CUSTOMER_NODE_INV_FORMAT...
Checking table CUSTOMER_NODE_LIST...
Checking table CUSTOMER_NODE_LIST_ITEM...
Checking table CUSTOMER_NODE_TYPE...
Checking table CUSTOMER_QUERY_TYPE...
Checking table DERIVED_ATTRIBUTE...
Checking table DERIVED_ATTRIBUTE_ARRAY...
Checking table DERIVED_ATTRIBUTE_HISTORY...
Checking table DERIVED_ATTRIBUTE_INDEX...
Checking table DERIVED_ATTRIBUTE_RESULT...
Checking table DERIVED_ATTRIBUTE_VALUE...
Checking table DIL...
Checking table DIL_HISTORY...
Checking table DIL_REFERENCE...
Checking table ENTITY_VALIDATION_LIST...
Checking table EQUIPMENT...
Checking table EQUIPMENT_HISTORY...
Checking table EQUIPMENT_TYPE...
Checking table ERROR_GROUP...
Checking table EXPRESSION_LIST...

<E50460> The expression list in the EXPRESSION_LIST table with
EXPRESSION_LIST_ID 80393 is not referenced by any rows in the
EXPRESSION_LIST_VALUE table.

<E50461> The row in the EXPRESSION_LIST table with
EXPRESSION_LIST_ID 80393 is not referenced by any other
entities.

Checking table EXPRESSION_LIST_VALUE...
Checking table FACILITY...
Checking table FACILITY_GROUP...
Checking table FACILITY_GROUP_HISTORY...
Checking table FAC_GROUP_COMPATIBILITY...

Disconnecting from 'ATP_TST'

Top 10 slowest queries are:

4 E50446
4 E50447
4 E50448
4 E50450
4 E50451
1 E51587
1 E50020
1 E50063
1 E50102
1 W50279

Done.
```

Identifying errors

Use the following tips to identify errors:

- Validation errors are listed after the name of the table in which they were identified.
- The error messages are prefixed with an error/warning code, such as `<E50460>`.
- Warnings are prefixed with an 'W'. Error messages are prefixed with an 'E'.
- The error/warning code can be queried within the client to see a full explanation of the error and recommended corrective actions to follow.
- To query error codes online, go to **Maintenance ⇨ Data Definition ⇨ Error Messages**. Change the *Search Field* to 'Error Message Id' and enter the digits of the error code only in the *Search Criteria*.

Correcting errors

Use the following tips to correct errors:

- The required fix may be as simple as filling in a mandatory value within a record. It may be more involved, requiring the deletion of unwanted records. Be sure to investigate the reported error.
- All data corrections should be made through the client, if possible.
- To search for the required record within the client, identify which record has the ID reported in the error message. For example, in the sample DVP error message below, determine 'which' expression list has an ID of 80393. An SQL search will have to be done within the database for this.

```
<E50460> The expression list in the EXPRESSION_LIST
  table with EXPRESSION_LIST_ID 80393 is not
  referenced by any rows in the
  EXPRESSION_LIST_VALUE table.
```

  The SQL query would look like…

```
SELECT *
    FROM EXPRESSION_LIST
   WHERE EXPRESSION_LIST_ID = 80393
```

- Once full details of the record are available, search for that record via the search forms within Singl.eView Convergent Billing, and appropriate corrective action can be performed to fix the validation error.
- Occasionally, the required corrective action must be performed within the database, because the client may not allow the required corrective action to be performed.

Correcting Errors for the Singl.eView IP/Data DVP

Errors will be returned when the code labels in the master reference types are invalid or when speed value records exist in the reference type instances that do not exist in the applicable master reference type. For information on valid code label formats and for how to update the master reference types and reference type instances, refer to *Chapter 22, Speed Value Management*.

When to Run the DVP

- The full DVP consumes significant resources and therefore should not be run while other key processes (such as rating or billing) are running.
- The larger the database, the more data there is to check and the longer the DVP will take.
- Choose a time outside of the peak operating hours of the system.

When not to run the DVP

The only time the full, customised, or Singl.eView IP/Data DVP should **not** be run is when data is being archived either into or out of the database. If the DVP is run during an archiving process, it may incorrectly report on data that is missing. It may just have been that the missing data was in the process of being archived in when the check was made.

Recommendations

- For new installations of Singl.eView Convergent Billing, the DVPs should be run daily.
- When the system has been operating smoothly without any errors, the DVPs can be run weekly (especially if performance is a key factor for that installation).
- It is recommended that the DVPs be run before a billing run so that any errors that could affect the billing results can be resolve.
- The DVPs should be run after large amounts of data are either loaded into the database or archived out of the database to ensure that the data integrity is still maintained.
- The DVPs should be run both before and after a Singl.eView ICP or Singl.eView IP/Data upgrade to ensure that the upgrade has not affected the existing data.

Chapter 12
Table Partitioning

This section covers the table partitioning functionality in Singl.eView ICP. The following information is included:

- How to create partitions
- Using schedules to rotate partitions
- Partition Maintenance.

Overview

Table partitioning is used to manage tables that tend to grow very large. This greatly reduces the time to access, archive and delete the data in these tables. The following is the list of tables that can be partitioned:

- `NORMALISED_EVENT`
- `CHARGE`.

The above tables play an important role in billing. Partitioning these tables helps improve the performance of the Billing engine.

The following are the three schedules that are specifically designed to manage partitions:

- Create Partition
- Partition Maintenance
- Rotate Partition.

Creating Partitions

Determining the number of partitions

The following requirements of Singl.eView ICP should be adopted when creating partitions for the `NORMALISED_EVENT` and `CHARGE` tables:

- Daily partition should be used for both tables.
- The number of partitions to be created is dependent on the number of days of on-line data that is required. Use the following formula to compute the number of partitions required:

  (No. of days of on-line data) + (Buffer) = No. of partitions

  where buffer = 10.

  Example

  If three months of data is to be kept online, the number of partitions required will be:

  (3 x 30 days) + 10 = **100 partitions**.

Creating partitions

The 'Create Partition' schedule type in Singl.eView ICP should be used to create the partitions (for more information, refer to *Chapter 7, Database Administration* in the *System Administration Guide for Singl.eView Convergent Billing*.

The following table lists the columns used by 'Create Partition' schedule to determine how the partitions should be split.

Table Name	Column Name
`CHARGE`	`CHARGE_START_DATE`
`NORMALISED_EVENT`	`CHARGE_START_DATE`

The following is the Singl.eView ICP specific information required when creating partitions:

- Before creating the partitions, archive off all billed data.
- The Start Date of the first partition should be smaller than the oldest `charge_start_date` of the data on the system.

- The number of partitions should be the number of partitions as computed from the formula in *Determining the number of partitions* earlier in this chapter.
- The duration of each partition should be . 1. and the units should be . Days.



If there is data on the system with a `charge_start_date` older than the value used for the Start Date in creating partitions, the data will be deleted in the process of partition creation.

Rotating Partitions

The 'Rotate Partition' schedule is used to recycle old partitions when the existing data has been billed and is ready to be archived. When 'Rotate Partition' is run, it essentially marks the selected partitions to be archived and recreated for a new range of dates. The 'Partition Maintenance' schedule is the schedule that actually performs the operation. The 'Rotate Partition' schedule should be run after a billing run and must be set up to rotate partitions that have already been billed. The following are the Singl.eView Convergent Billing requirements for the various parameters associated with the 'Rotate Partition' schedule:

- A 'Rotate Partition' schedule should be created for both the `CHARGE` and `NORMALISED_EVENT` tables that cover the same range of dates.
- Identify the date range of billed data that needs to be dropped. Based on the date range, identify the number of weeks of data to be kept on-line. In the column labelled. Drop partitions older than, enter the number of weeks/days of data that should remain online. Anything older than the number of weeks/days entered will be archived and dropped.

Example

Assume that data is to be retained online for 90 days. With 10 buffer partitions, a total of 100 one-day partitions would need to be created. One way of maintaining 90 days of online data, but no more, would be to set the 'Rotate Partition' schedule to run each day to rotate any partitions older than 90 days. If billing is done monthly, there would be no risk of archiving data that has not been billed because the oldest partition is 90 days old. Each day, any partition older than 90 days would be archived, dropped, and re-created as the newest partition.

A separate 'Rotate Partition' schedule would have to be created for the `CHARGE` and `NORMALISED_EVENT` tables with the parameters set to the following values:

- Drop partition older than = 90 days
- Duration of each partition = 1 day.

After the successful execution of the 'Rotate Partition' schedule, the 'Partition Maintenance' schedule will run to update the partitions. The oldest partitions will be archived and dropped.

Note: The 'Rotation Partition' schedule does not perform the rotation itself, rather it just marks the partitions for rotation. The 'Partition Maintenance Schedule' linked to the 'Rotation Partition' schedule actually performs the rotation.

The old partitions will be archived to the directory specified in the partition maintenance schedule that is run. The format of the archive file is:

```
[tablename]_[partition no].exp.[from date].  [to date]
```

eg.

```
CHARGE_P0001.exp.19990902000000.19990902235959
```

Partition Maintenance

The 'Partition Maintenance' schedule is the workhorse that performs the following operations:

- Create Partition
- Drop Partition
- Sort Partition
- Archive Partition.

Create and drop partition

This is normally performed using the 'Rotate Partition' schedule, which in turn calls the 'Partition Maintenance' schedule to perform the required operation.

Sort partition

Sorting partitions needs to be performed on all of the partitions that are included in a bill run before running the billing. Failure to do so significantly impacts the performance of the bill run.

For example, if billing is run once a week and all the events being billed that night are from the past seven (7) days of partitions, those partitions should be marked for sorting that night. The 'Partition Maintenance' schedule that actually performs the sorting should be run before the night's billing run.

Archiving partition

Whenever a 'Rotate Partition' schedule is run, it will first archive the old partition before recreating a new partition. The archiving of a partition could also be called separately by marking a partition to be archived and running the 'Partition Maintenance' schedule.

Chapter 13
General Housekeeping

This chapter describes the various housekeeping tasks in Singl.eView ICP that need to be performed on a regular basis. The following information is included:

- A list of log files to be switched
- A list of log files and event files to be purged.

Overview

The many processes within Singl.eView Convergent Billing tend to generate output files for logging and debugging purposes. Singl.eView Convergent Billing provides the following schedule types to maintain the growth of these files:

- 'Switch Files', which renames files matching a specified file pattern within a directory. The new filename is the old filename with the date appended.
- 'Purge Files', which deletes all the files that match a specified file pattern and that are older than a certain date, from the specified directory.

Switching files

The following file pattern ('*' denotes wildcard) from `$ATA_DATA_SERVER_LOG` directory should be switched on a regular basis:

```
*.out
*.sts
*.scm
*.trc
*.log
```

Ideally, one (or more) Switch Files schedules should be set up to run every night to rename the above mentioned files.

Purging server files

Server Log Files

Files of the following pattern ('*' denotes wild card) from the $ATA_DATA_SERVER_LOG directory should be purged on a periodic basis, such as weekly or monthly:

```
*.out.*
*.sts.*
*.scm.*
*.trc.*
*.log.*
ULOG.*
```

The frequency at which files of the above pattern are deleted varies based on the business requirements.

Event Files

The $ATA_DATA_SERVER_ARCHIVE directory is where all of the event files are placed after being processed. This directory should contain sub-directories for each ENM process configured on the system. A separate schedule needs to be set up for each of the individual sub-directories for purging files since a single 'Purge Files' schedule can only purge the files from a single directory. For example, a 'Purge Files' schedule may have to be set up for the following directories:

```
$ATA_DATA_SERVER_ARCHIVE/switch1
$ATA_DATA_SERVER_ARCHIVE/switch2
$ATA_DATA_SERVER_ARCHIVE/rentals
```

Purging client files

Client Log Files

The following file in $ATA_DATA_SERVER_CLIENT directory maintains log information on client sessions:

```
log.out
```

This file should also be switched and purged on a regular basis. Log files are also generated locally on the disks of each client PC, or to a network drive. These files should also be periodically cleaned out.

Invoice Image (PDF) Files

Whenever the Singl.eView Convergent Billing client is used to view an invoice, a pdf file containing the invoice image is generated so that it can be viewed in Adobe Acrobat Reader.

The directory in which these pdf files are created can be configured to point to a network drive instead of the PC's local hard disk. This process greatly simplifies the administration overhead involved in periodically cleaning up these files, because all clients can be configured to generate the pdf files in the same directory on the server machine. A Purge Files task can then be used to periodically delete old pdf files from this directory, thus eliminating the manual task of deleting the files from the local disks of many PCs.

Chapter 14
Backup and Recovery

This section describes the suggested backup and recovery model for Singl.eView Convergent Billing. The following information is included:

- Points to consider for a backup and recovery strategy
- Disaster scenarios
- Types of backup strategy
- Singl.eView ICP specific issues.

Overview

Backup and recovery is one of the most critical aspects of a production system. It is mandatory that the backup and recovery processes are completely tested in a simulated production environment. Disk head crashes, circuitry failure and even catastrophic loss of data centres are all a fact of life for which a Database Administrator (DBA) must plan.

The emphasis of this chapter is helping the DBA to decide on, plan and test backup strategies. Ease of database recovery is directly related to the effort invested in putting an effective backup strategy in place.



The backup strategies outlined in this document are only suggested models of operations based on Singl.eView ICP requirements. Each individual site may have to tailor its backup and recovery strategies based on requirements.

Backup Strategy

Points to consider

The DBA must consider the needs of the business; for example, frequency and types of backups are determined by the business needs. The following are some questions that a DBA needs to consider when developing a backup strategy:

- Do the business needs demand that the database be available 24 hours a day, 7 days a week?
- Can the database be shut down periodically (eg. every night, once a week, or once a month)?
- How much down time can the business afford in the event of a disaster?

Possible disaster scenarios

Some of the most common types of failure that can occur on a database are described below.

Instance failure

Instance failure occurs when a problem arises that prevents an instance (system global area and background processes) from continuing work. Instance failure may result from a hardware problem such as power outage, or a software problem such as an operating system crash. When an instance failure occurs, the data in the buffers of the system global area is not written to the data files.

User or application error

User errors can require a database to be recovered at a time before the error occurred. A user might accidentally delete data from a table that is still required (for example, payroll taxes). To allow recovery from user errors and accommodate other unique recovery requirements, ORACLE provides for exact point-in-time recovery. For example, if a user accidentally deletes data, the database can be recovered to the instant in time before the data was deleted.

Media (disk) failure

An error can arise when trying to write or read a file that is required to operate the database. This is called disk failure because there is a physical problem reading or writing physical files on disk. A common example is a disk head crash, which causes the loss of all files on a disk drive. Different files may be affected by this type of disk failure, including the data files, the redo log files, and the control files. Also, because the database instance cannot continue to function properly, the data in the database buffers of the system global area cannot be permanently written to the data files.

Singl.eView ICP Specific Backup Issues

Singl.eView Convergent Billing has several characteristics that will influence the backup strategy. These characteristics will warrant careful examination of the backup process. The following are some points to consider:

- Tablespaces may be very large (100GB+).
- Spare disk capacity will be insufficient to store temporary copies of large tablespaces.
- Time available for backup is likely to be limited.

To achieve good sequential access out of the tablespace and efficient writing to tape, it is desirable to perform the backups when reads to the tablespace are minimal. If this is not possible, the next best approach would be to reduce the I/O by copying the data files to a secondary disk on a different controller. The data is then periodically transferred to tape from the secondary disk.

It is important to consider a scalable backup architecture to facilitate the addition of extra tape drives to reduce the duration of the backup process.

If the backup does not complete within the idle periods defined by the daily operation of Singl.eView ICP, the following options apply:

- Allow backup to continue into the CSR period possibly impacting the perceived performance for users.
- Stop backup at current tablespace. Tablespaces not backed up would be backed up the next day.

Types of Backup

The following is a list of options provided by ORACLE to do backups:

- Cold backup (off-line backup)
- Hot backup (on-line backup)
- Logical backup (using the export utility of ORACLE to perform on-line or off-line backup).

The modes of operations can be broken down to two categories:

- Medium to low availability
- High availability.

Medium to low availability

In this scenario, the availability of the system is inversely proportional to the frequency of backups performed. It is assumed that the database could be shutdown once a month at the very minimum. The suggested backup options are cold backups and logical backups.

A cold physical backup usually involves making a file level copy of all the necessary database elements. Before starting the backup, the DBA has to ascertain that the database can stay offline long enough for the backup to complete. The database needs to be running in an `ARCHIVELOG MODE`. The following is the list of elements to be included in the backup:

- Control files
- Data files
- Redo log files
- Archived log files
- `init.ora and config.ora`.

The following table outlines the required backup processes to perform a cold backup, based on the frequency of database shutdown.

Frequency of database shutdown	Frequency of Archivelog and Control files backup	Frequency of full logical backup
Once a day	Not required	Once a month
Once a week	Once a day	Once a month
Once a month	Once a day	Once a month

In the event of failure, the duration to recover from cold backup is inversely proportional to the frequency of cold backups performed.

Example

A cold backup is performed once a week on every Sunday and a database failure happens on Friday resulting from disk crash. The DBA needs to recover the database from cold backup.

The recovery would involve:

- Restoring the last cold backup performed (which would be from the past Sunday).
- Uploading the archive log from Monday to Thursday from tapes.
- Starting the database in recover mode and applying the archive logs to roll forward the database to Thursday.

Note: Depending on when the failure occurred on Friday, the new data from that day could be potentially lost.

In the above example, the time to recover would have been much shorter had cold backups been performed every night.

High availability

In this scenario, the database is up and running 24 hours a day, 7 days a week. Since the database is never offline, it is not possible to perform cold backups. The only option available in this case is to perform hot backups. The database needs to be running in `ARCHIVELOG MODE`.

The following is the list of elements to be included in the backup:

- Control files
- Data files
- Redo log files
- Archived log files
- `init.ora and config.ora`.

The frequency of hot backups performed is greatly dependent on the amount of activity on the system. The following are general guidelines to follow when using hot backups:

- Perform hot backups when reads to the tablespace are minimal
- Define a large Shared Global Area (SGA) in init.ora
- Backup archivelogs and control files every day
- Do a full logical export of the database at least once a month.

Chapter 15
Proactive Monitoring

By regularly monitoring the system, problems can be prevented before they occur and small problems identified and rectified before they become major problems.

This chapter looks at the proactive monitoring techniques that can be used to ensure that Singl.eView ICP and Singl.eView IP/Data run smoothly.

Machine Monitoring

Machine monitoring involves monitoring aspects of the machine on which the Singl.eView ICP and Singl.eView IP/Data server runs. Except for the Monitoring of the Singl.eView RADIUS Accounting Real Time Rating process, these monitors are not specific to Singl.eView ICP and Singl.eView IP/Data, but are general monitors that any UNIX System Administrator should be checking on a regular basis.

CPU activity

The overall CPU activity of the server machine should be monitored to ensure there are no rogue processes consuming valuable CPU resources.

Note: It is expected that some Singl.eView Convergent Billing server processes will utilise the full capacity of a CPU when they run. An example of this would be during rating.

Disk space

The disk space available in the mounted filesystems should be monitored. Monitoring disk space is especially important for the filesystems containing the database and the Singl.eView Convergent Billing server.

For example, the System Administrator could set up a script to run daily and check if certain filesystems are over 90% full.

Server Process Monitoring

Server process monitoring involves checking the Singl.eView Convergent Billing server processes to ensure that they are running smoothly.

Daemon processes

Many of the server processes run as daemons, meaning that they are started when the server is started and stay running until the server is shut down.

It is prudent to check periodically that all the server processes are running, including the TUXEDO processes.

This could be automated by writing one or more simple shell scripts to check that certain processes are running. These shell scripts could then be configured to run at regular intervals. For example, the rating engine might be configured so that four Event Normalisation (ENM) processes, one Event Rating (ERT) process, one Event Rating Output (ERO) process, and the Event Rating Broker (ERB) process are running. The following UNIX command will return the total number of rating processes running:

```
ps -ef | grep -v "grep" | grep -c -E "enm|ert|ero|bkr"
```

A simple shell script could then be written to check that this command returns 7, and an email message sent to a billing operations staff member if it does not.

Task output

Many tasks performed by the Singl.eView Convergent Billing system will be scheduled to run via the Singl.eView Convergent Billing scheduler. The output of these tasks should be checked to ensure that they have run smoothly. It is especially important to check them during the initial months of operation of the system and whenever Singl.eView ICP parameters are modified. Key tasks to check are those for:

- Rating
- Billing/Invoicing
- Invoice Printing
- Invoice Applying.

Note: Task output can be viewed via the Singl.eView Convergent Billing client and will also be sent in the email notification message when the task completes. The email notification message is sent to the recipients specified in the schedule when the task completes.

log.out

All server processes log important information such as warning and error messages to the system log file:

`$ATA_DATA_SERVER_LOG/log.out`.

This file should be periodically checked to ensure that server processes are not generating any warning or error messages.

For example, during the initial months of operations it is prudent to check the `log.out` file daily to ensure that all rating and billing processes completed successfully and did not generate any warnings or errors.

Any text editor can be used to view the `log.out` file, though the **`LOG`** command is provided so that the contents of the file can be viewed from any server directory; for example:

- **`LOG`** displays the most recent 23 lines written to the `log.out` file.
- **`LOG 100`** displays the most recent 100 lines written to the `log.out` file (a page at a time).
- **`LOG f`** displays the most recent 10 lines written to the `log.out` file and waits for further lines to be written. As more lines are written to the file, they are automatically displayed (press CTRL-C to exit).

ULOG

TUXEDO maintains its own log file called:

`$ATA_DATA_SERVER_LOG/ULOG.MMDDYY`

(where `MMDDYY` is the month, day, and year). TUXEDO processes write important messages to this file.

This file should be checked periodically to ensure that TUXEDO processes are not generating any warning or error messages. For example, whenever a user starts the Singl.eView Convergent Billing client and logs on, a message is written to this file, saying that a user has been authenticated.

TUXEDO licensing restricts the number of concurrent Singl.eView Convergent Billing clients that can be logged in. As the number of concurrent clients approaches this limit, warning messages will be written to the `ULOG` file. The warning indicates that you may need to purchase additional TUXEDO client licenses so that additional CSRs can be supported.

Any text editor can be used to view the current ULOG file (the ULOG file for the current day). The **ULOG** command is provided so that the contents of the file can be viewed from any server directory; for example:

- **ULOG** displays the most recent 23 lines written to the current ULOG file.
- **ULOG 100** displays the most recent 100 lines written to the current ULOG file (one page at a time).
- **ULOG f** displays the most recent 10 lines written to the current ULOG file and waits for further lines to be written. As more lines are written to the file, they are automatically displayed (press CTRL-C to exit).

Monitoring of the Singl.eView RADIUS Accounting Real Time Rating process

The following are components of the RADIUS Accounting Real Time Rating process that should be checked regularly. For more information, refer to *Chapter 18, RADIUS Accounting Real Time Rating Process*.

Controller

The Controller should be used on a regular basis to monitor the status of the connection between the RADIUS Accounting Real Time Rating process and the Singl.eView Real Time Rating ENM. When this connection is unavailable, real time rating will fail. Instead, the events are stored in the Batch Output files. These files are then used for batch processing of the RADIUS Accounting events.

To check the status of the connection, click the **Statistics** button on the *Controller* window. For more information, refer to *Controlling the RADIUS Accounting Real Time Rating process after start up* in *Chapter 18, RADIUS Accounting Real Time Rating Process*.

Batch Output Directory

The Batch Output Directory should be checked regularly for Batch Output files. These files will not be rated in real time, but are stored for batch processing when the connection between the RADIUS Accounting Real Time Rating process and the Singl.eView Real Time Rating ENM is unavailable.

Error Batch Output Directory

The Error Batch Output Directory should be checked regularly for Error Batch Output files. These are identical to the Batch Output files, but an error message is written first and on a separate line from the actual event.

Log File

The Log File (`rtrclient.log`) should be periodically checked for error or warning messages. Whenever the RADIUS Accounting Real Time Rating process is started, re-started, or shut down, a message is logged. The Log File is stored at `$ATA_DATA_SERVER_LOG/rtrclient.log`.

Database validation

The database validation processes (DVPs) should be run on a regular basis to perform integrity checks on database entities. If the DVPs find errors, they should be investigated and fixed immediately. For more information, refer to *Chapter 11, Database Validation.*

Client Monitoring

Error and warning messages that are generated by the Singl.eView Convergent Billing client can be logged for future reference.

For example, a CSR may get an error that they have never seen before and do not understand. The CSR may tell a supervisor about it later in the day, but forget exactly what the error message said. The supervisor can look up the client log file to determine the exact message that was displayed.

Logging client messages can be controlled in two ways:

1. Client Controlled

   Each Singl.eView Convergent Billing client can be configured to log messages to a file which could be situated on a local disk, or on a network drive. The full pathname of the file is specified in the Singl.eView Convergent Billing `.ini` file for the client. For example, the `.ini` file might specify to log messages to the file:

   `C:\ATLANTA\LOG.OUT`.

2. Server Controlled

   The Singl.eView Convergent Billing server can be configured in such a way that messages from all Singl.eView Convergent Billing clients will be logged to a single file on the server. This is in addition to client controlled logging described above. The difference is that messages from all clients are written to this one file. The server controlled method of logging client messages may be used for auditing purposes.

The server file that contains the messages generated by all clients should be configured on the server as the file:

`$ATA_DATA_CLIENT_LOG/log.out`.

The location of this file is configured in the file

`$ATA_DATA_CLIENT_CONFIG/wsenvfile.tpl)`.

Any text editor can be used to view the client `log.out` file, though the **`CLOG`** command is provided on the server so that the contents of the file can be viewed from any server directory; for example:

- **`CLOG`** displays the most recent 23 lines written to the client `log.out` file.
- **`CLOG 100`** displays the most recent 100 lines written to the client `log.out` file (a page at a time).
- **`CLOG f`** displays the most recent 10 lines written to the client `log.out` file and waits for further lines to be written. As more lines are written to the file, they are automatically displayed (press CTRL-C to exit).

TUXEDO Monitoring

TUXEDO provides a program that enables many TUXEDO statistics to be displayed. This program is called `tmadmin`. After running the program, you are presented with a command prompt at which you can enter commands. To get a list of the available commands, type **`help`**.

Two useful commands are:

- **`printclient`**, which shows all the TUXEDO clients connected to the system.
- **`printserver`**, which shows the status and statistics of all TUXEDO servers.

Printclient

The **`printclient`** command displays all the TUXEDO client sessions that are currently in use. Each Singl.eView Convergent Billing client establishes a TUXEDO client session when it starts, and relinquishes the session when it terminates. Therefore, this command is a useful way to get a list of the clients that are currently logged in.

A sample output of this command is shown below.

```
LMID         User Name       Client Name    Time    Status  Bgn/Cmmt/Abrt
-------------- -------------- -------------- -------- ------- -------------
site1                          tpsysadm        10:11:25 IDLE    0/0/0
site1                          tpsysadm        10:11:18 IDLE    0/0/0
site1                          tpsysadm        10:11:13 IDLE    0/0/0
site1                          tpsysadm        10:11:12 IDLE    0/0/0
site1                          tpsysadm        10:11:12 IDLE    0/0/0
site1           atp_dev        WSH              6:11:54 IDLE    0/0/0
site1           pjv            BsBr0114         4:18:51 IDLE/W  0/0/0
site1           atp_dev        tmadmin          0:00:02 IDLE    0/0/0
site1                          tpsysadm        10:11:28 IDLE    0/0/0
```

Note: Not all the lines shown represent Singl.eView Convergent Billing clients. Some of the server processes maintain TUXEDO client sessions, and will therefore show up in this listing. In fact, the only line in the listing above that represents a Singl.eView Convergent Billing client is:

```
site1           pjv            BsBr0114         4:18:51 IDLE/W  0/0/0
```

Printserver

The **`printserver`** command provides statistics on all the TUXEDO servers that are running.

A sample output of this command is shown below.

```
Prog Name      Queue Name  Grp Name      ID RqDone Load Done Current Service
---------      ----------  --------      -- ------ --------- ---------------
BBL            63532       site1          0     69      3450 (  IDLE )
trenodb        00003.00001 NODB           1      0         0 (  IDLE )
trerodb        00002.00001 RODB           1      0         0 (  IDLE )
trerwdb        00001.00001 RWDB           1      0         0 (  IDLE )
ORATMS         RWDB_TMS    RWDB       30001      0         0 (  IDLE )
trenodb        00003.00002 NODB           2      0         0 (  IDLE )
trerodb        00002.00002 RODB           2    271     13550 (  IDLE )
trerwdb        00001.00002 RWDB           2      0         0 (  IDLE )
ORATMS         RWDB_TMS    RWDB       30002      0         0 (  IDLE )
WSL            00003.00003 NODB           3      0         0 (  IDLE )
trerodb        00002.00003 RODB           3      0         0 (  IDLE )
trerwdb        00001.00003 RWDB           3      4       200 (  IDLE )
trerodb        00002.00004 RODB           4      3       150 (  IDLE )
TMUSREVT       00003.00004 NODB           4    259     12950 (  IDLE )
tresok         00002.00005 RODB           5      0         0 (  IDLE )
tresok         00002.00006 RODB           6      0         0 (  IDLE )
authsvr        00002.00099 RODB          99      9       450 (  IDLE )
trerwdb        00001.00020 RWDB          20      0         0 (  IDLE )
trerwdb        00001.00021 RWDB          21      0         0 (  IDLE )
trerwdb        00001.00022 RWDB          22      2       100 (  IDLE )
trerwdb        00001.00040 RWDB          40      0         0 (  IDLE )
trerwdb        00001.00041 RWDB          41      0         0 (  IDLE )
trerwdb        00001.00061 RWDB          61      0         0 (  IDLE )
trerwdb        00001.00062 RWDB          62      0         0 (  IDLE )
trerwdb        00001.00063 RWDB          63     26      1300 (  IDLE )
```

Note: The listing displays how many service requests have been processed by each TUXEDO server since it started. It also indicates the current service being processed or whether the server is currently idle.

Database Monitoring

ORACLE has many tools for monitoring the database. The database administrator should refer to the relevant ORACLE manuals for details on how to perform such tasks.

Chapter 16
Schedule Planner

To ensure smooth and efficient operation of Singl.eView ICP, a schedule plan should be constructed to fit the operational model of the business. Such a plan should be implemented up front, before putting the system into operation, and needs to be continually monitored and updated as the business evolves.

The schedule plan identifies the policies to be used to perform the key server functions of rating, billing, and invoicing, as well as other functions such as archiving, and backup and recovery. The schedule plan identifies the schedules required to perform these tasks and provides for adequate time windows in which failed tasks can be re-run.

Example Scenario

The example schedule plan described in this chapter is based on an ICP carrier with the following characteristics:

- 50,000 subscribers, with each subscriber generating 5 billable events per day.
- Files received from the ICP switches are rated on a daily basis during the night.
- Customers are invoiced monthly.
- The customers can be geographically separated into four regions, with each region containing approximately the same number of customers. These regions are:
  - North
  - South
  - East
  - West.

- They will print invoices internally, using four postscript printers. Each printer will be dedicated to printing the invoices for all the customers from a certain region (ie. one printer prints all the invoices for the North regions, another printer prints all the invoices for the South region, etc).
- Treatment is to be run nightly on all customers.
- The carrier is running Singl.eView Convergent Billing on a machine with 4 CPUs.
- The system does not require 24x7 operation. Therefore, the Singl.eView Convergent Billing server and the database will be shut down four times a month and a full backup of the database to tape will be performed. These backups are to be used to restore the database in the event that a system failure of some kind causes the database to be lost or corrupted. Daily archives of the database redo logs will also be performed so that the system can always be restored to a point where no more than 24 hours worth of data is lost. The daily archives are performed while the system is online (ie. the database is not shut down).
- Event data record information is to be kept online (in the database) for three months. After three months, the event data record information will be archived to file and purged from the database.
- CSRs are online and using the system during the hours of 8:00am to 6:00pm. No server tasks are scheduled to run during this time, so that the full resources of the system are devoted to servicing the CSRs

Rating considerations

The best rating performance will be achieved on a machine with 4 CPUs by having two rating streams (assuming no other major activity is being performed by the machine). This means the switch files must be split into two groups, with two Event Normalisation (ENM) processes running simultaneously, each processing files from a file group. A single Event Rating (ERT) process and Event Rating Output (ERO) process are also running.

Billing considerations

The best billing performance will be achieved on a machine with 4 CPUs by having four billing schedules running at the same time (assuming no other major activity is being performed by the machine).

To spread the billing load over the month, the carrier will have 14 invoice cycles. These invoice cycles are spread evenly over the month on alternate days. The invoice cycle dates for each month are 2nd, 4th, 6th, 8th, 10th, ..., 28th. The odd numbered days can be used to revoke and reprocess any billing schedules that may fail.

Four billing schedules are created for each invoice cycle. Each billing schedule will handle the billing and the invoice generation and printing of all the customers from a certain region. For example, the invoice cycle identified by the 2nd of the month will have four billing schedules; one for the North regions, one for the South region, one for the East, and one for the West.

So that CSRs know which billing schedule to assign a customer to, it is decided to assign customers to invoice cycles based on the first letter of their surname. Within an invoice cycle, the actual billing schedule to assign the customer to is based on the customers region. Customers are assigned to invoice cycles as follows:

Invoice Cycle	Surname Starts with
2nd	A, B
4th	C, D
6th	E, F
8th	G, H
10th	I, J
12th	K, L
14th	M, N
16th	O, P
18th	Q, R
20th	S
22nd	T
24th	U, V
26th	W, X
28th	Y, Z

If the 50,000 customers are spread reasonably evenly across all the invoice cycles, each invoice cycle will contain about 3,500 customers.

It is initially decided that the billing schedules will not print the invoices or apply the invoices to customer accounts. This allows the results of the billing schedules to be manually checked and the billing schedule to be revoked and re-run in the event that any errors are detected. Also, the Billing Reconciliation Report (RBRC_CLEC) is to be run after each billing schedule has completed.

For each billing schedule, a schedule will be set up to print the invoices produced by that schedule, another schedule will be set up to apply the invoices to the customer accounts, and a third will be set up to run the billing reconciliation report. These schedules will be configured to run on the day after the billing schedule is performed.



This means that if a billing schedule produces errors and needs to be revoked and re-run, this must be done before the tasks to print and apply the invoices on the next day are run. If this cannot be done in time, the tasks to print and apply the invoices will need to be delayed (run at a later date/time). The task that produces the billing reconciliation report should also be delayed.

After a period of time, the carrier may decide to configure the billing schedules to print and/or apply the invoices that they generate. This would only be done once the tariff and invoice configuration of the system is stable and the billing schedules have run smoothly for a number of months.

The following table provides the names of the billing schedules for the system. For each billing schedule, there is an invoice printing schedule, an invoice applying schedule, and a billing reconciliation report schedule with the same name. Remember that these schedules are configured to run a day after their corresponding billing schedule.

Invoice Cycle	Billing Schedule
2nd	02 - AB North
2nd	02 - AB South
2nd	02 - AB East
2nd	02 - AB West
4th	04 - CD North
4th	04 - CD South
4th	04 - CD East
4th	04 - CD West
6th	06 - EF North
6th	06 - EF South
6th	06 - EF East
6th	06 - EF West

Invoice Cycle	Billing Schedule
...	...
26th	26 - WX North
26th	26 - WX South
26th	26 - WX East
26th	26 - WX West
28th	28 - YZ North
28th	28 - YZ South
28th	28 - YZ East
28th	28 - YZ West

Schedules

The following table identifies the types of schedules that will be run by the carrier and whether they will be run daily, weekly or monthly. Note that there may be multiple schedules created for each type of schedule identified.

Schedule Type	Task Performed	Frequency
Rate Files	Rates event files.	Daily
Invoice Generation	Performs billing and generates customer invoices (but does not print or apply them).	Monthly
General Bulk Output Process	Prints invoice images.	Monthly
Apply Invoices	Applies invoices to customer accounts.	Monthly
Archive	Purges old data (eg. tasks, cancelled customers) from the database.	Monthly
Treatment Monitor Schedule	Performs treatment actions on customers.	Daily

Schedule Type	Task Performed	Frequency
Promise To Pay Monitor	Evaluates all open promises and determines whether they have been kept or not.	Daily
Full Database Validation	Checks for database integrity using the DVP. (Depending on the size of the database, this process may take too long to perform daily, so a partial DVP is performed daily and a full DVP is performed weekly or monthly).	Daily
Partition Maintenance	Sorts and archives the database partitions that have been marked for sorting/archiving.	Daily
Purge Files	System Housekeeping. Deletes files from a directory that are older than a certain date.	Weekly
Switch Files	System Housekeeping. Renames files in a directory by appending the date to the original filename.	Daily
CLEC Generate Month Line Taxes	Performs monthly line tax calculations.	Monthly
Global Deposit Interest	Calculates Deposit Interest for all active customers and generates credit adjustments for valid refunds.	Monthly
CLEC Update Capture Extraction	Extracts update captures information, formats it, and sends it to a file (or other output device).	Daily/ Weekly
RACR_CLEC	Accounts Receivable and Deposits Report.	Daily
RADJ_CLEC	Operator Adjustment Activity Report.	Weekly
RAGD_CLEC	Aged Debtors Report.	Weekly
RBRC_CLEC	Billing Reconciliation Report.	Monthly

Schedule Type	Task Performed	Frequency
Upload to General Ledger (CLEC)	Prepares data for the General Ledger Report. All unmarked account transactions are marked as being uploaded to the GL system.	Monthly
RFBA_CLEC	Final Billed Report.	Monthly
RFGL_CLEC	General Ledger Report.	Monthly
RDFD_CLEC	Data Feed Report.	Daily
RESV_CLEC	Event Error Summary by Service Report.	Daily
RSER_CLEC	Event Error Summary Report.	Daily
RINS_CLEC	Inventory Status Report.	Weekly
RINV_CLEC	Invoice Summary.	Monthly
RIPN_CLEC	Telephone Number Inventory Report.	Weekly
ROPA	Operator Profile Audit Report.	Weekly
RPAY_CLEC	Bank Deposit Report.	Daily
RQRY_CLEC	Query/Order Report.	Daily/ Weekly
RSHU_CLEC	High Usage Report.	Weekly
RTXE_CLEC	Taxing Errors Report.	Monthly
Re-analyze Tables	Recomputes optimiser statistics for all database tables. Ensures database performance remains at an optimum level as the database tables change/grow.	Weekly
Rotate Partitions	Cycles database table partitions.	Daily



There are many other Singl.eView Convergent Billing schedule types that are used to administer and operate the system. The ones identified in the table above are an example only, but are representative of the tasks that will need to be run on an ongoing basis. Many other schedules types are run as immediate tasks in response to exceptions that occur (eg. 'Revoke Invoices'), and are not usually set up as schedules to run on a regular basis.

Full database backups also need to be performed on a regular basis. Cold backups will be performed in which the Singl.eView Convergent Billing server and the database are shut down for the duration of the backup. Therefore, the backups cannot be scheduled by the Singl.eView Convergent Billing scheduler. They must be scheduled to run using some other scheduling mechanism (eg. the CRON utility provided with UNIX.

Full backups are to be performed four times every month. They are always performed on set days of the month and are on days when billing schedules are not being run. The full database backup is expected to take about six (6) hours.

Schedule Plan

The schedule plan is shown below. It is broken down into a monthly schedule plan (for those schedules performed monthly), a weekly schedule plan, and a daily schedule plan.

Remember that CSRs are online and using the system during the hours of 8:00am to 6:00pm. No server tasks are scheduled to run during this time. This does not mean that server tasks will never be run during this time. Exceptional circumstances may dictate that certain server processes need to be run during the day on a one-off basis only. It is important to note that this may effect CSR response time (depending on the nature of the server task).

Rating or billing should never be run during the day, as this has a big impact on CSR performance. However, running a one-off report may not produce any noticeable degradation in CSR response time.

Monthly schedule plan

Date	Time	Schedule	Description
1st	18:00	Archive	Purges old data (eg. tasks, cancelled customers) from the database.
	18:00	Upload to General Ledger (CLEC)	Prepares data for the General Ledger Report.
	19:00	General Ledger Report	Generates the General Ledger Report.
	20:00	Global Deposit Interest Calculation	Calculates Deposit Interest for all active customers.

Date	Time	Schedule	Description
	21:00	Final Billed Report	Reports on all customers that have received their final invoice.
	00:00	Full Database Backup	Performs a full database backup.
2nd	00:00	Generate Month Line Taxes	Performs monthly line tax calculations for a billing schedule.
	01:00	Billing Schedules: • 02 - AB North • 02 - AB South • 02 - AB East • 02 - AB West	Billing schedules for all customers with surnames starting with A or B.
	06:00	Taxing Errors Report	Report of tax errors incurred in billing schedule.
3rd	18:00	Invoice Summary Report, Invoice Printing, Invoice Applying, and Billing Reconciliation Report Schedules: • 02 - AB North • 02 - AB South • 02 - AB East • 02 - AB West	Related billing schedules for all customers with surnames starting with A or B.
4th	00:00	Generate Month Line Taxes	Performs monthly line tax calculations for a billing schedule.
	01:00	Billing Schedules: • 04 - CD North • 04 - CD South • 04 - CD East • 04 - CD West	Billing schedules for all customers with surnames starting with C or D.
	06:00	Taxing Errors Report	Report of tax errors incurred in billing schedule.

Date	Time	Schedule	Description
5th	18:00	Invoice Summary Report, Invoice Printing, Invoice Applying, and Billing Reconciliation Report Schedules: • 04 - AB North • 04 - AB South • 04 - AB East • 04 - AB West	Related billing schedules for all customers with surnames starting with C or D.
6th	00:00	Generate Month Line Taxes	Performs monthly line tax calculations for a billing schedule.
	01:00	Billing Schedules: • 06 - EF North • 06 - EF South • 06 - EF East • 06 - EF West	Billing schedules for all customers with surnames starting with E or F.
	06:00	Taxing Errors Report	Report of tax errors incurred in billing schedule.
7th	18:00	Invoice Summary Report, Invoice Printing, Invoice Applying, and Billing Reconciliation Report Schedules: • 06 - EF North • 06 - EF South • 06 - EF East • 06 - EF West	Related billing schedules for all customers with surnames starting with E or F.
8th	00:00	Generate Month Line Taxes	Performs monthly line tax calculations for a billing schedule.
	01:00	Billing Schedules: • 08 - GH North • 08 - GH South • 08 - GH East • 08 - GH West	Billing schedules for all customers with surnames starting with G or H.
	06:00	Taxing Errors Report	Report of tax errors incurred in billing schedule.

Date	Time	Schedule	Description
9th	18:00	Invoice Summary Report, Invoice Printing, Invoice Applying, and Billing Reconciliation Report Schedules: • 08 - GH North • 08 - GH South • 08 - GH East • 08 - GH West	Related billing schedules for all customers with surnames starting with G or H.
	00:00	Full Database Backup	Performs a full database backup.
10th	00:00	Generate Month Line Taxes	Performs monthly line tax calculations for a billing schedule.
	01:00	Billing Schedules: • 10 - IJ North • 10 - IJ South • 10 - IJ East • 10 - IJ West	Billing schedules for all customers with surnames starting with I or J.
11th	18:00	Invoice Summary Report, Invoice Printing, Invoice Applying, and Billing Reconciliation Report Schedules: • 10 - IJ North • 10 - IJ South • 10 - IJ East • 10 - IJ West	Related billing schedules for all customers with surnames starting with I or J.
12th	00:00	Generate Month Line Taxes	Performs monthly line tax calculations for a billing schedule.
	01:00	Billing Schedules: • 12 - KL North • 12 - KL South • 12 - KL East • 12 - KL West	Billing schedules for all customers with surnames starting with K or L.
	06:00	Taxing Errors Report	Report of tax errors incurred in billing schedule.

Date	Time	Schedule	Description
13th	18:00	Invoice Summary Report, Invoice Printing, Invoice Applying, and Billing Reconciliation Report Schedules: • 12 - KL North • 12 - KL South • 12 - KL East • 12 - KL West	Related billing schedules for all customers with surnames starting with K or L.
14th	00:00	Generate Month Line Taxes	Performs monthly line tax calculations for a billing schedule.
	01:00	Billing Schedules: • 14 - MN North • 14 - MN South • 14 - MN East • 14 - MN West	Billing schedules for all customers with surnames starting with M or N.
	06:00	Taxing Errors Report	Report of tax errors incurred in billing schedule.
15th	18:00	Invoice Summary Report, Invoice Printing, Invoice Applying, and Billing Reconciliation Report Schedules: • 14 - MN North • 14 - MN South • 14 - MN East • 14 - MN West	Related billing schedules for all customers with surnames starting with M or N.
	00:00	Full Database Backup	Performs a full database backup.
16th	00:00	Generate Month Line Taxes	Performs monthly line tax calculations for a billing schedule.

Date	Time	Schedule	Description
	01:00	Billing Schedules: • 16 - OP North • 16 - OP South • 16 - OP East • 16 - OP West	Billing schedules for all customers with surnames starting with O or P.
	06:00	Taxing Errors Report	Report of tax errors incurred in billing schedule.
17th	18:00	Invoice Summary Report, Invoice Printing, Invoice Applying, and Billing Reconciliation Report Schedules: • 16 - OP North • 16 - OP South • 16 - OP East • 16 - OP West	Related billing schedules for all customers with surnames starting with O or P.
18th	00:00	Generate Month Line Taxes	Performs monthly line tax calculations for a billing schedule.
	01:00	Billing Schedules: • 18 - QR North • 18 - QR South • 18 - QR East • 18 - QR West	Billing schedules for all customers with surnames starting with Q or R.
19th	18:00	Invoice Summary Report, Invoice Printing, Invoice Applying, and Billing Reconciliation Report Schedules: • 18 - QR North • 18 - QR South • 18 - QR East • 18 - QR West	Related billing schedules for all customers with surnames starting with Q or R.
	06:00	Taxing Errors Report	Report of tax errors incurred in billing schedule.
20th	00:00	Generate Month Line Taxes	Performs monthly line tax calculations for a billing schedule.

Date	Time	Schedule	Description
	01:00	Billing Schedules: • 20 - S North • 20 - S South • 20 - S East • 20 - S West	Billing schedules for all customers with surnames starting with S.
	06:00	Taxing Errors Report	Report of tax errors incurred in billing schedule.
21st	18:00	Invoice Summary Report, Invoice Printing, Invoice Applying, and Billing Reconciliation Report Schedules: • 20 - S North • 20 - S South • 20 - S East • 20 - S West	Related billing schedules for all customers with surnames starting with S.
22nd	00:00	Generate Month Line Taxes	Performs monthly line tax calculations for a billing schedule.
	01:00	Billing Schedules: • 22 - T North • 22 - T South • 22 - T East • 22 - T West	Billing schedules for all customers with surnames starting with T.
	06:00	Taxing Errors Report	Report of tax errors incurred in billing schedule.
23rd	18:00	Invoice Summary Report, Invoice Printing, Invoice Applying, and Billing Reconciliation Report Schedules: • 22 - T North • 22 - T South • 22 - T East • 22 - T West	Related billing schedules for all customers with surnames starting with T.
	00:00	Full Database Backup	Performs a full database backup.

Date	Time	Schedule	Description
24nd	00:00	Generate Month Line Taxes	Performs monthly line tax calculations for a billing schedule.
	01:00	Billing Schedules: • 24 - UV North • 24 - UV South • 24 - UV East • 24 - UV West	Billing schedules for all customers with surnames starting with U or V.
	06:00	Taxing Errors Report	Report of tax errors incurred in billing schedule.
25rd	18:00	Invoice Summary Report, Invoice Printing, Invoice Applying, and Billing Reconciliation Report Schedules: • 24 - UV North • 24 - UV South • 24 - UV East • 24 - UV West	Related billing schedules for all customers with surnames starting with U or V.
26th	00:00	Generate Month Line Taxes	Performs monthly line tax calculations for a billing schedule.
	01:00	Billing Schedules: • 26 - WX North • 26 - WX South • 26 - WX East • 26 - WX West	Billing schedules for all customers with surnames starting with W or X.
	06:00	Taxing Errors Report	Report of tax errors incurred in billing schedule.
27st	18:00	Invoice Summary Report, Invoice Printing, Invoice Applying, and Billing Reconciliation Report Schedules: • 26 - WX North • 26 - WX South • 26 - WX East • 26 - WX West	Related billing schedules for all customers with surnames starting with W or X.

Date	Time	Schedule	Description
28th	00:00	Generate Month Line Taxes	Performs monthly line tax calculations for a billing schedule.
	01:00	Billing Schedules: • 28 - YZ North • 28 - YZ South • 28 - YZ East • 28 - YZ West	Billing schedules for all customers with surnames starting with Y or Z.
	06:00	Taxing Errors Report	Report of tax errors incurred in billing schedule.
29st	18:00	Invoice Summary Report, Invoice Printing, Invoice Applying, and Billing Reconciliation Report Schedules: • 28 - YZ North • 28 - YZ South • 28 - YZ East • 28 - YZ West	Related billing schedules for all customers with surnames starting with Y or Z.

Weekly schedule plan

Day	Time	Schedule	Description
Saturday	18:00	Purges Files Dir1 Purge Files Dir2 Purge Files Dir3 ...	System Housekeeping. Deletes files from certain directories that are older than a certain date.
	18:15	Query/Order Report	Generates the Query/ Order Report.
	19:00	Re-analyze Tables	Recomputes optimiser statistics for all database tables.
Sunday	18:00	Operator Adjustment Activity Report	Generates the Operator Adjustment Activity Report.
	18:00	Aged Debtors Report	Generates the Aged Debtors Report.

Day	Time	Schedule	Description
	18:00	Telephone Number Inventory Report	Generates the Telephone Number Inventory Report.
	18:30	Inventory Status Report	Generates the Inventory Status Report.
	19:00	Operator Profile Audit Report	Generates the Operator Profile Audit Report.
	19:00	High Usage Report	Generates the High Usage Report.

Daily schedule plan

Time	Schedule	Description
18:00	-	CSR activity completes. A two hour buffer zone is provided until the first daily server tasks are scheduled to run. *Weekly and Monthly reports are scheduled to run in this time period. Also, invoice printing and applying is performed in this period.*
20:00	Rate Switch 1 Rate Switch 2	Two rating schedules so that the daily switch files are processed in two streams. Expected Running Time: 40mins.
21:00	Data Feed Report, Event Error Summary Reports	Reports to assist with the rating tasks and event management.
21:15	Partition Maintenance	Sorts and archives partitions that have been marked for sorting/archiving.
21:30	Partition Rotation	Rotates database partitions.
22:00	CLEC Update Capture Extraction	Extracts update capture information (from the XPSI_UPDATE_CAPTURE table), formats it, and sends it to a file (or other output device).

Time	Schedule	Description
23:00	Promise To Pay Monitor	Evaluates all open promises up to effective date and determines whether they have been kept or not.
	Treatment	Performs Treatment actions on customers.
	Accounts Receivable Report	Generates the Accounts Receivable and Deposits report.
11:30	Bank Deposit Report	Generates the Bank Deposit report.
00:00		Monthly Billing schedules are run during this time on certain days of the month. Expected Running Time: 3 hours On days when billing schedules are not being run, this window can be used revoke and re-process any billing runs that have errored, as well as run other one-off tasks that may be required. On certain days of the month (when billing schedules are not being run), a full database backup is performed. Expected Running Time: 6 hours
03:00	Full DVP	Runs a full DVP Expected Running Time: 3 hours On the days that full backups are being done, the Singl.eView ICP server and the database will be shut down. Therefore, the full DVP will not be performed at this time. The DVP will start as soon as the backup is complete and the database and Singl.eView ICP server have been restarted. This may cause the running of the DVP to spill over into CSR activity time. One approach to solve this would be not to perform the DVP on the days that full backups are performed. This could be achieved by:

Time	Schedule	Description
		• Deleting the DVP task for those days that the full backup is to occur. • Providing a wrapper script around the DVP script that checks the time and if it is later than a set time, to exit immediately and not run the DVP.
06:00		All major server tasks are expected to be completed by this time. This leaves a two hour buffer zone for any tasks to complete that have taken longer than usual (until CSRs come online at 8:00am). Other one-off tasks may also be scheduled in this period.
07:30	Switch Files Dir1 Switch Files Dir2 Switch Files Dir3 ...	System Housekeeping. Renames the current log files. By doing this daily, separate log files are created each day, with the log files from previous days identified by the date that is appended to the filename.

Daily time windows

With the monthly, weekly, and daily schedule plans described above, the typical daily operational time windows are as shown in the table below:

Time Window	Operation
8:00am - 6:00pm	CSR Activity.
6:00pm - 8:00pm	Free Buffer Zone, except for the days of the week/month on which weekly/monthly reports are run the day each week on which system housekeeping tasks are performed.
8:00pm - 10:00pm	Rating.
10:00pm - 10:30pm	Partition rotation and maintenance.
10:30pm - 12:00am	Treatment and other daily reports.

Time Window	Operation
12:00am - 3:00am	Billing, only on certain days of the month. Bill revoking and re-running can be performed, if necessary, on those days for which billing isn't scheduled.
12:00am-6:00am	Full database backup. Done once a week, on a day when billing is not scheduled.
3:00am-6:00am	Full DVP, except for the days on which full backups are performed.
6:00am-8:00am	Free Buffer Zone, with some daily system housekeeping also performed. The daily system housekeeping is negligible in terms of CPU usage, and other tasks could be run in this time window if necessary.

Chapter 17
Payment Interfaces and Schedule Types

This chapter contains operations information on all schedule types related to payments. See the appendixes for information on the inbound and outbound payment file formats. This chapter contains the following sections:

- Overview of Outbound Payments and Confirmation Letter Schedules
- PAP Auto Payment Generation Schedule Type
- PAP Payment File Generation Schedule Type – Pre-authorised Payments
- PAP Revoke Process Schedule Type
- PAP Confirm Letter Generation Schedule Type
- PAP Confirm Letter Revoke Schedule Type
- Process Lockbox Files – Inbound Payments
- FTP Exchange Schedule Type.

Overview of Outbound Payments and Confirmation Letter Schedules

The pre-authorised payment schedules are very closely related. It is recommended that operators who run any of the schedules for pre-authorised payments be familiar with the relationship among the schedules. Figure 17-1 shows which schedules can be triggered by the PAP Auto Payment Generation schedule and which revoke processes are related to the pre-authorised payments schedules. The PAP Auto Payment Generation schedule generates payment batches for customers who have requested to pay through pre-authorised direct debit and credit card payments. This schedule can automatically trigger the PAP Payment File Generation schedule, which creates a payment file to be transferred to a bank or clearinghouse.

In addition, the PAP Auto Payment Generation schedule can automatically trigger the PAP Confirm Letter Generation schedule. This schedule generates confirmation letters to be sent to customers who have set up pre-authorised payments, had a change to the details of these payments (including termination), suspended the payments, or re-instated the payments.

The PAP Revoke Process schedule can be used to reverse the effects of the PAP Auto Payment Generation and PAP Payment File Generation schedules. Similarly, the PAP Confirm Letter Revoke process can be used to reverse the effects of the PAP Confirm Letter Generation schedule.

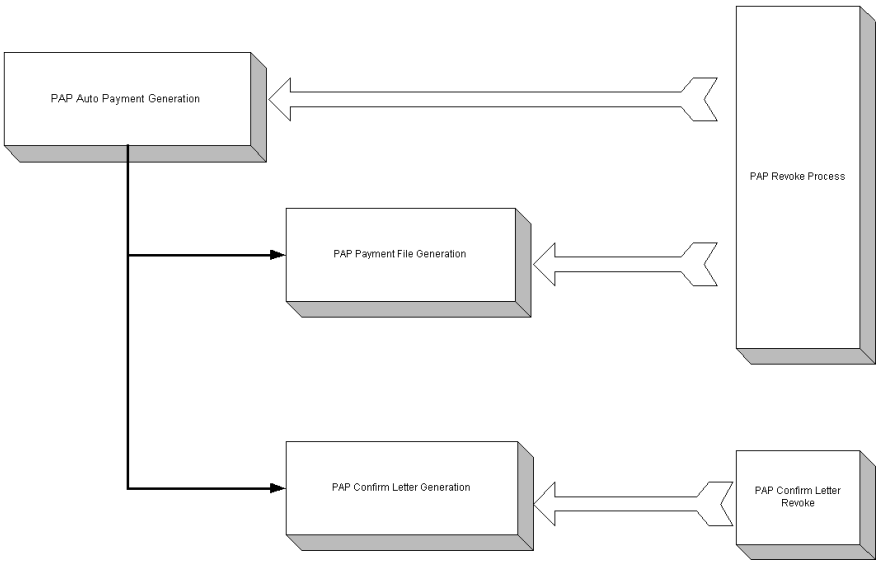


Figure 17-1 Visual Representation of Relationship among the Pre-authorised Payments Schedules

PAP Auto Payment Generation Schedule Type

The PAP Auto Payment Generation schedule type can be used to automatically generate payment batches for customers that have specified that they want to make pre-authorised payments (PAP) by credit card or by direct debit from a bank account. The full process of creating an outbound file to be transferred to a bank or clearinghouse involves two schedule types:

1. The PAP Auto Payment Generation schedule type generates the pre-authorised payment batches.
2. The PAP Payment File Generation schedule type, which can be automatically triggered by the first schedule, creates the files containing the payment details to be transferred. (To transfer the files via FTP, the FTP Exchange schedule can be used.)

This section covers the first schedule. For information about the second schedule, see *PAP Payment File Generation Schedule Type* later in this chapter.

Scheduling

It is recommended that the PAP Auto Payment Generation schedule be run once per day in order to guarantee that all customers whose payments are due will have payments generated in a timely manner. Attempting to run the schedule at the precise time that each customer's payments are due would be overly complicated. This is principally because different customer types can be configured to have different due dates relative to the date of the billing cycle. For example, if Bill Cycle 10 is run on the 15th of every month, the payments for residential customers could be due within 15 days, within 10 days for business customers, and within 20 days for small office/home office customers. Figure 17-1 illustrates how the combination of many bill cycles that include many customer types can create multiple due dates to account for. As a result, it is simpler to run the schedule once per day.



The PAP Auto Payment Generation schedule can be run no more than once per day.

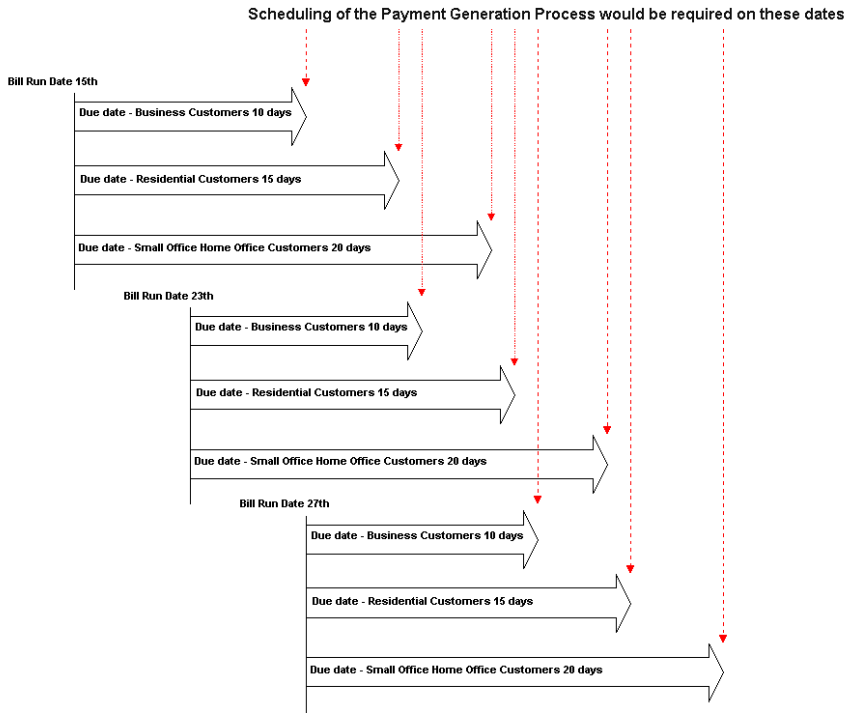


Figure 17-1 Multiple Payment Due Dates

Scheduling parameters

When the PAP Auto Payment Generation schedule type is configured, two optional parameters may be specified on the *Parameters* tab of the *Schedule Definition* form:

1. The *Extra Days* field may be used to indicate a payment due date that is used as the criteria for processing payments in any particular run. If it is left blank, the payment due date that is used will default to the effective date of the run. If a number of days is specified, then the due date that is used will be the effective date of the run plus the number of days specified.

2. The *Run File Generation Process* field may also be used to specify whether the PAP Payment File Generation process should be automatically launched when the PAP Auto Payment Generation task is finished. If errors or other complications are expected for the run, then the task queue results may be consulted before the second schedule type is launched. For further information, refer to *PAP Payment File Generation Schedule Type* later in this chapter.

Actions triggered by running the PAP Auto Payment Generation schedule

When the PAP Auto Payment Generation schedule is run, payment batches are automatically generated for customers who have previously requested to pay through pre-authorised direct debit or credit card payment methods. Once all payment batches have been created, they will be posted against the customer's accounts.

Singl.eView Convergent Billing payment batches cannot include payments of more than one payment type. As a result, if the PAP Auto Payment Generation schedule type generates both credit card and direct debit payments in one run, separate payment batches will be created for each type. A naming convention will identify these batches as belonging to the same run. For example, PAP-000123-C could be the payment batch number for pre-authorised credit card payments from a particular run, and PAP-000123-D could be the payment batch number for pre-authorised direct debit payments from the same run. The naming convention for the credit card and direct debit payment types must be defined in conjunction with the ADC deployment team on the *Payment Batch Format* field of the *Payment Type Definition* form. For more information on configuring payment types, refer to *Chapter9, Accounting Elements* in the *Customer Care Configuration Guide for Singl.eView Convergent Billing*.

Source of customer care PAP information

Pre-authorised payments are set up for individual customers by the Customer Service Representative (CSR) on the *Pre-Authorised Details* tab of the *Customer Detail* form. The details indicated on this form can include the following:

Detail Type	Values
Payment Method	
	Pre-authorised Credit Card
	Pre-authorised Direct Debit
Suspension information (for temporarily suspending pre-authorised payments)	
	Pre-authorised Credit Card – Suspended
	Pre-authorised Debit Card – Suspended

Pre-authorised credit card details	
	Credit Card Type
	Credit Card Number
	Credit Card Expiry Date
Pre-authorised direct debit details	
	Banking Name
	Branch
	Bank Account Number
	Bank Account Name
	Payment Limit (optional maximum limit for pre-authorised direct debit payments)

Criteria for generating pre-authorised payments

Payment batches are generated for pre-authorised payments due on the day that the schedule is run or within the number of days specified in the *Extra Days* field on the *Parameters* tab of the *Schedule Definition* form. The payment due date used as the criteria for processing payments in any particular run defaults to the day that the run is scheduled unless the number of *Extra Days* is specified. In that case, the payment due date that is used is the effective date of the run, plus the number of days in the *Extra Days* field. In addition, only payments for customers with the following criteria will be processed:

- A Report Level of Invoice on the Invoicing/Reporting tab of the *Customer Detail* form.
- A Payment Method of Pre-authorised Direct Debit or Pre-Authorised Credit Card on the Pre-Authorised Details tab of the *Customer Detail* form.

Payments will not be processed in the following cases:

- The customer has a Payment Method of Pre-authorised Credit Card – Suspended or Pre-authorised Direct Debit – Suspended on the *Pre-Authorised Details* tab of the *Customer Detail* form. This indicates that the CSR has temporarily suspended pre-authorised payments.
- The customer has a Credit balance.

Amount of payment

Once all payments have been created, they will be posted against the customer's accounts. The amount of the payment detail that is created will depend in part on any adjustments, other payments, and disputed amounts.

The amount of each payment generated also depends on the type of pre-authorised payment.

If the customer has chosen to pay with pre-authorised credit card, the payment will be created for the entire amount owing.

If the customer has chosen to pay by direct debit from a bank account, the customer has an option to specify a Payment Limit. For further information, refer to *Source of customer care PAP information* earlier in this section. The amount of the payment will be created with either the total amount owing or the pre-authorised limit, whichever is less.

For example, if the customer has set $100.00 as the pre-authorised direct debit limit and the customer's invoice is $87.56, the payment will be for $87.56. However, if the invoice was $187.56, the payment created will be $100.00.

Output results

The success or failure of the run can be determined by checking the *Task Status* field and the *Results* tab of the *Task Details* form. The task ID is also displayed on the *Results* tab and should be noted, in case the task needs to be revoked and re-submitted. For more information on how to access this data, refer to *Using Schedules* in *Chapter 2, Scheduling Tasks*. Alternatively, the output results can be sent by email. For information about this feature, refer to *Fields* in *Chapter 2, Scheduling Tasks.*

PAP Payment File Generation Schedule Type – Pre-authorised Payments

The PAP Payment File Generation schedule type is the second of two schedule types that are used in succession to automatically generate payment batches for pre-authorised payments, and create an outbound file to be transferred to a bank or clearinghouse:

1. The PAP Auto Payment Generation schedule type generates the pre-authorised payment batches.
2. The PAP Payment File Generation schedule type, which can be automatically triggered by the first schedule, creates the files containing the payment details to be transferred. (To transfer the files via FTP, the FTP Exchange schedule can be used.)

This section discusses the second schedule type. For information on the first schedule type, refer to *PAP Auto Payment Generation Schedule Type* earlier in this chapter.

Scheduling

It is recommended that the PAP Auto Payment Generation schedule be run once per day (for more information, refer to *Scheduling* in the section *PAP Auto Payment Generation Schedule Type* earlier in this chapter).

The second schedule type, the PAP Payment File Generation process, can be automatically triggered by the first schedule and thus would be run once per day as well.

It is also possible to run this second process separately. In that case, the schedule type should be run each time the first process is run.

Actions triggered by running the PAP Payment File Generation schedule for pre-authorised payments

The PAP Payment File Generation schedule type will create an outbound file or files that contain payment details for all the pre-authorised payments created by the PAP Auto Payment Generation. The format of the standard file with no customisations is compatible with tools, such as CreditNow and ICVERIFY that are used to transfer the outbound files to a bank or clearinghouse. The file format can be customised to be compatible with other third party software.

The file or files can be transferred using the FTP Exchange schedule type.

Outbound file options

The dCMN_PAP_Filename$ derived attribute determines the type and number of outbound files that are created and also determines the naming convention for the files. This derived attribute will be configured for each client by the ADC deployment team. The process can create three types of outbound files:

1. A single file with embedded batches containing all pre-authorised credit card and direct debit payments
2. Individual files broken down by the type of payment.
3. Individual files broken down by the type of payment and further broken down by bank or credit card type.

For more information on the format of the outbound files, refer to *Appendix B, Outbound File Format for Pre-authorised and Credit Card Payments*.

Rejected payments

After the outbound files are transferred to the bank or clearinghouse, if any payments are rejected, these must be manually rejected in Singl.eView Convergent Billing. For instructions on how to reject a payment, refer to the Online Help (Reject a Payment procedure under How Do I/Customer Care/Payments). To reject the payment, the payment batch must be searched for. The applicable payment batch should be identified by a payment batch number with a standard format for pre-authorised credit card or pre-authorised debit card payments. The naming convention for the credit card and direct debit payment types must be defined in conjunction with the ADC deployment team on the *Payment Batch Format* field of the *Payment Type Definition* form. For more information on configuring payment types, refer to *Chapter 9, Accounting Elements* in the *Customer Care Configuration Guide for Singl.eView Convergent Billing.*

Output results

The success or failure of the run can be determined by checking the *Task Status* field and the *Results* tab of the *Task Details* form. The task ID is also displayed on the *Results* tab and should be noted, in case the task needs to be revoked and re-submitted. For more information on how to access this data, refer to *Using Schedules* in *Chapter 2, Scheduling Tasks,*. Alternatively, the output results can be sent by email. For information about this feature, refer to *Fields* in *Chapter 2, Scheduling Tasks.*

In addition, the following information will be displayed on the *Results* tab:

- Names of the files created
- Total amount for the payments processed
- Subtotals for each payment type, for each batch, and for each file.

PAP Revoke Process

In case of an error, the PAP Revoke Process schedule may be run. For more information on this schedule type and the applicable restrictions, refer to *PAP Revoke Process Schedule Type* later in this chapter.

PAP Revoke Process Schedule Type

The PAP Revoke Process schedule type can be run if there are errors encountered when running the PAP Auto Payment Generation schedule and the PAP Payment File Generation schedule or if these schedules fail. For more information on checking for errors in a run or for failed runs, refer to this section and the *PAP Payment File Generation Schedule Type* section under *Output results*.



The PAP Revoke Process schedule type can only be run after both the PAP Auto Payment Generation and the PAP Payment File Generation schedule types have been run.

Scheduling

The PAP Revoke Process schedule can be run as an immediate task when errors are encountered. For more information on how to schedule an immediate task, refer to *Scheduling an immediate one-off task* in *Chapter 2, Scheduling Tasks*.

Scheduling parameters

To run this schedule, one mandatory parameter must be specified on the *Parameters* tab of the *Schedule Definition* form. The task ID of the PAP Auto Payment Generation schedule to be revoked must be specified on the *Schedule to output* field. The appropriate task ID can be chosen from the drop-down list.

Actions triggered by running the PAP Revoke Process schedule

When this schedule is run, the effects of the PAP Auto Payment Generation and PAP Payment File Generation schedules are reversed:

1. Any batch payments that have been created are removed.
2. Any outbound files that have been created are deleted.
3. If no outbound files are found, the process will fail. Consequently, the PAP Payment File Generation schedule must be run before the PAP Revoke Process schedule is run.

Criteria for running the PAP Revoke Process schedule

The PAP Revoke Process will fail in the following situations:

- If any payments have been included on a subsequent invoice
- When no output files are found in the database
- If the database fails to delete the records from the tables.

Output results

The success or failure of the run can be determined by checking the *Task Status* field and the *Results* tab of the *Task Details* form. The task ID is also displayed on the *Results* tab and should be noted, in case the task needs to be revoked and re-submitted. For more information on how to access this data, refer to *Using Schedules* in *Chapter 2, Scheduling Tasks,*. Alternatively, the output results can be sent by email. For information about this feature, refer to *Fields* in *Chapter 2, Scheduling Tasks.*

PAP Confirm Letter Generation Schedule Type

The PAP Confirm Letter Generation schedule generates confirmation letters to be sent to customers who have set up pre-authorised payments, had a change in the details of these payments (including termination), suspended the payments, or re-instated the payments. Once generated, the confirmation letters are stored in the database. The General Bulk Output Process schedule must be run to print the letters.

Scheduling

The PAP Confirm Letter Generation schedule can be automatically triggered by the PAP Auto Payment Generation schedule. This is dependent on the entry in the *Server Command* field on the *Schedule Type Definition* form for the PAP Auto Payment Generation schedule type. (papgenpay –p= generate letters automatically, papgenpay=do not generate letters automatically).

If there is a need for letters to be reprinted, the PAP Confirm Letter Generation schedule can be launched independently after the PAP Confirm Letter Revoke schedule is run. For more information, refer to *PAP Confirm Letter Revoke Schedule Type* in the next section.

To re-generate and re-print a batch of letters, these steps must be followed:

- Run the PAP Confirm Letter Revoke schedule using the task ID for the original task.
- Re-submit the original task for the PAP Confirm Letter Generation schedule type.
- Run the General Bulk Output Process to print the letters. For more information, refer to *Chapter 7, Schedule Types* of the *System Operations Guide for Singl.eView Convergent Billing*.

The task ID for the original task must be used to revoke and re-submit the task and can be found on the *Results* tab of the *Task Details* form for the task that originally launched this schedule. For more information on how to access the task ID, refer to *Using Schedules* in *Chapter 2, Scheduling Tasks*. For more information on re-submitting a task, refer to *Re-submitting a task* in *Chapter 6, Scheduling Server Tasks* of the *System Operations Guide for Singl.eView Convergent Billing*.

Scheduling parameters

Before this schedule is launched, either automatically by the PAP Auto Payment Generation schedule or independently, the following mandatory and optional parameters should be configured on the *Parameters* tab of the *Schedule Definition* form:

1. A code that identifies the letter template to be used in creating the letter should be selected in the *Letter Template* field (mandatory). One letter template is provided as part of the core software and others can be configured on request as part of a deployment package.
2. It must be indicated whether letters should be generated for adds, changes and/or suspends in the *Process Adds*, *Process Changes*, and *Process Suspends* fields (mandatory).
3. The size of the run can be limited by specifying the earliest pre-authorised details for which letters should be processed in the *Earliest Date* field (optional). This field defaults to one week before the current (system) date because it is assumed that the process will be run about once per week. However, this date can be changed. The date is based on the date the details were inserted into the Update Capture, not on the effective or termination date of the detail.

Actions triggered by running the PAP Confirm Letter Generation schedule

Running the PAP Confirm Letter Generation schedule, either automatically as part of the PAP Auto Payment Generation schedule or independently, will generate the printing of confirmation letters for customers who meet any of the following criteria:

1. They have been set up for pre-authorised credit card or direct debit payments (adds).
2. They have changed some of the details of those payments. An example would be if a customer changed the *Payment Limit* for pre-authorised direct debit payments (changes).
3. They have terminated arrangements for those payments (changes).
4. They have suspended those payments or re-instated the payments after a suspension (suspends).

Letters will be generated for any records that have not been processed by previous runs of the schedule and that fall within the parameters set on the *Parameters* tab of the *Schedule Definition* form.

Source of customer care pre-authorised payment information

Pre-authorised payments are set up for individual customers by the CSR on the *Pre-Authorised Details* tab of the *Customer Detail* form. For information on the details indicated on this form, refer to *PAP Auto Payment Generation Schedule Type* earlier in this chapter.

Output results

The success or failure of the run can be determined by checking the *Task Status* field and the *Results* tab of the *Task Details* form. The task ID is also displayed on the *Results* tab and should be noted, in case the task needs to be revoked and re-submitted. For more information on how to access this data, refer to *Using Schedules* in *Chapter 2, Scheduling Tasks.* Alternatively, the output results can be sent by email. For information about this feature, refer to *Fields* in *Chapter 2, Scheduling Tasks.*

PAP Confirm Letter Revoke schedule type

In case of an error, the PAP Confirm Letter Revoke schedule may be run. For more information on this schedule type, refer to *PAP Confirm Letter Revoke Schedule Type* in the next section.

PAP Confirm Letter Revoke Schedule Type

In case of an error or failure of the PAP Confirm Letter Generation schedule, this schedule can be run to reverse the effects of the letter generation process. The errors can then be corrected, and the PAP Confirm Letter Generation process can be run again.

Scheduling

Typically, this schedule is run as an immediate task when errors are encountered. For more information on how to schedule an immediate task, refer to *Scheduling an immediate one-off task* in *Chapter 2, Scheduling Tasks*.

Scheduling parameters

To launch the PAP Confirm Letter Revoke process, one mandatory parameter must be specified on the *Parameters* tab of the *Schedule Definition* form. The task ID for the schedule that needs to be revoked must be specified in the *Task to Revoke* field.

Actions triggered by running the PAP Confirm Letter Revoke schedule

When this schedule is run, the effects of the PAP Confirm Letter Generation schedule are reversed. Any letters that have been created are deleted from the database.

Output results

The success or failure of the run can be determined by checking the *Task Status* field and the *Results* tab of the *Task Details* form. The task ID is also displayed on the *Results* tab and should be noted, in case the task needs to be revoked and re-submitted. For more information on how to access this data, refer to *Using Schedules* in *Chapter 2, Scheduling Tasks*. Alternatively, the output results can be sent by email. For information about this feature, refer to *Fields* in *Chapter 2, Scheduling Tasks.*

Process Lockbox Files Schedule Type – Inbound Payments

Unlike the schedules discussed in the previous sections for outbound payments (such as credit card, pre-authorised credit card and pre-authorised direct debit), the Process Lockbox Files schedule is used to process an inbound file that contains records for payments that were made through a third party (such as a bank). The transfer of the lockbox file from the third party to Singl.eView Convergent Billing can be done through the FTP Exchange schedule type.

There is a derived attribute, reference type, and function that must be set up and maintained before this schedule type is configured. The derived attribute and associated reference type identify the type and format of incoming files that will be processed by this schedule. The function contains fundamental default information for processing lockbox payments. For more information, refer to *Appendix D, Configuration for Process Lockbox Files Schedule Type*.

Scheduling

Typically, the Process Lockbox Files schedule type is run as an immediate task after an inbound file has been received in Singl.eView Convergent Billing. For more information on how to schedule an immediate task, refer to *Scheduling an immediate one-off task* in *Chapter 2, Scheduling Tasks*.

Scheduling parameters

To run the Process Lockbox Files schedule, two mandatory parameters must be specified on the *Parameters* tab of the *Schedule Definition* form:

1. The name of the incoming file must be entered in the *Lock Box File Name* field.
2. The type of incoming file (usually identified by the name of the third party that collected the payments included in the file) must be selected from the drop-down list in the *Lockbox File Type* field.

Actions triggered by running the Process Lockbox Files schedule

When the Process Lockbox Files schedule is launched, the payment records contained in the flat file that has been received from the third party are processed. Batch payments are created, and after several checks are performed, including a check to ensure that the designated account exists in the Singl.eView Convergent Billing database, the payments are posted to the appropriate customer accounts. Regardless of the file format of the incoming file, the mandatory data needed to complete a successful payment includes the following:

Data	Description
Account number	Alphanumeric code that identifies the customer account to which the payment will be posted.
Payment date	Date when the third party payment.
Amount	Dollar amount of the payment.
Cheque bank identifier	Alphanumeric code that identifies the financial house.
Cheque number	Identifying number printed on the cheque.
Invoice number	Invoice that the payment is associated with.

Future-dated payments

Some of the payments received in the lockbox file will be for future dated payments. These payments will be posted to the customer's account as of the date the file is processed by the Process Lockbox Files schedule, not as of the payment date. Given that the payment will be posted as soon as it is successfully processed, the account balance will not be accurate. However, the history record for the payments will be accurate.

Rejected payments

After the lockbox file is processed, if any payments are subsequently rejected, they must be manually rejected in Singl.eView Convergent Billing. For instructions on how to reject a payment, refer to the Online Help (Reject a Payment procedure under How Do I/Customer Care/Payments). To reject the payment, the payment batch must be searched for. The applicable payment batch should be identified by a payment batch number with a standard format for lockbox payments. The standard format will be determined for each implementation in conjunction with the ADC deployment team.

Output results

The success or failure of the run can be determined by checking the *Task Status* field and the *Results* tab of the *Task Details* form. The task ID is also displayed on the *Results* tab and should be noted, in case the task needs to be revoked and re-submitted. For more information on how to access this data, refer to *Using Schedules* in *Chapter 2, Scheduling Tasks*. Alternatively, the output results can be sent by email. For information about this feature, refer to *Fields* in *Chapter 2, Scheduling Tasks.*

Errors and clean-up

If any errors are encountered during a run, the Process Lockbox Files schedule will automatically fail and any batches created as a result of the schedule will be deleted. However, the process will not terminate on the first error that is encountered. Instead it will continue to process the file in an attempt to detect any additional errors.

The *Results* tab of the *Task Details* form will display information on the errors encountered, the reason the run failed, and the number of successful records.

In case of failure, clean-up of the records is automatically performed. As a result, after the errors have been investigated and arrangements have been made to correct the failed payment file, the task can be re-submitted. For more information on re-submitting a task, refer to the *System Operations Guide, Chapter 6, Scheduling Server Tasks, Working with schedules, Re-submitting a task.*

Duplicate checking

The Process Lockbox Files schedule will check for duplicate records at three levels:

1. File name
2. File header (where applicable)
3. Batch header (where applicable)

No duplicate checks at the transaction level will be performed. For more information on the standard format and components of incoming lockbox files, refer to *Appendix C, Inbound File Format for Lockbox*.

FTP Exchange Schedule Type

The File Transfer Protocol (FTP) Exchange schedule type can be used to transfer any file or group of files from an external system into Singl.eView Convergent Billing or to transfer a file or group of files from Singl.eView Convergent Billing to an external system. This schedule is described in *Chapter 17, Payment Interfaces and Schedule Types* because it is useful for transferring inbound lockbox payments files, outbound pre-authorised payments, and credit card payments files. However, it can also be used for non-payment files. FTP is the only medium (eg. tape, CD) for file transfer for which Singl.eView Convergent Billing provides a schedule type.

To configure the FTP Exchange schedule type, a directory rather than a file name is specified. A mask is also specified so that only files of a specific file type will be transferred. The transfer formats available are ASCII and binary.

A user name and password must be set up for access to the remote machine or server before using this schedule to transfer inbound or outbound files.

An archive directory with sufficient file space must be set up on the local machine or server when the FTP Exchange schedule is used for outbound transfers. The transferred files will be saved to this directory when the transfer is complete.

Scheduling

Typically, this schedule is run as an immediate task when there is an incoming or outgoing file to be transferred. For more information on how to schedule an immediate task, refer to *Chapter 2, Scheduling Tasks, Using Schedules, Scheduling an immediate one-off task*.

Scheduling parameters

To run the FTP Exchange schedule, several mandatory parameters must be specified on the *Parameters* tab of the *Schedule Definition* form:

1. The direction of the transfer must be indicated in the *Transfer Direction* field.
2. It must be indicated whether incoming files should be decompressed or whether outgoing files should be compressed in the *Compress/Decompress?* field. If compression is needed, the *Compression Method* and *Compression File Extension* must also be indicated.

The *Compression Method* field specifies the command that will be executed on the UNIX server to compress or decompress files. The value entered in this field should be the full UNIX command line, but without the file name. The file name will be appended to the value specified when the compression program is run.

3. A *Transfer Format* of ASCII or Binary must be indicated.
4. The local and remote machine names and directories must be specified in the respective fields.
5. A *User Name* and *Password* should be specified for logging on to the remote machine or server.
6. As mentioned earlier in this section, a *File Mask* must be specified to indicate the types of files in the local or remote directory that should be transferred.
7. The *Directory Listing* field can be used to indicate directories that already contain transferred files. Then when the schedule is run, any files contained in these directories will not be transferred.
8. As mentioned earlier, an *Archive Directory* must be specified.
9. The *Retries* field specifies the number of times the process will retry the transfer of a file, if it fails to be successfully transferred the first time.



For more information on each parameter, refer to the context-sensitive help. On the *Parameters* tab of the *Schedule Definition* form for the FTP Exchange schedule type, place the cursor in any field and press the **F1** key. A help topic for that field will be displayed.

Actions triggered by running the FTP Exchange schedule

When the FTP Exchange schedule is run, a file or group of files located in a specified directory on a remote or local server or machine are transferred. If the first attempt to transfer the files is not successful, the process will be re-launched the number of times specified in the *Retries* field on the *Parameters* tab of the *Schedule Definition* form. For example, if 3 is specified in the *Retries* field, and the schedule fails the first time it is run, it will be launched up to three more times or until the transfer is complete, whichever comes first. For this reason, a list of directories that contain transferred files must be specified in the *Directory List* field. Any files that have successfully been transferred and which are contained in these directories will not be transferred in subsequent runs.

Output results

The success or failure of the run can be determined by checking the *Task Status* field and the *Results* tab of the *Task Details* form. The task ID is also displayed on the *Results* tab and should be noted, in case the task needs to be revoked and re-submitted. For more information on how to access this data, refer to *Using Schedules* in *Chapter 2, Scheduling Tasks.* Alternatively, the output results can be sent by email. For information about this feature, refer to *Fields* in *Chapter 2, Scheduling Tasks.*

Part 2
Specific Topics for Singl.eView IP/Data Functionality

This part covers topics that apply only to the Singl.eView IP/Data functionality. It consists of the following five chapters:

- *Chapter 18, RADIUS Accounting Real Time Rating Process*
- *Chapter 19, Singl.eView IP/Data Updates for Vertex*
- *Chapter 20, Automatic Provisioning: Troubleshooting Guide*
- *Chapter 21, IP/Data Event Feeds and Normalised Event Layouts*
- *Chapter 22, Speed Value Management*.

Chapter 18
RADIUS Accounting Real Time Rating Process

This chapter provides an overview and detailed information about the RADIUS Accounting Real Time Rating process. The following topics are covered:

- Introduction to the RADIUS Server and Real Time Rating
- The RADIUS Accounting Real Time Rating Process and Singl.eView Convergent Billing
- Internal and External Components of the RADIUS Accounting Real Time Rating Process
- Sockets and the RADIUS Accounting Real Time Rating Process
- File Components of the RADIUS Accounting Real Time Rating Process
- Understanding and Running the RADIUS Accounting Real Time Rating Process.

Introduction to the RADIUS Server and Real Time Rating

RADIUS is software loaded into a server that acts as the authentication server for remote users. This RADIUS method of authentication is often used in conjunction with second generation firewall security to form the second layer of a company's network security. The primary method is the perimeter router security that channels unidentified and unwanted traffic away from its network.

In addition to authentication, the RADIUS Server generates accounting start and stop records that track connection time for each event. This information is often used for billing purposes. When the user is authenticated and the session has been configured according to the authorisation information, an accounting start record is created. When the user's session is terminated, an accounting stop record is created. The stop record includes the Acct-Session-Time record.

Figure 18-1 shows the RADIUS server in context with other network system components.

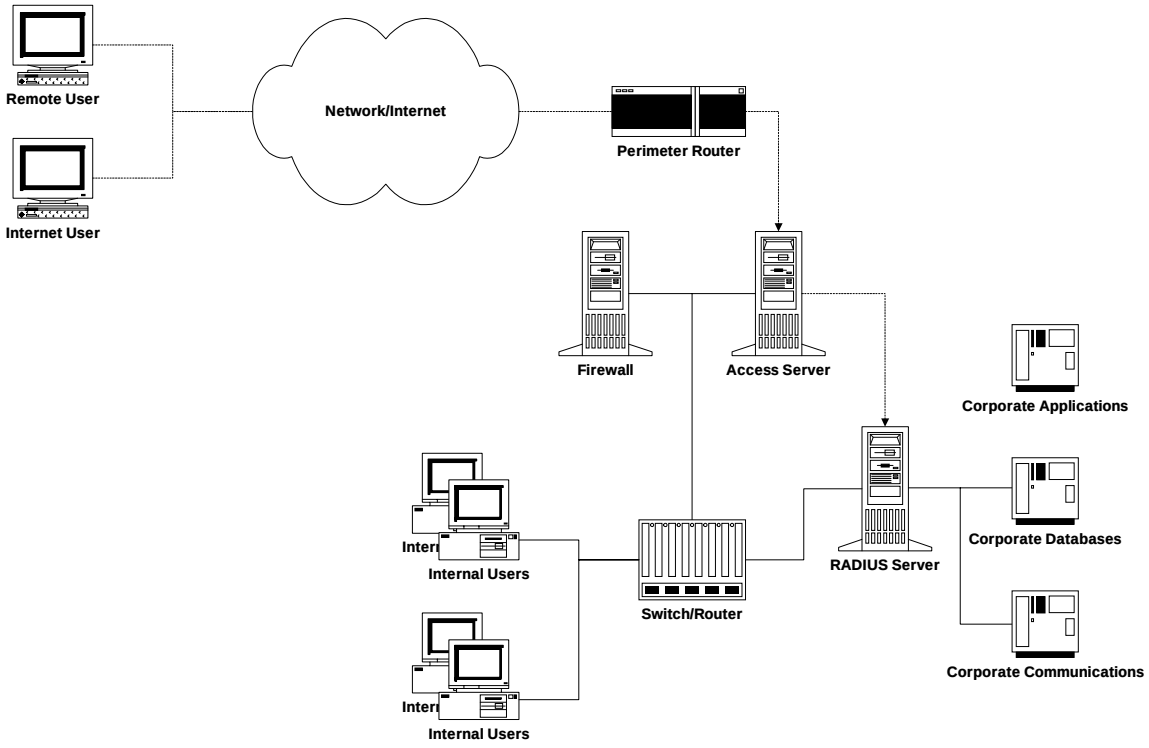


Figure 18-1 RADIUS Server in Relation to Other Network Components

The RADIUS server and real time rating

The greatest business need for RADIUS Accounting Real Time Rating is to perform real time rating on data and IP based usage.

The explosion of the Internet and other IP based services has created the need for ISP and Service Providers to track and cost data services in real time.

For example, if an internet usage plan allows the first $100 of internet usage at no cost and charges $2.50 per hour thereafter, the provider would need to provide up to the minute information to its customers on their current consumption.

RADIUS protocol

The network access server(s) accepts a user's connection request and forwards each request to the RADIUS server. The RADIUS server contains a database for authenticating remote connection requests. The approved or denied results of the validation process are returned to the Network Access server. The network access server is the router or access server that allows or disallows access.

The purpose of the RADIUS server is dual; the primary purpose is authentication, and the secondary purpose is to track connection time. It is this very important connection information that is used to create RADIUS Accounting start and stop records. Together the start and stop records are similar to an event detail record that is generated by traditional telecommunication switches.

The RADIUS Accounting Real Time Rating Process and Singl.eView Convergent Billing

The RADIUS Accounting Real Time Rating Process is an external process to the billing system, but is a composite part of the Singl.eView Convergent Billing product. Real-time rating of IP/Data services is critical in today's fast paced telecommunications industry.

The network access server(s) identifies access requests and passes them on to the RADIUS server. Figure 18-2 depicts only one example of how the RADIUS Accounting Real Time Rating Process can be implemented. Other possible variations of the RADIUS Real Time Rating Process are to have the it running on a network access server, or on the Singl.eView Convergent Billing server.

Figure 18-2 illustrates how the RADIUS Accounting Real Time Rating Process fits in with the Singl.eView Convergent Billing Rater and the network access server.

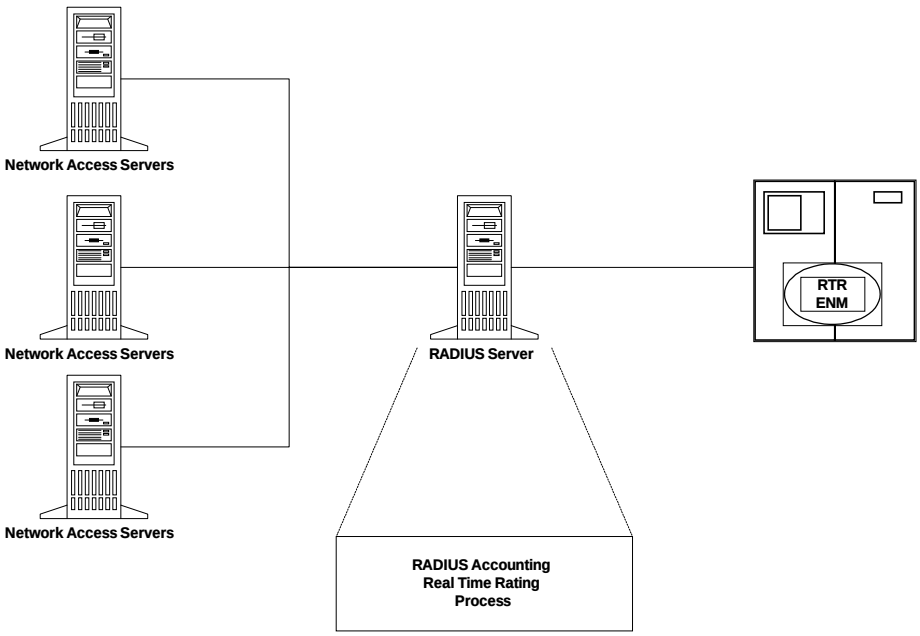


Figure 18-2 RADIUS Accounting Real Time Rating Process in Relation to Network Access Servers and Singl.eView Convergent Billing Rater

The RADIUS server's normal process of authenticating users and then storing their connection information in the server's memory storage is enhanced to directly pass the start and stop accounting records onto Singl.eView Convergent Billing. The enhancement is made possible with the introduction of the Real Time Rating process. The details of this process will be explained later; however, the more important point to note here is that the Real Time Rating process will enable a provider or ISP to rate usage in real-time.

While traditional event capture is performed real-time but recorded/stored en masse and processed in the same manner, the addition of the RADIUS Real Time Rating to the RADIUS server process will cause each valid authenticated event to be sent to the billing system for immediate processing.

RADIUS Accounting start and stop records

The following are examples of accounting start and stop records. The attributes in these records are defined in the Data Dictionary. For more information, refer to *File Components of the RADIUS Accounting Real Time Rating Process* later in this chapter. For more information on RADIUS event feeds, refer to *Non-Native Event Feeds* in *Chapter 21, IP/Data Event Feeds and Normalised Event Layouts*.

RADIUS accounting start and stop records for login user

The following code sample is an example start record in a RADIUS server detail file.

```
Tue Jul 30 14:48:18 1996
Acct-Session-Id = "AC000004"
User-Name = "jamie"
NAS-IP-Address = 172.16.64.91
NAS-Port = 1
NAS-Port-Type = Async
Acct-Status-Type = Start
Acct-Authentic = RADIUS
Service-Type = Login-User
Login-Service = Telnet
Login-IP-Host = 172.16.64.25
Acct-Delay-Time = 0
Timestamp = 838763298
```

The `Acct-Status-Type` attribute in the record indicates whether the record was sent when the connection began ('Start') or when it ended ('Stop'). The `Acct-Session-Id` is listed at the beginning of the record.

Note: This value matches the `Acct-Session-Id` of the stop record on the following page, indicating that these records correspond to the same session.

User-Name specifies the username, in this case, 'jamie'. NAS-IP-Address specifies the IP address of the RADIUS server. NAS-Port-Type specifies that this is an asynchronous connection. Acct-Authentic specifies that jamie is authenticated via RADIUS. Service-Type and Login-Service specify that jamie is a login user using Telnet. Login-IP-Host specifies the host into which user jamie logged in.

The following code sample is an example stop record that is associated with the start record on the previous page. The Acct-Session-Id of the stop record matches that of the start record on the previous page, indicating that these records correspond to the same session.

```
Tue Jul 30 14:48:39 1996
Acct-Session-Id = "AC000004"
User-Name = "jamie"
NAS-IP-Address = 172.16.64.91
NAS-Port = 1
NAS-Port-Type = Async
Acct-Status-Type = Stop
Acct-Session-Time = 21
Acct-Authentic = RADIUS
Acct-Input-Octets = 22
Acct-Output-Octets = 187
Acct-Terminate-Cause = Host-Request
Service-Type = Login-User
Login-Service = Telnet
Login-IP-Host = 172.16.64.25
Acct-Delay-Time = 0
Timestamp = 838763319
```

In the stop accounting record, Acct-Session-Time specifies that user jamie's connection lasted 21 seconds. Acct-Input-Octets indicates that 22 bytes of incoming traffic were received; Acct-Output-Octets indicates that 187 bytes of outgoing traffic were sent.

The Acct-Terminate-Cause indicates that a Host-Request terminated the session, meaning that jamie logged off the host or that the host logged him off. The Acct-Delay-Time is 0 seconds, indicating that the RADIUS accounting server received the accounting-request on the first try.

RADIUS accounting start and stop records for PPP user

The following is an example of a start record in a RADIUS server detail file. The start record is for an ISDN PPP connection.

```
Tue Jul 8 08:44:17 1997
Acct-Session-Id = "1A00014E"
User-Name = "wallace"
NAS-IP-Address = 192.168.32.1
NAS-Port = 0
NAS-Port-Type = Async
Acct-Status-Type = Start
Acct-Authentic = RADIUS
Connect-Info = "33600 LAPM/V42BIS"
Called-Station-Id = "5557026"
Calling-Station-Id = "5105550285"
Service-Type = Framed-User
```

```
Framed-Protocol = PPP
Framed-IP-Address = 192.168.32.35
Acct-Delay-Time = 0
Timestamp = 868376657
```

The `NAS-Port-Type` specifies that the user wallace has an asynchronous connection. `Called-Station-Id` and `Calling-Station-Id` specify the destination and source of the call. `Service-Type` and `Framed-Protocol` indicate that user wallace is a framed user establishing the connection via PPP.

This example is the stop record associated with the start record on the previous page. The stop record indicates that the `Acct-Session-Time` for user wallace was 67 seconds. The `Acct-Input-Octets` and `Acct-Output-Octets` indicate that the incoming traffic for this session was 5877 bytes, and outgoing traffic was 2418 bytes.

```
Tue Jul 8 08:45:24 1997
Acct-Session-Id = "1A00014E"
User-Name = "wallace"
NAS-IP-Address = 192.168.32.7
NAS-Port = 0
NAS-Port-Type = Async
Acct-Status-Type = Stop
Acct-Session-Time = 67
Acct-Authentic = RADIUS
Connect-Info = "33600 LAPM/V42BIS"
Acct-Input-Octets = 5877
Acct-Output-Octets = 2418
Called-Station-Id = "5557026"
Calling-Station-Id = "5105550285"
Acct-Terminate-Cause = User-Request
Service-Type = Framed-User
Framed-Protocol = PPP
Framed-IP-Address = 192.168.32.35
Acct-Delay-Time = 0
Timestamp = 868376724
```

Internal and External Components of the RADIUS Accounting Real Time Rating Process

RADIUS Accounting Real Time Rating process components overview

The RADIUS Accounting Real Time Rating Process is made up of the following components:

External components

- Network Access Servers
- TRE service via socket server
- Real Time Rating ENM.

Internal components

- Controller
- Real Time Rating Socket Connection Manager
- Input Thread
- Output Thread
- Controller Thread
- Queue.

Socket components

- UDP Socket
- Real Time Rating Socket
- Controller Socket.

File components

- Configuration File
- Secrets File
- Data Dictionary
- Restart Data File
- Log File
- Batch File
- Error Batch File.

Figure 18-3 shows all of the essential components for the RADIUS Real Time Rating Process.

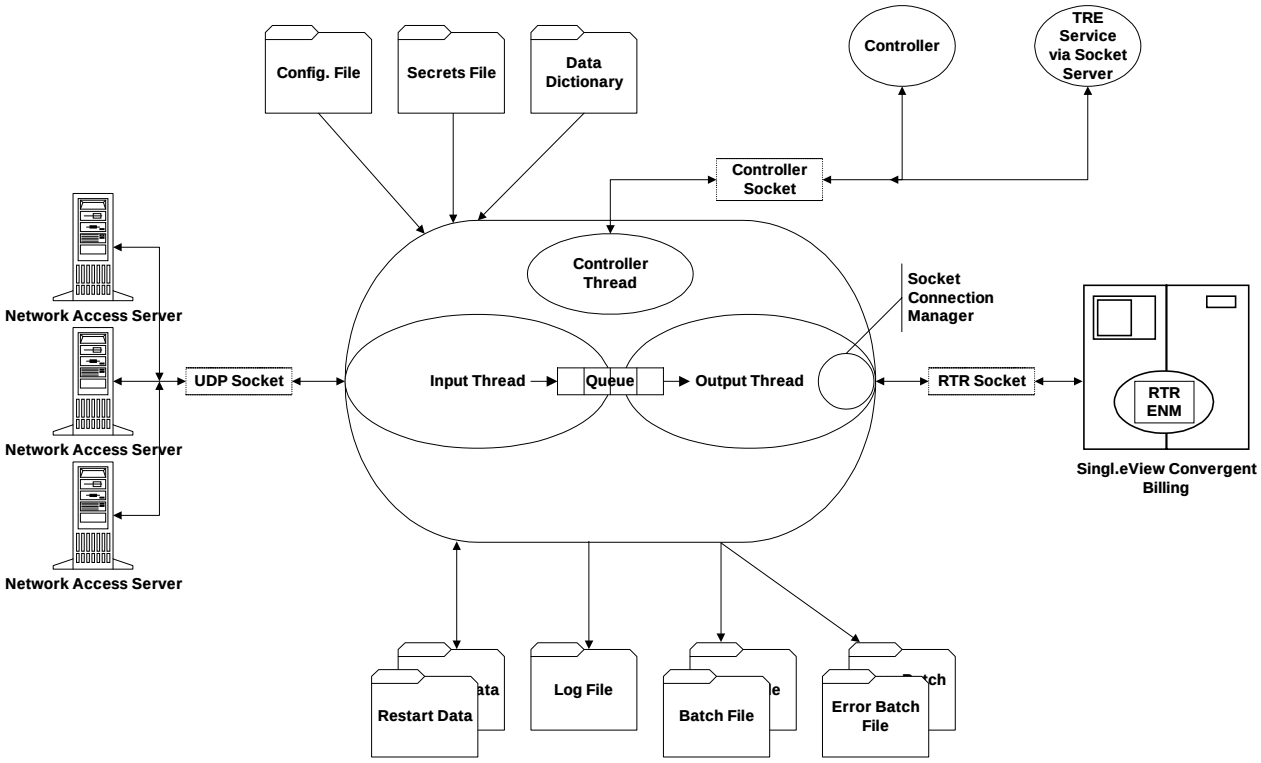


Figure 18-3 Components of RADIUS Accounting Real Time Rating

External components

Network access servers

The Network Access Servers are generally external systems that allow or disallow remote connections to the network. It is these servers that start the real-time rating process in motion by sending an accounting request when a user attempts to connect and again when the user has disconnected.

TRE service via socket server

The Controller is used to interact with the RADIUS Accounting Real Time Rating Process. Future releases of Singl.eView IP/Data will contain a prototype for using the Transaction (TRE) Server to interface with the RADIUS Accounting Real Time Rating Process. The TRE Service is an external component to the RADIUS Accounting Real Time Rating Process, and is therefore not critical to the operation installation.

Real time rating Event Normalisation Module (ENM)

This component is the input process of Singl.eView Convergent Billing Real-time Rating (RTR) process. The Real Time Rating ENM is the process that will rate each valid RADIUS Accounting event. Within Singl.eView Convergent Billing, an ENM process can be configured to accept input directly from a socket or sources other than a RADIUS server. For more information on configuring ENM processes, refer to *Chapter 5, Configuring Server Processes* and *Chapter 12, Real Time Rating* in the *System Administration Guide for Singl.eView Convergent Billing*.

Internal components

The RADIUS Accounting Real Time Rating Process is a multi-threaded process consisting of:

- Input Thread
- Output Thread
- Controller Thread.

The Input and Output Threads are linked via an internal queue data structure. Also included in the Internal Components are the Controller and Real Time Rating Socket Connection Manager. The Controller is an interface between the user and the RADIUS Accounting Real Time Rating Process, and the Real Time Rating Socket Connection Manager is an interface between the RADIUS Accounting Real Time Rating Process and the Singl.eView Convergent Billing Real Time Rating ENM. These components are shown in figure 18-3.

Controller

Not to be confused with the Controller Thread, the Controller is an applet used to control the operation of the RADIUS Accounting Real Time Rating Process. Its function is to provide Status, Refresh, Reconnect, and Shutdown actions and services to the system operator via a GUI interface.

Real Time Rating Socket Connection Manager

The Real Time Rating Socket Connection Manager is responsible for connecting, and monitoring the connection with the Singl.eView Convergent Billing Real Time Rating ENM. Should the connection attempt fail or if the connection is lost, the Real Time Rating Socket Connection Manager attempts to reconnect with the socket using a timing strategy.

Input Thread

The Input Thread will be discussed in greater detail later in this chapter. It is an essential part of the RADIUS Accounting Real Time Rating Process that handles the receiving of the accounting request packets, decoding and authenticating them. The Input Thread also handles the construction of the response packet.

In the event of errors or system failure, the Input Thread writes its current requests into the restart log. This allows the RADIUS Accounting Real Time Rating Process to re-initialise with the restart log data in place. A response is then sent and a request is placed into the Queue.

Output Thread

The Output Thread will be discussed later in the chapter. Its primary focus is to monitor the Queue and process the accounting request information received. It sends all requests to the Singl.eView Convergent Billing Real-Time Rating ENM Process, receives the acknowledgments, and writes the acknowledgments to the restart log. When the RADIUS Accounting Real Time Rating Process is not connected to the Singl.eView Convergent Billing Real Time Rating ENM process, the accounting requests are received and processed as if the Real Time Rating ENM was available. The only difference is that the requests are written to a batch file and sent to the Real Time Rating ENM when the connection is established.

Controller Thread

The Controller Thread is responsible for listening on the configured port for incoming controller connection requests.

Queue

The Queue is a structured data storage area used to pass accounting requests from the Input Thread to the Output Thread.

Sockets and the RADIUS Accounting Real Time Rating Process

A socket is one endpoint of a two-way communication link between two programs running on the network. A socket is bound to a port number so that the TCP layer can identify the application that data will be sent to.

In client-server applications, the server provides some service, such as processing database queries or sending out current stock information. The client uses the service provided by the server, either displaying database query results to the user or making stock transactions with an investor. The communication that occurs between the client and the server must be reliable. That is, no data can be dropped, and it must arrive on the client side in the same order in which the server sent it.

TCP provides a reliable, point-to-point communication channel that client-server applications on the Internet use to communicate with each other. To communicate over TCP, a client program and a server program establish a connection to one another. Each program binds a socket to its end of the connection. To communicate, the client and the server each reads from and writes to the socket bound to the connection.

Normally, a server runs on a specific computer and has a socket that is bound to a specific port number. The server waits, listening to the socket for a client to make a connection request.

The client knows the hostname of the machine on which the server is running and the port number to which the server is connected. To make a connection request, the client tries to rendezvous with the server on the server's machine and port. Figure 18-4 depicts the connection request from the client to the server.

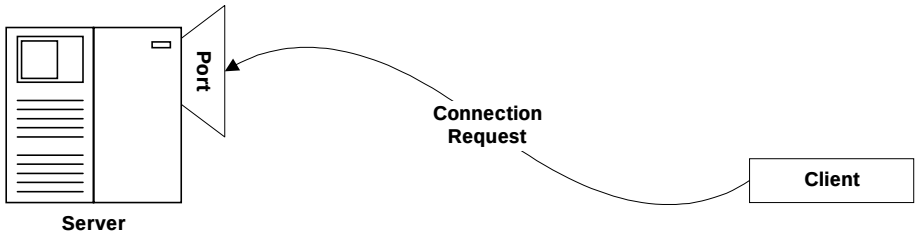


Figure 18-4 Client to Server Connection Request

If the connection is successful, the server accepts the connection. Upon acceptance, the server gets a new socket bound to a different port. It needs a new socket (and consequently a different port number) so that it can continue to listen to the original socket for connection requests, while tending to the needs of the connected client. Figure 18-5 shows that the client is connected to the server via a new socket, not via the original socket that listened for the request.

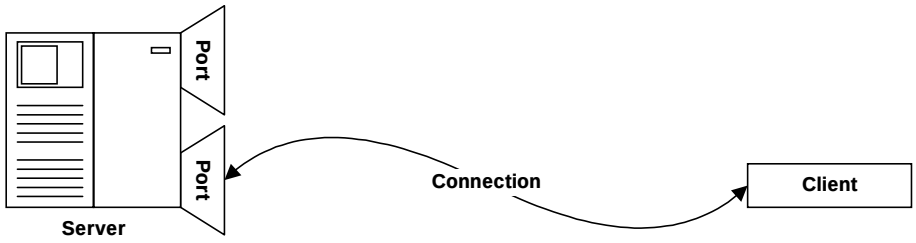


Figure 18-5 Client to Server Connection with Establishment of New Socket

If the connection is accepted, a socket is successfully created, and the client can use the socket to communicate with the server.

Note: The socket on the client side is not bound to the port number used to rendezvous with the server. Rather, the client is assigned a port number local to the machine on which the client is running.

Socket Components

UDP Socket

The UDP Socket shown in Figure 18-3 that listens for access requests from the network access server is a type of connectionless socket. Essentially, a permanent connection is not required or established. The packets are addressed for the destination and sent over the network, similar to a broadcast message. For reference, the RADIUS Accounting socket port is 1813.

Real Time Rating Socket

The Real Time Rating Socket (shown in Figure 18-3) is associated with the Real Time Rating ENM and listens for incoming requests to process the RADIUS accounting start and stop records. The Real Time Rating Socket is a normal, standard socket as shown in figures 18-4 and 18-5. The Singl.eView Convergent Billing Real Time Rating ENM process will have to be configured to listen on a particular port; and the output thread must be configured for the Singl.eView Convergent Billing Host and RADIUS Accounting Real Time Rating Process' port.

Controller Socket

The Controller Socket (shown in Figure 18-3) is also a standard socket. The RADIUS Accounting Real Time Rating process will have to be configured to listen on a particular port, which the controller will specify when making the connection.

File Components of the RADIUS Accounting Real Time Rating Process

All of the file components are shown in figure 18-3.

Configuration File

The Configuration File is used to store information necessary to the RADIUS Accounting Real Time Rating process; this includes socket ports numbers, file paths, and file sizes. For an example, refer to *Appendix A, Configuration File Reference Information and Example*.

Secrets File

The Secrets File is used by the network access server and RADIUS Server to validate that request and response packets have originated from trusted sources, and that nothing has tampered with the packets.

Data Dictionary

The Data Dictionary contains the RADIUS Packet attributes and values, which are used to convert a Datagram Packet into a RADIUS Packet. The accounting attributes appear in the start and stop accounting records. For examples of the attributes in accounting start and stop records, refer to *The RADIUS Accounting Real Time Rating Process and Singl.eView Convergent Billing* earlier in this chapter.

Restart Data

The Restart Data file is an external store. In the event that an abnormal termination occurs after a response has been sent but a Real Time Rating acknowledgment has not been received, this file ensures that request information is not lost. The Restart Data file is intended to assist in recovering from unexpected disconnections or shutdowns that may occur prior to completing all of the connection requests.

Log File

The Log File is used to capture any error or informational messages produced by the RADIUS Accounting Real Time Rating Process, or error messages received from the Real Time Rating ENM Process. This auditing process is managed via a Logger and LoggerListener object.

Batch File

Batch files are created as a backup storage mechanism for the requests in case the Real Time Rating ENM connection is unavailable. A new file is created after a specific number of requests or a specific amount of time has elapsed. The parameters can be maintained through the Configuration File (for more information, refer to *Configuration File* earlier in this section).

Error Batch File

Similar to the Batch File, the purpose of the Error Batch File is to act as a backup storage mechanism for the events that have been rejected by the RADIUS Accounting Real Time Rating Process. Each event that errors out will also have a corresponding error message recorded into the file.

Understanding and Running the RADIUS Accounting Real Time Rating Process

The RADIUS Accounting Real Time Rating Process is started by entering a command on the RADIUS server's command line. The Controller or GUI interface only controls the RADIUS Accounting Real Time Rating process; it is not used to start the process. Once the process begins, it goes through a series of checks and validations before the actual threads are started. Figure 18-6 illustrates the preliminary set up that the process performs each time that it is invoked.

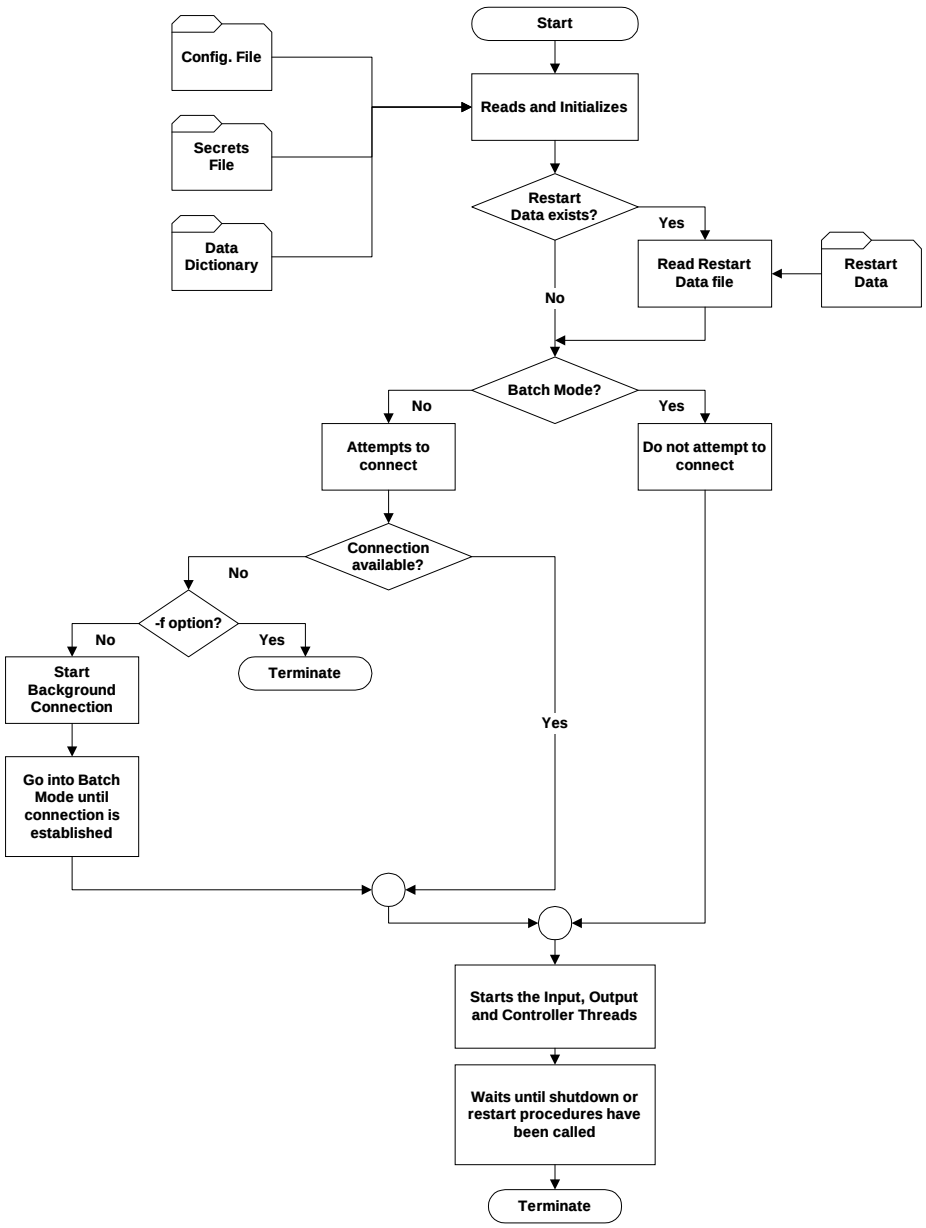


Figure 18-6 RADIUS Accounting Real Time Rating Preliminary Set-Up Process

Starting the RADIUS Accounting Real Time Rating process

To invoke the RADIUS Accounting Real Time Rating process, type the following script onto the server command line:

```
java com.adc.express.rtrclient.RadiusAccountingServer [-n <num>] [-b|-f] <config_dir>
```

where:

`<-n num>`	is the instance sequence number, which identifies the sequence number of the file to be used for configuration. This parameter defaults to one unless it is specified. The instance files are used to customise the configuration of the RADIUS Accounting Real Time Rating Process and ensure that Singl.eView Convergent Billing configuration is not overwritten. For more information, refer to *Appendix A*.
`-b`	indicates that the RADIUS Accounting Real Time Rating Process should be run in Batch Mode. If this parameter is specified, the process will not attempt to connect to the Singl.eView Convergent Billing Real Time Rating ENM.
`-f`	indicates that if a connection is not available to Singl.eView Convergent Billing Real Time Rating ENM, the RADIUS Accounting Real Time Rating Process should fail. If this parameter is not specified during start up, and a connection to the Singl.eView Convergent Billing Real Time Rating ENM is not available, a background process is started to continually try to connect with the Real Time Rating ENM.
`<config_dir>`	is the Configuration directory, which is the location of the RADIUS Accounting Real Time Rating Process Configuration File, Secrets File, and RADIUS packet Data Dictionary.

Controlling the RADIUS Accounting Real Time Rating process after start up

An applet is available with Singl.eView IP/Data to control the RADIUS Accounting Real Time Rating Process. The applet is referred to as the Controller and is used to control the RADIUS Accounting Real Time Rating Process after it has been started. For more information, refer to *Starting the RADIUS Accounting Real Time Rating Process* earlier in this section.

For information on proactive monitoring of the real time rating process, refer to *Monitoring of the Singl.eView RADIUS Accounting Real Time Rating process* in *Chapter 15, Proactive Monitoring*.

Figure 18-7 shows the RADIUS Accounting Server Controller GUI interface.

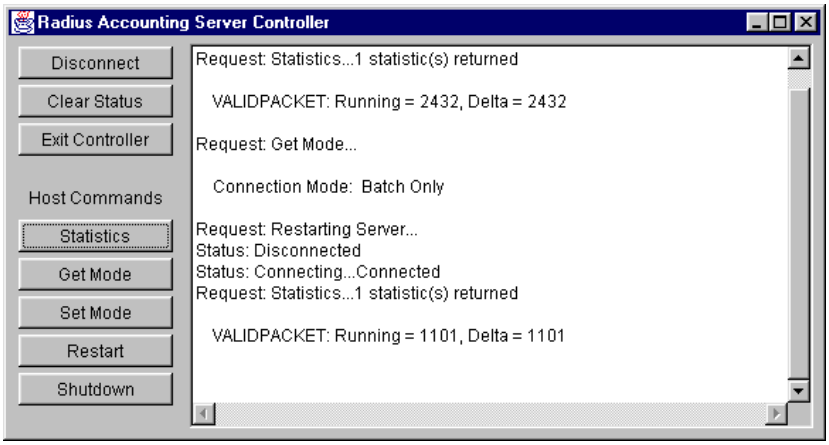


Figure 18-7 RADIUS Accounting Server Controller GUI Interface

Due to security reasons inherent in applets, if the Controller is run in a web browser, it must be served off the same machine that is running the RADIUS Accounting Real Time Rating Process. This restriction does not apply if the applet is run as a separate Java application.

The Controller contains two sets of command buttons, Controller command buttons and Host command buttons:

Controller/ Host	Button	Description	
Controller			
	Connect/ Disconnect	A toggle button that connects/disconnects from the RADIUS Accounting Real Time Rating Process.	
	Clear Status	Clears the status window of all previous display information.	
	Exit Controller	Terminates the Controller applet.	
Host			
	Statistics	Displays the current RADIUS Accounting Real Time Rating processing statistics in the display window. These statistics include the status of the connection with the Singl.eView Convergent Billing Real Time Rating ENM Process as monitored by the Real Time Rating Socket Connection Manager.	
	Get Mode	Displays the current mode of the RADIUS Accounting Real Time Rating Process:	
		1	SOCKETMODE (Real Time Rating only).
		2	ETHERMODE (If connection to Singl.eView Convergent Billing Real Time Rating ENM is successful, Real Time mode. If initial connection to Singl.eView Convergent Billing Real Time Rating ENM fails, Batch mode with background reconnect.).
		3	BATCHMODE (Batch mode only with no real time connection).

Controller/ Host	Button	Description
	Set Mode	Invokes a new window that allows the user to specify the mode of operation for the RADIUS Accounting Real Time Rating Process. See *Get Mode* for the different values. Figure 18-8 shows the new window that is displayed when the Set Mode button is pressed.
	Restart	Terminates the current session and restarts a new session, re-reading the configuration and secrets files.
	Shutdown	Terminates the current RADIUS Accounting Real Time Rating session and disconnects the Controller once shutdown is complete.

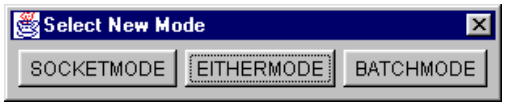


Figure 18-8 Select New Mode Window is Displayed When the Set Mode Button is Pressed

RADIUS Accounting Real Time Rating process Input Thread

The Input Thread's main purpose is to receive and process datagram packets. Figure 18-9 details the steps and processes of the Input Thread.

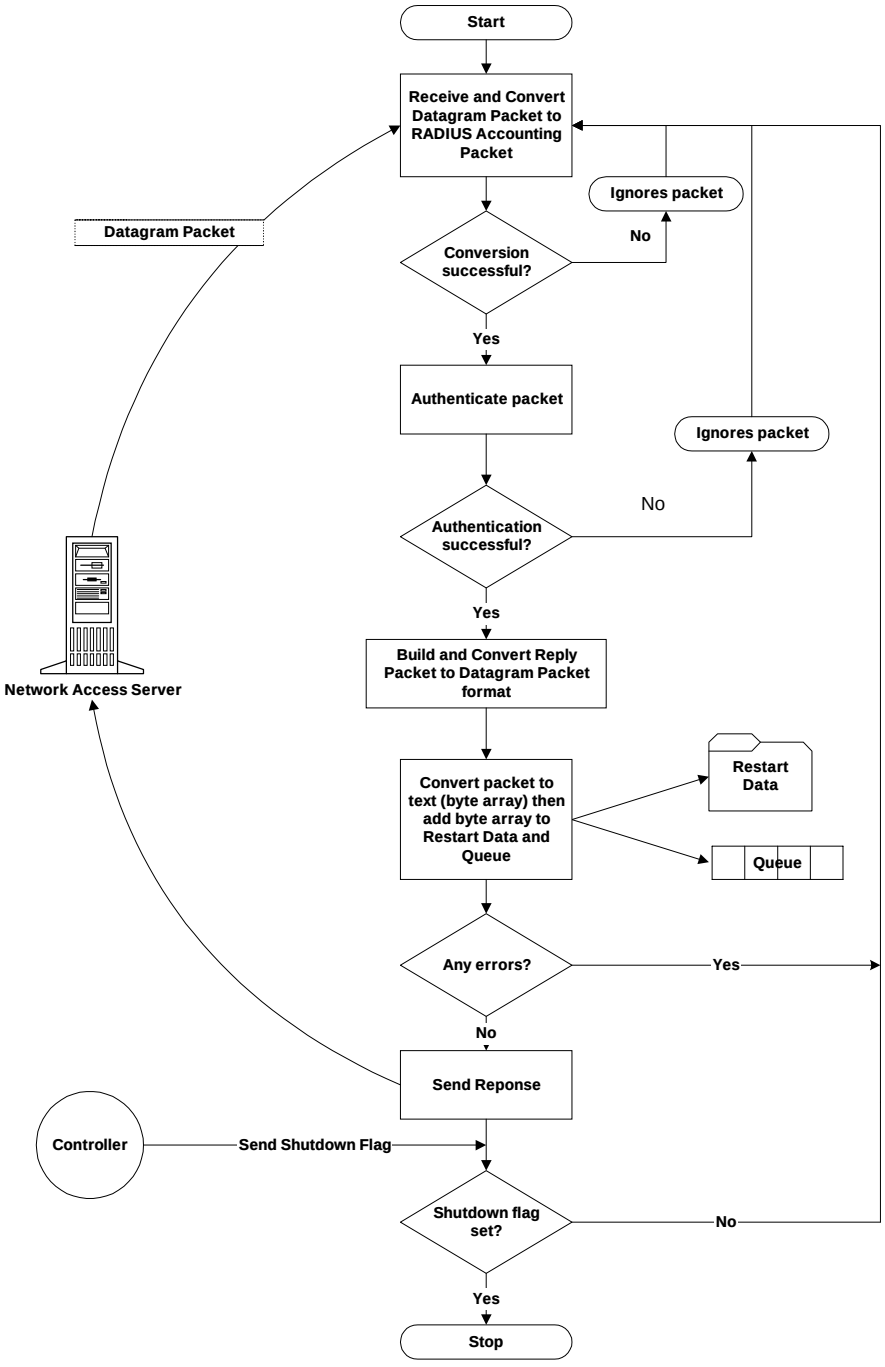


Figure 18-9 Input Thread

The Input Thread continuously processes datagram packets until there are no longer any packets to process, or until the shutdown flag is received from the Controller. If the shutdown flag is received and the Input Thread is waiting for the next packet, the entire process is interrupted. It is possible to initiate multiple threads.

RADIUS Accounting Real Time Rating process Output Thread

The Output Thread's main purpose is to receive events from the Queue and pass them on to the Singl.eView Convergent Billing Real Time Rating process for processing.

Figure 18-10 details the steps and processes of the Output Thread.

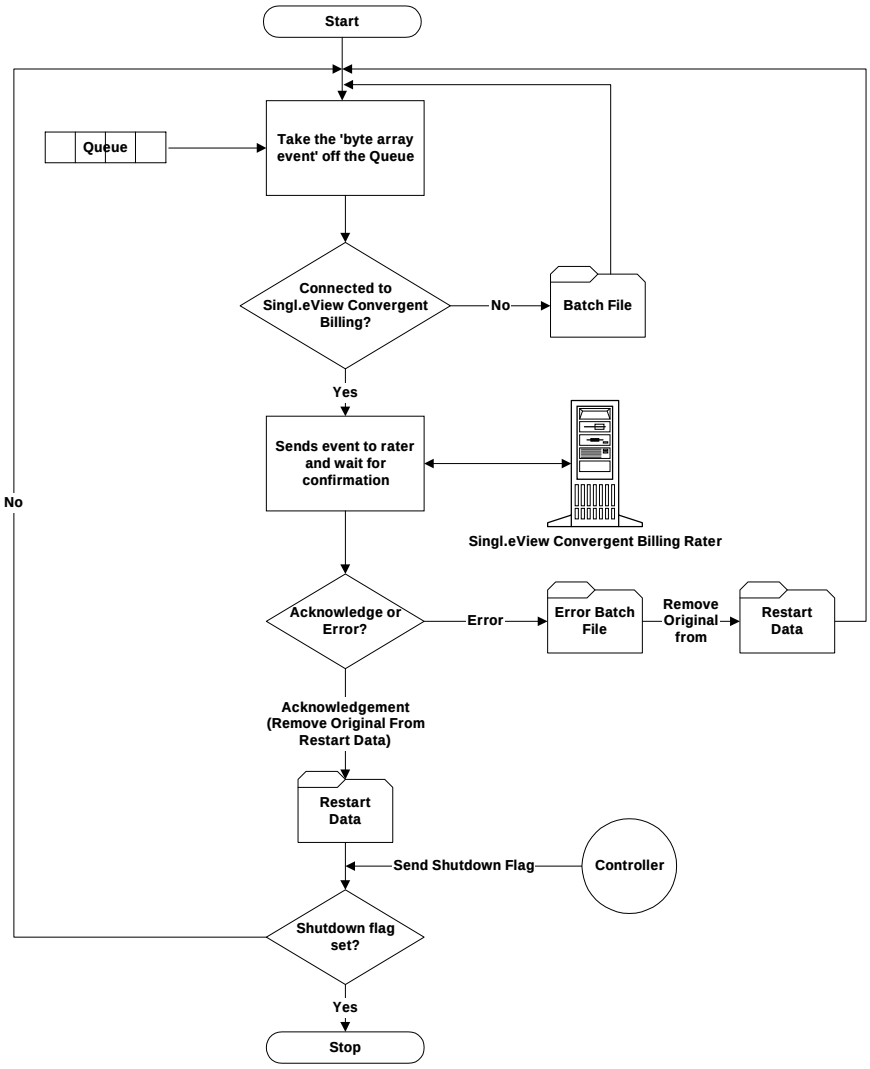


Figure 18-10 Output Thread

When the Controller Thread receives a shutdown request by the Controller, the Controller Thread sends messages to the Input and Output Thread. The Controller Thread is responsible for ensuring the Input Thread completes prior to shutting down the Output Thread. The Output Thread does not shutdown until the Queue is emptied of all queued requests.

If there is a communication problem or error (eg. the socket connection is unavailable or lost prior to receiving the acknowledgement or error message), the event records are sent to the Batch Output and Error Streams.

RADIUS Accounting Real Time Rating process Controller Thread

The Controller Thread acts as a socket listener, waiting for an incoming socket connection. It is responsible for coordinating the commands between the Controller and the Input and Output Threads. It is also responsible for allowing only one connection to be established, thereby preventing conflicts.

The incoming messages from the Controller are decoded from the TRE format and the appropriate action taken for each command.

- Statistics – queries the statistics of the current processes and returns the information
- Get Mode – determines the current mode from the Real Time Rating Socket Manager and returns this value
- Set Mode – allows the user to specify the mode of operation for the RADIUS Accounting Real Time Rating Process
- Restart – is similar to shutdown; however, it will start the process again once shutdown is complete
- Shutdown – begins the shutdown sequence, waits for notification that the shutdown has basically been completed, returns the acknowledgement that the shutdown has been received, closes the connection, and ends.

RADIUS Accounting Real Time Rating Socket Manager

Acting as the link between the RADIUS Accounting Real Time Rating Process and the Singl.eView Convergent Billing Real Time Rating process, the RADIUS Accounting Real Time Rating Socket Manager manages all socket connect and disconnect requests.

Figure 18-11 illustrates how the Real Time Rating Socket Manager handles connection requests.

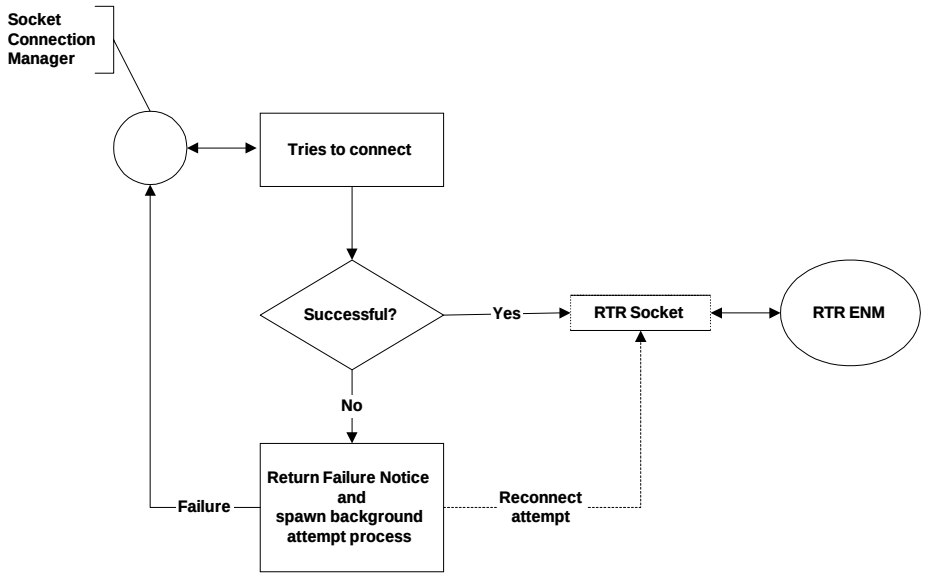


Figure 18-11 Management of Connection Requests by the RADIUS Accounting Real Time Rating Socket Manager

While not connected, the Output Thread queries the Connection Manager to determine the connection status, prior to writing the text to the batch file. If the Controller requests to connect again while waiting for a retry, the connection is attempted immediately with the retry timing being reset. The timing can be set through the use of an object class, which is similar to a specialised file with timing details within.

During a connection attempt, if the Real Time Rating Socket Manager receives a request to disconnect, the connection attempt is terminated, and if it is already connected, a normal disconnect procedure occurs.

Using the Statistics button on the Controller window, the user can query the Socket Manager for one of the following statuses:

- Connected
- Not connected but attempting
- Not connected.

Restart Data File

The RADIUS Accounting Real Time Rating Process is equipped to restart in the event of an abnormal termination. This is achieved through a re-startable collection process. The collection process utilises two files: the Restart Data File and the Transaction Log. The Transaction Log is not the same as the Log File. The RADIUS Accounting Real Time Rating Process uses the Log File for general logging. For more information about the Log File, refer to the *Log File* section later in this chapter.

Figure 18-12 illustrates the process used to ensure that data is not lost in case of an abnormal termination.

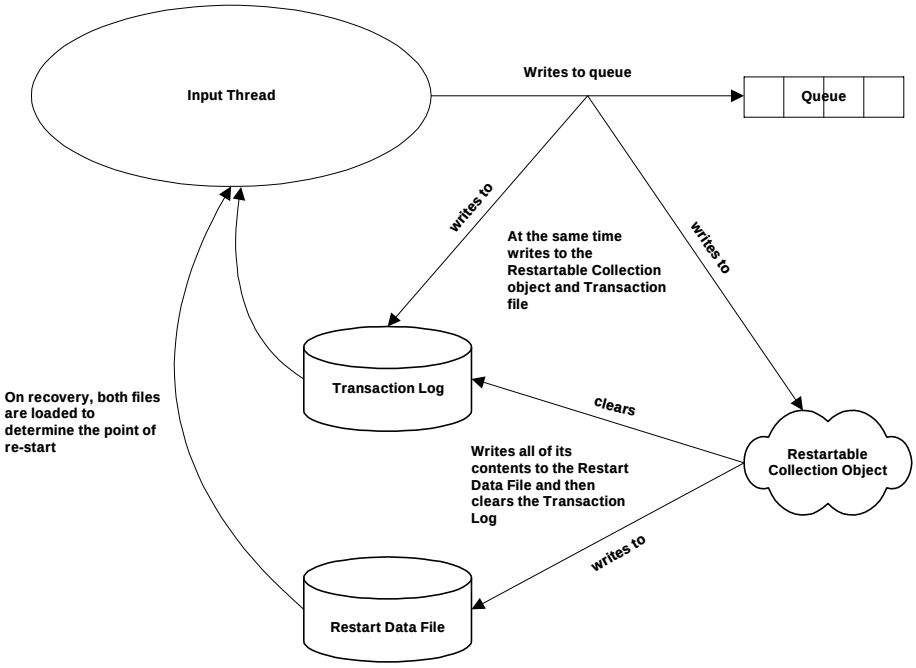


Figure 18-12 Recovery Process Using the Restart Data File and Transaction Log

Every time the Input Thread processes a request event, it is added to the Restartable Collection Object, at the same time recording its transaction in the Transaction Log. At regular intervals, a background process synchronises by writing the contents stored in the Restartable Collection Object into the Restart Data File, and then clears the Transaction Log.

In the event of an abnormal termination, the Restart Data File and the Transaction Log are used during recovery. No explicit save of remaining data is required during an abnormal termination. This is made possible by using the Transaction Log that is regularly cleared during synchronisation. In this way, only the most recent transactions are retained in the Transaction Log.

During re-initialise, the saved state of the Restart Data File is re-loaded and the saved transactions from the transaction log are re-applied to recover to the point before the abnormal termination. In addition, all of the entries contained with the Restart Data are added back to the Queue for the Output Thread to process.

Batched output and error streams

The Batch Output Files are created to store each request event received by the RADIUS Server when the connection to the Singl.eView Convergent Billing Real Time Rating ENM process is not available. This file can then be used for batch processing of the RADIUS Accounting events. The Error Batch Output Files are identical, with the exception of the error message, which is written first and on a separate line from the actual event.

The file size and timing settings are determined and set in the Configuration File, together with the names of the files, which are determined using a base string, timestamp, and optional extensions.

Log File

The Log File is used for recording general transactions, unlike the Transaction File that is within the Restartable Collection Object. The Log is used to maintain counts of silently discarded packets, failed authorisation packets, and valid packets received. The Input Thread logs these statistics on a periodic basis, and the Controller Thread logs the statistics during shutdown. The Controller Thread also queries this file when a statistic request is received.

Silently discarded packets are those that are discarded without further processing. This situation is created, for example, when a user logs on but is not authenticated. The error is logged, including the contents of the silently discarded packet. The event should also be recorded in the statistical counter (eg. after 3 failed attempts to log on are recorded in the statistics, log on access will be barred).

Archiving RADIUS Accounting start and stop records on the RADIUS server

It is recommended that RADIUS accounting records be archived on a regular basis (eg. weekly or monthly). For typical installations, allocate a minimum of 50MB per 1000 users per month. Keep in mind that allocating too much space is preferable to allocating too little because usage can vary. In addition, if the RADIUS server is being used to archive the records, it is recommended to allocate more storage than this minimum.

For example, if an ISP has 1000 users, one port for every 10 users, an average connection time per user of 1 hour, and all ports in use around the clock, one month of logs would require 50 MB of disk space. This assumes that the record for each session would be 700 bytes.

Chapter 19
Singl.eView IP/Data Updates for Vertex

Currently, Vertex products are unable to handle tax compliance for IP or Data Products and Services. Vertex has announced that their products will be IP/Data capable by the end of 2000.



This chapter is only meant as a guide to aspects of Singl.eView Convergent Billing that impact Vertex. This chapter should be used in conjunction with the CommTax 21 Reference Manual published by Vertex. In addition, a tax expert should be consulted to ensure that all taxes are set up correctly.

In the meantime, as an interim step, the following solution is offered:

- Vertex CommTax 21 works based on assigning the proper service code, category code, and geocode. These two codes are not currently defined by Vertex for IP or data services. As a result, a transaction file must be built to be used by the CTX150 program to update the CommTax 21 data modules with IP and data taxation information. The service code and category codes and the associated taxes and rates must be defined.
- The Tax code must also be defined to ensure CommTax 21 can identify and calculate the appropriate tax.
- In addition, the percent tax rate applicable for each State and/or locality must be defined.
- No solution has been provided for federal level taxes because currently there are no federal taxes for IP and data services.

This chapter covers the following topics:

- Singl.eView IP/Data Category, Service, and Tax Codes
- CTX150 Program
- Using the Transaction File Generator Spreadsheet and Running CTX150
- Configuration Checklist for Updates to Vertex.

Singl.eView IP/Data Category, Service, and Tax Codes

CommTax 21 organises tax data into categories and services. Similar services are grouped into one category for ease of use. Services are also classified by the type of service offered. This section lists and defines the Categories and Services to be used for Singl.eView IP/Data.

Category and service codes

Vertex reserves categories thirteen through fifteen to be used for 'flexible' services. Singl.eView IP/Data will use category 15. Because all Data and Internet Services/Products receive the same tax treatment, Category 15 will be used for all of them (eg. Internet, DSL, ATM, and Frame Relay). Table 19-1 shows the service codes for category 15.

Table 19-1 Service Codes for Category 15

Number	Service	Description
01	Network Access	Charge for Access to the Data or Internet Network
02	Usage: Data	Charge for upload/download of Data packets
03	Usage: Voice intrastate	Charge per minute for Voice usage for intrastate calls (eg. VoIP or VoDSL)
04	Usage: Voice interstate	Charge per minute for Voice usage for interstate calls (eg. VoIP or VoDSL)
05	Service activation/ deactivation	A charge for activating or deactivating a service.

Tax codes

CommTax 21 provides for a wide variety of tax types. Table 19-2 shows the applicable codes for current IP/Data services and products taxes:

Table 19-2 Tax Type Codes for IP/Data Taxes

Tax Type Code	Description
50	Excise Tax
64	Gross Receipts Tax
99	Sales Tax

CTX150 Program

CommTax 21 provides businesses with the ability to tailor tax information via the CTX150 program. CTX150 uses a transaction file to make updates to, or add to, decision and rate files. Rate files contain tax rates associated with each of the taxing jurisdictions. Decision files contain switches and flags to control the calculation for each jurisdiction. For example, the decision file could be set up so that the application of taxes for a particular jurisdiction would depend on the service code, category code, and whether the affected accounts were residential or business.

The CTX150 program reads the transaction file, which is a sequential file prepared by the user. The rate and decision files are then updated or added to by the program.

Transaction File Generator spreadsheet

To facilitate the creation of the transaction file, a spreadsheet (Transaction File Generator.xls) is available into which the relevant information can be entered. The transaction file that is generated is then sent via FTP onto the server on which CommTax 21 resides. Command line information is entered. If the CTX150 run is successful, CommTax 21 is updated. The following is a section of the transaction file for Hawaii:

```
R12009500500000005O1X
040001996010100000000000000000000000000000000000

R12009500700000005O1X
040001996010100000000000000000000000000000000000

R12009501600000005O1X
040001996010100000000000000000000000000000000000

R12009501700000005O1X
040001996010100000000000000000000000000000000000

D120000000000150150 1X RIAAA000000000000000000000000000

D120010000000150150 1X RIAAA000000000000000000000000000

D120010005000150150 1X RIAAA000000000000000000000000000

D120010005000150150 1X RIAAA000000000000000000000000000

D120010007000150150 1X RIAAA000000000000000000000000000

D120010007000150150 1X RIAAA000000000000000000000000000

D120010007000150150 1X RIAAA000000000000000000000000000
```

Using the Transaction File Generator Spreadsheet and Running CTX150

The Transaction File Generator.xls Excel™ spreadsheet consists of a number of worksheets (tabs on the Excel spreadsheet file). Table 19-3 lists these worksheets and their function:

Table 19-3 Worksheets in the Transaction File Generator Spreadsheet

Worksheet	Function
Transaction Filter	Provides a list of the various columns in the transaction file and a sample of previous spreadsheet entries for various states.
Transaction List	Contains multiple columns in which taxation rules for a particular state are entered.
Description 1	Geocodes, city and county information for each state are pasted here from the relevant alphabetical worksheet.
A-F	Contains a listing of geo-codes associated with states that have first letters starting with A through F.
G-L	Contains a listing of geo-codes associated with states that have first letters starting with G through L.
M	Contains a listing of geo-codes associated with states that have first letters starting with M.
N	Contains a listing of geo-codes associated with states that have first letters starting with N.
O-Z	Contains a listing of geo-codes associated with states that have first letters starting with N.

Generating the transaction file

To generate the transaction file, first become familiar with the Transaction File Generation.xls spreadsheet and each of the worksheets within it. For more information on the worksheets, refer to the beginning of this section.



As previously mentioned, this chapter should be used in conjunction with the CommTax 21 Reference Manual published by Vertex. A tax expert should be consulted to ensure that all taxes are set up correctly.



- It is recommended that one transaction file be created for each state. The files can then be processed one at a time. This will avoid overloading system resources or very slow performance times when the transaction file is loaded into the CTX150 program.
- Before beginning this procedure, set up a directory where the transaction file can be stored before it is re-named and sent via FTP to the server on which CommTax 21 resides.

To generate the transaction file:

1. Gather the information needed to set up IP/Data taxes for the first state.
1. Open the Transaction File Generation.xls spreadsheet using Microsoft Excel™.
3. Select the relevant alphabetical worksheet tab for the state.
4. Select the information for the state.
5. Select the **Edit ⇨ Copy** option.
6. Select the *Description 1* worksheet tab.
7. Select a cell in the *Description 1* worksheet.
8. Select the **Edit, Paste** option.
9. Select the *Transaction List* worksheet tab.
10. Populate the rows with the tax information for the first state.
11. Specify the directory to which the transaction file will be generated in the *Existing Directory Path* field (column E, row 9).
12. Specify the timestamp for the filename in the *Timestamp* field (column B, row 7). The resulting filename will be `C2TRANS` + timestamp. For example, if the timestamp is `08312000`, the resulting filename will be `C2TRANS08312000`. This timestamp can be used to designate any label that will help identify different transaction files that are generated.

13. After the Transaction List is completed, select the **File, Save** option.

14. To generate the transaction file, press the **Transaction File** button. The file will be generated and can be found in the directory specified in step 11. Continue with the *Transferring the transaction file and running CTX150* procedure later in this section.

Transferring the transaction file and running CTX150

After the transaction file has been generated using the Transaction File Generation.xls spreadsheet, it must be re-named and transferred to the server where CommTax 21 resides. CTX150 can then be run to generate the updates and additions to the rate and decision files.



- Before the CTX150 program is run, create a copy of the CommTax 21 files. This copy will ensure that recovery from any errors is possible and will allow comparison of the data in the current and newly generated updates and files.
- CTX150 should be run from your own work directory on the UNIX server. If the run is successful, the files can then be copied into the CommTax 21 directory. Before running CTX150, ensure that all CommTax 21 files reside in your home directory. Errors will occur if you try to run CTX150 within the main CommTax 21 directory.

To transfer the transaction file and run CTX150:

1. After the transaction file has been created in the directory you specified, re-name the file `c2trans`. Use lower case letters.

2. Use the FTP Exchange schedule to transfer the `c2trans` file to the UNIX server where the CommTax 21 files reside. Use the ASCII *Transfer Format*. For more information, refer to *FTP Exchange Schedule Type* in *Chapter 17, Payment Interfaces and Schedule Types.*

3. Ensure the `c2trans` filename on the UNIX server is in lower case letters.

4. To run CTX150, enter **`ctx150`** on the UNIX command line.

5. Compare the backup copies that you created before running CTX150 with the new CommTax 21 `c2rate` and `c2dec` rate and decision files to find out what was added and updated.

Troubleshooting

CTX150 Load Problems

1. Ensure that Transfer Format of ASCII was used to transfer the `c2trans` file.
2. Ensure that `c2trans` file is in your home directory on the UNIX server and that the filename is in lower case letters.
3. Review the comparison between the backup copy that you created before running CTX150 with the new CommTax 21 rate and decision files to find out what was added and updated.

If taxes have not been updated

1. Use the *Expression Test* form to call the `biCommTaxFetch` function and evaluate the results of running the CTX150 program. Use sample data to find out whether the data modules have been updated with the new taxes for the state you have entered in the Transaction File Generator.xls spreadsheet. Figure 21-1 shows an example of an expression that could be used as a test for a particular state. The `biCommTaxFetch` function fetches all taxes that match the input parameters and is the interface to CommTax 21. For more information, refer to *Testing Expressions* in *Chapter 2 Expressions and Operators* and *Chapter 3, Functions* of the manual *Variables, Expressions, and Functions for Singl.eView Convergent Billing*.

   The following is an example of expression for testing results of Vertex update.

```
{
var request?{} := [
'A_PARTY_NPANXX', '203915',
'B_PARTY_NPANXX', '203915',
'C_PARTY_GEOCODE', '070090310',
'CATEGORY_CODE', 15,
'SERVICE_CODE', 01,
'TAXABLE_AMOUNT', 25000.00,
'CALL_MINUTES', 400.00,
'EFFECTIVE_DATE', to_date('31-12-1995', 'dd-mm-yyyy'),
'CUSTOMER_REFERENCE', 'TEST',
'NORMALISED_EVENT_ID', 888777666
];
var Result?{};
var lrc&;

lrc& := biCommTaxFetch&(request?{}, Result?{});
if (lrc& = 0) then {
print('ERROR: ' +
to_string(Result&{'RETURN_CODE'},'0'));
}
else {
print(Result?{});
}
}
```

2. Verify that the `c2rate` and `c2dec` files have been updated.

3. Perform a UNIX search using the **`grep`** command to determine whether the `c2reg` file has been populated with a normalised event ID for one of the records that have been processed in a bill run. The following is an example of a UNIX search:

   `grep 12345789 c2reg* >probs`

   where:

`grep`	is the UNIX command for performing a search.
`12345789`	is the normalised event ID.
`c2reg*`	is the Vertex output file that shows what records have been passed in from the billing run and what records Vertex has tried to evaluate. Records that are populated with normalised event ids have been evaluated by Vertex.
`probs`	is the output file for the search results.

4. Bring down the Transaction Engine (TRE) and bring it back up again.

Configuration Checklist for Updates to Vertex

The following is a checklist of entities within Singl.eView Convergent Billing that must be maintained once the CommTax 21 data modules have been updated and the tariffs and products for Singl.eView IP/Data have been configured. These entities should be maintained in consultation with ADC personnel.

✓ CLEC_NE TAXING_ CATEGORY reference type

The `CLEC_NE_TAXING_CATEGORY` reference type has been updated to contain the new Category 15 for IP/Data taxes. For more information on category codes, refer to *IP/Data Category, Service, and Tax Codes* earlier in this chapter.

Figure 19-1 shows the `CLEC_NE_TAXING_CATEGORY` reference type.

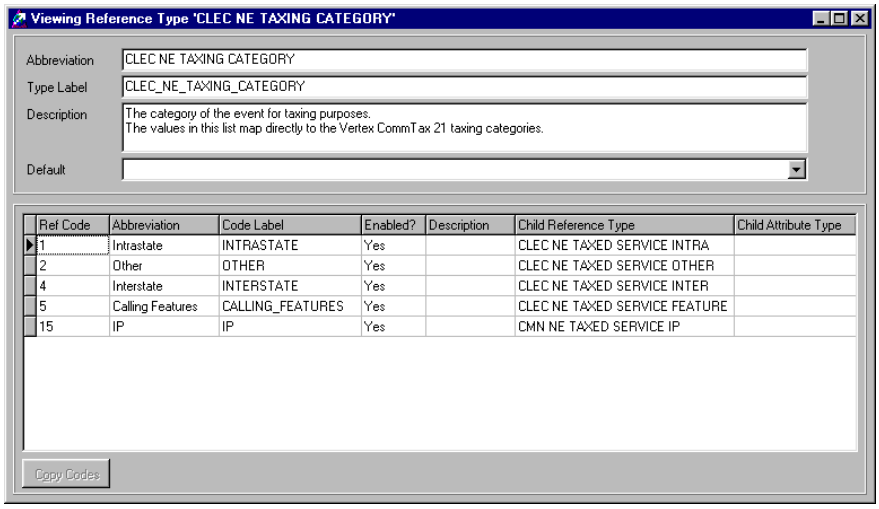


Figure 19-1 CLEC_NE_TAXING_CATEGORY Reference Type

✓ CLEC_NE_TAXED_SERVICE_IP reference type

The CLEC_NE_TAXING_CATEGORY reference type uses a child reference type for each category code. Category 15 uses CLEC_NE_TAXED_SERVICE_IP (abbreviation is CMN NE TAXED SERVICE IP). The CLEC_NE_TAXED_SERVICE_IP reference type contains all the service codes for Category 15.

Figure 19-2 shows the CLEC_NE_TAXED_SERVICE_IP reference type.

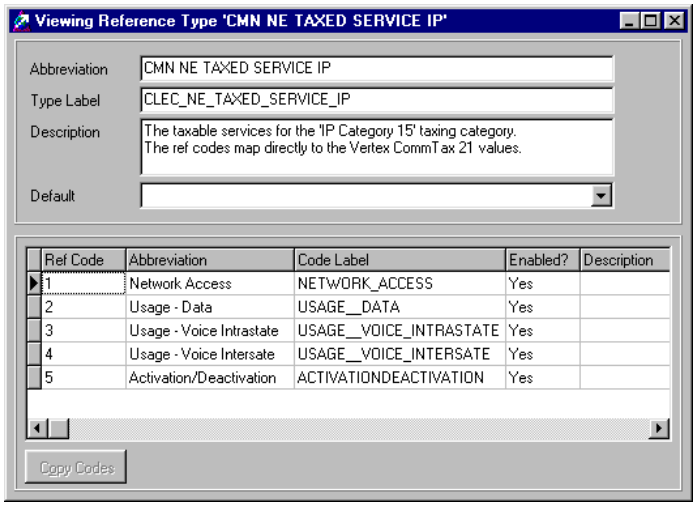


Figure 19-2 CLEC_NE_TAXED_SERVICE_IP Reference Type

✓ dCLEC_NE_TaxFields& derived attribute

The dCLEC_NE_TaxFields& derived attribute (shown in figure 19-3) must be updated for each category and service code. This derived attribute is called by rating to determine the service and category codes used by Vertex to calculate taxes. It uses Event Class, Event Type, Event Subtype, Additional Index 1, and Additional Index 2 as indices and returns *Category*, *Service* and *Taxable?* as results.

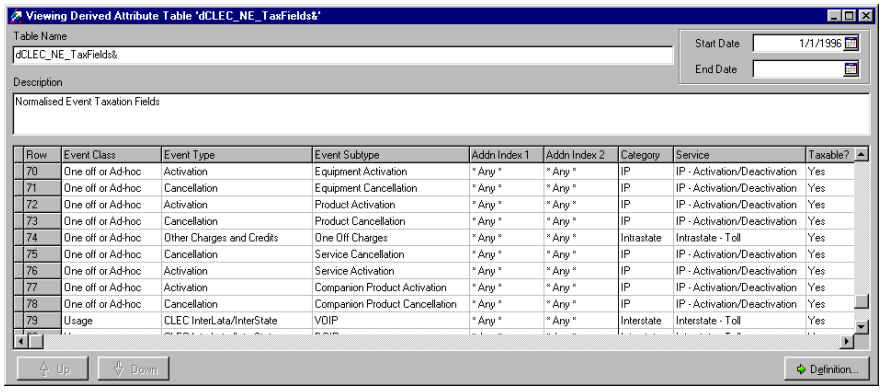


Figure 19-3 dCLEC_NE_TaxFields& Derived Attribute

✓fCMN_TAX_ GatherSvcChg function

For service level taxation, such as taxation for the activation or deactivation of a service, the `fCMN_TAX_GatherSvcChg&` function has been set up with records for five service codes in category 15. This function is called by rating to determine the service and category codes used by Vertex to calculate taxes. If any additional service codes are added to the `CLEC_NE_TAXED_SERVICE_IP` reference type, a subtotal must be configured for the tax, the `dCMN_SVC_TAX_Group$` derived attribute must be updated and this function must be updated to include the subtotal and the applicable service and category codes

Figures 19-4 and 19-5 show the text on the Function Definition tab and the content of the function. For more information on the `dCMN_SVC_TAX_Group$` derived attribute, refer to the next section.

```
#-------------------------------------------------------------------
--------
#
#   File:       fCMN_TAX_GatherSvcChg.epm
#   Created:    31-05-2000 11:37:00
#   Creator:    myates  (Michael Yates)
#   $Revision: 1.2 $
#   $Id: fCMN_TAX_GatherSvcChg.epm,v 1.2 2000/06/20 06:58:32 myates
Exp $
#
#       ============================================================
#               Copyright C - 2000 All rights reserved.
#                 ADC Software Systems Aust.  Pty Ltd.
#
#       No part of this program may be reproduced or adapted in any
#             form or by any means, electronic or mechanical,
#       without permission from ADC Software Systems Aust.  Pty Ltd.
#          This program is confidential and may not be disclosed,
#          decompiled or reverse engineered without permission in
#             writing from ADC Software Systems Aust.  Pty Ltd.
#       ============================================================
#
# NAME:
#   fCMN_TAX_GatherSvcChg&
#
# DESCRIPTION:
#   Gathers service charges, taxing categories and service
categories

#   into an array of hashes.
#
# PARAMETERS:
#   charges?[] - Array of charges.  Each item in the array is a hash
containing
#                the charge, tax category and service category
#
# RETURNS:
#   1
#
# REVISION HISTORY:
#   $Log: fCMN_TAX_GatherSvcChg.epm,v $
#   Revision 1.2  2000/06/20 06:58:32  myates
#   Fixed null value passing bug
#
#   Revision 1.1  2000/06/19 11:04:48  csmyth
#   Subtotals added for each service code.
#
#-------------------------------------------------------------------
--------
```

Figure 19-4 fCMN_TAX_GatherSvcChg Function (continued in figure 19-5)

```
fCMN_TAX_GatherSvcChg&(var charges?[]) = {

    # Add values to array
    if (!is_defined(sS_CMN_TAX_SVC_Network#)) {
        charges?[0]{} :=
['CHARGE',0.0,'CATEGORY_CODE','15','SERVICE_CODE','1'];
    }
    else {
        charges?[0]{} :=
['CHARGE',sS_CMN_TAX_SVC_Network#,'CATEGORY_CODE','15','SERVICE_CODE
','1'];
    }
    if (!is_defined(sS_CMN_TAX_SVC_UsageData#)) {
        charges?[1]{} :=
['CHARGE',0.0,'CATEGORY_CODE','15','SERVICE_CODE','2'];
    }
    else {
        charges?[1]{} :=
['CHARGE',sS_CMN_TAX_SVC_UsageData#,'CATEGORY_CODE','15','SERVICE_CO
DE','2'];
    }
    if (!is_defined(sS_CMN_TAX_SVC_VoiceIntState#)) {
        charges?[2]{} :=
['CHARGE',0.0,'CATEGORY_CODE','15','SERVICE_CODE','3'];
    }
    else {
        charges?[2]{} :=
['CHARGE',sS_CMN_TAX_SVC_VoiceIntState#,'CATEGORY_CODE','15','SERVIC
E_CODE','3'];
    }
    if (!is_defined(sS_CMN_TAX_SVC_VoiceIntrState#)) {
        charges?[3]{} :=
['CHARGE',0.0,'CATEGORY_CODE','15','SERVICE_CODE','4'];
    }
    else {
        charges?[3]{} :=
['CHARGE',sS_CMN_TAX_SVC_VoiceIntrState#,'CATEGORY_CODE','15','SERVI
CE_CODE','4'];
    }
    if (!is_defined(sS_CMN_TAX_SVC_ActDeact#)) {
        charges?[4]{} :=
['CHARGE',0.0,'CATEGORY_CODE','15','SERVICE_CODE','5'];
    }
    else {
        charges?[4]{} :=
['CHARGE',sS_CMN_TAX_SVC_ActDeact#,'CATEGORY_CODE','15','SERVICE_COD
E','5'];
    }

    return(1);

}


#
#       End (of fCMN_TAX_GatherSvcChg.epm)
#
```

Figure 19-5 fCMN_TAX_GatherSvcChg Function (continued from figure 19-4)

✓dCMN_SVC_ TAX_Group$

The dCMN_SVC_TAX_Group$ derived attribute has been set up with records for five service codes in category 15. This derived attribute defines what subtotals are applicable to each service code. If any additional service codes are added to the CLEC_NE_TAXED_SERVICE_IP reference type, then a subtotal must be configured for the tax, the dCMN_SVC_TAX_Group$ derived attribute must be updated and the fCMN_TAX_GatherSvcChg function must be updated to include the subtotal and the applicable service and category codes. Figure 19-6 shows the dCMN_SVC_TAX_Group$ derived attriubute. For more information on the fCMN_TAX_GatherSvcChg function, refer to the previous section.

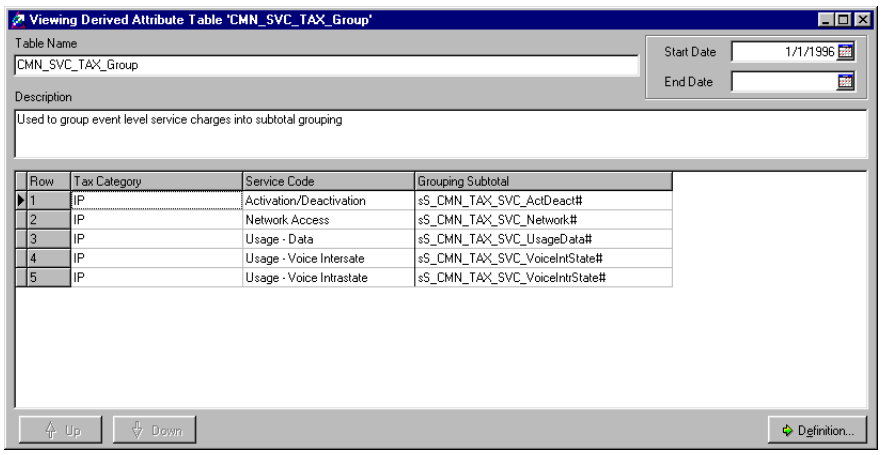


Figure 19-6 dCMN_SVC_TAX_Group$ Derived Attribute

Singl.eView IP/Data Updates for Vertex

Chapter 20
Automatic Provisioning: Troubleshooting Guide



Singl.eView IP/Data includes automatic provisioning for email, dialup and user web hosting services. Domain web hosting services are provisioned manually. This troubleshooting chapter contains information about the reference implementation. Some of this information may change depending on the way automatic provisioning is set up for a particular implementation.

Some of the factors that may vary from implementation to implementation include the following:

- In this reference implementation, it is the creation of services for the Dialup products which trigger automatic provisioning. When a dialup product is sold and the service is created, three services are actually provisioned: email (primary service), dialup, and user web hosting. In each implementation, this could be set up differently. For example, there could be a product group for RADIUS authentication products and services. Selling these products and creating the associated service could trigger automatic provisioning of RADIUS authentication services only.
- The primary service for the reference implementation, the service that triggers the creation of a user account on each remote host, is the email service. At a minimum, the user account consists of a username and password, and can also include a password expiry date, home directory, and disk quota. Depending on what products and services have been set up for automatic provisioning, the primary service can be changed. For example, in a particular implementation, if RADIUS authentication rather than Dialup products are being provisioned, the primary service may be RADIUS authentication.

Troubleshooting Aids for Automatic Provisioning

If automatic provisioning fails, pop-up error messages and Provisioning Error queries will be created to alert the user that there is a problem. This guide provides information about what actions are carried out with successful automatic provisioning, what the common error messages are, and how to interpret these messages. The workforce that is in charge of managing provisioning can then address the problem. Most often, this will involve manual provisioning of the remote host on which provisioning is being carried out (eg. the RADIUS server or email server).

This chapter contains the following sections:

- How Automatic Provisioning is Triggered in Singl.eView Convergent Billing
- Troubleshooting Aids: Pop-up Error Messages
- Troubleshooting Aids: UNIX Log Directory
- Troubleshooting Aids: Customer Hierarchy Form
- Troubleshooting Aids: Service Detail Form
- Troubleshooting Aids: Provisioning Error Queries.

How Automatic Provisioning is Triggered in Singl.eView Convergent Billing

The following tables provide information about how actions performed by the CSR relate to automatic provisioning actions in the reference implementation. Information is provided for provisioning of services associated with dialup products, provisioning of email aliases, and provisioning of alternative email accounts. An example of an alternative email account is when one member of a household has dialup service, email, and user web hosting services and other members of the same household are given their own email accounts. However, these other members have no access to Internet and web hosting services except through the primary household member's accounts.

Another example of an alternative email account is when there is a main dialup username for a business, but each employee of the business has a username for his or her own email account. Table 20-1 covers provisioning of dialup products. Table 20-2 covers provisioning of email aliases, and table 20-3 covers provisioning of alternative email accounts.



In the following tables in the user action column, it is assumed that after performing the action described, the relevant form is saved by pressing the **Save** button.

Table 20-1 Automatic Provisioning of Dialup Products

User Action	Automatic Provisioning Action	Tasks Carried Out by Automatic Provisioning
Selling Dialup product to customer and creation of Dialup service	insert	• Insert the user account on each remote host. Use the following variables: username, password, password expiry date, home directory, disk quota, and framed ipaddress. • Create email inbox. • Lock account. • Create user web hosting directory in user's home directory. • Change permissions of home directory to enable world execute and reading. • Insert huntgroups (these determine the number of modems a customer has access to).
	activate	• Activate password for user account.
Changing the *Status* field on the *Product Instance Detail* form from 'Suspended' to 'Reactivation Required'	activate	Activate password for user account.

User Action	Automatic Provisioning Action	Tasks Carried Out by Automatic Provisioning
Changing the *Status* field on the *Product Instance Detail* form from 'Active' to 'Suspension Required'	deactivate	Password for user account is disabled. This prevents use of the dialup service. It does not prevent customers from receiving email messages in their in-box or from displaying web pages in their home directory.
Changing the *Status* field on the *Product Instance Detail* form from 'Active' or 'Suspended' to 'Cancellation Required'	delete	Remove user account from each remote host. Remove user web hosting directory. Remove email inbox and aliases. This completely removes dialup, e-mail, and user web hosting services for the customer.
Changing details on the *Service Detail* form, such as password	update	Re-provision any details that are provisioned, such as password. Details such as download quota and connect time are not provisioned in the reference implementation.

Table 20-2 Automatic Provisioning of Email Aliases

User Action	Automatic Provisioning Action	Tasks Carried Out by Automatic Provisioning
*Inserting record(s) in the Email Address list on the *Lists* tab of the *Service Detail* form. (To create an alias, the same user name as is in the *Username* field is entered in the *Username* column.)	insert	Add alias(es) to the email account.

User Action	Automatic Provisioning Action	Tasks Carried Out by Automatic Provisioning
Changing the *Status* field on the *Lists* tab of the *Service Detail* form from Active to Cancellation Required	delete	Remove alias(es) from the email account.
Changing email alias details on the *Lists* tab of the *Service Detail* form.	update	Re-provision email alias details.

* This can be done at the same time as the dialup service is created.

Table 20-3 Provisioning of Alternative Email Accounts

User Action	Automatic Provisioning Action	Tasks Carried Out by Automatic Provisioning
*Inserting record(s) in the Email Address list on the *Lists* tab of the *Service Detail* form. (To create an alternative email account, the entry in the *Username* column should be different from the entry in the *Username* field on the *Service Detail* form.) In addition, to create more than one alternative account, an email pack companion product must be sold to the customer.	insert acitvate	• Create email inbox. • Create email aliases. • Lock account. Activate email account. Alternative email accounts have no access to the World Wide Web or to user web hosting except through the primary user's account.
Changing the *Status* column on the *Lists* tab of the *Service Detail* form from 'Suspended' to 'Reactivation Required'	activate	Activate password for alternative email account.

User Action	Automatic Provisioning Action	Tasks Carried Out by Automatic Provisioning
Changing the *Status* column on the *Lists* tab of the *Service Detail* form from 'Active' to 'Suspension Required'	deactivate	Password for alternative email account is disabled. This does not prevent customers from receiving email messages in their in-box. To prevent the use of dialup sevices for an alternative email account, the *Service Status* on the *Service Detail* form must be changed from 'Active' to 'Suspension Required'. This will suspend access for the primary user as well.
Changing the *Status* field on the *Lists* tab of the *Service Detail* form from 'Active' to 'Cancellation Required'.	delete	Remove alternative email accounts, including inbox and aliases.
Changing details on the *Lists* tab of the *Service Detail* form, such as password or email aliases.	update	Re-provision password or email alias details for the alternative email account.

* This can be done at the same time as the dialup service is created.

Troubleshooting Aids: Pop-up Error Messages

After any of the user actions described in the last section are performed and the **Save** button is pressed, if automatic provisioning fails, a pop-up message will appear describing the error and giving a reference to the Provisioning Error query that relates to this error. For more information on Provisioning Error queries, refer to *Troubleshooting Aids: Provisioning Error Queries* later in this chapter. Figure 20-1 shows an example of a pop-up error message for automatic provisioning.

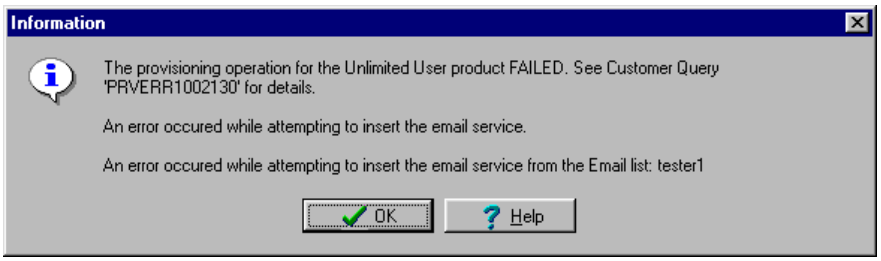


Figure 20-1 Pop-up Error Message for Automatic Provisioning

Note that the error message contains the following information:

- The product that was being provisioned.
- The Provisioning Error query number that was generated.
- The type of provisioning action that failed and the specific service that was being provisioned (eg. insert of email service and insert of alternative email service from the email list).

Read the pop-up message and click the **OK** button. The *Customer Hierarchy* form, Provisioning Error Query and the *Lists* tab of the *Service Detail* form (for provisioning of alternative email accounts) can then be reviewed. For more information on these troubleshooting aides, refer to their respective sections later in this chapter.

Troubleshooting Aids: UNIX Log Directory

In some exceptional cases, no pop-up message will appear even though an error has occurred. If it appears from the status settings that automatic provisioning has completed successfully but the service has not in fact been provisioned on the remote host, the UNIX Log, which contains the most recent status messages, can be checked. On the UNIX command line, enter **LOG**. A screen such as that shown in figure 20-2 will be displayed. The second to the last entry shows an example of an error message.

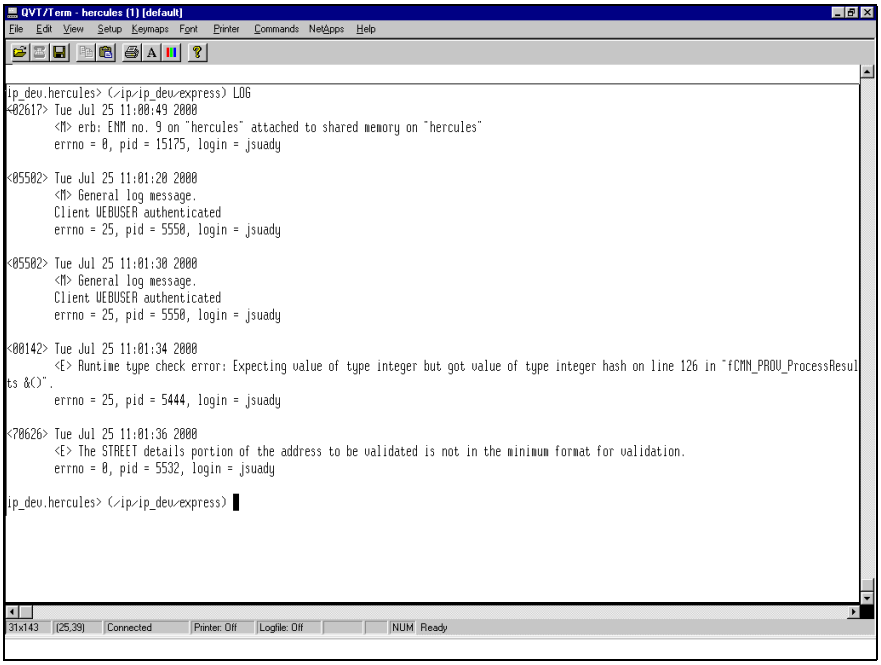


Figure 20-2 UNIX Log Directory

Troubleshooting Aids: Customer Hierarchy Form

Another source of troubleshooting information is the *Customer Hierarchy* form for the customer whose products are being provisioned. This form shows the status of the product and the related service and will show any Provisioning Error queries that have been created. For more information on Provisioning Error queries, refer to *Troubleshooting Aids: Provisioning Error Queries* later in this chapter. Figure 20-3 shows an example of a *Customer Hierarchy* form with the product and service status values and the Provisioning Error query records.

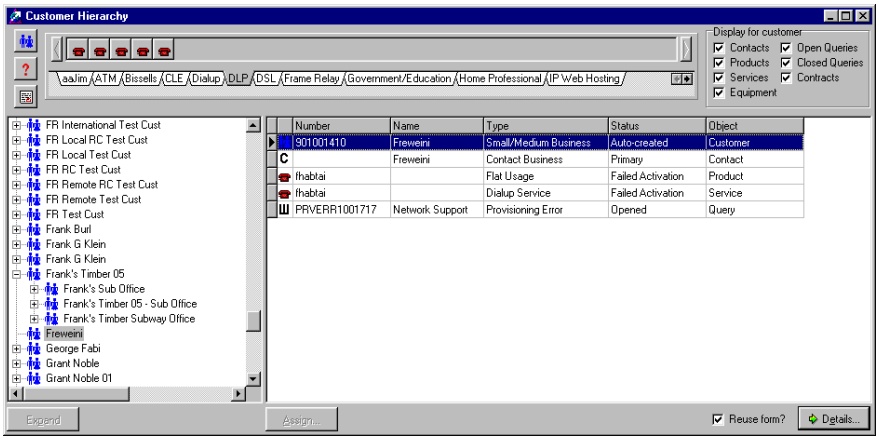


Figure 20-3 Customer Hierarchy Form with Provisioning Information



To see all the records on the *Customer Hierarchy* form, ensure that all the check boxes are selected in the *Display for customer* group. If the *Customer Hierarchy* form is already open when the service has been created or changed, refresh the form to see the current statuses. To refresh the form, de-select one of the checkboxes and then re-select it. The new statuses will be displayed. For example, if a service has been created, the status could initially show 'Provisioning Required' and then show 'Failed Activation' when the form is refreshed.

Troubleshooting Aids: Service Detail Form

If the provisioning error occurred when an email alias or alternative email account was being provisioned, the statuses are not displayed on the *Customer Hierarchy* form. To see these statuses, the Email list on the *Lists* tab of the *Service Detail* form (shown in figure 20-4) must be viewed.

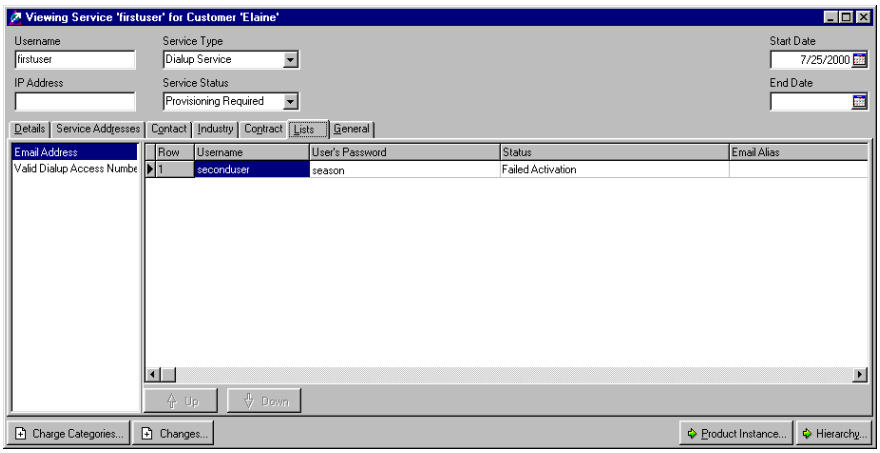


Figure 20-4 Email List on Lists Tab of Service Detail Form with Status of Provisioning for Alternative Email Account

Troubleshooting Aids: Provisioning Error Queries

In most cases, if automatic provisioning fails, a Provisioning Error query will be generated. As mentioned in earlier sections, the pop-up error message that is displayed after the service is created and saved refers to the provisioning error query that has been created and gives the identification number for the query. In addition, the *Customer Hierarchy* form for the customer whose service is being provisioned will show a record of the query.

Provisioning Error queries contain detailed information about what errors occurred and can be forwarded to a provisioning workforce to resolve the problem. Most often, this will involve manual provisioning of the remote host on which provisioning is being carried out (eg. the RADIUS server or email server). For more information about queries in Singl.eView Convergent Billing, refer to *Chapter 8, Recording and Resolving Customer Enquiries* in the *Customer Service Representative User Guide for Singl.eView Convergent Billing*.



Whenever details that require provisioning are updated on the *Service Detail* form or on the *Lists* tab of the *Service Detail* form, automatic provisioning is triggered. However, in the case of errors, no Provisioning Error queries will be generated. A pop-up error message should appear. For tips on troubleshooting, refer to *Troubleshooting Aids: UNIX Log Directory* earlier in this chapter.

To understand the error messages that are displayed on the *Request* tab of the *Customer Query Detail* form, it is useful to understand the structure of the Provisioning Framework that triggers automatic provisioning and that generates the error messaages that are displayed on the *Request* tab. Figure 20-5 illustrates this structure.

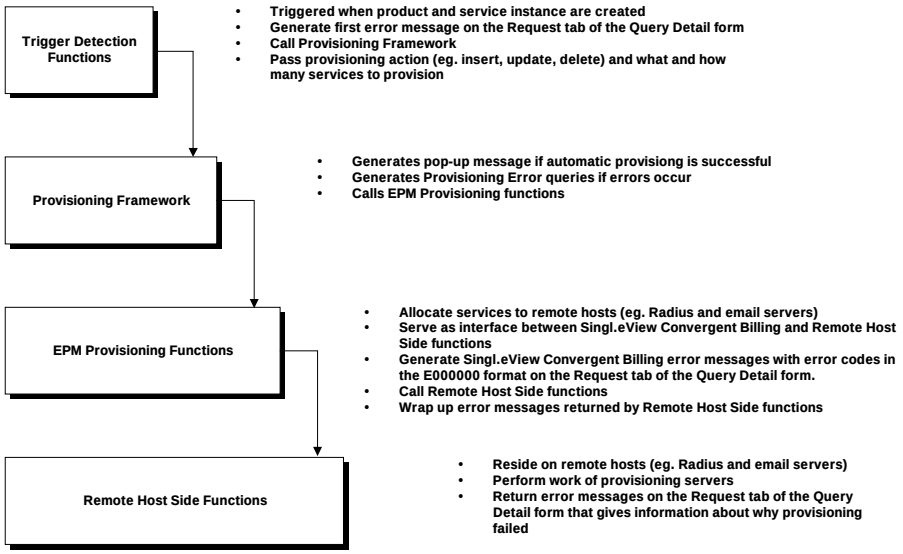


Figure 20-5 Four Levels of Provisioning Framework

Error Messages Generated by EPM provisioning functions

As shown in figure 20-5 above, error codes that are displayed on the *Request* tab of the *Customer Query Detail* form that are in the E000000 format are generated by the EPM Provisioning Functions. The EPM Provisioning Functions perform the following:

- Allocate services to remote hosts (eg. RADIUS and email servers).
- Serve as the interface between Singl.eView Convergent Billing and the Remote Host Side Functions.
- Call the Remote Host Side Functions.
- Wrap up error messages returned by Remote Host Side Functions.

Table 20-4 lists the most common error messages that are generated at this level. Figure 20-6 shows an example of error messages on the *Request* tab of the *Customer Query Detail* form.

The error messages shown in figure 20-6 are generated by both the EPM Provisioning Functions and the Remote Host Side Functions. For more information, refer to *Error messages generated by the Remote Host Side Functions* in the next section.

Table 20-4 Common Error Messages Generated by the EPM Provisioning Functions

Error Code	Example	Description
E75350 through E75353	<E75353> Provisioning with request insert-email failed on all hosts.	Indicates that the Remote Host Side Functions have failed. In general, the messages say which provisioning action failed on which remote hosts. These messages follow the more detailed error information that is returned by the Remote Host Side Functions.
E75354	<E75354> Unsupported Action copy email.	Indicates that the action submitted to the EPM Provisioning Functions is not a recognised provisioning action in the Provisioning Framework. Supported actions include insert, activate, update, deactivate, delete, and fetch.
E75355	<E75355> Required attribute missing 'username'.	Indicates that a required attribute was not provided in the request for provisioning that is passed by the Trigger Detection Functions. In the reference implementation, the only required attribute is the username.
E75357	<E75357> IPUMP provisioning error. Details: Insert failed: email. **Note:** The part of this message that follows the word 'Details' is returned by the Remote Host Side Functions.	Displayed in front of the more detailed error messages returned by the Remote Host Side Functions.

Error Code	Example	Description
E75361	<E75361> Unable to encode ipump command for host insert email. Details: see error E00168. **Note:** The part of this message that follows the word 'Details' provides the error code of the exception that is thrown if encryption fails.	Indicates that the EPM Provisioning Functions are unable to build the command to call the Remote Host Side Functions. This error usually indicates that there has been a problem encrypting the password to send to the Remote Host Side Functions. The problem could be in the TRE crypt service function, a function that is external to the Provisioning Framework. Alternatively, the problem could be that no cryptor (secret) has been associated in the encryption function with the remote host on which the service is being provisioned

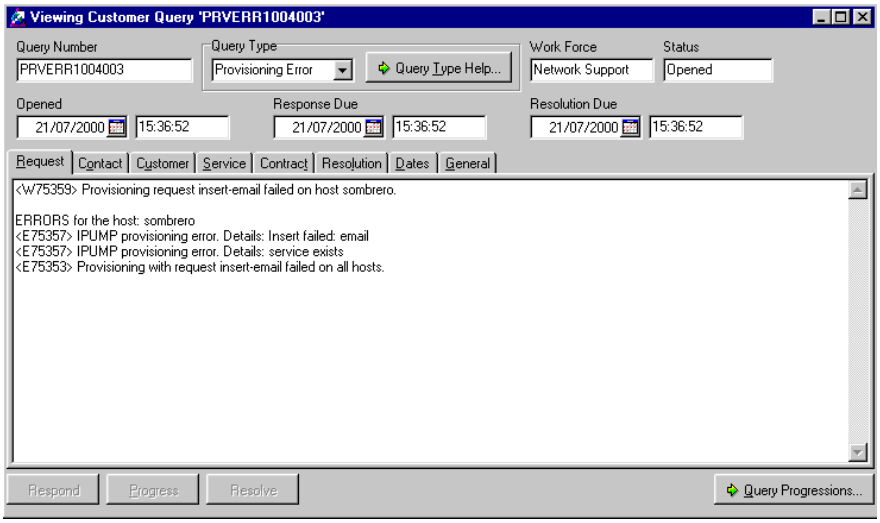


Figure 20-6 Error Messages on Request Tab of Customer Query Detail Form

Figure 20-6 shows an example of a Provisioning Error query with error messages generated by the Remote Host Side. The Remote Host Side error information follows the word 'Details'. The first line in this example tells what provisioning operation failed for what service: 'Insert failed: email'. The second line gives the reason for the failed provisioning action: 'service exists'. This means that a CSR created a Dialup service (which includes email as the primary service) for a username that already has an account on the email server.

Table 20-5 shows the common error messages generated by the Remote Host Side functions.

Table 20-5 Common Error Messages Generated by the Remote Host Side Functions

Type of Error	Examples	Description
Parameter Parsing Errors (built in)	Service parameter not defined or not passed. Action parameter not defined or not passed. Unknown service requested: 'bob' Action 'copy' not defined for service 'email' Syntax error in parameters. Bad parameter: 'action'	These error messages are generated at the first stage of receiving parameters from the EPM Provisioning Functions. In the first stage of processing, the parameters are sanitised or validated to ensure that the record format is valid.
Parameter Parsing Errors (will vary with the implementation)	Syntax error in parameters: ''-fre#d' contains illegal character: #' Mandatory parameter missing or undefined: username username 1rewr begins with non-alpha character parameter 'test%5' contains illegal character: % decrypted password 'frog/star' contains illegal character: /	These error messages are similar to the built in Parameter Parsing Errors but involve processing rules that can be set during the implementation

Type of Error	Examples	Description
Service/Action Logic Errors		The error explained after figure 20-6 is an example of a Service/Action Logic error. The first line of the error message (after 'Details' in the E75357 message) says what provisioning operation failed for what service. The second line gives the reason for the failure.
When the user account does not exist on the remote host	Normally, if the user account does not exist, insert will work. Here is an example of an error message that would occur if the inserted service is not the primary service: Insert failed: radius service does not insert user (is not the primary service) User account does not exist Error messages for other actions: Activate failed: web User account does not exist Delete failed: web User account does not exist.	

Type of Error	Examples	Description
When the user account exists, but not the service	Insert should not cause an error. Other actions should not work: Activate failed: email service does not exist Deactivate failed: email service does not exist Update failed: email service does not exist	
When the user and the service exist	All actions except insert should work. Insert failed: email service exists	
Service/Action System Errors	Fetch failed: email Error in GetMail /var/spool/mail/test100 does not exist This example shows: • Provisioning operation that failed for which service • Reason for the failure: error in the function GetMail • Further details about the error: mail spool file does not exist.	The first line of the error message says what provisioning operation failed for what service. The second line gives the reason for the failure. The third line can contain more detailed causes for the error. In some cases, these error messages also provide the content of the error returned by the remote host operating system and the command call to the remote host.

Type of Error	Examples	Description
	Update failed: email user->Update error Error in SetUserDetails /usr/sbin/usermod: invalid option --q usage: usermod [-u uid [-o]] [-g group] [-G group, . . .] [-d home [-m]] [-s shell] [-c comment] [-l new_name] [-f inactive] [-e expire] [-p passwd] [-L | -U] name: /usr/sbin/usermod -c"100 Test, , " -q wrong test100 2>&1 This example shows: • Provisioning operation that failed for which service • Reason for the fauilure • Further details about the error: error in the function SetUserDetails • Error returned by the remote host operating system • Command call to the server (in bold).	

Chapter 21
IP/Data Event Feeds and Normalised Event Layouts

This chapter provides information on event feeds for IP/Data services. All data feeds except those for Dialup and DSL Internet services are based on the generic Singl.eView Convergent Billing Native format. *Appendix E, DILs and Configuration Entities for IP/Data Event Feeds* contains information on the DILs that are available for the Dialup and DSL Internet event feeds and also has information on configuration entities that affect rating of IP/Data events. This chapter includes the following sections:

- Non-Native Event Feeds
- Normalised Event Fields – All Usage Events
- Normalised Event Fields – Dialup Usage
- Normalised Event Fields – DSL Internet Usage
- Normalised Event Fields – Data Services Usage
- Normalised Event Fields – GVAS
- Normalised Event Fields – Web Hosting
- Normalised Event Fields – Payments Gateway.

Non-Native Event Feeds

RADIUS Accounting Server Log Data Feed

The RADIUS Accounting Server Log data feed provides raw records for both dialup and DSL events. The general record layout is the same for records passed to the Singl.eView Convergent Billing real-time rating ENM and records contained in the Batch Output Files that are batch processed when the connection between the RADIUS Accounting Real Time Rating Process and Singl.eView Convergent Billing is unavailable.

Further examples of RADIUS Accounting start and stop records can be found in *Chapter 18, RADIUS Accounting Real Time Rating Process.*

RADIUS Accounting Server Log Data Feed records typically have the following layout:

```
<Date-and-Time>
      <Attribute-Name> = <Attribute-Value>
      <Attribute-Name> = <Attribute-Value>
      <…>
```

The following shows an example of start and stop accounting records for one RADIUS event:

```
Thu Feb  3 16:36:33 2000
      NAS-IP-Address = 10.72.8.22
      Acct-Authentic = RADIUS
      Acct-Status-Type = Start
      User-Name = "user5"
      Acct-Session-Id = "102"
      Timestamp = 949559793
      Request-Authenticator = Verified

Thu Feb  3 16:37:24 2000
      NAS-IP-Address = 10.72.8.22
      Acct-Terminate-Cause = User-Request
      Acct-Output-Octets = 55625412
      Acct-Status-Type = Stop
      User-Name = "user5"
      Acct-Input-Octets = 292871
      Acct-Session-Id = "102"
      Timestamp = 949559844
      Request-Authenticator = Verified
```

There may be additional attributes (eg. `Timestamp`, `Request-Authenticator`). The order of the attribute list may vary as may the attributes displayed.

Singl.eView IP/Data will provide DIL specifications to support RADIUS accounting log file data feeds. It is expected that during deployment, slight modifications to this specification will be needed to support different RADIUS accounting server implementations. The DIL specification will support arbitrary ordering of attributes and the presence of non-recognised attributes.

XACCT RADIUS Accounting CSV Data Feed

The XACCT RADIUS Accounting CSV data feed is rated using batch processing only. This data feed is produced using the XACCT scheduled reporting mechanism with the options shown in Table 21-1. XACCT removes the complexity of supporting different versions of RADIUS by providing a consistent interface to both RADIUS authentication and accounting functionality.

Table 21-1 Options used to Produce the XACCT RADIUS Accounting CSV Data Feed

Parameter Type	Parameter	Option
Report Parameters		
	Media	File.
	File Name	Must use variable parameters to generate a unique file name.
	Format	CSV (Comma Separated Values).
Schedule Parameters		
	Incremental Report	Yes.

The format of the XACCT RADIUS Accounting CSV Data Feed is line-based, with each line having a comma-separated series of fields. What the fields represent, and their order, are defined during configuration of the XACCT query and report. All lines in a batch will have the same field layout.

The first line may be a comma-separated list of column names. The DIL specification has a configuration option to accept this line and use it for verification purposes. However, the position and number of fields is pre-

defined for the DIL specification. The DIL specification must be modified when changes in the layout occur.

Both CR/LF (Windows-style) and LF (UNIX-style) line endings are supported.

Support for embedded commas in fields will also be supported, using the Microsoft Excel 97 mechanism. Any field that contains commas or double quotes is double quoted. Double quotes in the field are escaped by a preceding double quote. For example, the field

```
ABC and D,"E&F"
```

will appear in the CSV file as:

```
"ABC and D,""E&F"""
```

DILs for IP/Data Event Feeds

For information on the DILs that are provided with Singl.eView IP/Data, refer to *Appendix E, DILs and Configuration Entities for IP/Data Event Feeds*. For general information on DILs, refer to *Chapter 13, Data Interface Language* of the *System Administration Guide for Singl.eView Convergent Billing*.

Normalised Event Fields – All Usage Events

The following table shows the fields from the Normalised Event and how they are used for incoming IP/Data raw events. These fields correspond to the direct variables on the `Normalised_Event_V` view. The fields on the incoming raw events must be mapped to these direct variables, either through the DILs (for raw events in non-native formats), or through the built in decoding methods (for raw events in Singl.eView Convergent Billing native format). For further information on the direct variables corresponding to the normalised event fields, refer to *Appendix C, Direct Variables* of the manual *Variables, Expressions, and Functions for Singl.eView Convergent Billing*.



If the *Field Usage* column for any normalised event field says 'Event type specific', consult the individual normalised event layouts for the different types of usage events later in this chapter.

Table 21-2 Normalised Event Fields for All Usage Events

Normalised Event Field Name	Field Usage	Data Type
NORMALISED_EVENT_ID	Automatically populated	Integer (10)
LAST_MODIFIED	Automatically populated	Date
PARENT_NORMALISED_EVENT_ID	Not used	Integer (10)
NORMALISED_EVENT_FILE_ID	Automatically populated	Integer (8)
FILE_RECORD_NR	Automatically populated	Integer (10)
ORIGINAL_RECORD_NR	Populated with the number of the record in the original event. If the original record does not specify a record number, then this field is left blank.	Integer (10)
A_PARTY_ID	Event type specific	String (30)
A_PARTY_NETWORK_ID	Not used	String (30)
A_PARTY_NAME	Not used	String (40)
A_PARTY_TON_CODE	Not used	Integer (8)
A_PARTY_CELL_ID	Not used	String (20)
A_PARTY_CARRIER_CODE	Not used	Integer (8)
A_PARTY_LOCATION_CODE	Not used	Integer (8)
A_PARTY_ROUTE	Not used	String (80)
B_PARTY_ID	Event type specific	String (30)
B_PARTY_NETWORK_ID	Not used	String (30)
B_PARTY_NAME	Not used	String (40)
B_PARTY_TON_CODE	Not used	Integer (8)
B_PARTY_CELL_ID	Not used	String (20)
B_PARTY_CARRIER_CODE	Not used	Integer (8)

Normalised Event Field Name	Field Usage	Data Type
B_PARTY_LOCATION_CODE	Not used	Integer (8)
B_PARTY_ROUTE	Not used	String (80)
C_PARTY_ID	Event type specific	String (30)
C_PARTY_NETWORK_ID	Not used	String (30)
C_PARTY_NAME	Not used	String (40)
C_PARTY_INTERNAL_ID	Not used	Integer (10)
C_PARTY_TON_CODE	Not used	Integer (8)
C_PARTY_CELL_ID	Not used	String (20)
C_PARTY_CARRIER_CODE	Not used	Integer (8)
C_PARTY_LOCATION_CODE	Not used	Integer (8)
C_PARTY_ROUTE	Not used	String (80)
FULL_PATH	Not used	String (40)
CASCADE_CARRIER_CODE	Not used	Integer (8)
SWITCH_START_DATE	Not used	Date
CHARGE_START_DATE	from feed	Date
PERIOD_START_DATE	Not used	Date
PERIOD_END_DATE	Not used	Date
EVENT_SOURCE	Event type specific	String (40)
EVENT_CLASS_CODE	Set to one of the following based on the record in the event feed: 'Usage' or 'One-off/Ad-hoc'.	Integer (2)

Normalised Event Field Name	Field Usage	Data Type
EVENT_TYPE_CODE	Set to one of the following: • GVAS • Dialup Internet • Dialup VPN • DSL Internet • DSL VPN • Data PVC Usage • Web Hosting • Payments Gateway.	Integer (8)
EVENT_SUB_TYPE_CODE	Event type specific	Integer (8)
DURATION	Event type specific	Real
VOLUME	Event type specific	Integer
PULSES	Not used	Integer (8)
CHARGE	Event type specific	Real (15, 6)
CURRENCY_ID	Defaulted or set as required	Integer (8)
RATE_BAND	Not used	String (40)
GENERAL_1	Event type specific	String (255)
GENERAL_2	Event type specific	String (255)
GENERAL_3	Event type specific	String (255)
GENERAL_4	Not used	String (255)
GENERAL_5	Not used	String (255)
GENERAL_6	Not used	String (255)
GENERAL_7	Not used	String (255)
GENERAL_8	Event type specific	String (255)
GENERAL_9	Taxing Category	String (255)
GENERAL_10	Taxable Service	String (255)
GENERAL_11	Event Reference (NORMALISED_EVENT_ID)	String (255)
GENERAL_12	Not used	String (255)

Normalised Event Field Name	Field Usage	Data Type
GENERAL_13	Not used	String (255)
GENERAL_14	Not used	String (255)
GENERAL_15	Not used	String (255)
GENERAL_16	Not used	String (255)
GENERAL_17	Not used	String (255)
GENERAL_18	Reserved. Used to store part of the original record in case it needs to be extracted later.	String (255)
GENERAL_19	Reserved. Used to store part of the original record in case it needs to be extracted later.	String (255)
GENERAL_20	Reserved. Used to store part of the original record in case it needs to be extracted later.	String (255)

Normalised Event Fields – Dialup Usage

Usage feeds from RADIUS and XACCT are supported. For information on these usage feeds, refer to *Non-Native Event Feeds* earlier in this chapter. In the case of RADIUS, both batch and real-time processing of the records are supported. For XACCT, only the batch processing of records is supported.

Rating of dialup event records will vary based on the type of dialup plan the customer has signed up for. With some types of plans, customers are charged for the total amount of time that they spend online. In other cases, the customer pays a set fee for a certain number of hours per billing period and any hours that exceed this threshold are then rated as usage. Other customers may have unlimited usage for one set fee but may pay a charge for any excess data downloaded.

The following algorithms apply to the different types of plans:

Table 21-3 Usage Tariffs for Dialup Services

Usage Tariff	Algorithm
Flat Usage Tariff	Event Charge = Usage Charge Rate x Event Duration
Subscription Tariff	Total Charge = Fixed Charge + (Usage Charge Rate x Excess Usage) + (Download Charge Rate x Excess Download)
Flat Rate Tariff	Total Charge = Fixed Charge + Download Charge Rate x Excess Download

In addition the tariff will vary according to the type of phone line the customer has access to, either normal or premium (with access to more modems) and whether the customer has used a roaming number to access the Internet from a remote location.



After the DIL for the RADIUS Accounting Server Log or XACCT RADIUS Accounting CSV Data Feed maps the `NAS-Identifier` or `NAS-IP-Address` to the `EVENT_SOURCE` field, the decoding expression that is defined in the Normalised Event File Type calls one of two derived attributes to map the `EVENT_TYPE_CODE` as either Dialup Internet, Dialup VPN, DSL Internet, or DSL VPN. A third derived attribute is called to map the `EVENT_SUB_TYPE_CODE` for Dialup Internet events as either 'Standard Usage' or 'Roaming Usage' and for other Dialup and DSL events as 'Standard Usage'. For more information on these derived attributes, refer to *Appendix E, DILs and Configuration Entities for IP/Data Event Feeds.*

Table 21-4 Normalised Event Fields for Dialup Services

Normalised Event Field Name	Detail
`A_PARTY_ID`	Calling-Station-ID.
`B_PARTY_ID`	Called-Station-ID. The Called Station ID can be used in rating to determine whether the source of the record is a normal phone line, a premium phone line (with access to more modems), or a national roaming facility. This will determine which tariff is used to rate the event.
`C_PARTY_ID`	This is used to guide the event to the customer account. The C Party ID must match the *Username* on the *Service Detail Form*.
`EVENT_SOURCE`	`NAS-Identifier` **or** `NAS-IP-Address` (appropriately prefixed).
`EVENT_SUB_TYPE_CODE`	2220 or 2221 (Standard Usage or Roaming Usage).
`DURATION`	`Acct-Session_Time` measured in seconds.
`VOLUME`	Sum of `Acct-Input-Octets` (volume of uploaded data) and `Acct-Output-Octets` (volume of downloaded data) measured in MB.
`GENERAL_1`	`Acct-Input-Octets`.
`GENERAL_2`	`Acct-Output-Octets`.
`GENERAL_3`	`Framed-IP-Address` (user's IP address) This field is used for the purpose of security reporting.

Related reports

The Dialup Trace Report (RDIAL_IP schedule type) provides call history details to help in detecting fraud. For information on this report, refer to *Part 2, Reports* of the *Reports Guide for Singl.eView Convergent Billing ICP and Singl.eView Convergent Billing IP/Data*.

Normalised Event Fields – DSL Internet Usage

Usage feeds from RADIUS and XACCT are supported. For information on these usage feeds, refer to *Non-Native Event Feeds* earlier in this chapter. In the case of RADIUS, both batch and real-time processing of the records are supported. For XACCT, only the batch processing of records is supported.

A typical model for charging usage charges for DSL Internet services involves charging for excess traffic usage.

Maximum throughput per month is defined as a Usage Threshold of 5GB downloaded and 1GB uploaded.

Any usage beyond this is charged at an Excess Volume Rate of $5.00 per GB.

The Volume Threshold tariff will use the `Volume` field in the normalised event record to calculate the charge using the following algorithm:

```
Excess Charge = (Total Usage – Usage Threshold) x Excess
Volume Rate
```



After the DIL for the RADIUS Accounting Server Log or XACCT RADIUS Accounting CSV Data Feed maps the `NAS-Identifier` or `NAS-IP-Address` to the `EVENT_SOURCE` field, the decoding expression that is defined in the Normalised Event File Type calls one of two derived attributes to map the `EVENT_TYPE_CODE` as either Dialup Internet, Dialup VPN, DSL Internet, or DSL VPN. A third derived attribute is called to map the `EVENT_SUB_TYPE_CODE` for Dialup Internet events as either standard or roaming and for other Dialup and DSL events as standard. For more information on these derived attributes, refer to *Appendix E, DILs and Configuration Entities for IP/Data Event Feeds*.

Table 21-5 Normalised Event Fields for DSL Internet Services

Normalised Event Field Name	Detail
C_PARTY_ID	Used to guide the event to the customer account. The C Party ID must match the *Username* on the *Service Detail Form*.
EVENT_SOURCE	NAS-Identifier or NAS-IP-Address (appropriately prefixed).
EVENT_SUB_TYPE_ CODE	2220 (Standard).
DURATION	Acct-Session_Time.
VOLUME	Sum of Acct-Input-Octets (volume of uploaded data) and Acct-Output-Octets (volume of downloaded data)
GENERAL_1	Acct-Input-Octets
GENERAL_2	Acct-Output-Octets
GENERAL_3	Framed-IP-Address (user's IP address) This field is used for the purpose of security reporting.

Normalised Event Fields – Data Services Usage

The usage feed for data services is based on the Singl.eView Convergent Billing Native format. Two events per measurement period are generated for PVC services represented as two half circuit PVC services while one event per measurement period is generated for PVC services represented as a single full circuit PVC service. For half circuit events, the total volume reported in each event shall be the same and equal to half the volume that would be reported for an equivalent full circuit event. Half circuit events allow costs to be split between cost centres for one customer's PVC or for two customers with a data link between them to split the cost.

One event per billing period per PVC service can be generated. Alternatively, multiple events may be generated with each event representing a configurable period of time, such as one day. The minimum period duration is one hour and the maximum is one month. Usage events must be generated for a usage rated PVC for any periods in which the PVC has had non-zero usage.

The event charge is equal to the usage charge rate multiplied by the volume.

Table 21-6 Normalised Event Fields for Data Services

Normalised Event Field Name	Detail
C_PARTY_ID	PVC Identifier. For full circuit events, this must match the *Service Name* on the *Service Detail* form. For half circuit events, this must match the *A End Port* or *B End Port* on the *General* tab of the *Service Detail* form.
EVENT_CLASS	Usage.
EVENT_SUB_TYPE_CODE	Data PVC ATM Usage or Data PVC Frame Usage.
DURATION	Period of time in seconds that the event represents.

Normalised Event Field Name	Detail
VOLUME	Volume of data in GB that was successfully transferred over the PVC in either direction during the period that the usage record represents. For example, if 3 GB were transferred in the A to B direction and 2 GB transferred in the B to A direction, then the Volume would be reported as 5 GB. Fractions of GB are supported. If the event is for a half circuit service, the total volume is divided by two. In the above example, the volume would be 2½ GB.

Related reports

The PVC Usage Variance Report (RPVC_IP schedule type) provides information that allows service providers to view the variance between usage in a target month and a comparison month. This may aid in detecting missing usage data. For information on this report, refer to *Part 2, Reports* of the *Reports Guide for Singl.eView Convergent Billing ICP and Singl.eView Convergent Billing IP/Data*.

Normalised Event Fields – GVAS

The usage feed for Generic Value Added Service (GVAS) is based on the Singl.eView Convergent Billing Native format. Each transaction generates one event. Rather than being a base product, GVAS is a companion product that may be added to Dialup and DSL products. Rates for GVAS products can be determined by GVAS category, which is the EVENT_SUB_TYPE_CODE on the normalised event layout. The GVAS categories are set up in the IP_NE_GVAS_Category reference type. For more information, see *Appendix E, DILs and Configuration Entities for IP/Data Event Feeds*.

The charge for a GVAS event can be included in the event itself. If no charge is included, then the units and charge per unit can be derived from the charging framework. The event can then be rated using the applicable tariff for the GVAS category. The event charge will equal the charge per unit multiplied by the volume.

Table 21-7 Normalised Event Fields for GVAS Services

Normalised Event Field Name	Detail
C_PARTY_ID	Used to guide the event to the customer account. The C_Party_ID must match the Username on the *Service Detail Form* for the DSL or Dialup service.
EVENT_SUB_TYPE_CODE	One of the following: • Miscellaneous (eg. real-time stock market quotes, interactive tutorials) • Games Server • News Server • Games Extra These must match the GVAS categories defined in the IP_NE_GVAS_Category reference type.
CHARGE	The charge can be included in the event itself. If this field is null, the VOLUME field must be populated and a tariff will be used to rate the event.

Normalised Event Field Name	Detail
VOLUME	Number of units of the service used. The units and charge per unit vary by GVAS service category. This field is used in rating and is only populated if the CHARGE field is not populated.
GENERAL_8	Optional additional detail text that can be included on the invoice, such as the name of the game, the name of the tutorial, or the score.

Normalised Event Fields – Web Hosting

There are two possible event feeds for Web Hosting services:

- A data feed for traffic usage that exceeds the threshold shown in the *Web Traffic* field on the *General* tab of the *Service Detail* form.
- A data feed for disk space usage that exceeds the threshold shown in the *Disk Space* field on the *General* tab of the *Service Detail* form.

Whether these feeds will be used or will result in charges will depend on the type of web hosting products. For example, products intended for advanced small business users may have a threshold for both disk space and traffic. Dedicated web hosting products intended for use by medium sized businesses would have a threshold for traffic usage only because the customer is effectively paying for the amount of disk space through their rental of the server.

Both the traffic and disk usage data feeds are based on the Singl.eView Convergent Billing Native format and contain one event per web hosting service per billing period (typically one month).

Usage charges are generated when the customer exceeds the thresholds for disk usage or traffic usage. The excess disk or traffic usage charge is calculated using the following algorithm:

```
Excess Charge = (Total Usage–Usage Threshold) x Excess Rate
```

If no excess usage charge applies to a web hosting product, excess disk or traffic usage events are still rated successfully with a zero charge.

Table 21-8 Normalised Event Fields for Web Hosting Traffic Usage

Normalised Event Field Name	Detail
C_PARTY_ID	URL for the web hosting service. This field needs to match the URL of a web hosting service stored on the *Service Detail* form.
EVENT_SUB_TYPE_CODE	Web Hosting Traffic Usage.
CHARGE_START_DATE	Start date and time of the period that the event represents.
CHARGE_END_DATE	End date and time of the period that the event represents.
DURATION	Volume of traffic measured in MB for the service for the billing period.

Table 21-9 Normalised Event Fields for Web Hosting Disk Usage

Normalised Event Field Name	Detail
C_PARTY_ID	URL for the web hosting service. This field needs to match the URL of a web hosting service stored on the *Service Detail* form.
EVENT_SUB_TYPE_CODE	Web Hosting Disk Usage.
CHARGE_START_DATE	Start date and time of the period that the event represents.
CHARGE_END_DATE	End date and time of the period that the event represents.
DURATION	Disk usage measured in MB for the service for the billing period.

Normalised Event Fields – Payments Gateway

Each raw event represents a payment transaction and is based on the Singl.eView Convergent Billing Native format. The raw events are fed in via an interface between the E-commerce server that processes the transactions and the Singl.eView Convergent Billing rating engine.

The frequency of the Payments Gateway usage feed is configurable. However, the billing cycle for the Payments Gateway product must be monthly.

The usage charge can be based on tiered rating of the total number of transactions in a month. Using a tiered rating scheme, the usage rate per transaction would vary with increasing levels of transactions. For example, the usage rate for 0 to 10,000 transactions in an event might be $0.50, but for 10,001 to 20,000 transactions, it might be $0.45. The rate for an event is the same whether the payment was accepted or rejected by the bank.

Table 21-10 Normalised Event Fields for Payments Gateway Services

Normalised Event Field Name	Detail
C_PARTY_ID	Merchant ID code that uniquely identifies the retailer to the ISP. This code is recorded on the *Service Detail* form and is used to guide the transaction event to the retailer's account. It is issued by the bank when a retailer sets up a business account.
EVENT_SUB_TYPE_ CODE	Payments Gway Transaction Summary or Payments Gway Transaction Detail
GENERAL_1	Card t ype code that identifies the credit card or debit card used in the transaction.
CHARGE_START_DATE	Transaction date and time.
GENERAL_2	Transaction number for the retailer. This is assigned by the bank for the payment transaction.
CHARGE	Dollar value of the transaction.
GENERAL_3	Flag that indicates whether the transaction was approved or declined by the bank.

Chapter 22
Speed Value Management

Speed values that indicate the speed of data transfer are defined for many IP/Data services. Because these speeds use a wide range of measurements and cover a wide range of values, it is necessary to maintain the values in a master reference type (one for single speeds and one for ADSL dual speeds). To create the drop-down lists of speed values that are part of different IP/Data services, copies of this master can be taken and only those record rows enabled that are applicable to that speed type and service. Figures 22-1 and 22-2 show examples of speed value drop-down lists. The first shows an ATM PVC service with single-speed values in the drop-down list (in this case for the Sustained Cell Rate (SCR) for the A to B half circuit). The second shows a DSL Internet service with dual-speed values.

When updates or additions are needed, the master reference type must be updated. Then one of two schedule types can be run to update the individual reference type instances for each type of speed value. In order for these schedules to update all existing individual reference type instances, each instance must be entered in the `IP_SPEED_INSTANCES` reference type or in the `IP_DUALSPEED_INSTANCES` reference type.

This organisation of speed values in the system allows for functions at both the EPM and SQL levels to be used to compare speed values of different formats (eg. 64kbit/s < 2Mbit/s) and to calculate equivalent values for different formats (eg. 64 kbit/s =64000 bit/s or 0.064 Mbit/s).

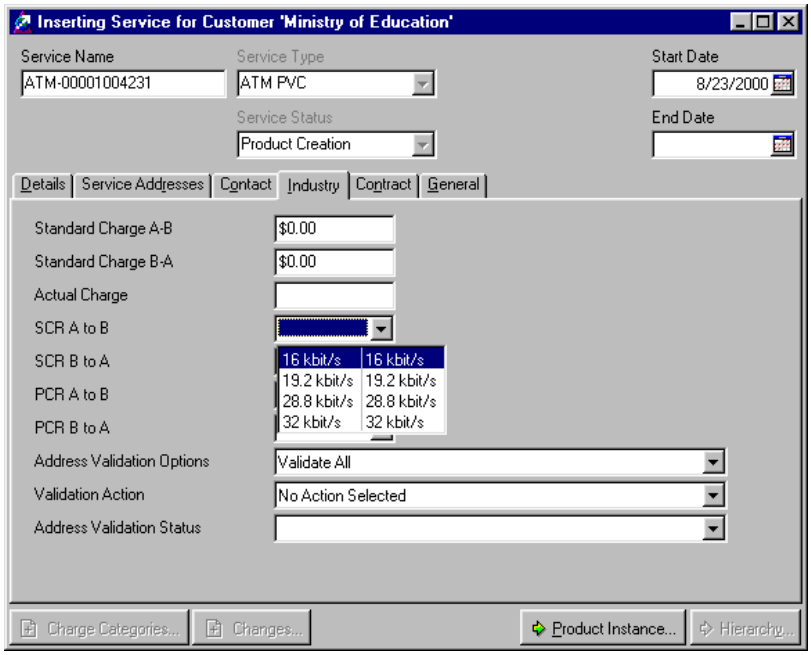


Figure 22-1 Example of Single-speed Values in a Drop-down List for an ATM PVC Service Instance

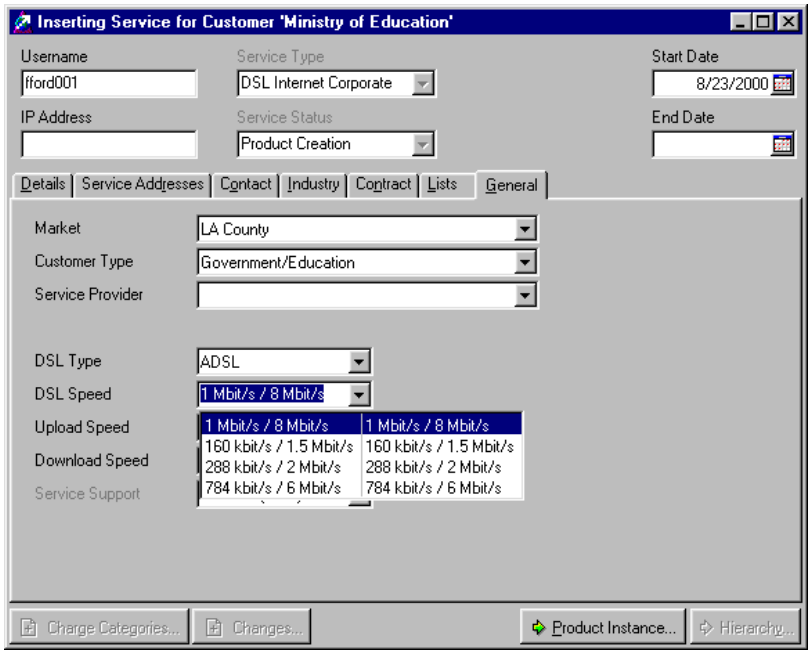


Figure 22-2 Example of Dual-speed Values in a Drop-down List for a DSL Internet Service Instance

This chapter contains the following sections:

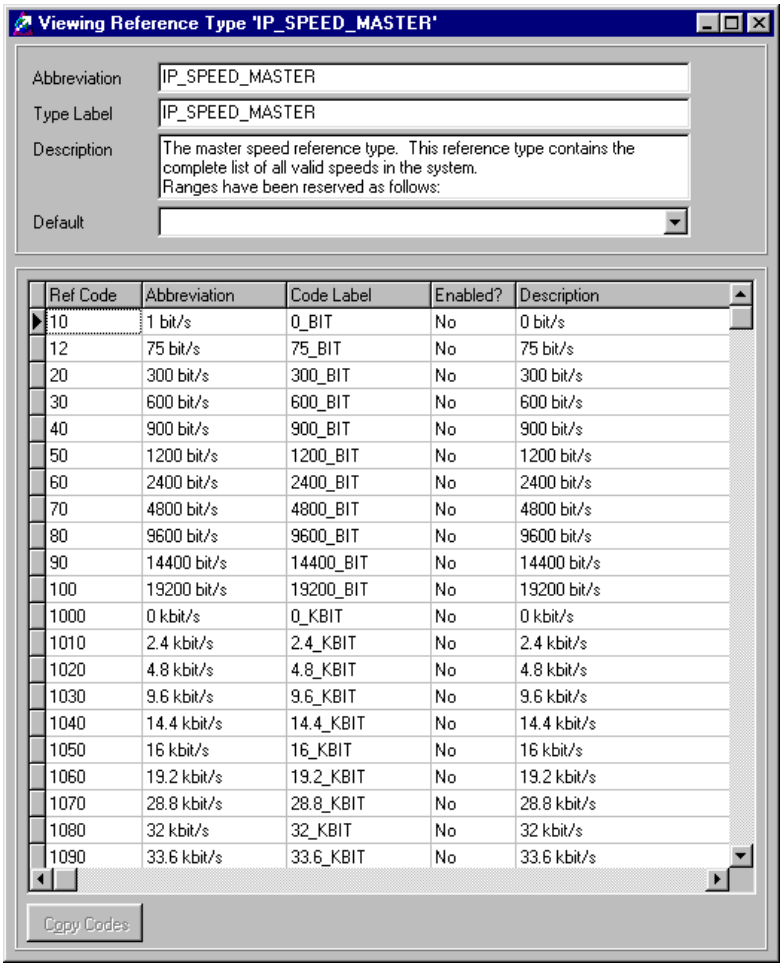
- Master Reference Types for Speed Values
- IP_SPEED_INSTANCES and IP_DUALSPEED_INSTANCES Reference Types
- Speed Reference Type Instances
- Adding New Speed Value Reference Type Instances
- Adding and Changing Speed Values
- Update Speed Reference Types Schedule Types.

Master Reference Types for Speed Values

All speed values in the system must be maintained using one of two Master Reference Types:

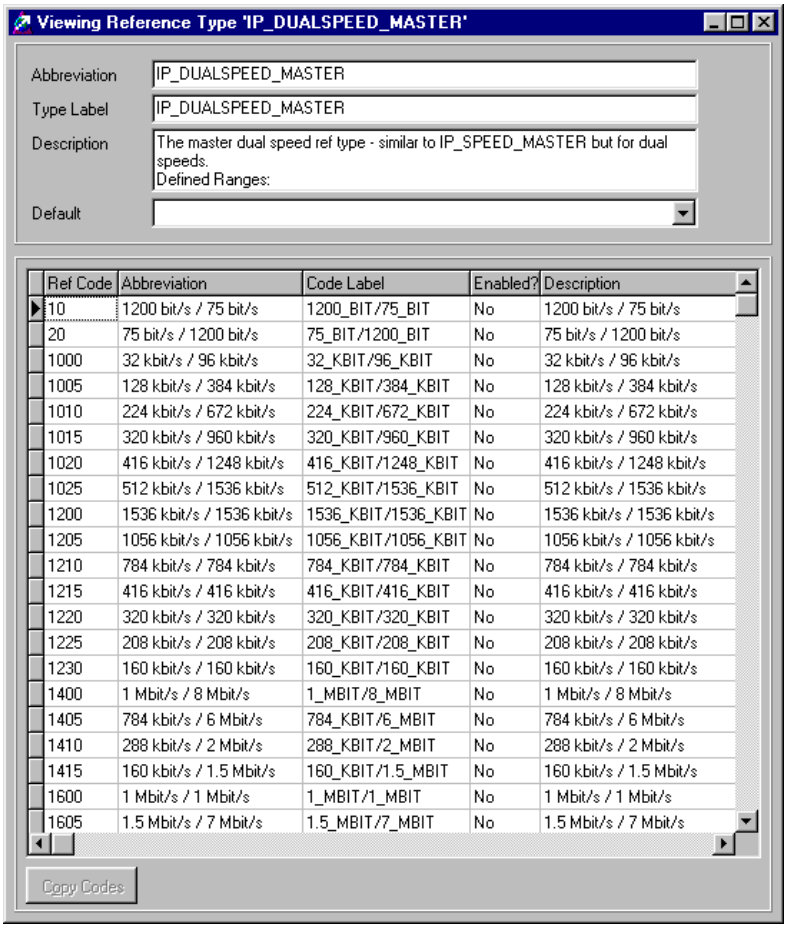
- Single-speed values are maintained in the IP_SPEED_MASTER reference type.
- Dual-speed values are maintained in the IP_DUALSPEED_MASTER reference type.

Figure 22-3 shows the IP_SPEED_MASTER reference type populated with sample values. Figure 22-4 shows the IP_DUALSPEED_MASTER reference type populated with sample values.



Ref Code	Abbreviation	Code Label	Enabled?	Description
10	1 bit/s	0_BIT	No	0 bit/s
12	75 bit/s	75_BIT	No	75 bit/s
20	300 bit/s	300_BIT	No	300 bit/s
30	600 bit/s	600_BIT	No	600 bit/s
40	900 bit/s	900_BIT	No	900 bit/s
50	1200 bit/s	1200_BIT	No	1200 bit/s
60	2400 bit/s	2400_BIT	No	2400 bit/s
70	4800 bit/s	4800_BIT	No	4800 bit/s
80	9600 bit/s	9600_BIT	No	9600 bit/s
90	14400 bit/s	14400_BIT	No	14400 bit/s
100	19200 bit/s	19200_BIT	No	19200 bit/s
1000	0 kbit/s	0_KBIT	No	0 kbit/s
1010	2.4 kbit/s	2.4_KBIT	No	2.4 kbit/s
1020	4.8 kbit/s	4.8_KBIT	No	4.8 kbit/s
1030	9.6 kbit/s	9.6_KBIT	No	9.6 kbit/s
1040	14.4 kbit/s	14.4_KBIT	No	14.4 kbit/s
1050	16 kbit/s	16_KBIT	No	16 kbit/s
1060	19.2 kbit/s	19.2_KBIT	No	19.2 kbit/s
1070	28.8 kbit/s	28.8_KBIT	No	28.8 kbit/s
1080	32 kbit/s	32_KBIT	No	32 kbit/s
1090	33.6 kbit/s	33.6_KBIT	No	33.6 kbit/s

Table 22-3 IP_SPEED_MASTER Reference Type with Sample Values



Ref Code	Abbreviation	Code Label	Enabled?	Description
10	1200 bit/s / 75 bit/s	1200_BIT/75_BIT	No	1200 bit/s / 75 bit/s
20	75 bit/s / 1200 bit/s	75_BIT/1200_BIT	No	75 bit/s / 1200 bit/s
1000	32 kbit/s / 96 kbit/s	32_KBIT/96_KBIT	No	32 kbit/s / 96 kbit/s
1005	128 kbit/s / 384 kbit/s	128_KBIT/384_KBIT	No	128 kbit/s / 384 kbit/s
1010	224 kbit/s / 672 kbit/s	224_KBIT/672_KBIT	No	224 kbit/s / 672 kbit/s
1015	320 kbit/s / 960 kbit/s	320_KBIT/960_KBIT	No	320 kbit/s / 960 kbit/s
1020	416 kbit/s / 1248 kbit/s	416_KBIT/1248_KBIT	No	416 kbit/s / 1248 kbit/s
1025	512 kbit/s / 1536 kbit/s	512_KBIT/1536_KBIT	No	512 kbit/s / 1536 kbit/s
1200	1536 kbit/s / 1536 kbit/s	1536_KBIT/1536_KBIT	No	1536 kbit/s / 1536 kbit/s
1205	1056 kbit/s / 1056 kbit/s	1056_KBIT/1056_KBIT	No	1056 kbit/s / 1056 kbit/s
1210	784 kbit/s / 784 kbit/s	784_KBIT/784_KBIT	No	784 kbit/s / 784 kbit/s
1215	416 kbit/s / 416 kbit/s	416_KBIT/416_KBIT	No	416 kbit/s / 416 kbit/s
1220	320 kbit/s / 320 kbit/s	320_KBIT/320_KBIT	No	320 kbit/s / 320 kbit/s
1225	208 kbit/s / 208 kbit/s	208_KBIT/208_KBIT	No	208 kbit/s / 208 kbit/s
1230	160 kbit/s / 160 kbit/s	160_KBIT/160_KBIT	No	160 kbit/s / 160 kbit/s
1400	1 Mbit/s / 8 Mbit/s	1_MBIT/8_MBIT	No	1 Mbit/s / 8 Mbit/s
1405	784 kbit/s / 6 Mbit/s	784_KBIT/6_MBIT	No	784 kbit/s / 6 Mbit/s
1410	288 kbit/s / 2 Mbit/s	288_KBIT/2_MBIT	No	288 kbit/s / 2 Mbit/s
1415	160 kbit/s / 1.5 Mbit/s	160_KBIT/1.5_MBIT	No	160 kbit/s / 1.5 Mbit/s
1600	1 Mbit/s / 1 Mbit/s	1_MBIT/1_MBIT	No	1 Mbit/s / 1 Mbit/s
1605	1.5 Mbit/s / 7 Mbit/s	1.5_MBIT/7_MBIT	No	1.5 Mbit/s / 7 Mbit/s

Table 22-4 IP_DUALSPEED_MASTER Reference Type with Sample Values

Reference codes

The reference code is the internal identifier of a speed value. Reference code ranges should be defined for each speed format. For example, all bit/s values should be stored in one range (such as 1 to 100) and all kbit/s values should be stored in another range (such as 1000 to 1999). To allow for future expansion, the ranges should be relatively large, and the reference codes used within a range should not be consecutive. For example, each time a new reference code is added, the reference code should be incremented by ten (as shown in the reference codes in figure 22-3). This will allow future speed values to be added that lie between two existing values.

Abbreviation

The abbreviation is the name of the speed value that will be displayed on the drop-down list in the form (eg. 64 kbit/s).

The following is a list of the speed formats that are currently populated in the `Abbreviation` column:

1. Bit/s (bits per second)
2. Kbit/s (kilobits per second)
3. Mbit/s (Megabits per second)
4. Gbit/s (Gigabits per second)
5. Cell/s (Cells per second – a measure of ATM traffic speed)
6. kCell/s (Kilocells per second).

Code Label for single-speed values

The *Code Label* field is used to store a formal representation of the speed value. The *Code Label* is used by the EPM and SQL functions to process speed values. The format of the field is

`NNNNNNNN_FFFFF`

where

`NNNNNNNN` is a positive number representing the size of the speed. Leading zeroes are not required. The number may contain a decimal place. If there is a decimal place, there must be a digit on either side of the decimal place.

`FFFFF` is a code representing the format of the speed. Some codes are less than five characters. In these cases, the width of the field is reduced accordingly. The following are valid formats:

- BIT
- KBIT
- MBIT
- GBIT
- CELL
- KCELL.

The following are examples of valid code labels:

- 64_KBIT
- 2_MBIT
- 19.2_KBIT

Code Label for dual-speed values

Dual-speed code labels are formatted as two single-speed value code labels separated by a slash:

`NNNNNNNN_FFFFF/NNNNNNNN_FFFFF`



Each code label in the `IP_DUALSPEED_MASTER` reference type must correspond to existing code labels in the `IP_SPEED_MASTER` reference type. For example, do not enter a code label of `'2_MBIT/300_KBIT'` unless there are `'2_MBIT' and '300_KBIT'` code labels in the `IP_SPEED_MASTER` reference type.

Enabled?

The *Enabled?* field should be set to 'No' for all record rows in the master speed reference type. When the record is first added, this field will default to 'Yes'. It should be changed to 'No'. Then when a copy of the master speed reference type is taken to create a set of speed values, it is only necessary to select the speeds to enable rather than selecting the ones to disable.

Description

This field contains an optional description of the speed value.

IP_SPEED_INSTANCES and IP_DUALSPEED_INSTANCES Reference Types

Every time a copy is made of a master reference type to create a reference type instance that stores a particular set of speed values for particular kinds of IP/Data services, the new reference type instance must be added to the `IP_SPEED_INSTANCES` reference type or the `IP_DUALSPEED_INSTANCES` reference type. The first is used for single-speed value reference types and the second is used for dual-speed value reference types. When the Update Speed Reference Types and Update Dual Speed Reference Types schedule types are run, only those reference types listed in the `IP_SPEED_INSTANCES` and `IP_DUALSPEED_INSTANCES` reference types are updated with new or changed values.

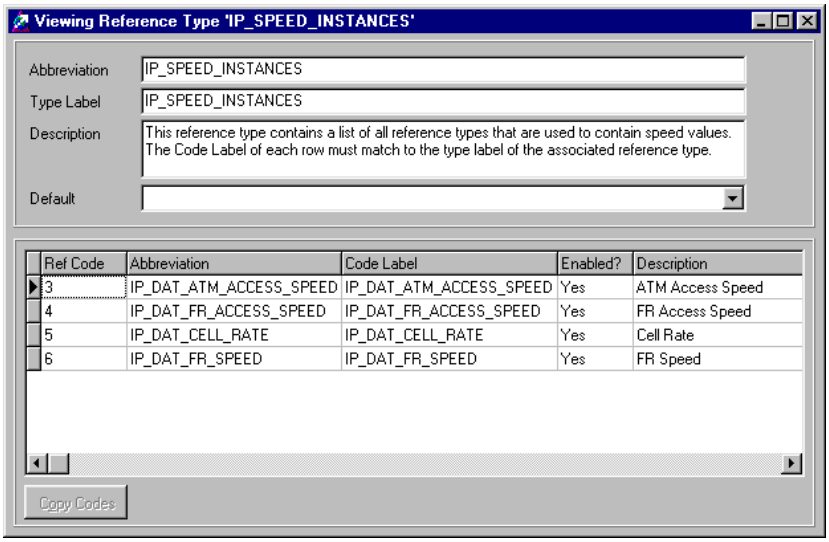


Figure 22-5 IP_SPEED_INSTANCES Reference Type with Sample Data

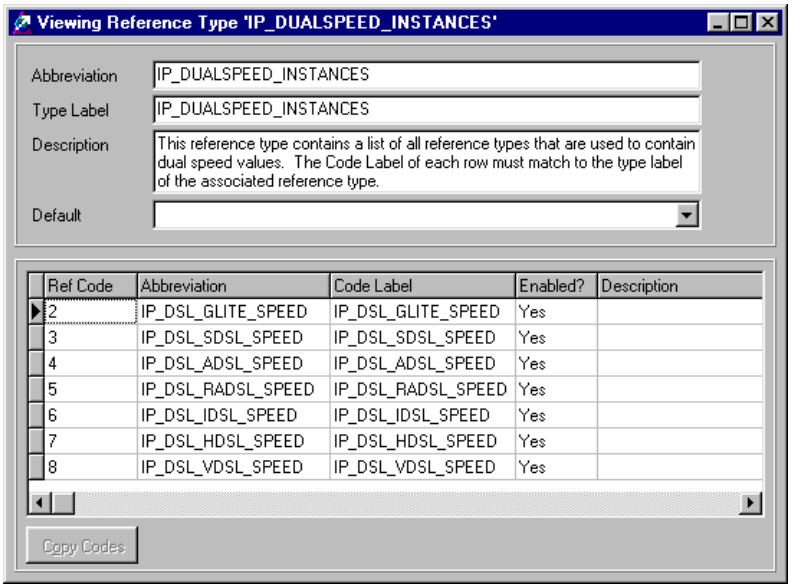


Figure 22-6 IP_DUALSPEED_INSTANCES Reference Type with Sample Data

Speed Reference Type Instances

To create a new set of speed values to be used in IP/Data services, a copy must be taken of the applicable speed master reference type. To create a reference type that contains single-speed values, copy the `IP_SPEED_MASTER` reference type. To create a reference type that contains dual-speed values, copy the `IP_DUALSPEED_MASTER` reference type. For more information, refer to *Adding New Speed Value Reference Type Instances* and *Adding and Changing Speed Values* later in this chapter. Figure 22-7 shows an example of a speed value reference type instance.



Any attribute types that point to a speed reference type instance should not be de-normalised. The EPM and SQL functions that process speed values rely on the speed Reference Code being stored rather than the Abbreviation text string. For more information, refer to *Chapter 5, Attribute Types* of the *Customer Care Configuration Guide for Singl.eView Convergent Billing*.

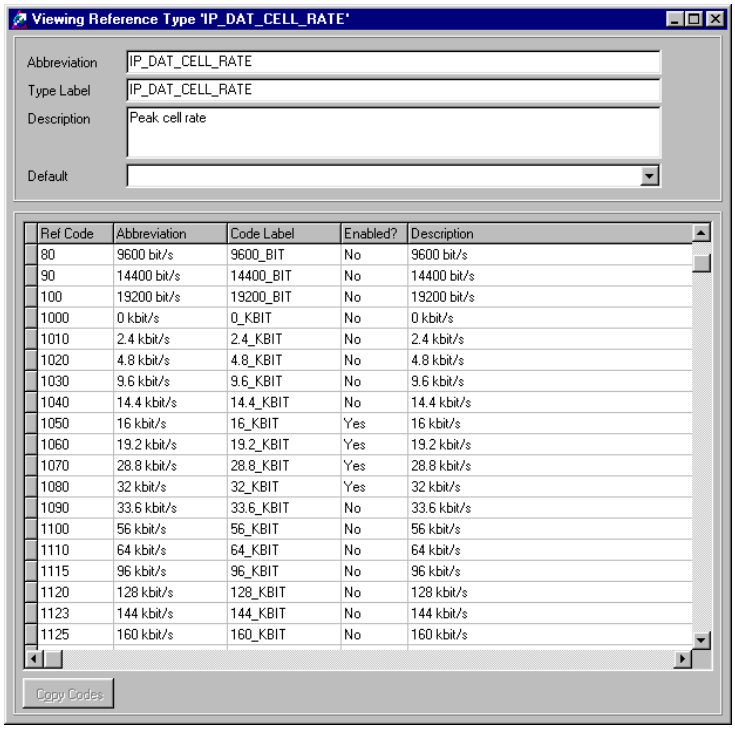


Figure 22-7 Example of a Speed Value Reference Type Instance



When any changes or additions to the set of values contained in a reference type instance are needed, the changes must be made to the applicable master reference type. Changes and additions should not be made directly to the reference type instance. Once the master reference type is updated, the Update Speed Reference Type or Update Dual-speed Reference Type schedule types should be run to update the reference type instances. For more information, refer to *Adding New Speed Value Reference Type Instances* and *Adding and Changing Speed Values* later in this chapter.

Adding New Speed Value Reference Type Instances

To create a new set of speed values to be used in IP/Data services, a copy must be taken of the applicable master speed reference type. For instructions on how to copy reference types, refer to *Chapter 4, Defining and Maintaining Reference Types* of the *Customer Care Configuration Guide for Singl.eView Convergent Billing*.



Any attribute types that point to a speed reference type instance should not be de-normalised. The EPM and SQL functions that process speed values rely on the speed Reference Code being stored rather than the Abbreviation text string. For more information, refer to *Chapter 5, Attribute Types* of the *Customer Care Configuration Guide for Singl.eView Convergent Billing*.

The following is a summary of the steps that should be followed to create a new speed value reference type instance. If the reference type instance being added contains speed values that are already records in the master reference type, follow steps 1 through 7. If the reference type instance being added contains new speed values that are not included in the master reference type, follow steps 1 through 14.

To add a new speed reference type instance:

1. To create a reference type that contains single-speed values, copy the `IP_SPEED_MASTER` reference type. To create a reference type that contains dual-speed values, copy the `IP_DUALSPEED_MASTER` reference type. For instructions on how to copy reference types, refer to *Chapter 4, Defining and Maintaining Reference Types* of the *Customer Care Configuration Guide for Singl.eView Convergent Billing.*

2. Once a new reference type instance (such as the one shown in figure 22-7) has been created, enter the *Abbreviation*, *Type Label* and *Description*.

3. Click the **Save** button on the Singl.eView Convergent Billing tool bar.

4. If speed values that are not already listed in the master reference type are needed to complete the new set of speed values in the reference type instance, skip to step 8. If the reference type instance being added contains speed values that are already records in the master reference type, leave the *Reference Type Definition* form open and click the **Update** button on the Singl.eView Convergent Billing tool bar.

5. Change the *Enabled?* column from 'No' to 'Yes' for the records that are included in the new set of speed values.

6. Click the **Save** button on the Singl.eView Convergent Billing tool bar.

7. If the new reference type instance contains single-speed values, add a record for the reference type instance to the `IP_SPEED_INSTANCES` reference type. If the new reference type instance contains dual-speed values, add a record for the reference type instance to the `IP_DUALSPEED_INSTANCES` reference type. For more information, refer to *IP_SPEED_INSTANCES and IP_DUALSPEED_INSTANCES Reference Types* earlier in this chapter. Stop here. The new reference type instance has been added.

8. If speed values that are not already listed in the master reference type are needed to complete the new set of speed values in the reference type instance, the master reference type should be updated.



Enabling or disabling rows is the only change that should be made directly to a reference type instance. When any changes or additions to the set of values contained in a reference type instance are needed, the changes must be made to the applicable master reference type. Once the master reference type is updated, the Update Speed Reference Types or Update Dual-speed Reference Types schedule types should be run to update the reference type instances.

9. Add any needed values to the master reference type. To add single-speed values, update the `IP_SPEED_MASTER`. To add dual-speed values, update the `IP_DUALSPEED_MASTER`. Ensure that when new records are added that the *Enabled?* column is set to 'No'. For more information, refer to *Master Reference Types for Speed Values* earlier in this chapter.

10. If the new reference type instance contains single-speed values, add a record for the reference type instance to the `IP_SPEED_INSTANCES` reference type. If the new reference type instance contains dual-speed values, add a record for the reference type instance to the `IP_DUALSPEED_INSTANCES` reference type. For more information, refer to *IP_SPEED_INSTANCES and IP_DUALSPEED_INSTANCES Reference Types* earlier in this chapter.

11. To update all reference type instances with the new speed values, run the Update Speed Reference Types schedule type for single-speed values or the Update Dual Speed Reference Types schedule type for dual-speed values. For information on these schedule types, refer to *Update Speed Reference Type Schedule Types* later in this chapter.

12. Open the *Reference Type Definition* form for the new reference type instance and click the **Update** button on the Singl.eView Convergent Billing tool bar.

13. Change the *Enabled?* column from 'No' to 'Yes' for the records that are included in the new set of speed values.

14. Click the **Save** button on the Singl.eView Convergent Billing tool bar.

Adding and Changing Speed Values

Whenever additions or changes are needed to speed values in the reference type instances, the applicable master reference type must be updated. The Update Speed Reference Type or Update Dual Speed Reference Type schedule types can then be run to update all existing reference type instances with the changes.



This procedure gives instructions on how to add and change speed values when no new reference type instances are being added. For instructions on how to add and change speed values when a new reference type instance is being added, refer to *Adding New Speed Value Reference Type Instances* in the previous section.



Enabling or disabling rows is the only change that should be made directly to a reference type instance. When any changes or additions to the set of values contained in a reference type instance are needed, the changes must be made to the applicable master reference type. Once the master reference type is updated, the Update Speed Reference Types or Update Dual Speed Reference Types schedule types should be run to update the reference type instances.

To add and change speed values:

1. Add or change any needed values in the master reference type. To add or change single-speed values, update the `IP_SPEED_MASTER`. To add or change dual-speed values, update the `IP_DUALSPEED_MASTER`. Ensure that when new records are added that the *Enabled?* column is set to 'No'. For more information, refer to *Master Reference Types for Speed Values* earlier in this chapter.
2. To update all reference type instances with the new speed values, run the Update Speed Reference Types schedule type for single-speed values or the Update Dual Speed Reference Types schedule type for dual-speed values. For information on these schedule types, refer to *Update Speed Reference Type Schedule Types* later in this chapter.

3. Enable rows on the applicable speed value reference type instances. Open the *Reference Type Definition* form for the first speed value reference type instance and click the **Update** button on the Singl.eView Convergent Billing tool bar.

4. Identify the added or changed speed values that should be enabled in the reference type instance. Change the *Enabled?* column from 'No' to 'Yes' for these records.

5. Click the **Save** button on the Singl.eView Convergent Billing tool bar.

6. Repeat steps 3 through 5 for all speed value reference type instances that contain speed values that must be enabled.

Update Speed Reference Types Schedule Types

There are two schedule types for updating speed reference type instances:

- The Update Speed Reference Type schedule type should be run after updating the `IP_SPEED_MASTER` reference type.
- The Update Dual Speed Reference Type schedule type should be run after updating the `IP_DUALSPEED_MASTER` reference type.

Running either schedule will transfer the changes made to the master reference type to each of the existing reference type instances.

Scheduling

These schedule types are typically run as immediate tasks after either the `IP_SPEED_MASTER` or `IP_DUALSPEED_MASTER` reference types have been updated. For more information on what steps are taken before the schedule is run, refer to *Adding New Speed Value Reference Type Instances* and *Adding and Changing Speed Values* earlier in this chapter. For more information on how to schedule an immediate task, refer to *Scheduling an immediate one-off task* in *Chapter 2, Scheduling Tasks*.

Scheduling parameters

To run this schedule, two mandatory parameters must be specified on the *Parameters* tab of the *Schedule Definition* form:

1. The *Test Mode?* field indicates whether the speed reference instances will actually be updated in the Singl.eView Convergent Billing database or whether the schedule will only generate a listing of which speed values in which reference type instances are in need of updating. To generate a detailed list, the *Verbose Reporting?* field must be set to 'Yes'. Selecting 'Yes' in the *Test Mode?* field indicates that only a listing will be generated. If the schedule is being run in test mode, any changes to the database will be revoked before the schedule terminates. Selecting 'No' indicates that the speed reference type instances will be updated.

2. The *Verbose Reporting?* field indicates whether the output for the task will include summary counts only or full details of which speed values were updated in which reference type instances . Selecting 'Yes' indicates that full details will be generated. Selecting 'No' indicates that only summary information will be generated. For more information, refer to *Output Results* later in this chapter.

In addition, two fields, the Master Reference Type and Instance Reference Type fields are automatically populated with the names of the applicable reference types. These parameters will be set to `IP_SPEED_MASTER` and `IP_SPEED_INSTANCES` for the Update Speed Reference Types schedule type and to `IP_DUALSPEED_MASTER` and `IP_DUALSPEED_INSTANCES` for the Update Dual Speed Reference Types schedule type.

Actions triggered by running the Update Speed Reference Types schedule types

When either the Update Speed Reference Type or Update Dual Speed Reference Type schedule types are run, the following actions are triggered:

1. A comparison is made between the applicable master reference type and any speed reference type instances listed on the applicable instance reference type.

2. Any missing rows in the speed value reference type instances are identified and automatically added. The *Enabled?* column in the records that are added is set to 'No'.

3. Any mismatches between the Code Labels in the applicable master reference type and speed value reference type instances are identified. For any mismatches that are found, the reference type instance is updated.

Output results

The success or failure of the run can be determined by checking the *Task Status* field and the *Results* tab of the *Task Details* form. The task ID is also displayed on the *Results* tab and should be noted in case the task needs to be viewed later. For more information on how to access this data, refer to *Using Schedules* in *Chapter 2, Scheduling Tasks*. Alternatively, the output results can be sent by email. For information about this feature, refer to *Fields* in *Chapter 2, Scheduling Tasks.*

The output on the *Results* tab will depend on whether the schedule has been run in Test mode or not and whether verbose reporting has been requested or not.

Test mode

If the schedule is run in test mode and verbose reporting is requested, the output details which speed values were updated on which reference type instances. If verbose reporting is not requested, the output contains only summary information stating how many rows were updated and the total added, deleted, and updated. The output is the same as it would be if the schedule were run in non-testing mode, but the actual updates are not present in the reference type instances when the schedule is complete. Any changes to the database are revoked before the schedule terminates.

Non-testing mode

If 'No' is selected in the *Test Mode?* field and verbose reporting is requested, the output details which speed values were updated on which reference type instances. Figure 22-7 shows an example of the text from the *Results* tab when the schedule is run in non-testing mode and verbose reporting is requested.

If 'No' is selected in the *Test Mode?* field and verbose reporting is not requested, the output contains only summary information stating how many rows were updated for each reference type instance and stating the total added, deleted, and updated. Figure 22-8 shows an example of the output on the *Results* tab when the schedule is run in non-testing mode and verbose reporting is not requested.

Troubleshooting

If the run fails to update all the reference type instances, check the following:

1. Ensure that all reference type instances have been listed in the `IP_SPEED_INSTANCES` reference type for single-speed values or in the `IP_DUALSPEED_INSTANCES` reference type for dual-speed values.
2. Ensure that 'No' is selected in the *Test Mode?* field.
3. Ensure that the changes made to the master reference type are accurate and that they have been saved.

```
<S90006> update_ref_types Started Sat Aug 26 12:50:06 2000.

Processing Reference Type IP_CIR_SPEEDS
Number of rows added: 1
Code            Code Label                  Abbreviation                Description
---- ------------------------------ ------------------------------ -----------
2840            160_MBIT                    160 Mbit/s                  160 Mbit/s

Processing Reference Type IP_DAT_ATM_ACCESS_SPEED
Number of rows added: 1
Code            Code Label                  Abbreviation                Description
---- ------------------------------ ------------------------------ -----------
2840            160_MBIT                    160 Mbit/s                  160 Mbit/s


Processing Reference Type IP_DAT_FR_ACCESS_SPEED
Number of rows added: 1
Code            Code Label                  Abbreviation                Description
---- ------------------------------ ------------------------------ -----------
2840            160_MBIT                    160 Mbit/s                  160 Mbit/s


Processing Reference Type IP_DAT_CELL_RATE
Number of rows added: 1
Code            Code Label                  Abbreviation                Description
---- ------------------------------ ------------------------------ -----------
2840            160_MBIT                    160 Mbit/s                  160 Mbit/s


Processing Reference Type IP_DAT_FR_SPEED
Number of rows added: 1
Code            Code Label                  Abbreviation                Description
---- ------------------------------ ------------------------------ -----------
2840            160_MBIT                    160 Mbit/s                  160 Mbit/s



Completed Processing
Total Number of rows deleted: 0
Total Number of rows added: 5
Total Number of rows updated: 0
<S90007> update_ref_types Completed Sat Aug 26 12:50:16 2000.
```

Figure 22-7 Output for Non-Testing Mode and Verbose Reporting

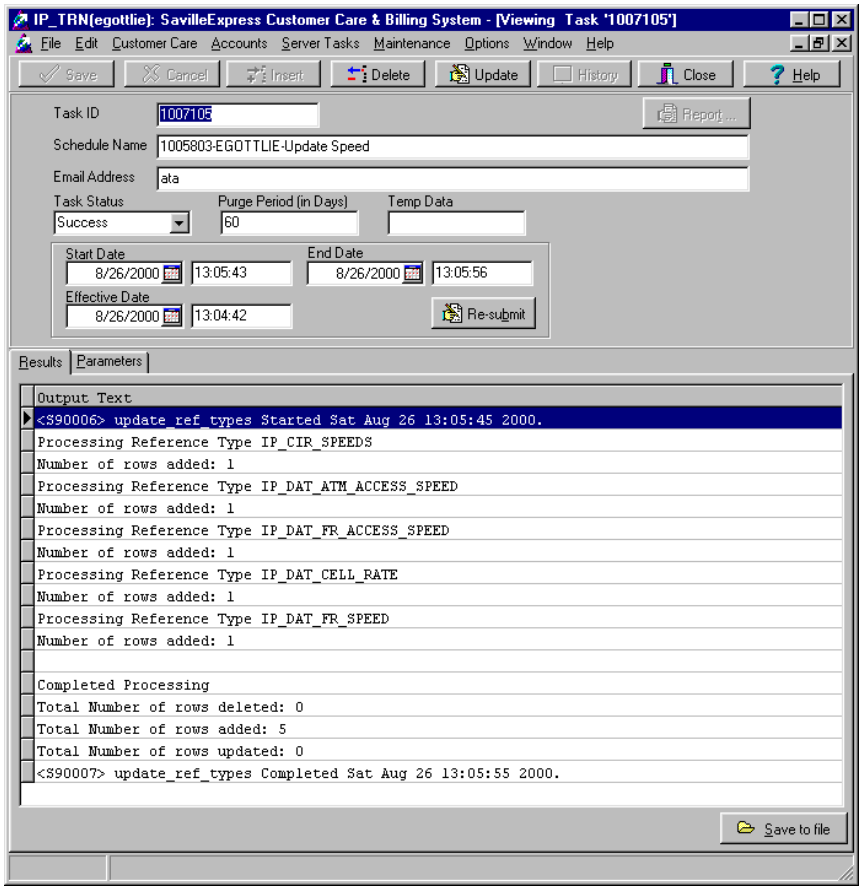


Figure 22-8 Output for Non-Testing Mode and Summary Report Only (No Verbose Reporting)

Speed Value Management

Part 3
Security for Web Enablement

This is a special supplement to the Singl.eView ICP and Singl.eView IP/Data Operational Support Guide and contains one chapter, *Chapter 23, Security for Web Enablement*.

Chapter 23
Security for Web Enablement

In order to enhance their levels of customer service, more and more telecommunications and internet service providers are giving their customers access to service and account information via the internet.

Specially developed interfaces between Singl.eView Convergent Billing and the internet provide web users with the ability to inquire about their customer details. Web users who are not customers, but who would like further information, can submit inquiries about products and services.

As well as functionality for the customer, web enablement must also address the issue of security. Users accessing the system via the internet must be authenticated and authorised to access data. Confidential information transferred across the internet needs to be sent by secure methods.

This chapter covers security for web enablement and contains the following sections:

- Security and Web Enablement
- Setting up Web Users
- Permissions
- Roles
- Encryption
- Auditing.

Security and Web Enablement

Companies that expose their business systems to the internet increase the risk that their systems may be accessed by unauthorised users. Therefore security is of paramount importance.

Singl.eView Convergent Billing web enablement uses many security features to guard against unauthorised access and to ensure data transferred across the internet is secure.

The following sections outline these features and how they are implemented for web enabled customers.

Authentication

A user wanting access to Singl.eView Convergent Billing customer and account data via the internet must first be authenticated as a user with privileges to that data.

The default mechanism to authenticate users is through the use of a user name and password in conjunction with a RADIUS server.

This is not the only method, however, and Singl.eView Convergent Billing allows for the integration of other methods and systems, depending on the service provider's requirements.

Passwords

Web user passwords are stored on the RADIUS server. They can be changed by the user or by a Customer Service Representative (CSR) through the Singl.eView Convergent Billing client.

Sessions

During the period after a web user is authenticated, and until the user logs out (or is logged out after a period of inactivity has passed), users do not need to be re-authenticated for any of the functions they perform. This period is known as a session.

In regard to sessions and security, the web interface to Singl.eView Convergent Billing performs the following functions:

- Starts a new session for each web user upon successful authentication of the user.
- Identifies requests as belonging to a particular session and to a particular web user.
- Allows a web user to terminate a session.
- Automatically terminates a session after a configurable period of inactivity.

Authorisation

After a web user is authenticated and allowed access to the system, there is still the question of what data they are authorised to access and what functions they are authorised to perform.

Web enablement for Singl.eView Convergent Billing uses a series of permissions, defined for each web user, to specify the data and the functions a user is authorised to use or access.

The different types of permission and how they are set up are outlined in the following sections.

Setting up Web Users

Customers do not have automatic access to their customer information via the internet. Web user access must be set up via the contact details for the customer.

A web username, web user role and web user password must be specified for the customer contact. The contacts who have access to customer information can then be specified, via the Customer details.

A sample *Person Detail* form that shows how to set up a web user is displayed in figure 23-1.

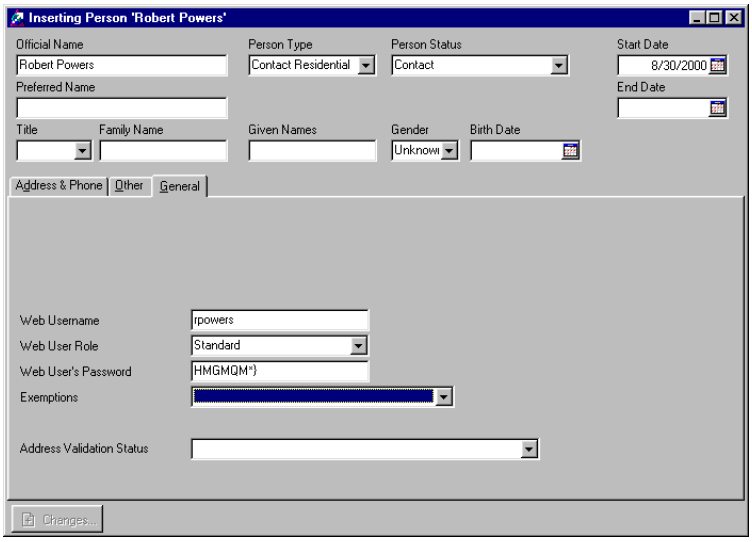


Figure 23-1 Person Detail Form with Web User Details

Note: On the *Person Detail* form shown in figure 23-1, the CSR would enter the appropriate web username and select a web user role. The *Web User's Password* field is completed automatically. The CSR can change this field if required.

Updates to the *Web User's Password* field are stored on the RADIUS server.

A sample *Customer Detail* form showing how to allow a web user access to customer information is displayed in figure 23-2.

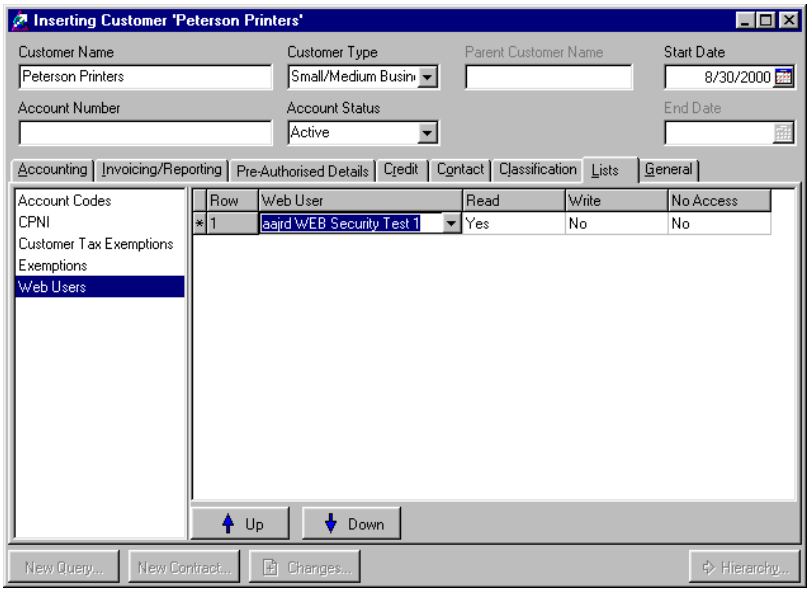


Figure 23-2 Lists Tab on Customer Detail Form with Web User List

Note: In figure 23-2, the CSR selects the appropriate web user from the drop-down list, and then selects whether the user will have 'Read', 'Write' or 'No Access' to their customer data.

Permissions

Permissions specify whether a user can access a particular function or item of data. Depending on what the user is trying to access, there are a number of different permission levels, including:

- No Access
- Execute
- Read
- Write
- Create
- Delete.

If permissions are not specifically set, users have no access. This is distinct from setting a 'No Access' permission, which can be used to override other permissions. Specifying 'No Access' ensures a user never gains access to a particular function or item of data; for example, refer to the permissions set out in figure 23-8.

In order for users to perform a particular function, they must have the right permissions at every level where permissions may be granted. A customer wishing to display a service, must have permission to:

- View that service
- View the service type
- View services
- Execute the function that retrieves and displays the information.

If a required permission is not granted at any level, the customer cannot perform the function. Just because permission is granted at one level, it does not mean permission is implied at another level.

Permissions may be assigned to the following elements:

- Servlets
- Entity classes
- Entity subclasses
- Entities.

These elements and the permissions that may be assigned are outlined in the following sections.

Servlets

Servlets are the web applications customers use to perform functions such as making a credit card payment, or retrieving service details. The following is a list of valid servlets:

- Customer Details
- Update Contact Details
- Account History
- Services (service listing screen)
- Enquiry (enquiry form)
- Tree (customer hierarchy control)
- Loading (loading page that is loaded on successful login)
- View (view control)
- Change Password
- Account Summary
- Update Contact Submit (submission of updated contact details)
- Invoice Details
- Invoice Image
- Make Payment
- Make Payment Confirm (confirmation of payment details)
- Make Payment Submit (submission of payment details)
- Enquiry Submit (submission of enquiries)
- Service Details
- Unbilled Usage.

The two permissions that must be set for servlets are:

- Execute
- No Access.

Access to servlets and other elements must be specifically granted; that is, users are not able to execute servlets unless they are granted 'Execute' access.

Entity classes

Entity classes include things like products, services, equipment, and customer nodes in general, rather than specific instances. The permissions that may be applied to entity classes include:

- Read
- Write
- Create
- Delete
- No Access.

Entity subclasses

Entity subclasses are types in Singl.eView Convergent Billing. Examples of entity subclasses include a dialup service or a residential customer type. The permissions that may be applied to entity subclasses include:

- Read
- Write
- Create
- Delete
- No Access.

Entities

Entities are the specific instances of types in Singl.eView Convergent Billing; for example, a specific customer node, Oxford Systems, is an entity. The permissions that may be applied to entities include:

- Read
- Write
- No Access.

Permissions for product, service, and child customer entities are inherited from the permissions for the parent customer entities, unless permission is explicitly set. For example, if a web user has 'Read' and 'Write' access to a parent customer node, and no other permissions are set, they also have 'Read' and 'Write' access to all of the child customer nodes, products, and services under the parent.

Figure 23-3 illustrates the concept of access inheritance for Entities.

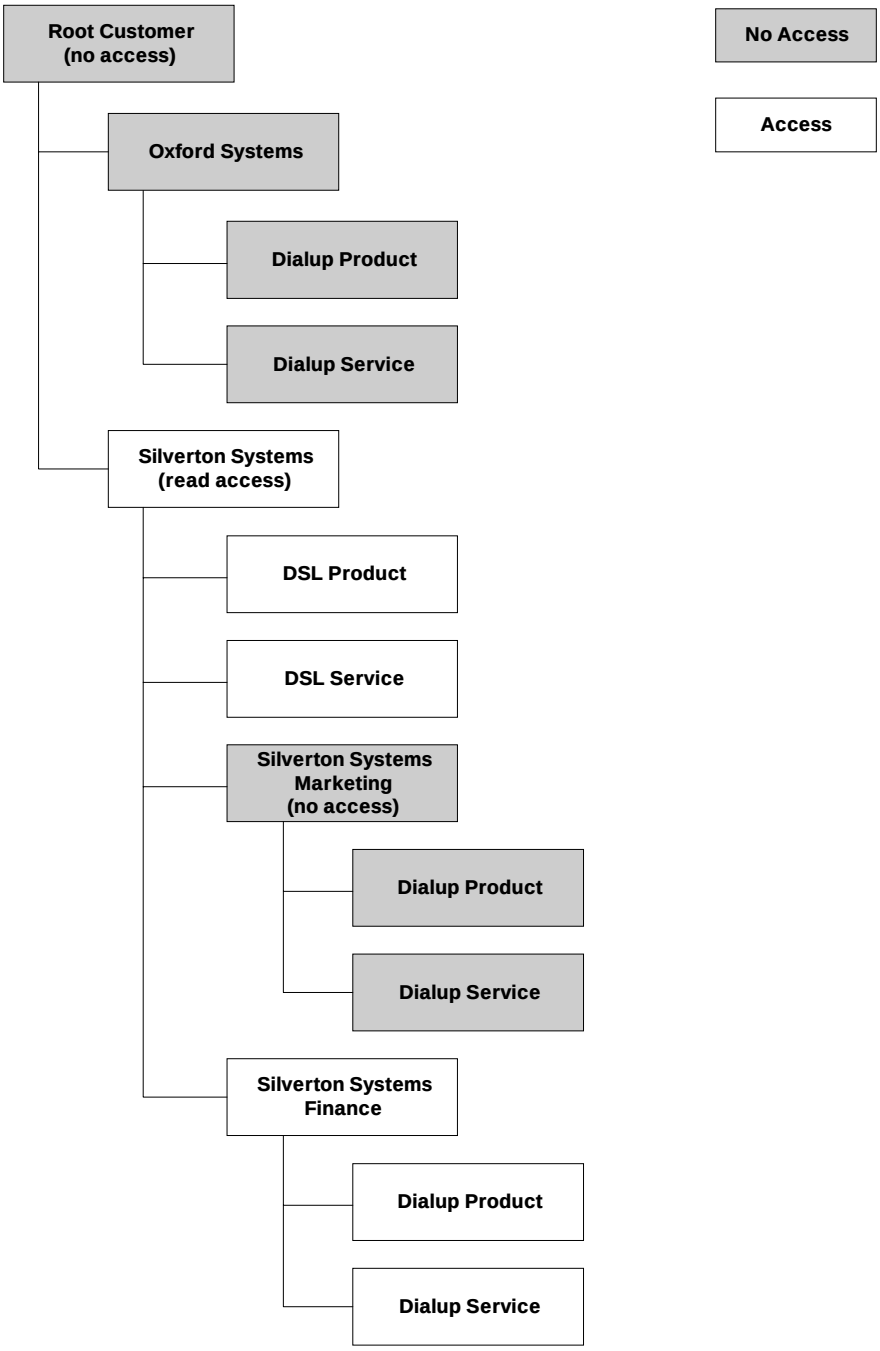


Figure 23-3 Access Inheritance

No permissions are set for the root node of the customer hierarchy. The permission for Silverton Systems in figure 23-8 is set to 'Read' access. Therefore, all customer, product and service entities under Silverton Systems inherit 'Read' access, except for Silverton Systems Marketing, which is explicitly set to 'No Access'.

Roles

Rather than set access permissions for each and every function and element of data for every user, permissions are grouped into roles. The role is then assigned to a web user.

For example, a Standard Web User role might exist, which provides the following kinds of permissions:

- 'Read' access for all entity classes and subclasses.
- No 'Write', 'Create', or 'Delete' access for entity classes and subclasses.
- 'Execute' access for all servlets.

A Super Web User role, on the other hand, might provide the following access:

- 'Read', 'Write', 'Create', and 'Delete' access for all entity classes and subclasses.
- 'Execute' access for all servlets.

Roles are defined through derived attributes and may be modified or created depending on the requirements of a particular installation.

Note: Web enablement roles are not to be confused with roles used in standard Singl.eView Convergent Billing security.

Derived attributes for role creation

Roles are defined using a series of derived attributes. The first, `dWEBRolePerms&`, is shown in figure 23-4.

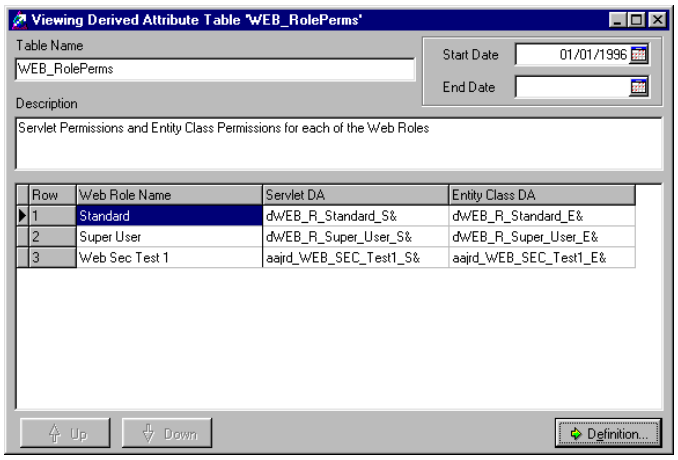


Figure 23-4 WEBRolePerms&

Note: In figure 23-4, there are three web roles. For each role there are corresponding servlet and entity class derived attributes. These define the servlet, entity class, and subclass permissions for each role.

The Standard Web Role derived attribute tables are shown in figures 23-5 and 23-6.

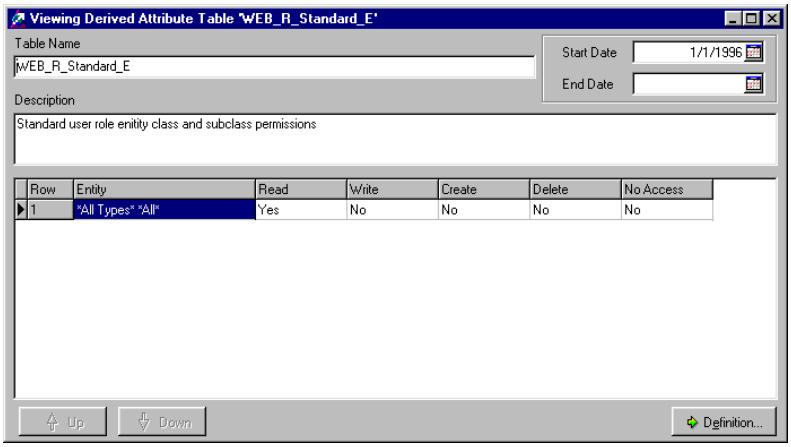


Figure 23-5 Entity Class and Subclass Permissions for Standard User Role

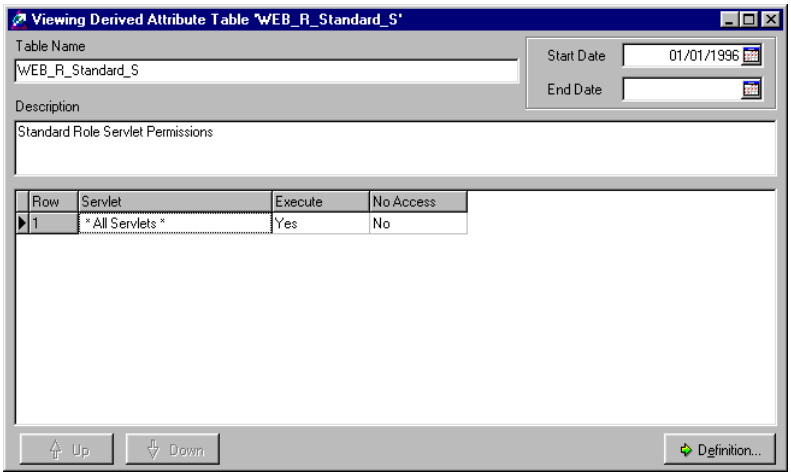


Figure 23-6 Servlet Permissions for Standard User Role

Note: The Standard Web User role has 'Read Only' access for all entity types, whereas the Super User role has 'Read', Write', 'Create', and 'Delete' permissions, as shown in figure 23-7.

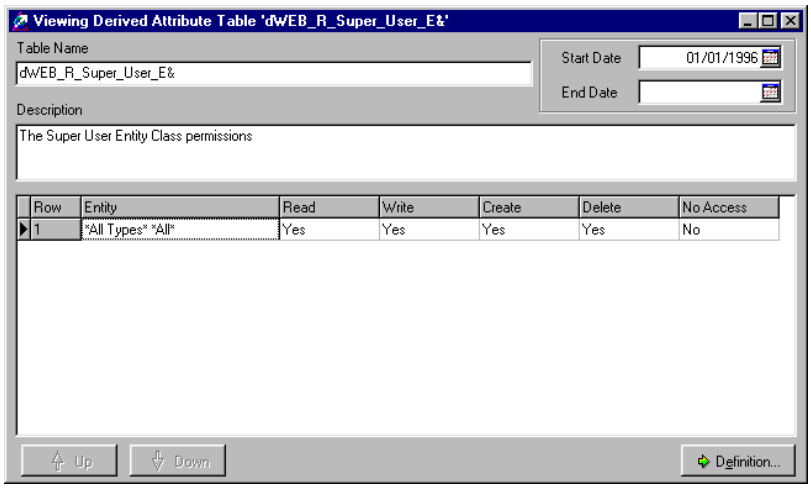


Figure 23-7 Entity Class and Subclass Permissions for Super User Role

Depending on the security requirements for an installation, custom permission roles can also be defined. An example of a customised web user role is shown in figure 23-8.

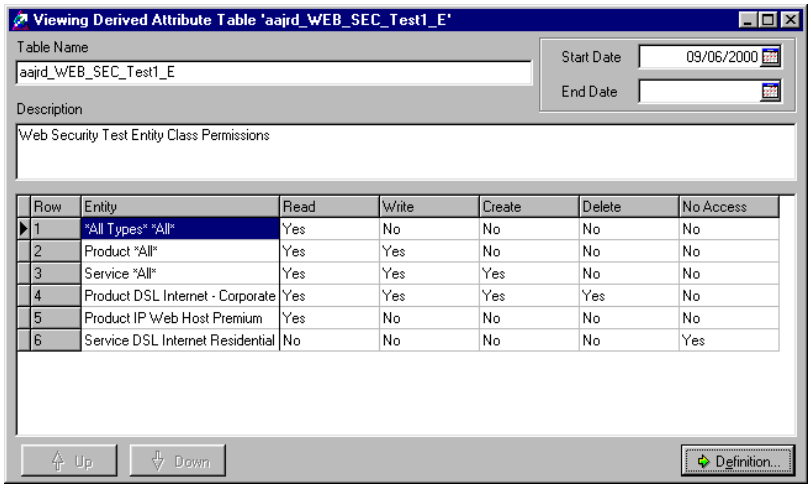


Figure 23-8 Entity Class and Subclass Permissions for a Sample Customised Web User Role

This web user role has a variety of permissions. In particular, web users assigned this role have 'No Access' to the DSL Internet Residential service.

The servlet permissions for this role are shown in figure 23-9.

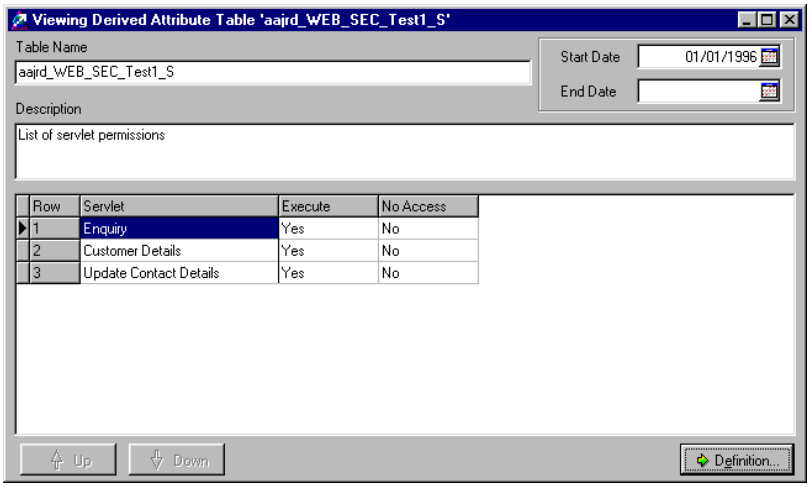


Figure 23-9 Servlet Permissions for a Sample Customised Web User Role

A web user with this role can only submit inquiries using the *Enquiry* form, inquire about customer details, and update contact details. They cannot perform any other functions.

Encryption

Customer information sent over the internet is typically confidential. To stop it from being intercepted or tampered with in transit, data must be encrypted.

The web interface for Singl.eView Convergent Billing makes use of the Secure Sockets Layer (SSL) for transmitting secure information across the internet. SSL is a widely implemented standard for providing secure data transmissions.

Auditing

Auditing web enabled user activity is required for the following reasons:

- Tracking of user trends
- Tracking of errors
- Tracking of single and repeated instances of unauthorised activities.

While the level of auditing will vary from site to site, at a minimum, information from the following events is captured:

- User login and the creation of their session
- Details of every incoming web request.

Likewise, the amount of information recorded for certain events varies according to the type of event. However, all audit records include the following information:

- An identifier for the system that generated the record
- Date and time of the event
- User associated with the event
- Session associated with the event
- Any other information required to describe the event or to track further information from other sources.

Appendix A
Configuration File Reference Information and Example

Configuration File Reference Information

The Configuration File is used to store the various settings required to run the RADIUS Accounting Real Time Rating Process. The file is in Extensible Markup Language (XML), which is used to assist in various implementations.

```
<!-- Global Configuration Document -->
<!ELEMENT ConfigFile (SubConfig*)>
<!ATTLIST ConfigFile
name CDATA #REQUIRED
version CDATA #IMPLIED>

<!-- Configuration attributes for a particular
module/process -->
<!ELEMENT SubConfig (GenAttr, InstAttr*)>
<!ATTLIST SubConfig
name CDATA #REQUIRED
version CDATA #IMPLIED>

<!-- General attributes (valid for all instances,
or used as defaults) -->
<!ELEMENT GenAttr (Attribute*)>

<!-- Instance attributes (particular instance
identified by the SeqNr) -->
<!ELEMENT InstAttr (Attribute*)>
<!ATTLIST InstAttr
SeqNr CDATA #REQUIRED>

<!-- Generic attribute element -->
<!ELEMENT Attribute EMPTY>
<!ATTLIST Attribute
name CDATA #REQUIRED
value CDATA #REQUIRED>
```

The root element of the document is *SavilleExpressConfig.xml*, noting that it must be the same as the actual filename. Additional modules to the *RdsAcctRTR* module may be added later to the DTD but are ignored by this process. This allows the flexibility of maintaining a single configuration file for all Singl.eView Convergent Billing modules or having separate configuration files.

A configuration loader object uses an XML parser to read the configuration from an input stream. It then makes the configuration available to the process, merging the general and relevant instance data to create a single set of attributes. An instance is identified by a sequence number that may be passed on the command line; if no sequence number is passed, the instance defaults to one. Instance attributes may override general attributes, and any attributes that have more than one value will be made available as a list of values. The process will know this to be the case in advance for the relevant attributes. Any type conversions that are required are also performed by the configuration loader.

Configuration File Example

```
<?xml version="1.0"?>
<!DOCTYPE ConfigFile SYSTEM "configuration.dtd">

<ConfigFile name="SavilleExpressConfig.xml"
version="1.0">

<SubConfig name="RdsAcctServer" version="1.0">
<GenAttr>

<Attribute name="GLOBAL_CONFIGPATH" value=""/>
<Attribute name="GLOBAL_TEMP" value=""/>

<Attribute name="OUTPUT_CMADDRESS"
value="hercules"/>
<Attribute name="OUTPUT_CMPORT" value="7536"/>
<Attribute name="OUTPUT_ERRORPATH" value=""/>
<Attribute name="OUTPUT_BATCHPATH" value=""/>
<Attribute name="RC_DATA" value="rd.dat"/>
<Attribute name="RC_TLOG" value="rd.log"/>
<Attribute name="RC_TLSIZE" value="1000"/>
<Attribute name="BATCH_BASESTRING" value="batch"/>
<Attribute name="BATCH_TIMESTAMP" value="true"/>
<Attribute name="BATCH_EXT" value=".evt"/>
<Attribute name="BATCH_TIMELIMIT" value="0"/>
<Attribute name="BATCH_SIZELIMIT" value="5000000"/>
<Attribute name="ERR_BASESTRING" value="error"/>
<Attribute name="ERR_TIMESTAMP" value="true"/>
<Attribute name="ERR_EXT" value=".log"/>
<Attribute name="ERR_TIMELIMIT" value="0"/>
<Attribute name="ERR_SIZELIMIT" value="0"/>
<Attribute name="LISTENER_PORT" value="8888"/>
<Attribute name="STATLOGGER_PERIOD" value="60000"/>

</GenAttr>
<InstAttr SeqNr="1">
<Attribute name="RC_TLSIZE" value="2000"/>
<Attribute name="BATCH_BASESTRING"
value="batcher"/>
<Attribute name="ERR_BASESTRING" value="err"/>
<Attribute name="ERR_TIMESTAMP" value="false"/>
<Attribute name="ERR_EXT" value=".out"/>
</InstAttr>
</SubConfig>

</ConfigFile>
```

Appendix B
Outbound File Format for Pre-authorised and Credit Card Payments

This section describes the format and content of the interface files produced by the credit card payments and pre-authorised payments interface process. The delivered format and content will be compatible with tools such as CreditNow and ICVERIFY. However, deployment teams will be able to easily modify the format and content.

The output file formats will be quote-comma delimited ASCII files. A comma separates the fields and each field is enclosed within quotes. A new line character will terminate all records.

The file names will contain a constant component to identify the file type and a date component based on the effective date of the run. The ID of the payment interface task will also be available for use by deployment teams in naming the files. Different file name components can be used for credit card payment files and pre-authorised payment files.

The delivered format and content will include a number of different record types. However, the presence or absence of a given record type will be controlled by deployment settings and file generation criteria. The following record types will be available:

- File header (optional pair with file trailer)
- File trailer (optional pair with file header)
- Batch header (optional pair with batch trailer, used for embedded batches)
- Batch trailer (optional pair with batch header, used for embedded batches)
- Transaction Detail (mandatory).

There may be one or more embedded batches in the one file, if this option has been selected.

The information that will be made available for each of the record types is given below. The inclusion of a field in the delivered output is indicated by the presence of a field sequence number. If the field is not sequenced then the information is available when producing the record, but will not be included in the actual delivered output. Deployment teams will be able to make use of the information if required. Some of the information will only be available for pre-authorised payments and for specific combinations of file generation criteria.

File Header

Seq.	Name	Example Data	Description
1	Record Type	"H"	Identifies the record as a header. May be changed during deployment (eg. to a numeric code).
	File record sequence number	"1"	Sequential numbers available for each record in the file. Header starts at 1.
2	File Identifier	"PAPAYM"	Derived from individual file generation criteria. Configurable. Defaults to a preset constant that can be modified during deployment. No restriction on size or type.
3	Client Identifier	"XPSCLT"	Preset constant field. May be modified during deployment. No restriction on size or type.
4	Destination Identifier	"CLRNHS"	Derived from the Card Type/Banking agency (pre-authorised payments). Configurable. Default is a preset constant field that can be modified during deployment. No restriction on size or type.

Seq.	Name	Example Data	Description
5	File effective date and time	“DD-MM-YYYY HH:MM:SS”	Effective date for the creation of the file. Output format may be modified during deployment.
	File creation date and time	“DD-MM-YYYY HH:MM:SS”	Actual date and time when the file was created. Output format may be modified during deployment.

Batch Header

Seq.	Name	Example Data	Description
1	Record Type	“S”	Identifies the record as a batch header. May be changed during deployment (eg. to a numeric code).
	File record sequence number	“2”	Sequential numbers available for each record in the file.
	Batch sequence number	“1”	Sequential numbers that increment for each batch in the file, starting at 1 for the first batch.
	Batch record sequence number	“1”	Similar to the file record sequence number value, but specific to the batch.
2	Batch identifier	“PAPAYM”	Derived from individual batch generation criteria. Configurable. Defaults to a preset constant that can be modified during deployment. No restriction on size or type.

3	Client Identifier	“XPSCLT”	Preset constant field. May be modified during deployment. No restriction on size or type.
4	Destination Identifier	“CLRNHS”	Derived from the Card Type/Banking agency. Configurable. The default is a preset constant field that can be modified during deployment. No restriction on size or type.
	File effective date and time	“DD-MM-YYYY HH:MM:SS”	Effective date for the creation of the file. Output format may be modified during deployment.
	File creation date and time	“DD-MM-YYYY HH:MM:SS”	Actual date and time when the file was created. Output format may be modified during deployment.

Transaction Detail

Seq.	Name	Example Data	Description
1	Record Type	“D”	Identifies the record as a transaction detail. May be changed during deployment (eg. to a numeric code).
	File record sequence number	“3”	Sequential numbers available for each record in the file.
	Batch sequence number	“1”	Sequential numbers that increment for each batch in the file, starting at 1 for the first batch.
	Batch record sequence number	“2”	Similar to the file record sequence number value, but specific to the batch.
2	Account Number	“000987345”	Primary Account ‘Name’ for the invoice node.
3	Payment Due Date	“DD-MM-YYYY”	Due date for the payment. Format may be modified during deployment.
4	Payment amount	“345.23”	The amount for the created payment.
5	Invoice Number	“07653242092”	Customer Invoice String for the invoice being paid.
6	Payment Type Code	“V” (Visa)	Derived from Payment Method and Credit Card Type for Credit Cards.
7	Bank Id	“3456789”	Derived from the banking agency (direct debit only). Left blank for credit cards.

8	Credit Card/Bank Account Number	“9992756434111666”	
9	Credit Card Expiry Date	“MM/YY”	Credit Cards only. Format may be modified during deployment. Left blank for Direct Debit.

Batch Trailer

Seq.	Name	Example Data	Description
1	Record Type	“E”	Identifies the record as a batch trailer. May be changed during deployment (eg. to a numeric code).
	File record sequence number	“3457”	Sequential numbers available for each record in the file.
	Batch sequence number	“9”	Sequential numbers that increment for each batch in the file, starting at 1 for the first batch.
	Batch record sequence number	“371”	Similar to the file record sequence number value, but specific to the batch.
2	Total number of batch records	“371”	Total number of records in the batch, including header and trailer.
3	Total payment amount of the batch.	“3000.50”	Total dollar amount for the batch.

File Trailer

Seq.	Name	Example Data	Description
1	Record Type	“T”	Identifies the record as a file trailer. May be changed during deployment (eg. to a numeric code).
	File record sequence number	“3”	Sequential numbers available for each record in the file.
2	Total number of records in the file	“345”	Total number of records in the batch, including header and trailer.
	Total number of batches in the file	“3”	Total number of batches in the file.
3	Total payment amount in the file.	“3000.50”	Total dollar amount for the file.

Outbound File Format for Pre-authorised and Credit Card Payments

Appendix C
Inbound File Format for Lockbox

This section describes the format options and expected content of the interface files processed by the Lockbox interface. The delivered process will allow easy modification by deployment teams of the input file format.

The expected input file format is fixed field length ASCII. The record length may vary based on record type, or alternatively may be space padded to a fixed length. A new line character will terminate all records. Integer field will be right justified and zero padded. String fields will have any leading or trailing blanks removed. Date/time fields are expected to have a format of some portion of "YYYYMMDDHHNNSS". Real values will have an implicit decimal point.

The file name of the interface file will be specified as a parameter to the interface process.

Support for processing a number of different record types within the same file will be provided. These record types are:

- File Header Record (optional, one per file)
- Batch Header Record (optional pair with Batch Trailer, one per batch, multiple batches per file)
- Transaction Detail Record (one or more per file and batch)
- Batch Trailer Record (optional pair with Batch Header)
- File Trailer Record (optional, one per file).

The expected content of each record type is given below. Only the transaction detail records are mandatory, and only some fields within these records are mandatory. Optional record types and fields will be used for verification purposes and as a source for additional stored information.

Example input file formats will be used during system testing, and a template parsing module and mapping module will be supplied as part of the release. These templates will be used to create the modules that parse the example test files.

File Header

Field Name	Description
Record Identifier	Alphanumeric value, identifying the record as a file header.
Record sequence number	Sequential integer value starting at 1 for the first record in the file.
File Type Identifier	Alphanumeric value indicating the file is a lockbox file.
Source Agency identifier	Alphanumeric value indicating the clearinghouse that created the file.
Client identifier	Alphanumeric value identifying the client.
File record count	Integer value equal to the number of records in the file, including headers and trailers.
Batch count	Integer value equal to the number of batches in the file.
File total payment amount	Real value (two implied decimal places) equal to the total amount for all payments within the file.
Creation date	Date value for when the payment file was created.

Batch Header

Field Name	Description
Record Identifier	Alphanumeric value, identifying the record as a batch header.
Record Sequence number	Sequential integer value starting at 1 for the first record in the file.
Batch Sequence Number	Sequential integer value starting at 1 for the first batch. Value is constant across all records within the batch.
Batch Record Sequence number	Sequential integer value starting at 1 for the first record in the batch.
Batch identifier	Alphanumeric value identifying the type of batch (eg. lockbox number, batch of certain payment types).
Source Agency identifier	Alphanumeric value indicating the clearinghouse or upstream agency for which the batch is for.
Client identifier	Alphanumeric value identifying the client.
File record count	Integer value equal to the number of records in the file, including headers and trailers.
File total payment amount	Real value (two implied decimal places) equal to the total amount for all payments within the file.
Effective date	Effective Date value for the payments within the batch.

Transaction Detail

Field Name	Description
Record Identifier	Alphanumeric value, identifying the record as a detail record.
Record Sequence number	Sequential integer value starting at 1 for the first record in the file.
Batch Sequence Number	Sequential integer value starting at 1 for the first batch. Value is constant across all records within the batch.
Batch Record Sequence number	Sequential integer value starting at 1 for the first record in the batch.
Account Number	Mandatory. Alphanumeric string. Matched with the primary account 'name' for an invoicing node.
Payment Date	Date value, mandatory unless supplied in the batch/file header record.
Cheque Amount	Real value with two decimal places implied. Mandatory.
Cheque Bank identifier	Alphanumeric value identifying the bank from the cheque details.
Cheque Number	Numeric string from the cheque details.
Invoice Number	Invoice associated with the payment (Customer Invoice String). This value will only be used for verification. Payments will be associated to invoices during the apply_invoice processing.

Batch Trailer

Field Name	Description
Record Identifier	Alphanumeric value, identifying the record as a batch trailer.
Record Sequence number	Sequential integer value starting at 1 for the first record in the file.
Batch Sequence Number	Sequential integer value starting at 1 for the first batch. Value is constant across all records within the batch.
Batch Record Sequence number	Sequential integer value starting at 1 for the first record in the batch.
Batch identifier	Alphanumeric value identifying the type of batch (eg. lockbox number, batch of certain payment types).
File record count	Integer value equal to the number of records in the file, including headers and trailers.
File total payment amount	Real value (two implied decimal places) equal to the total amount for all payments within the file.
Effective date	Effective Date value for the payments within the batch.

File Trailer

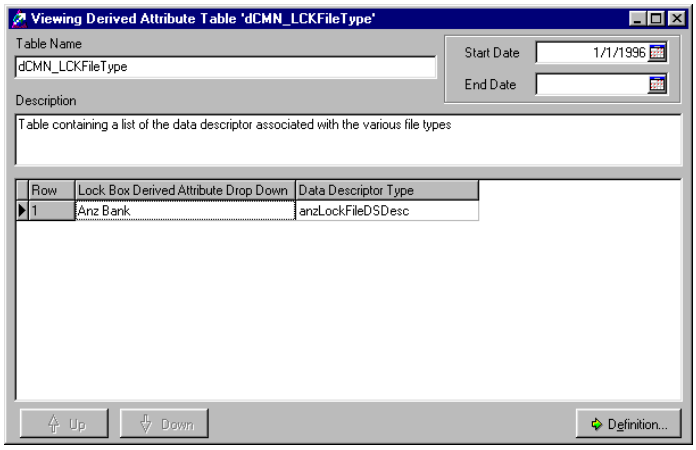
Field Name	Description
Record Identifier	Alphanumeric value, identifying the record as a file header.
Record sequence number	Sequential integer value starting at 1 for the first record in the file.
File Type Identifier	Alphanumeric value indicating the file is a lockbox file.
File record count	Integer value equal to the number of records in the file, including headers and trailers.
Batch count	Integer value equal to the number of batches in the file.
File total payment amount	Real value (two implied decimal places) equal to the total amount for all payments within the file.
Creation date	Date value for when the payment file was created.

Appendix D
Configuration for Process Lockbox Files Schedule Type

Before the parameters for the Process Lockbox Files schedule type can be defined, the `dCMN_LCKFileType` derived attribute, the `CMN_LCKBOX_FILE_TYPE` reference type, and the `fCMN_LockboxOptions&` function must be updated. This should be done in consultation with ADC personnel. For more information on configuration, refer to the *Customer Care Configuration Guide for Singl.eView Convergent Billing.*

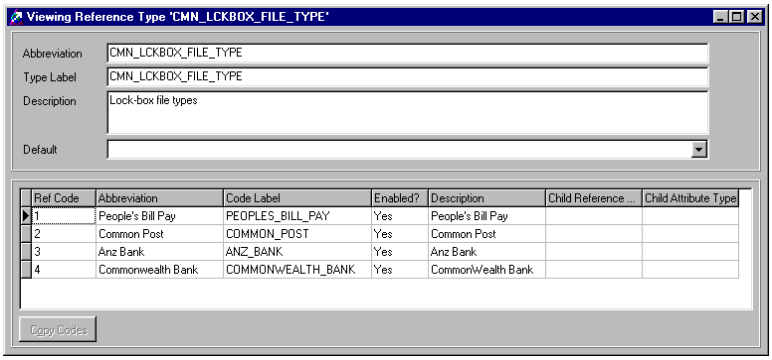
Updating the derived attribute and reference type is necessary so that Singl.eView Convergent Billing can recognize and process the particular types of lockbox payment files that your organization will be receiving. Any time that a new lockbox file format needs to be processed, these entities will also have to be updated.

The `dCMN_LCKFileType` derived attribute (shown with sample data in figure D-1) allows Singl.eView Convergent Billing to identify the incoming payment file and retrieve the associated data descriptor type. The data descriptor allows Singl.eView Convergent Billing to convert the incoming file to a standard format that can be used to create payment batches and allocate the payments to accounts.



D-1 dCMN_LCKFileType Derived Attribute

The CMN_LCKBOX_FILE_TYPE reference type (associated with the entity validation for the dCMN_LCKFileType derived attribute) defines the valid values for the *Lock Box File Type* field on the *Parameters* tab of the *Schedule Definition* form. It also defines the Index values to be used in the derived attribute to determine the data descriptor. Figure D-2 shows an example of the CMN_LCKBOX_FILE_TYPE reference type.



D-2 CMN_LCKBOX_FILE_TYPE Reference Type

The fCMN_LockboxOptions& function (shown in figure D-3) must also be maintained. It contains fundamental default information to be used in processing lockbox files, including where to pick up the lockbox flat file for processing, what to process the payment numbers with, and default text for the lockbox payments for the *Payment Description* and *Payment Type*.

```
#-------------------------------------------------------------------------------
#
#   File:       fCMN_LockboxOptions.epm
#   Created:    10-May-00 10:36:29
#   Creator:    mzj  (Jenny McCormack)
#   $Revision: 1.3 $
#   $Id: fCLEC_UpdateCaptureOptions.epm,v 1.3 1999/12/22 02:29:14 pjv Exp $
#
#       ============================================================
#       Copyright C - 1999 All rights reserved. Saville Systems PLC.
#
#       No part of this program may be reproduced or adapted in any
#             form or by any means, electronic or mechanical,
#       without permission from Saville Systems PLC. This program is
#           confidential and may not be disclosed, decompiled or
#           reverse engineered without permission in writing from
#                        Saville Systems PLC.
#       ============================================================
#
# NAME:
#   fCMN_LockboxOptions
#
# DESCRIPTION:
#   Provides all the client configurable parameters for Lockbox Interface.
#
# PARAMETERS:
#   var l_options?{} - This hash of unknown type will contain the values
#                      for the l_options listed above
#
# RETURNS:
#   1:  Always
#
# REVISION HISTORY:
#
#-------------------------------------------------------------------------------

fCMN_LockboxOptions&(var l_options?{}) = {
    l_options?{'WORKING_DIRECTORY'} :=
'/ip/ip_dev/express/atai/dev/src/payments/lockbox';#directory where to read lockbox file from
    l_options?{'PAYMENT_DESCRIPTION'} := 'LOCKBOX';#text to appear in payment description on
batch screen
    l_options?{'BATCH_TYPE'} := 'LOCKBOX BATCH';#batch type
    l_options?{'PAYMENT_LOCATION'} := 'PAYMENT_MADE_THROUGH_LOCKBOX';#default payment location
    l_options?{'RECIPT_NR_FORMAT'} := 'P';#variable to pre-fix receipt_nr associated with
payment
    l_options?{'PAYMENT_TYPE'} := 'Check';#default payment type to associate with all batches
if one is not specified in file

    l_options?{'ANZBANK_DEFAULT_SOURCE_AGENCY'} := '2222';#variable used in the creation of
dummy source agency if missing file header,batch header for Anz Bank
    l_options?{'ANZBANK_DEFAULT_CLIENT_ID'} := '3333';#variable used in the creation of dummy
client id if missing file header,batch header for Anz Bank
    l_options?{'ANZBANK_DEFAULT_DATE'} := 'SYSDATE';#date used for creating file header, file
trailer, batch header, batch trailer if this info is missing
                                                    #enter in format dd-mm-yyyy hr:mi:ss if you
don't want to use sysdate
    return 1;
}

#
#       End (of fCMN_LockboxOptions.epm)
#
```

Figure D-3 fCMN_LockboxOptions& Function

Configuration for Process Lockbox Files Schedule Type

Appendix E
DILs and Configuration Entities for IP/Data Event Feeds

This appendix contains the following information to supplement *Chapter 21, IP/Data Event Feeds and Normalised Event Layouts*:

- Generic Parser for RADIUS Accounting Server Log Data Feed
- Wrapper for Generic RADIUS Accounting Log Parser
- Generic Parser for CSV Event Feed
- Wrapper for Generic CSV Parser.

The event feed interface is broken down into two sections:

- A generic parser DIL parses the event feed and stores the information into a hash of values.
- A wrapper DIL validates the values in the hash and defines where they are to be stored in the normalised event.

The generic parser DIL can be common to all event types, whereas the wrapper is specific to each kind of event.

For Singl.eView IP/Data, two generic DILs will exist: one for processing XACCT RADIUS Accounting CSV Data Feeds and one for processing RADIUS Accounting Server Log Data Feeds. Wrappers that include these generic DILs and that take care of decoding the information from the hash can be implemented for each deployment. Two reference RADIUS Accounting wrapper DILs are provided that make use of the Generic CSV DIL and the Generic RADIUS Accounting Server Log DIL. Both wrappers are based on a standard wrapper template and can easily be customised to include the RADIUS Accounting LOG DIL instead of the General CSV DIL.

Generic Parser for RADIUS Accounting Server Log Data Feed

```
------------------------------------------------------------------------
-----

--

--  File:       genradlog.dil
--  Created:    10-03-2000 14:29:46
--  Creator:    myates  (Michael Yates)
--  $Revision: 1.6 $
--  $Id: genradlog.dil,v 1.6 2000/07/19 23:14:16 myates Exp $
--
--      ============================================================
--      Copyright C - 2000 All rights reserved. Saville Systems PLC.
--
--      No part of this program may be reproduced or adapted in any
--            form or by any means, electronic or mechanical,
--      without permission from Saville Systems PLC. This program is
--          confidential and may not be disclosed, decompiled or
--          reverse engineered without permission in writing from
--                          Saville Systems PLC.
--      ============================================================
--
-- DESCRIPTION:

--  Generic parser for RADIUS Accounting Log Feed. Validates event feed
and
--  stores values into a hash.
--

-- CAVEATS:

--

-- FILES:
--
-- NOTES:
--
-- SEE ALSO:
--
-- REVISION HISTORY:
--    $Log: genradlog.dil,v $
--    Revision 1.6  2000/07/19 23:14:16  myates
--    Moved DIL configuration parameters back to dil wrapper, since
these parameters
--    are specific to each individual wrapper
--
--    Revision 1.5  2000/06/19 23:47:04  myates
--    Initial release. Ready for system testing
--
--    Revision 1.4  2000/05/24 02:29:05  myates
--    Moved parameters semantics from wrapper to parser
--
--    Revision 1.3  2000/05/24 01:37:48  myates
--    Minor fixes
--
--    Revision 1.2  2000/05/24 01:02:57  myates
--    Added support for multi-valued attributes
--
--    Revision 1.1  2000/04/28 01:24:11  myates
--    Initial Version - prior to testing
```

```
--

-------------------------------------------------------------------------
-----

-------------------------------------------------------------------------
-----
--      DIL SPEC START
-------------------------------------------------------------------------
-----

-------------------------------------------------------------------------
-
--  EXPORTED TYPE DEFINITION
-------------------------------------------------------------------------
-
RECORD ::= SEQUENCE {
    parameters EMPTY,
    startrec   EMPTY,
    header$    HEADER, eol1 CHAR,
    seqattval  SEQOFATTVAL,
    predecode  DECCHC,
    endrec     INCCHC
}

-------------------------------------------------------------------------
-
--  SEMANTICS
-------------------------------------------------------------------------
-
startrec Semantics ::= {
    Action = "RS;Pre"
    Assignment = "{
                   undefine(recfields?{});
                   recignore& := 0;
                   recerror& := 0;
                  }"
}


header$ Semantics ::= {
    Conversion = "STRING"
    Action = "VE;Pre"
    Assignment = "{
                   var tmpstring$;

                   # raise error if header begins with whitespace
                   if (substr(header$,0,1) = ' ' or
                       substr(header$,0,1) = char(9) or
                       substr(header$,0,1) = char(10) or
                       substr(header$,0,1) = char(13)) {
                       recerror& := 1;
                   }
                   tmpstring$ := ltrim(rtrim(header$));

                   # raise error if header is empty
                   if (tmpstring$ = '') {
                       recerror& := 1;
                   }

                   if (!recerror&) {
                       recfields?{'recheader'} := tmpstring$;
                   }

                  }"

}
```

```
seqattval Semantics ::= {
    Action = "SM; Pre"
    Decoding = "Infinite"
}

errchk Semantics ::= {
    Action = "recerror&;FM;Post"
}


-- last record does not require a blank line to be a valid end
firstchar$ Semantics ::= {
    Conversion = "STRING"
    Action = "VE; Pre"
}

-- blank line marks the end of the current record
charchk Semantics ::= {
    Action = "firstchar$ = char(10);FM; Post"
}

-- cater for carraige returns
secondchar$ Semantics ::= {
    Conversion = "STRING"
    Action = "secondchar$ = char(10);FM; Post"
}


attribute$ Semantics ::= {
    Conversion = "STRING"
}

value$ Semantics ::= {
    Conversion = "STRING"
    Assignment = "{
                    var attribute1$;
                    var value1$;
                    var value2$;


                    attribute1$ := firstchar$ + secondchar$ +
attribute$;

                    #ensure attr-val pair begins with whitespace
matching defined format
                    if (position(attribute1$, options${'whitespace'})
!= 0) {
                        recerror& := 1;
                    }
                    else {
                        #trim surrounding whitespace from attribute and
value
                        #this also handles carraige return if it exists
                        attribute1$ := rtrim(ltrim(attribute1$));
                        value1$ := rtrim(ltrim(value$));
                        #trim off outer quotes on value if they exist
                        if (substr(value1$,0,1) = '\"' and
substr(value1$,length(value1$)-1,1) = '\"') {
                            value1$ := substr(value1$, 1,
length(value1$)-2);
                        }

                        # Insert into hash (supports multi valued
attributes)
                        # Case 1 -> Doesn't exist -> insert direct
                        if
(!fCMN_IsIn&(attribute1$,keys(recfields?{}))) {
                            recfields?{attribute1$} := value1$;
```

```
                        }
                        # Case 2 -> Does exist (single value) ->
extract value, add as array with new value
                        else if (is_string(recfields?{attribute1$})) {
                            value2$ := recfields${attribute1$};
                            recfields?{attribute1$}[] :=
[value2$,value1$];
                        }
                        # Case 3 -> Does exist (already array) -> add
value to end of array
                        else {
                            recfields?{attribute1$}[] :=
recfields?{attribute1$}[] + [value1$];
                        }
                    }
                    return(1);

                }"
}

eol2 Semantics ::= {
    Action = "VE;Post"
}

predecode Semantics ::= {
    Decoding = "recignore& = 1; parseIgnore;
                recerror&  = 1; parseError;
                Default;        decoder"
}

parseError Semantics ::= {
    Action = "EA;Post"
}

--
--  decoder Semantics ::= (defined in wrapper)
--


endrec Semantics ::= {
    Decoding = "recignore& = 1; wrapIgnore;
                recerror&  = 1; wrapError;
                Default;        include"
}

wrapError Semantics ::= {
    Action = "ER;Post"
}

include Semantics ::= {
    Action = "RE;Post"
}

------------------------------------------------------------------------
-
--  PRIVATE TYPE DEFINITIONS
------------------------------------------------------------------------
-
HEADER ::= KEYED STRING (TagIs("0A"))
SEQOFATTVAL ::= SEQUENCE OF attval ATTVAL
ATTVAL ::= SEQUENCE {
    errchk EMPTY,
    firstchar$ CHAR,
    charchk EMPTY,
    secondchar$ CHAR,
    attribute$  ATTRIBUTE,
    comparitor COMPARITOR,
```

```
    value$ VALUE,
    eol2  CHAR
}
ATTRIBUTE ::= KEYED STRING (TagIs("3D"))
COMPARITOR ::= CHAR
VALUE ::= KEYED STRING (TagIs("0A"))
DECCHC ::= CHOICE
{
    parseIgnore   EMPTY,
    parseError    EMPTY,
    decoder       EMPTY
}
INCCHC ::= CHOICE
{
    wrapIgnore    EMPTY,
    wrapError     EMPTY,
    include       EMPTY
}
CHAR ::= OCTET STRING (SIZE(1))

--
--      End (of genradlog.dil)
--
```

Wrapper for Generic RADIUS Accounting Log Parser

```
------------------------------------------------------------------------
-----

--
--  File:       radlog.dil
--  Created:    10-03-2000 14:30:00
--  Creator:    myates  (Michael Yates)
--  $Revision: 1.5 $
--  $Id: radlog.dil,v 1.5 2000/07/19 23:14:42 myates Exp $
--
--      ============================================================
--      Copyright C - 2000 All rights reserved. Saville Systems PLC.
--
--      No part of this program may be reproduced or adapted in any
--            form or by any means, electronic or mechanical,
--      without permission from Saville Systems PLC. This program is
--          confidential and may not be disclosed, decompiled or
--          reverse engineered without permission in writing from
--                          Saville Systems PLC.
--      ============================================================
--
-- DESCRIPTION:
--    Wrapper for generic RADIUS Accounting Log parser (DSL & Dialup).
--
-- CAVEATS:
--
-- FILES:
--
-- NOTES:
--
-- SEE ALSO:
--
-- REVISION HISTORY:
--    $Log: radlog.dil,v $
--    Revision 1.5  2000/07/19 23:14:42  myates
--    Moved DIL configuration parameters back to dil wrapper, since
these parameters
--    are specific to each individual wrapper
--
--    Revision 1.4  2000/06/19 23:47:20  myates
--    Initial release. Ready for system testing
--
--    Revision 1.3  2000/05/24 02:29:31  myates
--    Moved parameters semantics from wrapper to generic parser
--
--    Revision 1.2  2000/05/24 01:03:18  myates
--    Added support for multi-valued attributes
--
--    Revision 1.1  2000/04/28 01:25:08  myates
--    Initial Version - prior to testing
--
------------------------------------------------------------------------
-----

------------------------------------------------------------------------
-----
--  DIL SPEC START
------------------------------------------------------------------------
-----

RADLOG DEFINITIONS ::=
```

```
BEGIN

-------------------------------------------------------------------------
-----
--  INCLUDE FILES
-------------------------------------------------------------------------
-----

--Include "genradlog.dil"

-------------------------------------------------------------------------
-
--  MAIN SECTION
-------------------------------------------------------------------------
-
Start ::= SEQOFRECS
SEQOFRECS ::= SEQUENCE OF record RECORD

start Semantics ::= {
    Decoding = "Infinite"
}

record Semantics ::= {
    Error = "EI; 0A0A,0D0A0D0A"
}

-------------------------------------------------------------------------
-
--  CONFIGURATION PARAMETERS
-------------------------------------------------------------------------
-
parameters Semantics ::= {
    Assignment = "{

                    # Use the NAS-Identifier rather then then NAS-IP-
Address if both are present
                    options?{'useId'} := 1;

                    # White space indentation in RADIUS event feeds
                    options?{'whitespace'} := ' ';

                  }"
}

-------------------------------------------------------------------------
-
--  HASH DECODER & CHECKER
-------------------------------------------------------------------------
-
decoder Semantics ::= {

    Evaluation = "{
                    #Normalised event field assignment

                    # Set A & B Parties
                    if (recfields${'Calling-Station-Id'} != '') {
                        APartyId$ := recfields${'Calling-Station-Id'};
                    }
                    if (recfields${'Called-Station-Id'} != '') {
                        BPartyId$ := recfields${'Called-Station-Id'};
                    }
                    # Set Chargeable party
                    CPartyId$ := recfields${'User-Name'};

                    # Decoding of the date string from the header
                    var tmpDate$ := recfields${'recheader'};
                    ChargeStartDate~ :=  to_date(substr(tmpDate$,4,16)
```

```
+ substr(tmpDate$,length(tmpDate$)-4,4),'mmm d hh:nn:ss yyyy');

                    # Set Event Source
                    if (recfields${'NAS-Identifier'} != '') {
                        EventSource$ := 'Id: ' + recfields${'NAS-
Identifier'};
                        if (recfields${'NAS-IP-Address'} != '') {
                            if (options&{'useId'} != 1) {
                                EventSource$ := 'IP Addr: ' +
recfields${'NAS-IP-Address'};
                            }
                        }
                    }
                    else if (recfields${'NAS-IP-Address'} != '') {
                        EventSource$ := 'IP Addr: ' + recfields${'NAS-
IP-Address'};
                    }
                    # Set Event Class to Usage
                    EventClassCode& := 1;

                    # Other fields
                    Duration# := to_real(recfields${'Acct-Session-
Time'});
                    Volume# := to_real(recfields${'Acct-Input-Octets'})
+ to_real(recfields${'Acct-Output-Octets'});
                    General1$ := recfields${'Acct-Input-Octets'};
                    General2$ := recfields${'Acct-Output-Octets'};
                    General3$ := recfields${'Framed-IP-Address'};

                  }"

    Assignment = "{
                    #Hash value checking

                    if (recfields${'Acct-Status-Type'} != 'Stop') {
                        recignore& := 1;
                    }

                  }"
}

END

--
--      End (of radlog.dil)
--
```

Generic Parser for CSV Event Feed

```
------------------------------------------------------------------------
-----
--
--  File:       radlog.dil
--  Created:    10-03-2000 14:30:00
--  Creator:    myates  (Michael Yates)
--  $Revision: 1.5 $
--  $Id: radlog.dil,v 1.5 2000/07/19 23:14:42 myates Exp $
--
--      ============================================================
--      Copyright C - 2000 All rights reserved. Saville Systems PLC.
--
--      No part of this program may be reproduced or adapted in any
--            form or by any means, electronic or mechanical,
--      without permission from Saville Systems PLC. This program is
--          confidential and may not be disclosed, decompiled or
--          reverse engineered without permission in writing from
--                        Saville Systems PLC.
--      ============================================================

--
-- DESCRIPTION:
--    Wrapper for generic RADIUS Accounting Log parser (DSL & Dialup).
--
-- CAVEATS:
--
-- FILES:
--
-- NOTES:
--
-- SEE ALSO:
--
-- REVISION HISTORY:
--    $Log: radlog.dil,v $
--    Revision 1.5  2000/07/19 23:14:42  myates
--    Moved DIL configuration parameters back to dil wrapper, since
these parameters
--    are specific to each individual wrapper
--
--    Revision 1.4  2000/06/19 23:47:20  myates
--    Initial release. Ready for system testing
--
--    Revision 1.3  2000/05/24 02:29:31  myates
--    Moved parameters semantics from wrapper to generic parser
--
--    Revision 1.2  2000/05/24 01:03:18  myates
--    Added support for multi-valued attributes
--
--    Revision 1.1  2000/04/28 01:25:08  myates
--    Initial Version - prior to testing
--
------------------------------------------------------------------------
-----

------------------------------------------------------------------------
-----
--  DIL SPEC START
------------------------------------------------------------------------
-----
```

```
RADLOG DEFINITIONS ::=
BEGIN

-------------------------------------------------------------------------
-----
--  INCLUDE FILES
-------------------------------------------------------------------------
-----

--Include "genradlog.dil"

-------------------------------------------------------------------------
-
--  MAIN SECTION
-------------------------------------------------------------------------
-
Start ::= SEQOFRECS
SEQOFRECS ::= SEQUENCE OF record RECORD

start Semantics ::= {
    Decoding = "Infinite"
}

record Semantics ::= {
    Error = "EI; 0A0A,0D0A0D0A"
}

-------------------------------------------------------------------------
-
--  CONFIGURATION PARAMETERS
-------------------------------------------------------------------------
-
parameters Semantics ::= {
    Assignment = "{

                    # Use the NAS-Identifier rather then then NAS-IP-
Address if both are present
                    options?{'useId'} := 1;

                    # White space indentation in RADIUS event feeds
                    options?{'whitespace'} := ' ';

                  }"
}

-------------------------------------------------------------------------
-
--  HASH DECODER & CHECKER
-------------------------------------------------------------------------
-
decoder Semantics ::= {

    Evaluation = "{
                    #Normalised event field assignment

                    # Set A & B Parties
                    if (recfields${'Calling-Station-Id'} != '') {
                        APartyId$ := recfields${'Calling-Station-Id'};
                    }
                    if (recfields${'Called-Station-Id'} != '') {
                        BPartyId$ := recfields${'Called-Station-Id'};
                    }
                    # Set Chargeable party
                    CPartyId$ := recfields${'User-Name'};

                    # Decoding of the date string from the header
                    var tmpDate$ := recfields${'recheader'};
```

```
                    ChargeStartDate~ :=  to_date(substr(tmpDate$,4,16)
+ substr(tmpDate$,length(tmpDate$)-4,4),'mmm d hh:nn:ss yyyy');

                    # Set Event Source
                    if (recfields${'NAS-Identifier'} != '') {
                        EventSource$ := 'Id: ' + recfields${'NAS-
Identifier'};
                        if (recfields${'NAS-IP-Address'} != '') {
                            if (options&{'useId'} != 1) {
                                EventSource$ := 'IP Addr: ' +
recfields${'NAS-IP-Address'};
                            }
                        }
                    }
                    else if (recfields${'NAS-IP-Address'} != '') {
                        EventSource$ := 'IP Addr: ' + recfields${'NAS-
IP-Address'};
                    }
                    # Set Event Class to Usage
                    EventClassCode& := 1;

                    # Other fields
                    Duration# := to_real(recfields${'Acct-Session-
Time'});
                    Volume# := to_real(recfields${'Acct-Input-Octets'})
+ to_real(recfields${'Acct-Output-Octets'});
                    General1$ := recfields${'Acct-Input-Octets'};
                    General2$ := recfields${'Acct-Output-Octets'};
                    General3$ := recfields${'Framed-IP-Address'};

                }"

    Assignment = "{
                    #Hash value checking

                    if (recfields${'Acct-Status-Type'} != 'Stop') {
                        recignore& := 1;
                    }

                }"
}

END

--
--      End (of radlog.dil)
--
```

Wrapper for Generic CSV Parser

```
------------------------------------------------------------------------
-----

--
--  File:       radcsv.dil
--  Created:    10-03-2000 14:30:06
--  Creator:    myates  (Michael Yates)
--  $Revision: 1.5 $
--  $Id: radcsv.dil,v 1.5 2000/07/19 23:14:29 myates Exp $
--
--      ============================================================
--      Copyright C - 2000 All rights reserved. Saville Systems PLC.
--
--      No part of this program may be reproduced or adapted in any
--            form or by any means, electronic or mechanical,
--      without permission from Saville Systems PLC. This program is
--          confidential and may not be disclosed, decompiled or
--          reverse engineered without permission in writing from
--                          Saville Systems PLC.
--      ============================================================
--
-- DESCRIPTION:
--  Wrapper for generic CSV parser that decodes RADIUS accounting feeds
--  from Xacct
--
-- CAVEATS:
--
-- FILES:
--
-- NOTES:
--
-- SEE ALSO:
--
-- REVISION HISTORY:
--    $Log: radcsv.dil,v $
--    Revision 1.5  2000/07/19 23:14:29  myates
--    Moved DIL configuration parameters back to dil wrapper, since
these parameters
--    are specific to each individual wrapper
--
--    Revision 1.4  2000/06/19 23:43:11  myates
--    Initial release. Ready for system testing
--
--    Revision 1.3  2000/05/24 02:30:17  myates
--    Moved parameters semantics from wrapper to generic parser
--
--    Revision 1.2  2000/05/24 01:02:31  myates
--    Added support for multi-valued attributes
--
--    Revision 1.1  2000/04/28 01:24:51  myates
--    Initial Version - prior to testing
--
------------------------------------------------------------------------
-----

------------------------------------------------------------------------
-----
--  DIL SPEC START
------------------------------------------------------------------------
-----
```

```
RADCSV DEFINITIONS ::=
BEGIN

-------------------------------------------------------------------------
-----
--  INCLUDE FILES
-------------------------------------------------------------------------
-----

--Include "gencsv.dil"

-------------------------------------------------------------------------
-
--  MAIN SECTION
-------------------------------------------------------------------------
-
Start ::= SEQOFRECS
SEQOFRECS ::= SEQUENCE OF record RECORD

start Semantics ::= {
    Decoding = "Infinite"
}

record Semantics ::= {
    Error = "EI; 0A"
}

-------------------------------------------------------------------------
-
--  CONFIGURATION PARAMETERS
-------------------------------------------------------------------------
-
parameters Semantics ::= {
    Assignment = "{

                    # Use the NAS-Identifier rather then then NAS-IP-
Address if both are present
                    options?{'useId'} := 1;

                    # Array of names to use as hash keys
                    options?{'recKeys'}[] :=
['username','password','aparty','bparty','rectype','nasid','nasipaddres
s','duration','inputdata','outputdata','usripaddress'];

                    # Names of columns to verify against
                    options?{'colNamesExist'} := 1;
                    options?{'colNames'}[] := ['User-
Name','Password','Calling-Station-Id','Called-Station-Id','Acct-Status-
Type','NAS-Identifier','NAS-IP-Address','Acct-Session-Time','Acct-
Input-Octets','Acct-Output-Octets','Framed-IP-Address'];

                }"
}

-------------------------------------------------------------------------
-
--  HASH DECODER & CHECKER
-------------------------------------------------------------------------
-
decoder Semantics ::= {

    Evaluation = "{
                    #Normalised event field assignment

                    # Set A & B Parties
                    if (recfields${'Calling-Station-Id'} != '') {
                        APartyId$ := recfields${'aparty'};
```

```
                    }
                    if (recfields${'Called-Station-Id'} != '') {
                        BPartyId$ := recfields${'bparty'};
                    }
                    # Set Chargeable party
                    CPartyId$ := recfields${'username'};

                    # If the csv feed provides an event date of some
kind then
                    # that should be used instead of the current_date.
                    ChargeStartDate~ := current_date();

                    # Set Event Source
                    if (recfields${'nasid'} != '') {
                        EventSource$ := 'Id: ' + recfields${'nasid'};
                        if (recfields${'nasipaddress'} != '') {
                            if (options&{'useId'} != 1) {
                                EventSource$ := 'IP Addr: ' +
recfields${'nasipaddress'};
                            }
                        }
                    }
                    else if (recfields${'nasipaddress'} != '') {
                        EventSource$ := 'IP Addr: ' +
recfields${'nasipaddress'};
                    }
                    # Set Event Class to Usage
                    EventClassCode& := 1;

                    # Other fields
                    Duration# := to_real(recfields${'duration'});
                    Volume# := to_real(recfields${'inputdata'}) +
to_real(recfields${'outputdata'});
                    General1$ := recfields${'inputdata'};
                    General2$ := recfields${'outputdata'};
                    General3$ := recfields${'usripaddress'};


                }"

    Assignment = "{
                    #Hash value checking

                    if (recfields${'rectype'} != 'Stop') {
                        recignore& := 1;
                    }

                }"
}

END

--
--      End (of radcsv.dil)
```

dIP_NE_EventType_Prfx_IP$ Derived Attribute

After the DIL for the RADIUS Accounting Server Log or XACCT RADIUS Accounting CSV Data Feed maps the `NAS-Identifier` or `NAS-IP-Address` to the `EVENT_SOURCE` normalised event field, the `dIP_NE_EventType_IP_Prfx$` derived attribute is called by the decoding expression that is defined in the Normalised Event File Type. Any IP Address contained in the raw event can then be mapped to the `EVENT_TYPE_CODE` normalised event field.

The field is populated as one of the following:

- Dialup Internet
- Dialup VPN
- DSL Internet
- DSL VPN.

This derived attribute uses `IP Address` as the index and returns the `Event Type`.

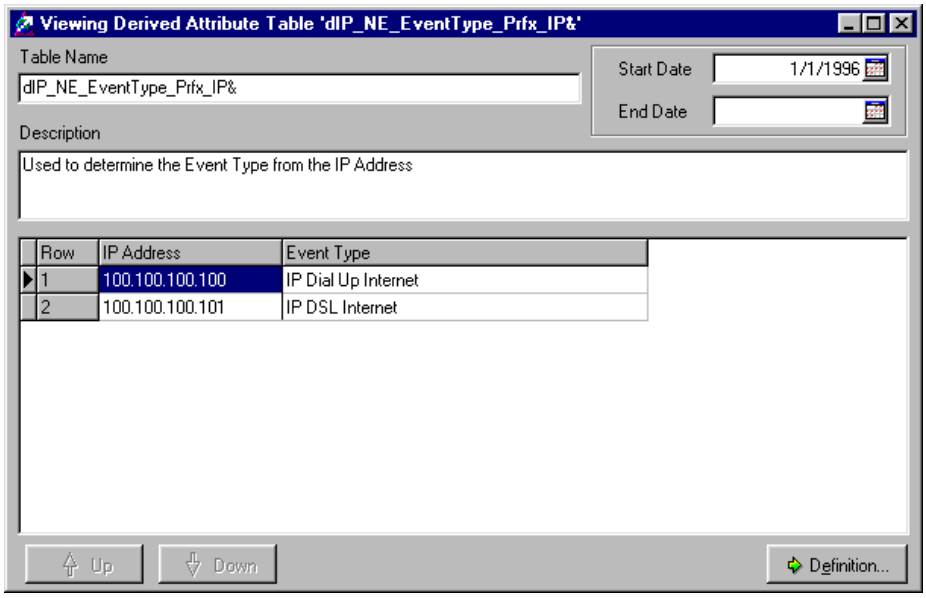


Figure E-1 dIP_NE_EventType_Prfx_IP& Derived Attribute

dIP_NE_EventType_Prfx_ID& Derived Attribute

After the DIL for the RADIUS Accounting Server Log or XACCT RADIUS Accounting CSV Data Feed maps the `NAS-Identifier` or `NAS-IP-Address` to the `EVENT_SOURCE` normalised event field, the `dIP_NE_EventType_Prfx_ID&` derived attribute is called by the decoding expression that is defined in the Normalised Event File Type. Any NAS-Identifier contained in the raw event can then be mapped to the `EVENT_TYPE_CODE` normalised event field.

The field is populated as one of the following:

- Dialup Internet
- Dialup VPN
- DSL Internet
- DSL VPN.

This derived attribute uses `NAS-Identifier` as the index and returns the `Event Type`.

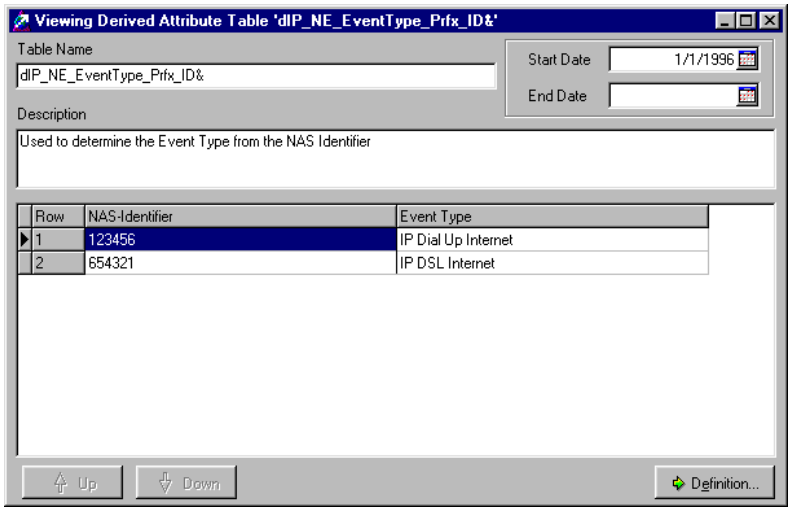


Figure E-2 dIP_NE_EventType_Prfx_ID&

dIP_NE_EventSubType_Roaming& Derived Attribute

After the DIL for the RADIUS Accounting Server Log or XACCT RADIUS Accounting CSV Data Feed maps the `NAS-Identifier` or `NAS-IP-Address` to the `EVENT_SOURCE` field, the decoding expression that is defined in the Normalised Event File Type, calls two derived attributes to map the `EVENT_TYPE_CODE` as either Dialup Internet, Dialup VPN, DSL Internet, or DSL VPN. These are the `dIP_NE_EventType_Prfx_IP&` and `dIP_NE_EventType_Prfx_ID&` derived attributes that are discussed earlier in this appendix. A third derived attribute, `dIP_NE_EventSubType_Roaming&` is called to map the `EVENT_SUB_TYPE_CODE` normalised event field for Dialup Internet events as either 'Standard Usage' or 'Roaming Usage' and for other Dialup and DSL events as 'Standard Usage'.

This derived attribute uses the `Freecall Prefix` as an index and returns the `Call Type`.

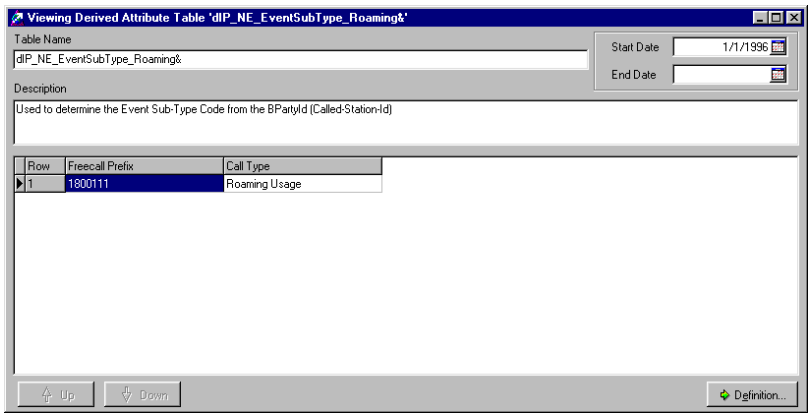


Figure E-3 dIP_NE_EventSubType_Roam& Derived Attribute

IP_NE_GVAS_CATEGORY Reference Type

The IP_NE_GVAS_CATEGORY reference type defines the Generic Value Added Service (GVAS) Category Codes for GVAS services. These category codes should be included in the raw event and are used to derive the units and charge per unit for the applicable tariffs. These codes are used to populate the EVENT_SUB_TYPE_CODE on the normalised event.

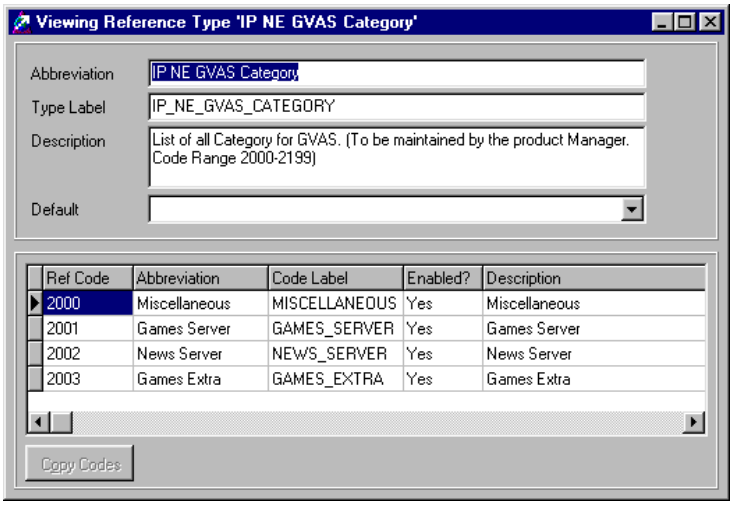


Figure E-4 IP_NE_GVAS_CATEGORY Reference Type